book 4

go 1.27

a complete language manual

Raeliosol

version 7.0, compiled 10 2026

Contents

1. toolchain and modules 8
   1.1. the module 8
   1.2. build tags, embed, generate 8
   1.3. build info 10
   1.4. the checks that gate every commit 10
2. lexical structure 11
   2.1. keywords, 25 and frozen 11
   2.2. identifiers and export 11
   2.3. literals 12
   2.4. semicolon insertion 13
   2.5. operators and precedence 14
3. types 16
   3.1. the inventory 16
   3.2. zero values, no uninitialized anything 16
   3.3. pointers, arrays, slices 17
   3.4. maps 18
   3.5. structs, embedding, and the 1.27 literal keys 20
   3.6. defined types, aliases, conversions 20
   3.7. strings 21
4. generics and generic methods 23
   4.1. type parameters and constraints 23
   4.2. generic types 23
   4.3. generic methods, the go 1.27 change 24
   4.4. function type inference, generalized in 1.27 26
5. functions 27
   5.1. multiple returns and named results 27
   5.2. defer 27
   5.3. closures and loop variables 28
   5.4. variadics 29
   5.5. methods and method sets 29
   5.6. init 30
6. concurrency 32
   6.1. goroutines 32
   6.2. channels 32
   6.3. select 33
   6.4. patterns 34
   6.5. the sync package 36
7. error handling 39
   7.1. the interface and the convention 39
   7.2. sentinels, wrapping, matching 39
   7.3. the typed nil trap 41
   7.4. panic and recover 42
8. program structure 44
   8.1. declarations 44
   8.2. if and for 44
   8.3. switch 46
   8.4. labels, break, continue, goto 47
   8.5. const and iota 47

8.6. packages and visibility ... 48
9. runtime ... 50
9.1. the scheduler ... 50
9.2. the garbage collector ... 50
9.3. profiling and debug ... 51
9.4. the goroutine lifecycle ... 52
10. allocation: mspans, size classes, escape analysis, sync.Pool ... 53
10.1. the fast path ... 53
10.2. the mspan ... 54
10.3. mcache, mcentral, mheap ... 54
10.4. the size class table ... 55
10.5. tiny and large ... 57
10.6. escape analysis ... 58
10.7. sync.Pool ... 59
10.8. the particle kernel ... 60
11. profiling: pprof, the tracer, benchstat, p99 ... 62
11.1. the profile set ... 62
11.2. pprof, read as text ... 62
11.3. the execution tracer ... 63
11.4. benchstat, honest comparison ... 64
11.5. runtime/metrics ... 65
11.6. gctrace, the raw feed ... 66
11.7. p99, the honest latency ... 67
11.8. the row kernel, ten million rows ... 68
11.9. which tool when ... 70
11.10. across the six books ... 71
12. stdlib tour 1: fmt, errors, slices, maps, cmp, time, slog ... 72
12.1. fmt ... 72
12.2. slices and maps ... 72
12.3. time ... 74
12.4. slog ... 75
13. stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io ... 77
13.1. json v2 ... 77
13.2. jsontext ... 79
13.3. uuid ... 80
13.4. net/http ... 81
13.5. crypto and simd ... 82
13.6. os and io ... 83
14. testing ... 85
14.1. the test function ... 85
14.2. examples are tests ... 85
14.3. fuzzing ... 86
14.4. benchmarks ... 87
14.5. testing/synctest ... 87
15. idioms ... 89
15.1. accept interfaces, return structs ... 89
15.2. functional options ... 89
15.3. zero values that work ... 90
15.4. errors are values, no hierarchies ... 91
16. htmx 4: html over the wire ... 92
16.1. fragments, not json ... 92

16.2. what changed in 4.0 ... 93
16.3. the core attributes ... 93
16.4. triggers and their modifiers ... 94
16.5. swaps and out of band ... 95
16.6. the response protocol ... 96
16.7. polling and server sent events ... 96
16.8. validation and errors ... 97
16.9. serving and pinning htmx ... 98
17. templ: compiled html components ... 100
17.1. the component is a function ... 100
17.2. codegen, with the artifact committed ... 101
17.3. composition and children ... 102
17.4. contextual escaping, honestly against html/template ... 102
17.5. rendering to strings for htmx swaps ... 104
17.6. wiring into net/http ... 104
17.7. when not to ... 105
18. http kernel ... 107
18.1. the handler contract ... 107
18.2. ServeMux as the route table ... 107
18.3. the error envelope ... 108
18.4. writing json responses with json/v2 ... 109
18.5. httptest the serverless server ... 110
18.6. the app skeleton and its wiring table ... 111
18.7. what the kernel proves ... 112
19. middleware ... 114
19.1. the chain type ... 114
19.2. recover ... 114
19.3. request-id ... 115
19.4. the access log ... 116
19.5. timeouts at two layers ... 117
19.6. wiring order and why ... 118
19.7. testing middleware ... 119
20. the user resource ... 121
20.1. the representation ... 121
20.2. decoding with strictness ... 122
20.3. validation from first principles ... 123
20.4. PATCH merge semantics ... 124
20.5. keyset pagination ... 125
20.6. the status contract ... 126
20.7. golden handler tests ... 127
21. authn ... 129
21.1. password storage with argon2id ... 129
21.2. login and timing flattening ... 130
21.3. server-side sessions ... 131
21.4. cookie mechanics ... 132
21.5. JWT from scratch with HMAC ... 133
21.6. JWT with EdDSA ... 134
21.7. the refresh rotation and reuse detection ... 135
21.8. testing auth without walls ... 136
22. authz and rbac ... 138
22.1. authentication versus authorization ... 138

22.2. roles and the grant table ... 138
22.3. policies as functions ... 139
22.4. 401 versus 403 ... 140
22.5. object-level authorization ... 141
22.6. deny by default with a route audit ... 142
22.7. the rbac matrix test ... 143
23. store ... 145
23.1. database/sql from first principles ... 145
23.2. migrations test driven ... 146
23.3. queries with context every time ... 147
23.4. the transaction idiom ... 148
23.5. isolation levels and what the engine actually gives you ... 149
23.6. the lost update demo and the CAS answer ... 150
23.7. the analytics query as an in-Go walk ... 151
23.8. store tests without docker ... 152
24. concurrency control ... 154
24.1. the lost update prevented ... 154
24.2. If-None-Match and 304 ... 155
24.3. idempotency keys ... 156
24.4. singleflight ... 157
24.5. the primitives revisited ... 158
24.6. errgroup with bounded fan-out ... 159
24.7. the race canary ... 160
25. caching ... 162
25.1. what caching promises ... 162
25.2. HTTP caching semantics for private APIs ... 162
25.3. LRU from first principles ... 163
25.4. TTL beside LRU ... 164
25.5. the stampede and its guard ... 165
25.6. invalidation on write ... 165
25.7. what not to cache ... 166
26. rate limiting ... 168
26.1. why limits exist ... 168
26.2. the token bucket from first principles ... 168
26.3. x/time/rate in production ... 169
26.4. the 429 contract ... 171
26.5. what to key on ... 172
26.6. limits under test ... 173
26.7. admission preview ... 174
27. observability ... 175
27.1. slog the real setup ... 175
27.2. structured beats interpolated ... 176
27.3. tracing from scratch ... 177
27.4. metrics and the prometheus text format ... 178
27.5. p99 done honestly ... 179
27.6. pprof on the live server ... 180
27.7. one request fully observed ... 181
28. load and p99 ... 183
28.1. the load harness ... 183
28.2. the report ... 184
28.3. benchstat over committed captures ... 184

28.4. graceful shutdown ... 185
28.5. testing shutdown without sleeps ... 186
28.6. admission control ... 187
28.7. the goroutine leak check ... 188
29. ship it ... 190
29.1. the image ... 190
29.2. compose one command ... 191
29.3. the docker lane as a gate ... 192
29.4. ci on GitHub Actions ... 194
29.5. helm the service envelope ... 195
29.6. terraform for this service ... 196
29.7. the shipped checklist ... 198
30. the test suite ... 199
30.1. what the suite already holds ... 199
30.2. properties as fuzz targets ... 200
30.3. the cache as a state machine ... 200
30.4. forgery resistance as a target ... 201
30.5. strictness without a framework ... 202
30.6. the testing.T surface and the generics truth ... 203
30.7. coverage without ceremony ... 204
30.8. what the suite proves ... 205
31. numerical statistics in go ... 207
31.1. why a game tool needs exact numbers ... 207
31.2. the binomial in log space ... 207
31.3. the 2x2 chi-square and its floor ... 209
31.4. midranks, the shared tie primitive ... 210
31.5. auc as a rank statistic ... 212
31.6. spearman, pearson on ranks ... 213
31.7. the deterministic bootstrap ... 214
31.8. correcting the family ... 215
31.9. the screening chain ... 218
32. fitting, backtesting, and guarded promotion ... 220
32.1. shrinkage toward a prior ... 220
32.2. the alpha the data chooses ... 221
32.3. completing the thin cells ... 223
32.4. masked-cell cross-validation ... 226
32.5. comparators that never feed the live score ... 227
32.6. a bagged blend under coordinate ascent ... 229
32.7. replaying drafts without lookahead ... 232
32.8. guarded promotion ... 234
32.9. the fit loop ... 237
33. the offline engine and its js twin ... 238
33.1. the offline frame ... 238
33.2. spacing plus a bucket ... 239
33.3. the client and its retries ... 240
33.4. fault-injecting fakes ... 242
33.5. the resumable backfill ... 244
33.6. the store decision ... 247
33.7. the go-to-js emitter ... 248
33.8. golden tests ... 249
33.9. the twin and the parity contract ... 250

34. capstone: concurrent crawler, indexer, and web ui 254
34.1. the fetch boundary 254
34.2. result, the generic method 255
34.3. the walk 255
34.4. the index side 257
34.5. the store, sqlite without cgo 259
34.6. the web ui 262
34.7. profiled, and why 265
34.8. what the capstone proves 265
34.9. the service part 266
35. appendices: coverage and sources 267
35.1. keyword coverage 267
35.2. stdlib coverage 267
35.3. pinned sources 268

1. toolchain and modules

Go ships as one command. `go build`, `go test`, `go vet`, `go fmt`, `go mod tidy`, `go generate`, `go doc` are all the same binary, which is why Go projects have no per-project build scripts: the module file plus the toolchain is the whole configuration surface. This book pins go 1.27, released august 2026, and every sample lives in one module at `go/samples/` with `go 1.27` in its `go.mod`.

1.1. the module

A module is a directory tree with a `go.mod` at the root declaring the module path and the language version:

```
module gobook

go 1.27
```

fragment, illustrative, not executed

The `go` line is a language version, not a toolchain request. Code in the module is compiled under 1.27 rules even when a newer toolchain builds it, so a language change like the promoted literal keys of chapter 3 is opt in per module. A separate `toolchain` directive, for example `toolchain go1.27.2`, requests a patch release, and `GOTOOLCHAIN=auto` (the default) downloads and switches to it transparently. Dependencies carry semantic import versions, `go get` records them in `go.mod` and hashes into `go.sum`, and `go mod tidy` adds what imports need and drops the rest, merging duplicate require blocks into direct and indirect since 1.27.

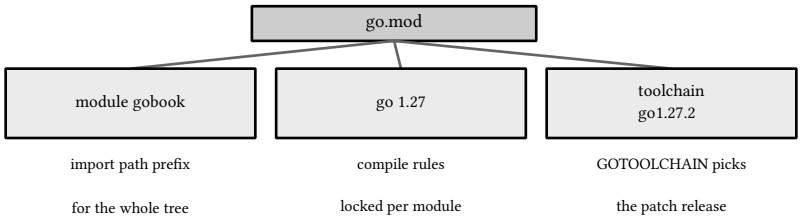


figure 1: one go.mod governs the whole tree: import paths, compile rules, patch release

NOTE: one module or many

One module per repository is the default stance, and this book keeps its samples in one. Multi module repositories exist for versioned public apis, and `go work` stitches several modules into one build for local development without editing their `go.mod` files.

1.2. build tags, embed, generate

Three directives run before, at, and after compilation, and the sample package uses all three.

Build tags choose which files compile. The `//go:build` constraint sits under the package clause, and exactly one of a pair of files enters the build:

```
//go:build !hardened

package ch01

// BuildFlavor is selected at compile time by build tags. The default
// build compiles this file; `go build -tags hardened` compiles the
// other one instead. Both declare the same symbol exactly once.
func BuildFlavor() string { return "standard" }
```

listing 1: the default variant compiles without tags (source: go/samples/ch01/flavor_default.go, lines 1-8)

`go build` takes this file, `go build -tags hardened` takes its twin `flavor_hardened.go`, and both declare `BuildFlavor` exactly once per build. This is how platform and build flavor differences are expressed, never through runtime flags. The constraints understand `GOOS`, `GOARCH`, `//go:build linux && amd64`, release tags like `go1.27`, and custom names like `hardened` above.

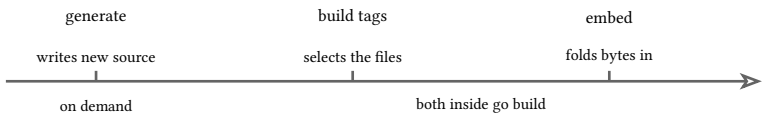


figure 2: three directives at three trigger points on the road from source to binary

`go:embed` folds a file into the binary at compile time:

```
//go:generate go run ./gen

import (
  _ "embed"
  "runtime"
  "runtime/debug"
)

//go:embed tagline.txt
var tagline string

// Tagline returns text folded into the binary at compile time by the
// go:embed directive. The file ships with the package.
func Tagline() string { return tagline }

// GoVersion reports the toolchain that built this binary.
func GoVersion() string { return runtime.Version() }
```

listing 2: generate directive, embed directive, and the metadata readers (source: go/samples/ch01/toolchain.go, lines 6-22)

The directive is an import of compile time data: the bytes of `tagline.txt` land in the binary verbatim, trailing newline included, with no file io at runtime. Embed accepts single files, patterns like `static/*.html`, and the `embed.FS` type for directory trees.

`go:generate` is a make for source. The directive runs a command with the package directory as working directory, and the convention is to commit both generator and output so reviewers see what ran:

```
//go:generate go run ./gen
// then, once:
go generate ./...
```

fragment, illustrative, not executed

The generator here writes `greeting_gen.go`, whose `GeneratedGreeting` is an ordinary function once generated, and the test suite asserts its value so a deleted or stale file fails `go test`, not production. Wire format marshaling, string tables, and code derived from schemas are the real world uses.

1.3. build info

A compiled binary knows what built it. `runtime.Version` answers during execution, and `debug.ReadBuildInfo` carries the same plus module versions and vcs state, which survives into installed binaries and is what `go version -m ./binary` prints from outside:

```
// survives into installed binaries where runtime.Version alone would
// still answer but Main.Path and friends would not.
func BuildInfoVersion() (string, bool) {
  info, ok := debug.ReadBuildInfo()
  if !ok {
    return "", false
  }
  return info.GoVersion, true
}
```

listing 3: build metadata, readable at runtime (source: go/samples/ch01/toolchain.go, lines 25-33)

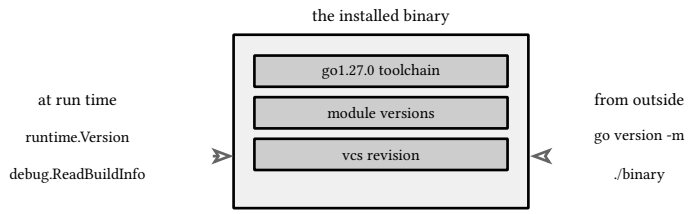


figure 3: what the binary knows about itself, read from inside and outside

1.4. the checks that gate every commit

`go vet ./...` runs the standard analyzers: printf argument mismatch, unreachable code, struct tags, lock copies, and `stringintconv`, the check that flags `string(65)` because it yields the rune `A` rather than `"65"`. `gofmt -l .` lists files whose formatting differs from canonical, and Go formatting is not configurable, which ends style debates by decree. `go fix -diff` previews the 1.27 modernizers, among them `atomictypes` for the atomic pointer types, `embedlit` and `slicesbackward`, and `go doc` now takes `package@version` to read a dependency's docs without cloning it. This repo runs vet plus tests for every go module it finds, under `make verify-go`.

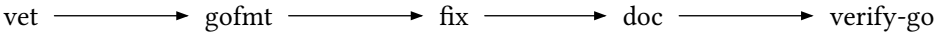


figure 4: the commit gate: vet analyzers, gofmt decree, fix modernizers, doc at version, one pipeline into make verify-go

sources: go.dev, go 1.27 release notes and modules reference, accessed 2026-09-08. Toolchain behavior verified live with go1.27.0 on windows, 26 tests across ch01 to ch03 green via `make verify-go`.

2. lexical structure

Go's lexical surface is the smallest of the four languages in this collection: 25 keywords, one identifier rule, semicolon insertion, and literals that read like C with the ambiguity removed. The sample package proves the rules by running `go/scanner` and `go/token`, the compiler's own front end, so nothing here is asserted from memory.

2.1. keywords, 25 and frozen

The complete reserved list:

```
func Keywords() []string {
  return []string{
    "break", "default", "func", "interface", "select",
    "case", "defer", "go", "map", "struct",
    "chan", "else", "goto", "package", "switch",
    "const", "fallthrough", "if", "range", "type",
    "continue", "for", "import", "return", "var",
  }
}
```

listing 4: every keyword in the language (source: go/samples/ch02/lexical.go, lines 14-22)

No keyword has been added since 1.0 kept this list stable, which is why new syntax in go 1.27 like generic methods reuses `func` rather than minting words. The bigger design point: the names programmers mistake for keywords are not. `len`, `cap`, `append`, `make`, `new`, `min`, `max`, `clear`, `print`, `true`, `false`, `nil`, `iota` are predeclared identifiers in the universe block, shadowable at any scope, and the test suite pins that `token.IsKeyword("len")` is false while `token.IsIdentifier("len")` is true.

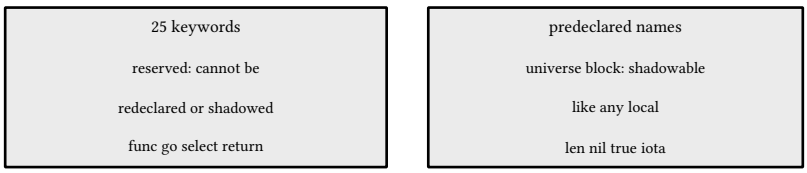


figure 5: 25 frozen keywords versus the shadowable predeclared universe

2.2. identifiers and export

An identifier is a letter followed by letters and digits, where letter means any unicode letter plus the underscore. Vietnamese and greek spellings are legal, digits cannot lead, and the blank identifier `_` discards. Export is a lexical rule with no keyword for it: a name is exported when its first character is a unicode uppercase letter, category `Lu`, and it lives in a package block or names a field or method. Lowercase names are private to the package, there is no `private` or `internal` keyword:

```
func Classify(word string) string {
  switch {
  case token.IsKeyword(word):
    return "keyword"
  case token.IsIdentifier(word):
    return "identifier"
  default:
    return "invalid"
  }
}

// IsExported applies the spec rule: the first character must be a
// unicode uppercase letter, category Lu.
func IsExported(name string) bool {
  return token.IsExported(name)
}
```

listing 5: classification through go/token, the compiler's own predicates (source: go/samples/ch02/lexical.go, lines 24-40)

The greek cases in the test are the sharp edge: capital `A` is `Lu` so `Aβ` exports, small `α` is `Ll` so `αβ` does not, and a leading underscore counts as lowercase, so `_x` never exports.

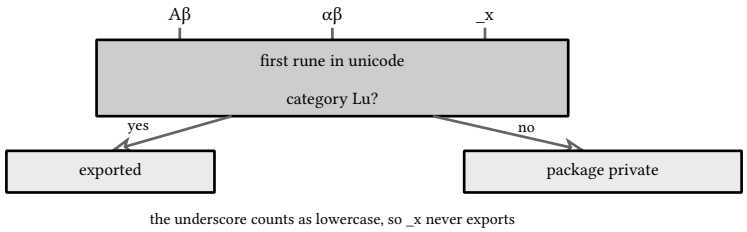


figure 6: export is one lexical test: the first rune decides

2.3. literals

Integers come in four bases with underscores as separators after the prefix and between digits. `0b101`, `0o600`, `0xBadFace`, `4_2`, and `0x_67_7a_2f` are all legal, while `42_` and `4__2` are not, and a leading plain zero is legacy octal, so `0600` is 384:

```
func ParseNumber(lit string) (int64, error) {
  return strconv.ParseInt(lit, 0, 64)
}
```

listing 6: parse with base 0, exactly what the compiler does (source: go/samples/ch02/lexical.go, lines 79-81)

Rune literals quote one code point with four escape forms: `\x` plus two hex digits, `\u` plus four, `\U` plus eight, and three octal digits, plus the single character escapes `\n`, `\t`, `\a`, `\b`, `\f`, `\r`, `\v`, `\\`. In rune literals `\'` is legal and `\"` is not, in string literals the reverse.

Strings have exactly two forms, and the difference is bytes versus characters:

```
const (
  // RawMultiline: back quote literal, real newlines allowed.
  RawMultiline = `first
second`
  // RawBackslash: inside back quotes a backslash is just a character.
  RawBackslash = `a\nb`
  // SlashN: inside double quotes \n is the newline escape.
  SlashN = "a\nb"
  // EscapeByte: \x and octal escapes name single bytes.
  EscapeByte = "\xff"
  // EscapedChar: \u names a code point, encoded as utf-8.
  EscapedChar = "ÿ"
)

const (
  HexA      = '\x41'
  Octal377  = '\377'
  UnicodeE9 = 'é'
  HanBen    = '本'
)
```

listing 7: raw versus interpreted strings, and rune escapes (source: go/samples/ch02/lexical.go, lines 83-102)

Back quote strings are raw: newlines and backslashes are literal characters and the only forbidden content is a back quote itself. Double quote strings interpret escapes, and here the byte boundary matters: `\xff` and octal escapes name single bytes, ÿ names a code point and becomes its two byte utf-8 encoding, so `len(EscapeByte)` is 1 while `len(EscapedChar)` is 2 and only the second equals the string "ÿ".

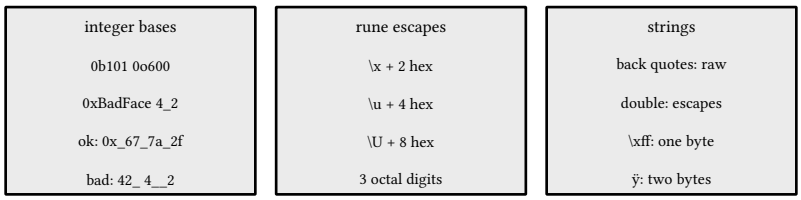


figure 7: the literal grammar: four integer bases, rune escape widths, two string forms

2.4. semicolon insertion

Go has semicolons in the grammar and none in the source. Two rules insert them. A semicolon appears after a line's final token when that token is an identifier, a literal, one of `break`, `continue`, `fallthrough`, `return`, or one of `++`, `--`, `)`, `]`, `}`. And a semicolon may be omitted before a closing `)` or `}` so a complex call fits one line. The scanner output is the proof, and it carries a fingerprint: an inserted semicolon reports its literal as `"\n"` while a typed one reports `";"`:

```
func Tokenize(src string) []Tok {
  var fset token.FileSet
  file := fset.AddFile("snippet.go", fset.Base(), len(src))
  var s scanner.Scanner
  s.Init(file, []byte(src), nil /* no error handler */, scanner.ScanComments)

  var toks []Tok
  for {
    pos, tok, lit := s.Scan()
    if tok == token.EOF {
      return toks
    }
    p := fset.Position(pos)
    text := lit
    if text == "" {
      text = tok.String()
    }
    toks = append(toks, Tok{
      Line: p.Line, Column: p.Column,
      Type: tok.String(),
      Text: text,
    })
  }
}

// ParseNumber parses an integer literal the way the compiler does,
```

listing 8: the compiler's scanner, with insertion visible (source: go/samples/ch02/lexical.go, lines 52-77)

The classic wound this design inflicts is a `return` alone on its line. The semicolon lands immediately after it, the expression on the next line becomes a separate statement, and the function returns zero values. The test pins the token stream `return ; x` exactly, and the same trap bites `break` and `continue`, and any literal or identifier ended line inside a call argument list. The closing brace on its own line after a slice literal is not just style: it is what makes the comma before it legal.

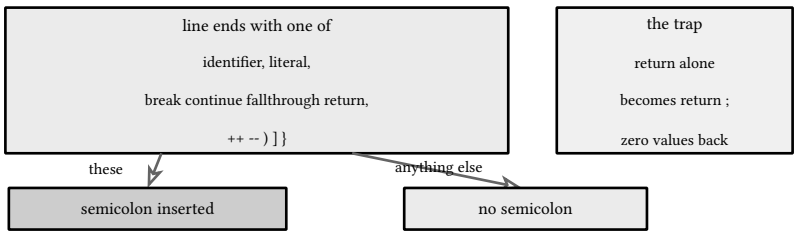


figure 8: semicolon insertion: the line-final token rule, and the return trap

2.5. operators and precedence

Five binary levels, tightest first:

```
5    *  /  %  <<  >>  &  &^
4    +  -  |  ^
3    ==  ≠  <  <=  >  ≥
2    &&
1    ||
```

fragment, illustrative, not executed

Unary operators bind tighter than all of these, comparisons yield untyped booleans, and `&^` is bit clear, `a &^ b` clears the bits of `a` that `b` sets. Untyped means the value has no type until context fixes one, a mechanism chapter 8's const section owns. When in doubt, parenthesize: unlike C there is no assignment-expression tower, and the fixed table above is the whole story.

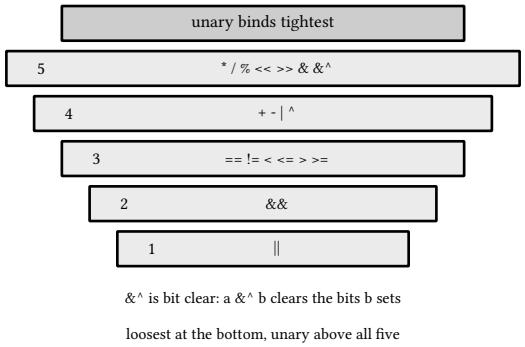


figure 9: five binary levels with unary above them all, comparisons yield untyped bool

sources: go.dev/ref/spec, lexical elements section, accessed 2026-09-08, and go 1.27 release notes. Scanner behavior verified by `go/samples/ch02` tests through go/scanner and go/token.

3. types

Go's type system is nominal with structural moments: types are declared, not inferred shapes, conversions are always explicit, and the two composite workhorses, slices and maps, are reference typed headers whose mutation semantics every Go bug report eventually touches. The sample package exercises each rule with a test.

3.1. the inventory

Booleans, signed `int` plus `int8` through `int64`, unsigned `uint8` through `uint64` and `uintptr`, `float32` and `float64`, `complex64` and `complex128`, strings, arrays, slices, maps, structs, pointers, functions, channels, interfaces. Two aliases matter: `byte` is `uint8` and `rune` is `int32`, a name for a code point, not a type of its own. `int` is 64 bit on every platform this book targets and `strconv.IntSize` says so portably:

```
func Sizes() map[string]int {
  var (
    b    byte
    r    rune
    i    int
    i64  int64
    f32  float32
    c128 complex128
  )
  return map[string]int{
    "byte":       int(unsafe.Sizeof(b)),
    "rune":       int(unsafe.Sizeof(r)),
    "int":        int(unsafe.Sizeof(i)),
    "int64":      int(unsafe.Sizeof(i64)),
    "float32":    int(unsafe.Sizeof(f32)),
    "complex128": int(unsafe.Sizeof(c128)),
  }
}
```

listing 9: sizes from unsafe.sizeof, rune is 4 bytes as int32 (source: go/samples/ch03/types.go, lines 32-50)

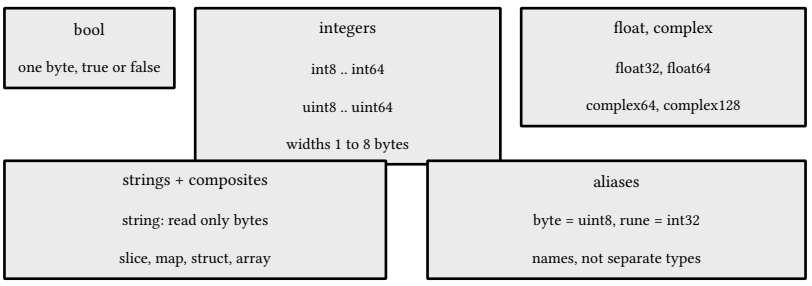


figure 10: the inventory by kind: widths, composites, and two aliases

3.2. zero values, no uninitialized anything

There is no uninitialized state in Go. Storage is zeroed, every type has a zero value, and declaration without assignment means zero. This is why Go has no constructors as a language feature: a struct whose zero value is usable needs none.

```
type Defaults struct {
  Bool   bool
  Int    int
  Float  float64
  String string
  Ptr    *int
  Slice  []int
  Map    map[string]int
  Func   func()
  Chan   chan int
  Iface  error
}

func ZeroValues() Defaults {
  var d Defaults
  return d
}
```

listing 10: one field per kind, all zero (source: go/samples/ch03/types.go, lines 14-30)

The consequence reaches deep: `sync.Mutex` is a usable zero value, `bytes.Buffer` too, and a nil slice or map is a readable, rangeable empty one, with the map write exception below.

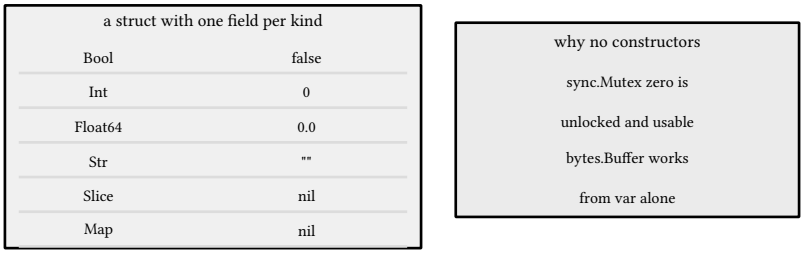


figure 11: zero values: every field of a fresh struct starts at zero

3.3. pointers, arrays, slices

Pointers exist, `*T`, with no arithmetic. Arrays are values: passing one copies it, and `[3]int` and `[4]int` are different types. Slices are the everyday type, a three word header of pointer, length, and capacity over an array, and the header is the value while the backing array is shared:

```go
func OverwriteViaAppend() (after [3]int) {
  a := [3]int{1, 2, 3}
  s := a[:2]
  _ = append(s, 99) // fits in capacity, no reallocation happens
  return a
}

// ShareBacking returns two slices pointing into the same backing array.
// Writing through the second is visible through the first.
func ShareBacking() (first, second []int) {
  s := []int{1, 2, 3}
  first = s[:2]
  second = s[1:]
  second[0] = 20 // s[1], and first[1]
  return first, second
}
```

listing 11: append writes through the header, two slices share one array (source: go/samples/ch03/types.go, lines 54-69)

`OverwriteViaAppend` is the canonical surprise: the slice has length 2 and capacity 3, so `append` does not reallocate, it writes index 2 of the original array, and `[3]int{1, 2, 3}` comes back as `[1, 2, 99]`. `ShareBacking` shows the same aliasing between two slices cut from one. Every slice gotcha, from `append` surprises to `sort` mutating the caller's data, is this one fact.

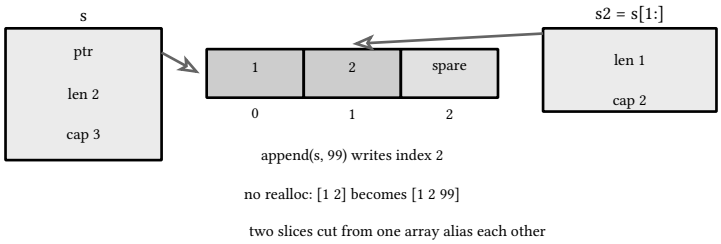


figure 12: a slice is a three word header over a shared array

3.4. maps

A map is a hash table with no declared element order, iteration order is deliberately randomized, and the three verbs are read with comma ok, `delete`, and since go 1.21 `clear`:

```
func MapOps() (present bool, afterDelete, afterClear int) {
  m := map[string]int{"a": 1, "b": 2}
  _, present = m["a"]
  delete(m, "a")
  afterDelete = len(m)
  clear(m)
  afterClear = len(m)
  return present, afterDelete, afterClear
}

func NilMapWritePanics() (panicked bool) {
  defer func() {
    if recover() != nil {
      panicked = true
    }
  }()
  var m map[string]int // nil map: reads work, writes panic
  m["k"] = 1
  return false
}

func NilMapRead(key string) (int, bool) {
  var m map[string]int
  v, ok := m[key]
  return v, ok
}

// Point embeds into Node. Since go 1.27 a composite literal key may be
// any valid field selector for the struct type, so the promoted names X
// and Y can be set directly. Qualified keys like Point.X stay illegal,
```

listing 12: comma ok, delete, clear, and the nil map rules (source: go/samples/ch03/types.go, lines 71-100)

The nil map is the exam question: reads succeed and report missing, writes panic. The test covers both sides. Map keys must be comparable, which excludes slices, maps, and functions, and the comparability of any struct type is itself queryable at run time:

```
func ComparableReports() map[string]bool {
  withSlice := struct{ S []int }{}
  withMap := struct{ M map[int]int }{}
  withFunc := struct{ F func() }{}
  plain := struct {
    A int
    B string
  }{}
  return map[string]bool{
    "slice": reflect.TypeOf(withSlice).Comparable(),
    "map":   reflect.TypeOf(withMap).Comparable(),
    "func":  reflect.TypeOf(withFunc).Comparable(),
    "plain": reflect.TypeOf(plain).Comparable(),
  }
}
```

listing 13: reflect reports which structs can sit behind == (source: go/samples/ch03/types.go, lines 117-131)

A struct with a slice, map, or func field cannot be a map key or be compared with ==, and `reflect.TypeOf(v).Comparable()` says so before the compile error does for values built at run time.

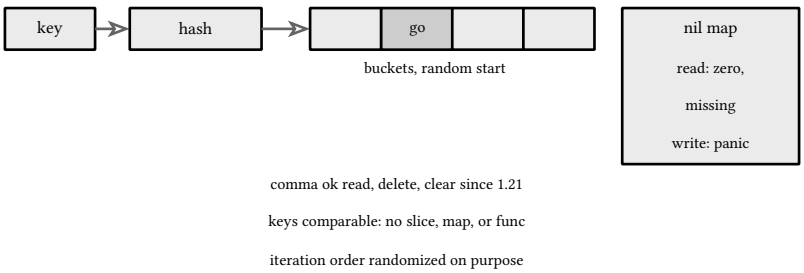


figure 13: a map is a hash table: hash to a bucket, nil is readable but unwritable

3.5. structs, embedding, and the 1.27 literal keys

Structs are field records with no inheritance and no methods in the declaration. Composition is embedding, an anonymous field whose fields and methods are promoted to the outer type:

```
type Point struct {
  X, Y int
}

func (p Point) Norm2() int { return p.X*p.X + p.Y*p.Y }

type Node struct {
  Point
  Label string
}

func NodeFromPromotedKeys() Node {
  return Node{X: 1, Y: 2, Label: "origin"}
}
```

listing 14: embedding with promotion, and go 1.27 promoted literal keys (source: go/samples/ch03/types.go, lines 102-115)

`Node` embeds `Point`, so `n.X`, `n.Y`, and `n.Norm2()` all resolve through the promotion. Before 1.27 a literal had to nest explicitly, `Node{Point: Point{X: 1, Y: 2}, Label: "origin"}`, asymmetric with the dotted access the same value supports. Go 1.27 fixed the asymmetry: a composite literal key may now be any valid field selector for the struct type, and the shortest selector for a promoted field is its plain name, so `Node{X: 1, Y: 2, Label: "origin"}` compiles. Qualified spellings like `Point.X` remain invalid, promotion conflicts at the same depth stay errors, and mixing a promoted key with an explicit `Point:` entry that sets the same field is a duplicate field error.

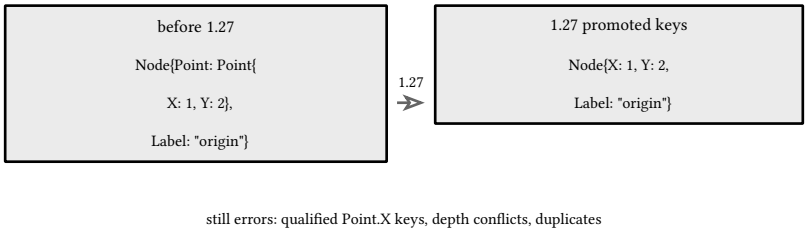


figure 14: composite literal keys before and after 1.27 promotion

3.6. defined types, aliases, conversions

`type Celsius float64` creates a new type with the same underlying representation but a distinct identity: no implicit conversion between them in either direction, methods attach to `Celsius` alone. `type Alias = float64` creates nothing, it is another name for the same type, usable interchangeably:

```go
// Celsius is a defined type: distinct from float64, needs a conversion.
// Alias is an alias: the same type as float64 under another name.
type Celsius float64

type Alias = float64

var AliasTemperature Alias = 1.5

func Convert(c Celsius) float64 { return float64(c) }

func IsAliasOfFloat64(v any) bool {
  return reflect.TypeOf(v) == reflect.TypeFor[float64]()
}

// StringFacts returns byte length and rune count of a mixed script
```

listing 15: defined type versus alias, distinguished at run time (source: go/samples/ch03/types.go, lines 133-147)

The general rule has no exceptions and no user defined conversions: values convert only where assignment compatibility holds, identical underlying types, named and unnamed pairs, or an explicit `T(x)`. Numeric conversions truncate, and `string(int)` converts the integer as a code point, which is why vet flags it.

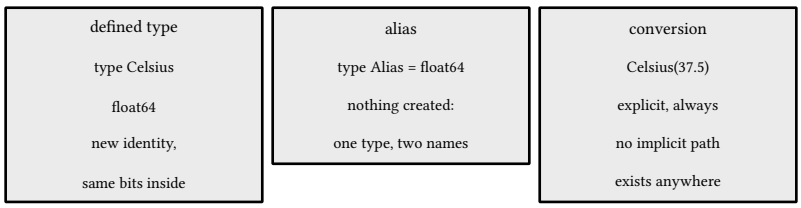


figure 15: three relations: defined type, alias, explicit conversion

3.7. strings

A string is a read only slice of bytes, utf-8 by convention not by type, and indexing yields bytes while `range` yields runes:

```go
func StringFacts() (bytes, runes int) {
  s := "héllo" + "thế"
  return len(s), utf8.RuneCountInString(s)
}
```

listing 16: len is bytes, rune count is characters (source: go/samples/ch03/types.go, lines 149-152)

`"héllo" + "thế"` is 11 bytes and 8 runes: two accents cost one extra byte each, one costs two. Conversions move between worlds: `[]byte(s)` copies into a mutable buffer, `[]rune(s)` decodes once into code points, `string(r)` encodes a single rune, and `string(bs)` where `bs` is `[]byte` copies back. Invalid utf-8 does not panic any of this, it decodes as `U+FFFD`, the replacement character, one per bad byte.

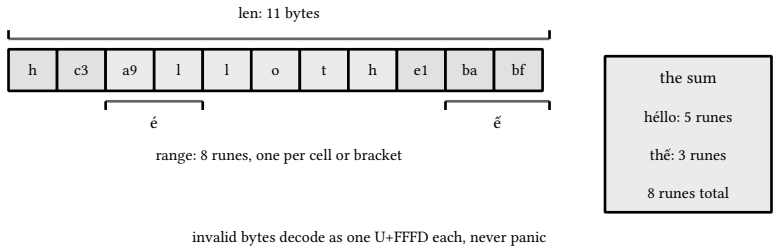


figure 16: eleven bytes, eight runes: indexing walks bytes, range decodes runes

sources: go.dev/ref/spec, types, composite literals, and conversions sections, go 1.27 release notes for the promoted literal keys, accessed 2026-09-08. Behavior verified by `go/samples/ch03` tests, including the compiler acceptance of promoted keys on go1.27.0.

4. generics and generic methods

Generics arrived in go 1.18 and stopped growing until now. Go 1.27 ships the change the proposal process spent years on: methods may declare their own type parameters, plus a generalized function type inference. This chapter covers the whole mechanism as it stands, and every listing runs in the sample module against go1.27.0.

4.1. type parameters and constraints

A function declaration lists its parameters in square brackets, each with a constraint, and the constraint is an interface:

```
func Max[T cmp.Ordered](a, b T) T {
  if a > b {
    return a
  }
  return b
}
```

listing 17: one constraint, the standard ordered set (source: go/samples/ch04/generics.go, lines 12-18)

`any` is alias for `interface{}`, a no constraint. `comparable` is the constraint of types that support ==. The standard constraints live in the `cmp` and `slices` packages, `cmp.Ordered` above covers numbers and strings. Constraints are ordinary interfaces, so they carry methods for behavior and type sets for capability:

```
type Grams int

func Sum[N ~int | ~float64](xs ...N) N {
  var total N
  for _, x := range xs {
    total += x
  }
  return total
}
```

listing 18: approximation terms accept defined types (source: go/samples/ch04/generics.go, lines 38-46)

The tilde is the operator that matters: `~int` means any type whose underlying type is int, so `Grams`, a defined int, passes where a bare `int` constraint would reject it. Union terms combine sets, `~int | ~float64` above. Call sites infer the argument, `Sum(1, 2)` instantiates `Sum[int]`, and `Sum(Grams(100), Grams(250))` instantiates `Sum[Grams]` because the variadic pack fixes `N`.

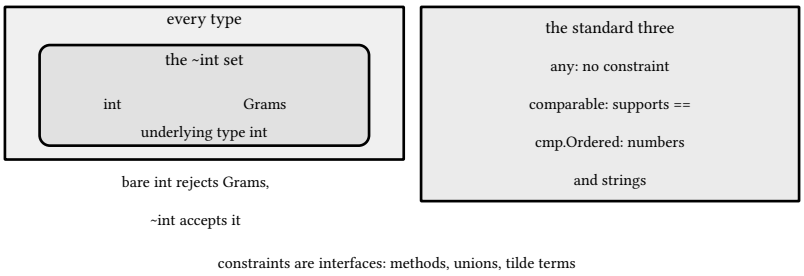


figure 17: constraints are interfaces: type sets, with the tilde opening the door

4.2. generic types

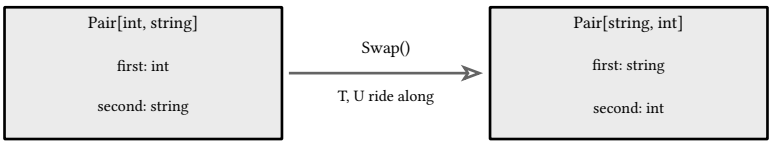
Types take parameters the same way, and methods on them use the receiver's parameters for free. Receivers and method sets are chapter 5's subject, here the receiver's parameters simply ride along:

```
type Pair[T, U any] struct {
  First  T
  Second U
}

func NewPair[T, U any](first T, second U) Pair[T, U] {
  return Pair[T, U]{First: first, Second: second}
}

// Swap needs no explicit type arguments at the call site: the result
// type Pair[U, T] follows from the receiver.
func (p Pair[T, U]) Swap() Pair[U, T] {
  return Pair[U, T]{First: p.Second, Second: p.First}
}
```

listing 19: two arguments, and swap with inferred result type (source: go/samples/ch04/generics.go, lines 20-33)



p.Swap() infers everything: zero explicit brackets

figure 18: the receiver's parameters ride along: swap with no explicit instantiation

No explicit instantiation appears at `p.Swap()`: the receiver fixes `T` and `U`, and the result `Pair[U, T]` follows. That inference is why generic types feel native rather than templated, and a type parameter's zero value is available as `var zero T`, the substitute for a default constructor:

```
func Zero[T any]() (T, bool) {
  var zero T
  return zero, false
}

func Double[N ~int | ~float64](x N) N { return x + x }
```

listing 20: zero value without constructors, and a one line generic (source: go/samples/ch04/generics.go, lines 103-108)

4.3. generic methods, the go 1.27 change

Until 1.27 a method could use only the type parameters of its receiver. Workarounds were package level functions taking the receiver as the first argument, which is why `slices.Sort(s)` exists alongside `s.Sort` that never could. Go 1.27 lifts the restriction: a method may declare fresh type parameters, used in its parameters, results, and body:

```
type Set[T comparable] struct {
  items map[T]struct{}
}

func NewSet[T comparable](xs ...T) *Set[T] {
  s := &Set[T]{items: make(map[T]struct{})}
  for _, x := range xs {
    s.items[x] = struct{}{}
  }
  return s
}

func (s *Set[T]) Map[U any](f func(T) U) []U {
  out := make([]U, 0, len(s.items))
  for x := range s.items {
    out = append(out, f(x))
  }
  return out
}
```

listing 21: a generic type whose method adds its own parameter (source: go/samples/ch04/generics.go, lines 51-70)

`Map` transforms the set's elements to any type `U` chosen per call, `Map[int]` and `Map[string]` both work on the same `*Set[int]`. The same liberty extends to plain receivers:

```
type Registry struct {
  mu sync.Mutex
  m  map[string]any
}

func NewRegistry() *Registry {
  return &Registry{m: make(map[string]any)}
}

func (r *Registry) Put(key string, value any) {
  r.mu.Lock()
  defer r.mu.Unlock()
  r.m[key] = value
}

func (r *Registry) Get[T any](key string) (T, bool) {
  r.mu.Lock()
  defer r.mu.Unlock()
  v, ok := r.m[key]
  if !ok {
    var zero T
    return zero, false
  }
  t, ok := v.(T)
  return t, ok
}

// Zero returns the zero value of any type parameter, the generic
```

listing 22: a plain type with a generic getter (source: go/samples/ch04/generics.go, lines 74-101)

Two restrictions survive from the design discussion, and they are the same fact: interface methods may not declare type parameters, and interface methods cannot be implemented by generic methods. A method set must

be knowable from the interface alone, before any call site picks arguments. The standard library's own example is `math/rand/v2`, whose `Rand.N` is now a generic method, `func (r *Rand) N[Int intType](Int) Int`, matching the package level `N`.

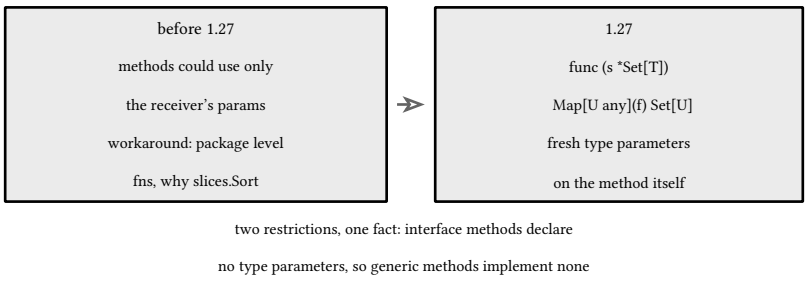


figure 19: generic methods, the 1.27 change: fresh parameters on the method

NOTE: when a function beats a method

Even with generic methods available, `slices.Sort(s)` style functions stay idiomatic when the operation reads as a utility over a standard shape. Methods earn their keep when the receiver carries state, like `Set.Map` above or a registry keyed by type.

4.4. function type inference, generalized in 1.27

Before 1.27, inference happened at calls, and assigning a generic function to a variable of function type required spelling the instantiation, `var f = Double[int]`. Go 1.27 generalizes inference to every context where a generic function meets a matching function type:

```
func Double[N ~int | ~float64](x N) N { return x + x }

var f func(int) int = Double     // 1.27: inferred as Double[int]
var g func(float64) float64 = Double
```

fragment, illustrative, not executed

Both assignments in the test suite compile and return 42 and 3. This is the feature that makes generic functions drop in where closures or passed functions were the only option, callbacks, handlers, and table driven dispatch all accept generic functions directly now.

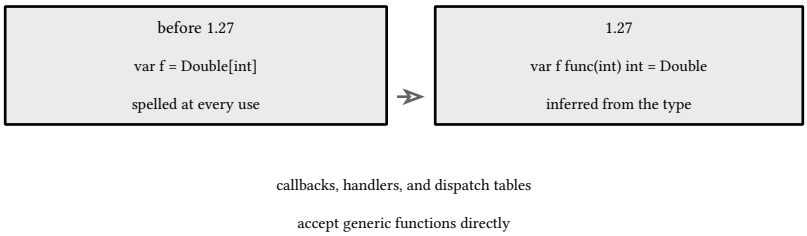


figure 20: function type inference, generalized in 1.27

sources: go.dev, go 1.27 release notes, language changes section, and proposal issue 9859's sibling discussion for generic methods, accessed 2026-09-08. All syntax verified by compiling and running `go/samples/ch04` on go1.27.0.

5. functions

Functions are go's unit of composition, and the language bets on having few features done thoroughly: multiple returns instead of tuples, defer instead of finally, closures instead of classes, methods as syntax over functions. Every rule below is exercised by the sample package's tests.

5.1. multiple returns and named results

A function returns as many values as it declares, and go's error convention rides on this: the last return is an error, callers check it, and there is no exception to intercept. Named results document the slots and stay in scope for the whole body:

```
func Divide(a, b int) (q, r int) {
  q, r = a/b, a%b
  return
}
```

listing 23: quotient and remainder, named (source: go/samples/ch05/functions.go, lines 9-13)

Naked `return` with named results compiles, but explicit values read better beyond two slots. The second use of named results is the serious one: they are variables the function owns, so deferred code can set them, which is how a defer recovers a panic into an error return.

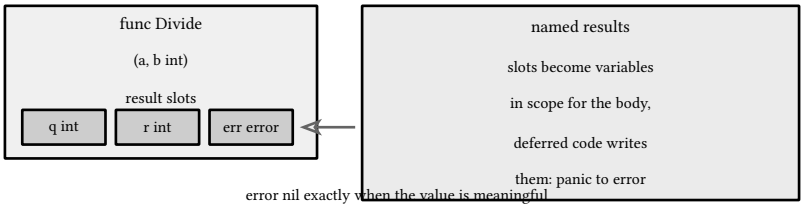


figure 21: results are slots the function owns, named ones stay in scope

5.2. defer

`defer` registers a call to run when the function returns, whoever returns however. Three rules, all pinned by tests:

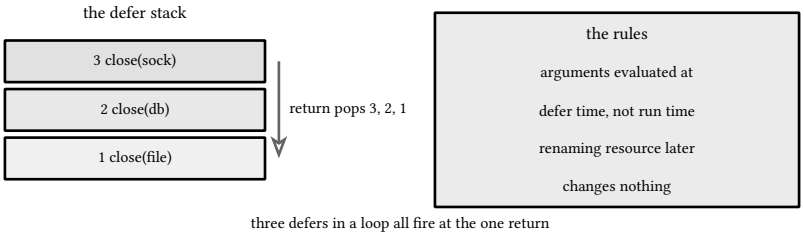


figure 22: defers stack up lifo, arguments already captured at registration

First, defers run last in, first out:

```
func DeferredOrder() (events []string) {
  events = append(events, "acquired: file")
  resource := "file"
  defer AppendLater(&events, "closed: "+resource)
  defer AppendLater(&events, "released: locked")
  events = append(events, "body")
  resource = "renamed"
  return
}

// DeferredInLoop shows defers piling up: three are registered inside
// the loop and all fire at return, after the loop long finished. The
```

listing 24: lifo order, and arguments evaluated at defer time (source: go/samples/ch05/functions.go, lines 22-33)

Second, the deferred call's arguments are evaluated when the defer statement executes. `resource` is renamed after registration and the closed message still says `"closed: file"`, because the string was already built. Third, defers accumulate, they do not nest:

```
func DeferredInLoop() (n int) {
  for range 3 {
    defer func() { n++ }()
  }
  return
}
```

listing 25: three defers fire at one return (source: go/samples/ch05/functions.go, lines 35-41)

A defer inside a loop holds its resource until the function ends. For per iteration cleanup, wrap the loop body in a function or use the pattern `func() { defer close(f); ... }()` per turn. Loop contexts and transactions are where this bites.

5.3. closures and loop variables

A function literal captures its free variables by reference, so closures over the same variable share state:

```
func Tally() (inc, dec func(), value func() int) {
  count := 0
  inc = func() { count++ }
  dec = func() { count-- }
  value = func() int { return count }
  return
}

// LoopCaptures captures the loop variable. Since go 1.22 each
// iteration has a fresh variable, so the functions return 0, 1, 2;
```

listing 26: three closures, one count (source: go/samples/ch05/functions.go, lines 43-52)

The historical trap was the loop variable: one `i` for the whole loop meant every closure saw the final value. Go 1.22 changed the semantics, each iteration gets a fresh variable, and 1.27 keeps it:

```go
func LoopCaptures() []func() int {
  var fns []func() int
  for i := range 3 {
    fns = append(fns, func() int { return i })
  }
  return fns
}
```

listing 27: per iteration capture, 0 1 2 since go 1.22 (source: go/samples/ch05/functions.go, lines 54-60)

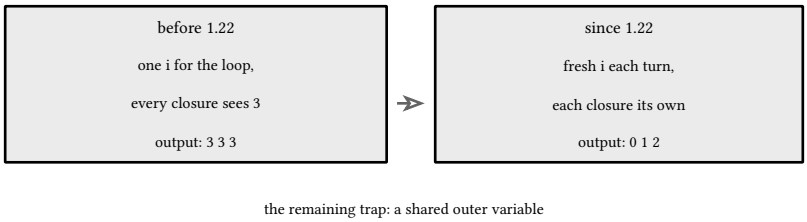


figure 23: loop capture before and after 1.22: one variable versus one per iteration

PITFALL: capturing range value versus index

`for _, x := range xs` also gives a fresh `x` per iteration since 1.22, but `&x` taken during iteration still points at that iteration's copy, which is what you want. The bug that remains is capturing a shared outer variable you meant to pass as a parameter.

5.4. variadics

The last parameter may collect, `nums ...int` is a slice inside the function and accepts zero or more arguments, or one slice with `...`:

```go
func Sum(nums ...int) int {
  total := 0
  for _, n := range nums {
    total += n
  }
  return total
}
```

listing 28: the variadic collect (source: go/samples/ch05/functions.go, lines 62-69)

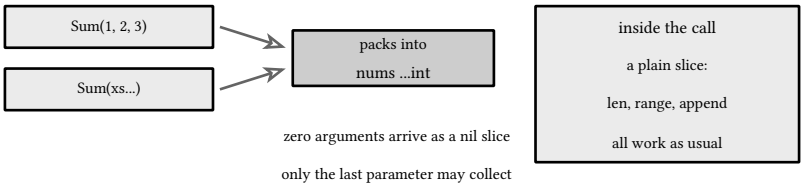


figure 24: variadics: two call shapes pack into one slice parameter

5.5. methods and method sets

A method is a function with a receiver. The receiver choice is the design decision: value receivers get a copy and cannot mutate, so stateful types use pointer receivers, and the choice decides which interfaces a type satisfies:

```go
type Incrementer interface {
  Increment() int
}

type Counter int

func (c Counter) Increment() int { return int(c) + 1 }

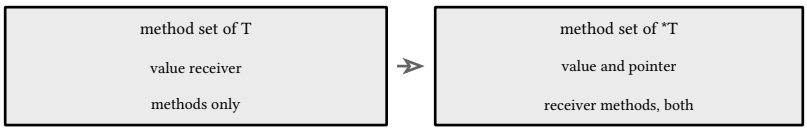
type Locked struct {
  n int
}

func (l *Locked) Increment() int {
  l.n++
  return l.n
}

// LockedImplements and CounterImplements expose the method set rule
```

listing 29: one interface, two satisfaction outcomes (source: go/samples/ch05/functions.go, lines 73-90)

The method set of `T` contains value receiver methods only, the method set of `*T` contains both. `Counter` satisfies `Incrementer`, `Locked` does not, `*Locked` does, and the test reads the verdicts from reflect rather than trusting the prose. Interface satisfaction is compile time, implicit, and there is no `implements` clause.



the verdicts, read from reflect

Counter	Inc on T	satisfies Incrementer
Locked	Inc on *T	does not satisfy
*Locked	both methods	satisfies Incrementer

implicit at compile time, no implements clause exists

figure 25: the receiver decides the method set, the method set decides satisfaction

5.6. init

A package may define `init` functions, parameterless, uncallable, running once at program start after imported packages initialized and variable initializers ran. They register drivers and formats, and beyond that they are a smell: test suites cannot reach them, so prefer explicit constructors:

```go
var (
  // set by init below, before any function in this package runs
  PackageInitialized bool
  InitializedAt      string
)

func init() {
  PackageInitialized = true
  InitializedAt = "init"
```

listing 30: the init pair the tests observe (source: go/samples/ch05/functions.go, lines 98-106)

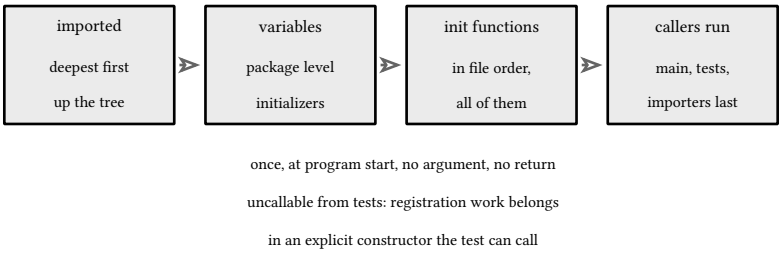


figure 26: package initialization runs once, in order, before any caller

sources: go.dev/ref/spec, function declarations, defer statements, and method sets sections, accessed 2026-09-08, plus the go 1.22 release notes for loop variable semantics. Verified by `go/samples/ch05` tests.

6. concurrency

Go's pitch since launch: concurrency built into the language, not a library bolted on. `go` starts work, channels move values between it, `select` waits on many things at once, and the `sync` package covers what channels should not. The sample package exercises each primitive deterministically, no sleeps anywhere.

6.1. goroutines

`go f(x)` runs `f` concurrently and forgets about it: no handle, no result, no way to wait on the function itself. Every coordination mechanism below exists because the `go` statement provides none. A goroutine costs a few kilobytes of stack that grows and shrinks, so hundreds of thousands are ordinary. The scheduler multiplexes them onto `GOMAXPROCS` os threads with work stealing, and since go 1.14 goroutines yield preemptively, so a tight loop stops starving others, and the scheduler underneath is chapter 9's subject.

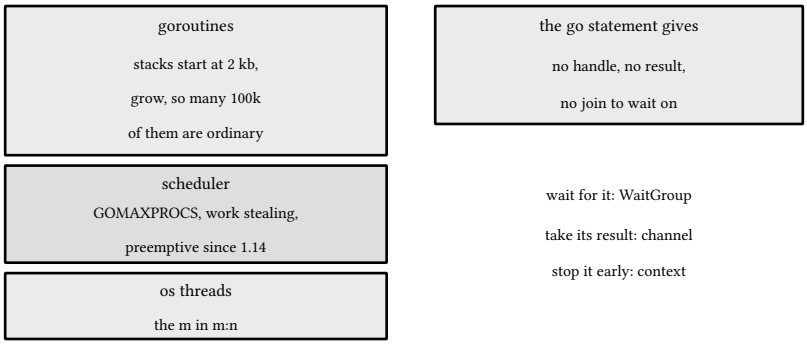


figure 27: goroutines layer over the scheduler, the primitives add what go lacks

6.2. channels

A channel is a typed conduit with a capacity. Capacity zero is a rendezvous, the send completes only when a receiver takes the value, which is synchronization, not just transport:

```
func TestUnbufferedHandoffSynchronizes(t *testing.T) {
  // an unbuffered channel is a rendezvous: send completes only when a
  // receiver is ready, so ordering is deterministic
  ch := make(chan int)
  done := make(chan []int, 1)
  go func() {
    var got []int
    for v := range ch {
      got = append(got, v)
    }
    done <- got
  }()
  ch <- 10
  ch <- 20
  close(ch)
  got := <-done
  if !slices.Equal(got, []int{10, 20}) {
    t.Fatalf("handoff = %v, want [10 20]", got)
  }
}
```

listing 31: unbuffered handoff, sender and receiver meet (source: go/samples/ch06/concurrency_test.go, lines 97-116)

Capacity n is a buffer: n sends proceed without a receiver, and the n+1st blocks until a receive frees a slot. The capstone in chapter 34 caps its simultaneous fetches with a buffered `chan struct{}` of capacity workers, because a buffered channel used this way is the semaphore pattern.

Closing is the completion signal. A receive on a closed channel returns immediately, the zero value and `false` from the comma ok form, and `range` drains then terminates:

```
func Drain(ch <-chan int) []int {
  var out []int
  for v := range ch {
    out = append(out, v)
  }
  return out
}

// NilChannelFallsToDefault proves a nil channel never becomes ready:
// the select falls through to default immediately.
func NilChannelFallsToDefault() bool {
  var ch chan int
  select {
  case <-ch:
    return false
  default:
    return true
  }
}
```

listing 32: drain semantics, and the nil channel rule (source: go/samples/ch06/concurrency.go, lines 77-95)

A nil channel blocks forever. That sounds like a bug and is a feature: a select case whose channel is nil is switched off, so a state machine disables a direction by nil ing its channel. Sends on closed channels panic, a double close panics too, and only the sender should close.

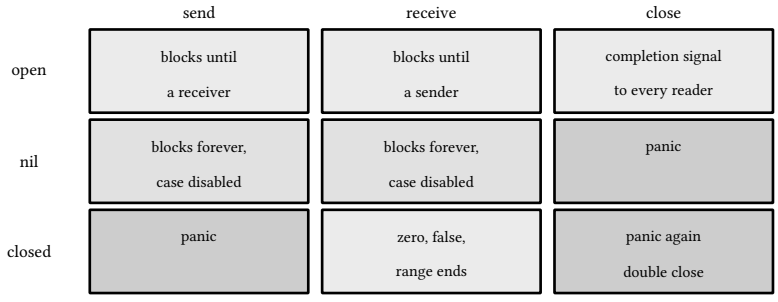


figure 28: channel semantics in nine cells: state against operation

PITFALL: who closes

The writer closes, never the reader, and with many writers a coordinating `WaitGroup` closes once after all finish, the pattern `FanInSum` below shows. Closing is broadcast, not cleanup: it means no more values are coming.

6.3. select

`select` waits on many channel operations and takes one that is ready, random among the ready:

```
func SelectFirst(a, b <-chan string) string {
  select {
  case v := <-a:
    return v
  case v := <-b:
    return v
  }
}
```

listing 33: first delivery wins (source: go/samples/ch06/concurrency.go, lines 100-107)

With `default` it becomes a non blocking attempt, and `select {}` blocks forever. Timeouts read naturally, `case <-time.After(sec)`, and those timer channels are unbuffered since go 1.23, with no `asynctimerchan` compat flag remaining in 1.27.

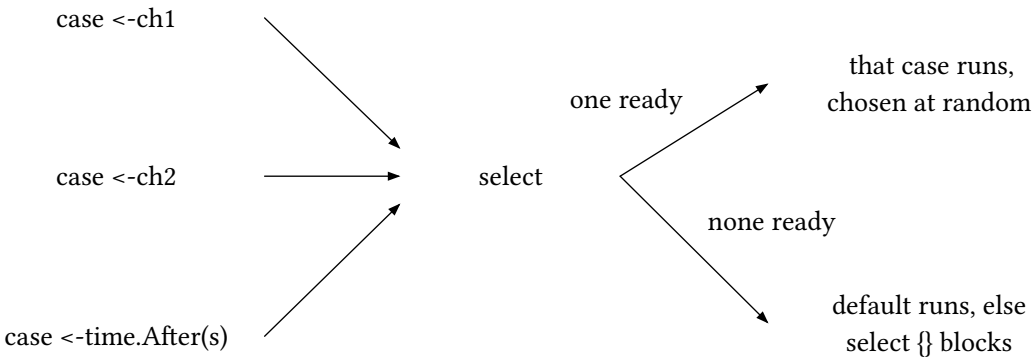


figure 29: select waits on every case, runs one ready case at random

6.4. patterns

The pipeline: each stage owns its output channel, closes it when its input drains, and `range` carries termination downstream. Its input is an `iter.Seq[int]`, a function iterator, chapter 8's range over func form:

```
func Pipeline(seq iter.Seq[int]) []int {
  nums := make(chan int)
  go func() {
    defer close(nums)
    for n := range seq {
      nums <- n
    }
  }()

  squares := make(chan int)
  go func() {
    defer close(squares)
    for n := range nums {
      squares <- n * n
    }
  }()

  var out []int
  for s := range squares {
    out = append(out, s)
  }
  return out
}

// FanInSum splits nums into chunks, sums each chunk in its own
// goroutine, and totals the partials. The closer goroutine waits for
```

listing 34: gen, square, consume, close chains (source: go/samples/ch06/concurrency.go, lines 16-41)

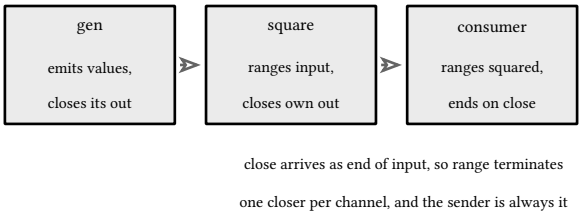


figure 30: the pipeline: each stage owns its output channel and closes it

The fan in: workers compute partials, a `WaitGroup` tracks them, one closer goroutine shuts the results channel, the consumer ranges. Since go 1.25 `wg.Go(f)` both starts and tracks, replacing the `Add(1)` plus `go func` with `defer wg.Done` dance:

```
func FanInSum(nums []int, workers int) int {
  results := make(chan int)
  var wg sync.WaitGroup

  chunk := (len(nums) + workers - 1) / workers
  for w := range workers {
    lo := w * chunk
    hi := min(lo+chunk, len(nums))
    if lo ≥ hi {
      break
    }
    part := nums[lo:hi]
    wg.Go(func() {
      partial := 0
      for _, n := range part {
        partial += n
      }
      results ← partial
    })
  }

  go func() {
    wg.Wait()
    close(results)
  }()

  total := 0
  for partial := range results {
    total += partial
  }
  return total
}
```

listing 35: fan in with a single closer (source: go/samples/ch06/concurrency.go, lines 43-75)

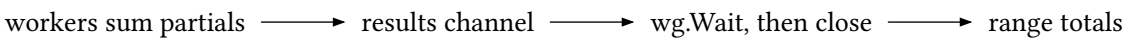


figure 31: fan in: many producers, one close, one consumer

6.5. the sync package

Channels pass ownership of data. When data stays put and many goroutines touch it, use a lock:

```
type LockedCounter struct {
  mu sync.Mutex
  n  int
}

func (c *LockedCounter) Inc() {
  c.mu.Lock()
  defer c.mu.Unlock()
  c.n++
}

func (c *LockedCounter) Value() int {
  c.mu.Lock()
  defer c.mu.Unlock()
  return c.n
}
```

listing 36: mutex guarded counter, zero value usable (source: go/samples/ch06/concurrency.go, lines 111-126)

The mutex embeds by value and its zero state is unlocked, so `LockedCounter{}` works without a constructor, the zero value theme of chapter 3 again. `sync.Once` runs a function exactly once under concurrency, the standard lazy initialization:

```
var (
  once  sync.Once
  calls atomic.Int64
)

func OnceInit() func() {
  return func() {
    once.Do(func() { calls.Add(1) })
  }
}

func OnceCalls() int { return int(calls.Load()) }
```

listing 37: once, with an atomic count as proof (source: go/samples/ch06/concurrency.go, lines 128-139)

For a single word, `sync/atomic` skips the lock, and `atomic.Int64` with `Add` and `Load` is the type to use, the test runs a thousand concurrent adds and reads 1000. The memory model guarantee behind all of this is happens before: a send happens before its receive completes, an unlock happens before the next lock. A data race is two goroutines touching the same memory, at least one access a write, and no happens before edge between them. Go's race detector, `go test -race`, enforces the discipline at run time, worth running on every test suite that touches this chapter's material.

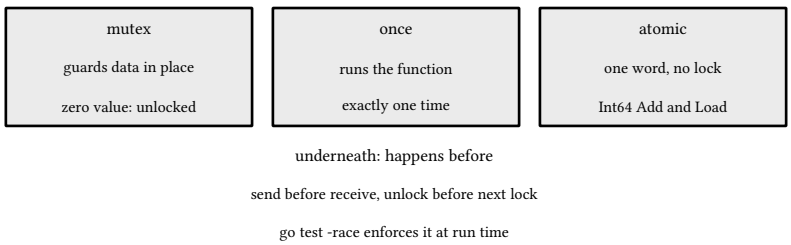


figure 32: channels move ownership, the sync package guards data that stays put

Stopping work early is context's job, the promise the goroutines diagram made. A `context.Context` carries cancellation: `Done` is a channel closed on cancel, `Err` reports `context.Canceled`, and `context.WithCancel` creates one from a parent. The capstone's crawler in chapter 34 stops its walk through exactly this channel.

sources: go.dev/ref/spec and go.dev/doc/effective_go, concurrency sections, go 1.27 release notes for wg.Go lineage and timer channel cleanup, accessed 2026-09-08. Verified by `go/samples/ch06` tests under `go test` and `go vet`.

7. error handling

Go has no exceptions. An error is a value of the two method interface `error`, returned as the last result, checked by the caller. Everything else in this chapter, wrapping, matching, joining, recovering, is machinery over that one decision, and it is why go code reads top to bottom with the failure path visible beside the happy one.

7.1. the interface and the convention

```
type error interface {
    Error() string
}
```

fragment, illustrative, not executed

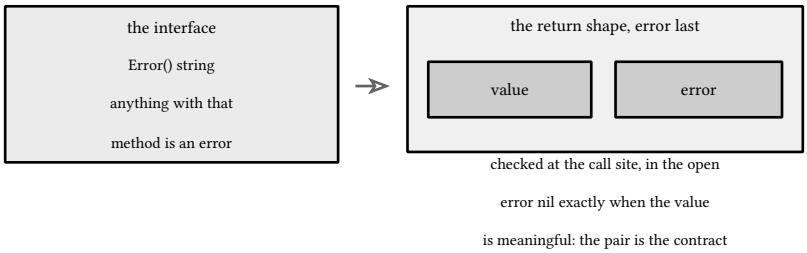


figure 33: one method, returned as the last slot, checked by the caller

Anything with an `Error() string` method is an error, and the convention is positional: functions return `(value, error)`, the error is nil exactly when the value is meaningful, and callers handle it at the call site:

```
func Divide(a, b int) (int, error) {
  if b == 0 {
    return 0, fmt.Errorf("divide %d by zero", a)
  }
  return a / b, nil
}
```

listing 38: the standard shape (source: go/samples/ch07/errors.go, lines 11-17)

`fmt.Errorf` builds wrapped messages, `errors.New` builds bare ones, and neither is more than a struct with a string.

7.2. sentinels, wrapping, matching

A sentinel is an exported error value that names a condition, and `%w` is the verb that chains errors together:

```
var ErrNotFound = errors.New("not found")

var table = map[string]int{"answer": 42}

// Lookup wraps the sentinel with %w, which chains the original into
// the new error so errors.Is can walk the chain.
func Lookup(key string) (int, error) {
  v, ok := table[key]
  if !ok {
    return 0, fmt.Errorf("lookup %q: %w", key, ErrNotFound)
  }
  return v, nil
}
```

listing 39: sentinel plus percent w wrap (source: go/samples/ch07/errors.go, lines 20-33)

The wrap is what makes matching work. `errors.Is(err, ErrNotFound)` walks the chain of wrapped errors and compares each link, so the sentinel stays comparable no matter how many layers of context surround it. Use `Is` for sentinel identity, and when the match must be by type because the error carries data, use `errors.As`:

```
type SyntaxError struct {
  Line, Col int
}

func (e *SyntaxError) Error() string {
  return fmt.Sprintf("syntax error at %d:%d", e.Line, e.Col)
}

func Parse(src string) error {
  for i, r := range src {
    if r == 'x' {
      return &SyntaxError{Line: 1, Col: i + 1}
    }
  }
  return nil
}
```

listing 40: a typed error found with As (source: go/samples/ch07/errors.go, lines 36-51)

`As` requires a pointer to the concrete type and finds the first match in the chain, giving the caller the `Line` and `Col` payload. Since go 1.20 `errors.Join` carries a tree of failures, and every branch stays findable:

```go
var (
  ErrEmptyName = errors.New("empty name")
  ErrTooOld    = errors.New("too old")
)

// Validate shows errors.Join carrying several failures at once, each
// findable with errors.Is.
func Validate(name string, age int) error {
  var errs []error
  if name == "" {
    errs = append(errs, ErrEmptyName)
  }
  if age > 150 {
    errs = append(errs, ErrTooOld)
  }
  return errors.Join(errs...)
}
```

listing 41: join keeps every branch Is-able (source: go/samples/ch07/errors.go, lines 53-69)

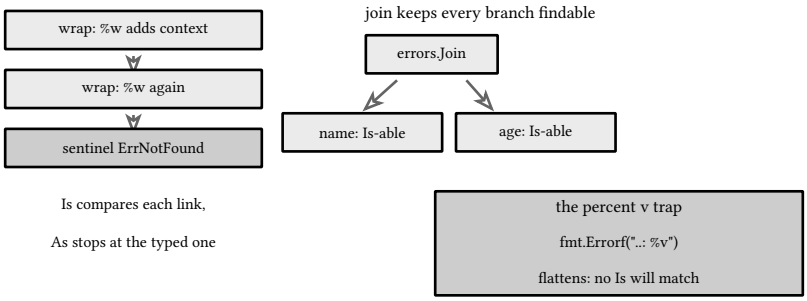


figure 34: the wrap chain and the join tree, walked by Is and As

Validation is the natural use: collect every field problem, return one error, and callers test each condition independently.

PITFALL: fmt.Errorf with percent v loses the chain

`%w` wraps, `%v` formats. `fmt.Errorf("lookup %q: %v", k, ErrNotFound)` produces an error no `errors.Is` call will ever match, because the sentinel was flattened into text. Vet cannot catch this one, review has to.

7.3. the typed nil trap

An interface value is a pair of type and pointer. Storing a typed nil pointer into an error interface fills the type half, and the pair compares non nil:

```go
type MyErr struct{}

func (e *MyErr) Error() string { return "my error" }

// TypedNilIsError stores a nil *MyErr into the error interface. The
// interface now holds a type and a nil pointer, so comparing it to
// nil reports false: the error is not nil. This is the classic trap.
func TypedNilIsError() bool {
  var typed *MyErr
  var err error = typed
  return err ≠ nil
}

// NilInterfaceIsNil is the well behaved form: a plain nil interface
// compares equal to nil.
func NilInterfaceIsNil() bool {
  var err error
  return err == nil
}
```

listing 42: the trap and the well behaved form, both return true (source: go/samples/ch07/errors.go, lines 71-90)

The test asserts both: the typed nil reports `err ≠ nil`, and the plain nil interface reports `err == nil`. The trap fires when a function returns `*MyErr` as an `error` on a failure path that produced a nil pointer, the caller sees a non nil error holding nothing. The defenses are returning a real error, or returning the concrete type and letting `errors.As` handle absence.

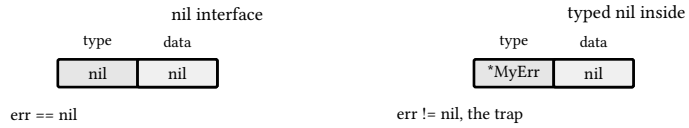


figure 35: one interface value is two words, and nil means both are empty

7.4. panic and recover

Panic is for programmer error and unrecoverable states, invariant violations, index bounds, nil dereferences, impossible switches. Recover exists so a package can keep a broken call from taking down the process, at library boundaries and server request handlers:

```go
func SafeDivide(a, b int) (result int, err error) {
  defer func() {
    if r := recover(); r ≠ nil {
      err = fmt.Errorf("recovered: %v", r)
    }
  }()
  return a / b, nil
}
```

listing 43: recover in the panicking frame's defer (source: go/samples/ch07/errors.go, lines 94-101)

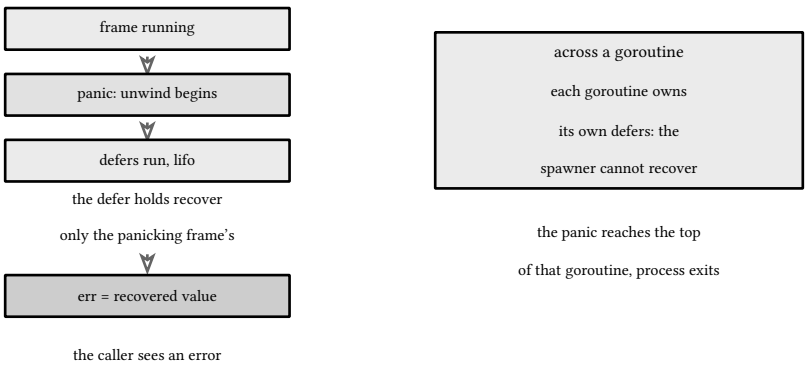


figure 36: panic unwinds the defers, recover catches, named results carry the error out

`recover` only works inside a deferred function of the panicking frame, the recovered value is what panic received, and converting it to an error through named results is the standard bridge back to the value world. A panic that crosses a goroutine boundary cannot be recovered by the spawner, each goroutine owns its own defers, which is why library code documents its panics and server frameworks wrap every handler.

NOTE: when not to panic

Ordinary expected failure, a missing file, a bad request, a closed connection, is an error value. A panic there forces every caller into recover blocks or crashes, and the stack unwinding throws away the structured context an error would have carried.

sources: go.dev/ref/spec and go.dev/blog, error handling posts plus pkg.go.dev/errors, accessed 2026-09-08. Verified by `go/samples/ch07` tests, including the typed nil behavior asserted live.

8. program structure

Program structure in go is packages, declarations, and scope, plus the statement level machinery: for, switch, labels, iota. Nothing here is exotic, and the design bet is visible: fewer statement forms than any language in this collection, each carrying more weight.

8.1. declarations

Four kinds: `var`, `const`, `type`, `func`. All accept grouping, functions nest none of them, and package level declarations may appear in any order across any file of the package, the compiler sorts out dependencies. Blocks come in four widths, universe, package, file, and local, from chapter 2's predeclared identifiers down to any pair of braces. Scope is lexical and block structured:

```
func ShadowDemo() (inner, outer int) {
  outer = 10
  if true {
    inner := outer * 2
    _ = inner
  }
  return 20, outer
}
```

listing 44: := inside a block declares, it never assigns (source: go/samples/ch08/structure.go, lines 126-134)

`inner` is a new variable that dies at the closing brace, and `outer` keeps its value. Shadowing an outer variable is legal and occasionally a bug, which is why loops that need to modify an outer variable declare it first. The blank identifier discards at any scope, and `_ = v` is the idiom for an intentional discard.

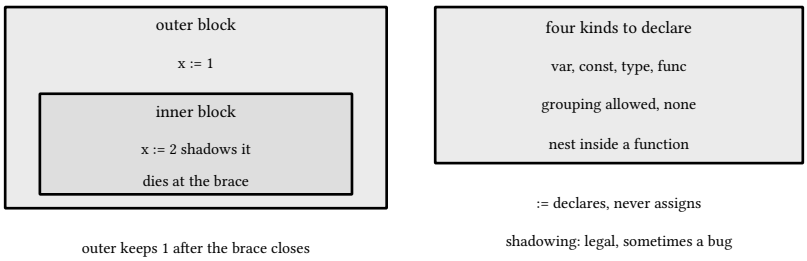


figure 37: blocks nest, a := inside declares a name that dies at the brace

8.2. if and for

`if` and `for` accept an init statement, `if v, ok := m[k]; ok`, which scopes the variable to the statement and kills the leak of temporary names. The grade chain is the everyday shape:

```
func Grade(score int) string {
  if score ≥ 90 {
    return "a"
  } else if score ≥ 80 {
    return "b"
  } else if score ≥ 70 {
    return "c"
  } else if score ≥ 60 {
    return "d"
  }
  return "f"
}
```

listing 45: else if chain, no ternary exists in go (source: go/samples/ch08/structure.go, lines 7-19)

There is no conditional expression in the language, the chain above is the replacement, and go's formatting vertically aligns it. For has one keyword and three forms:

```
func ForThreeForm() int {
  sum := 0
  for i := 1; i <= 5; i++ {
    sum += i
  }
  return sum
}

func ForRange() int {
  sum := 0
  for _, n := range []int{1, 2, 3, 4, 5} {
    sum += n
  }
  return sum
}

func ForInfinite() int {
  sum := 0
  i := 1
  for {
    if i > 5 {
      break
    }
    sum += i
    i++
  }
  return sum
}
```

listing 46: three form, range form, infinite form (source: go/samples/ch08/structure.go, lines 57-84)

Since go 1.22 `for n := range 10` counts, and since 1.23 `for f := range seq` iterates any function matching `iter.Seq`, which is how the pipeline sample of chapter 6 consumed a sequence. Range over slices gives index and value, over maps key and value, over channels values until close, over strings runes, and over integers counts.

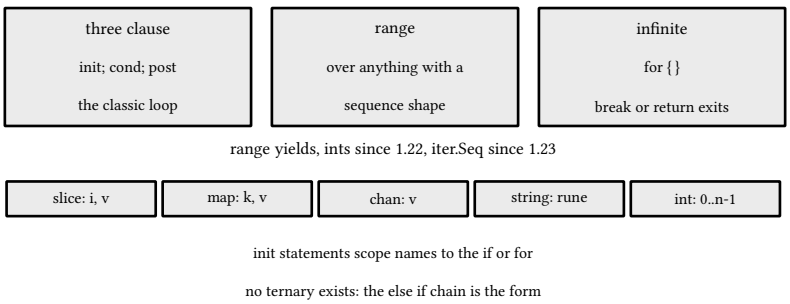


figure 38: one for keyword, three forms, and what range yields

8.3. switch

Go's switch does not fall through by default, the historical default reversed because implicit fallthrough was C's most productive bug source. Cases therefore need no break, values may be arbitrary expressions, and the operandless form is the cleanest boolean dispatch:

```
func VowelOr(c rune) string {
  switch {
  case c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u',
    c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U':
    return "vowel"
  case c == 'y' || c == 'Y':
    return "sometimes"
  case c ≥ 'a' && c <= 'z' || c ≥ 'A' && c <= 'Z':
    return "consonant"
  }
  return "other"
}

// Describe is a type switch over an interface value, the runtime
```

listing 47: operandless switch, comma separated cases (source: go/samples/ch08/structure.go, lines 22-35)

The type switch tests an interface's dynamic type and binds the converted value in each case, `nil` included:

```
func Describe(v any) string {
  switch v := v.(type) {
  case int:
    return "int"
  case float64:
    return "float64"
  case string:
    return "string"
  case bool:
    return "bool"
  case []int:
    return "slice"
  case nil:
    return "nil"
  default:
    _ = v
    return "unknown"
  }
}
```

listing 48: type switch over any (source: go/samples/ch08/structure.go, lines 37-55)

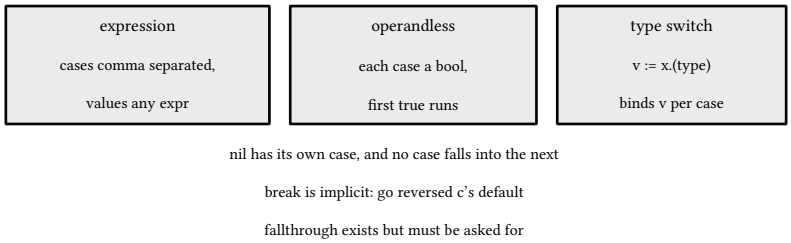


figure 39: three switch kinds, none of them falling through

8.4. labels, break, continue, goto

`break` and `continue` take an optional label to name which loop they mean, and `goto` jumps to a label in the same function, legal but rare, mostly generated code:

```
func FirstNegative(rows [][]int) int {
outer:
  for _, row := range rows {
    for _, v := range row {
      if v < 0 {
        return v
      }
      if v > 100 {
        break outer
      }
    }
  }
  return 0
}
```

listing 49: labeled break leaving both loops (source: go/samples/ch08/structure.go, lines 88-101)

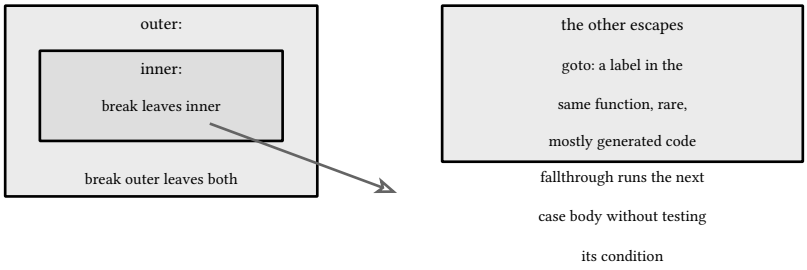


figure 40: the label names which loop the break leaves

`fallthrough` executes the next case's body unconditionally, without evaluating its condition, which is why the vowel function above does not need it and almost nothing does.

8.5. const and iota

Constants are compile time untyped values until context fixes their type, and `iota` is the row index of a const block, the entire enum mechanism the language has:

```
type Weekday int

const (
  Sunday Weekday = iota
  Monday
  Tuesday
  Wednesday
  Thursday
  Friday
  Saturday
)

const (
  _  = 1 << (10 * iota) // skip the first row, 1<<0
  KB                    // 1 << 10
  MB                    // 1 << 20
  GB                    // 1 << 30
)
```

listing 50: weekday enum and shifted sizes (source: go/samples/ch08/structure.go, lines 105-123)

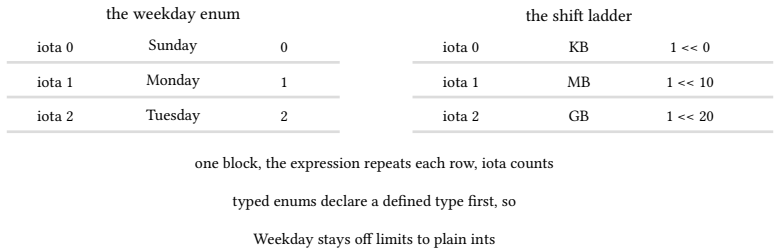


figure 41: iota is the row index, the expression repeats, two enums from one mechanism

One const block, repetition of the previous expression, `iota` counting rows: this builds the weekday enum and the kb, mb, gb ladder from identical machinery. Typed enums declare a defined type first, which is what makes `Weekday` printable and type safe against plain ints.

8.6. packages and visibility

A package is a directory, imported by path, compiled as a unit, and re-exporting nothing. The export rule from chapter 2 decides the boundary, and the sample's `inner` subpackage exists so the test file can stand outside it:

```
// Package inner exists so chapter 8's tests can stand outside it and
// observe the export boundary: Answer is reachable, hidden is not.
package inner

const Answer = 42

const hidden = 7

func Describe() string {
  // inside the package both spellings are ordinary names
  return "answer is " + itoa(Answer) + ", hidden is " + itoa(hidden)
}
```

listing 51: answer crosses the boundary, hidden does not (source: go/samples/ch08/inner/inner.go, lines 1-12)

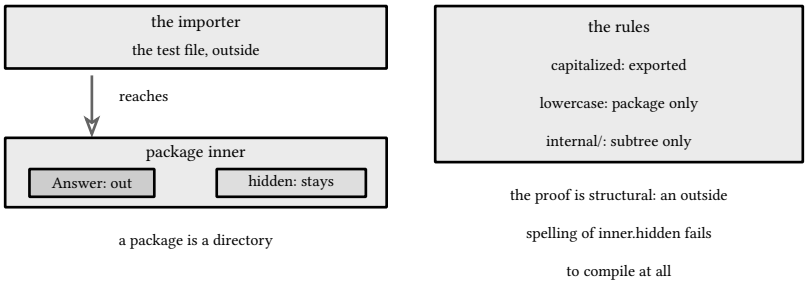


figure 42: the export boundary as a stack: capitalization crosses, lowercase stays

`inner.Answer` and `inner.Describe()` are reachable from the test, `inner.hidden` and `inner.itoa` are not, and the proof is structural: any attempt to spell them from outside fails to compile, so the test file cannot contain one. The `internal` directory name adds a second rule enforced by the toolchain, packages under `internal/` import only within their subtree, which is how a module hides its implementation packages.

sources: go.dev/ref/spec, declarations and statements, and effective go, accessed 2026-09-08. Verified by `go/samples/ch08` tests with the inner subpackage compiling the visibility boundary.

9. runtime

Go programs run on a small operating system, the runtime: a scheduler multiplexing goroutines onto threads, a concurrent garbage collector, growing stacks, and a profiler wired into every event. None of it needs configuration to work, and the realistic skill is knowing which knob exists and what it trades.

9.1. the scheduler

The model is M:N, many goroutines onto fewer os threads, with three players: `G` the goroutine, `M` the os thread, `P` the processor slot that owns a run queue. `GOMAXPROCS` sets the number of P slots and defaults to the machine's core count. Work steals balance the queues, network io integrates through netpoller so a goroutine waiting on a socket parks its G and frees the thread, and since go 1.14 preemption is async, a tight loop no longer starves the scheduler:

```
func Gomaxprocs() int {
  return runtime.GOMAXPROCS(0)
}
```

listing 52: the read form, argument zero (source: go/samples/ch09/runtime.go, lines 13-16)

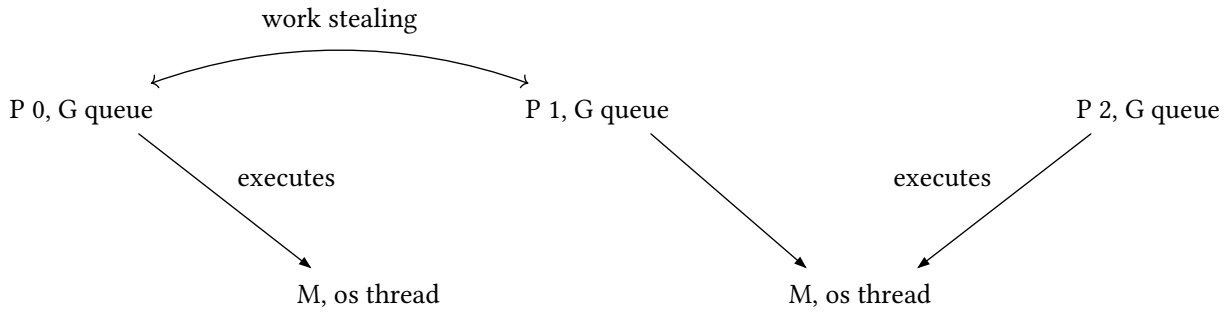


figure 43: G, M, P: goroutines wait in P run queues, M threads execute them, idle queues steal

Goroutine stacks start at 2 kib and grow by copying, up to the 1 gib per-goroutine ceiling (`debug.SetMaxStack` moves it), and past the cap the runtime kills the program with a fatal stack overflow error:

```
func SumTo(n int) int64 {
  if n == 0 {
    return 0
  }
  return int64(n) + SumTo(n-1)
}
```

listing 53: ten thousand frames deep, no configuration (source: go/samples/ch09/runtime.go, lines 36-41)

The test recurses 10000 deep and sums to 50005000. The cost model that follows from cheap stacks plus small allocations: prefer passing values, let escape analysis keep them on stack frames, and reach for object pools only when a profile says so.

9.2. the garbage collector

Concurrent tricolor mark and sweep, non generational, non compacting. The pacer targets a heap ratio: `GOGC=100`, the default, collects when the heap doubles since the last live set. Two knobs matter in production, `GOGC` for the ratio and `GOMEMLIMIT` for a soft memory ceiling, and the api route to the first is `debug.SetGCPercent`:

```
func GCAndCount() uint32 {
  runtime.GC()
  var stats runtime.MemStats
  runtime.ReadMemStats(&stats)
  return stats.NumGC
}

// HeapAlloc reads the live heap size in bytes from the memory
// statistics snapshot.
func HeapAlloc() uint64 {
  var stats runtime.MemStats
  runtime.ReadMemStats(&stats)
  return stats.HeapAlloc
}
```

listing 54: forced collection, count, and live heap (source: go/samples/ch09/runtime.go, lines 19-33)

`runtime.GC` forces a full cycle, `NumGC` counts them, `HeapAlloc` is the live heap the collector just measured. There are no generations and no separate large-object heap: objects over 32 kib skip the size classes and come from the page allocator, a split chapter 10 walks. Go 1.27 shaved up to 30 percent off sub 80 byte allocations through size specialized allocation routines, a runtime change invisible to code but visible in allocation profiles.

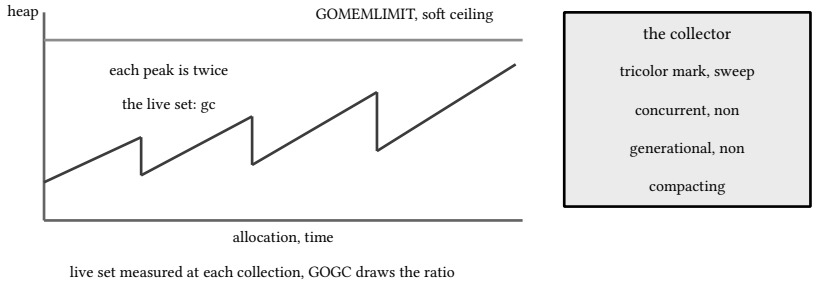


figure 44: the pacer: gogc collects when the heap doubles, gememlimit draws the ceiling

NOTE: what to actually do about the gc

Defaults first. If a service needs tighter memory, `GOMEMLIMIT` answers ceiling questions better than `GOGC` tuning, and the collection that hurts is usually caused by allocation rate, which `pprof` heap profiles attribute by allocation site. Escape analysis, `go build -gcflags=-m`, explains why a value escaped.

9.3. profiling and debug

Every profile hangs off `runtime/pprof`, cpu, heap, goroutine, block, mutex, threadcreate, and `net/http/pprof` serves them over http on a debug port. Go 1.27 adds `goroutineleak`, a detector for goroutines blocked on primitives that no runnable goroutine can reach, promoted from an experiment into the standard profile set:

```
func LeakProfileAvailable() bool {
  return pprof.Lookup("goroutineleak") ≠ nil
}
```

listing 55: the 1.27 leak profile, registered and queryable (source: go/samples/ch09/runtime.go, lines 46-48)

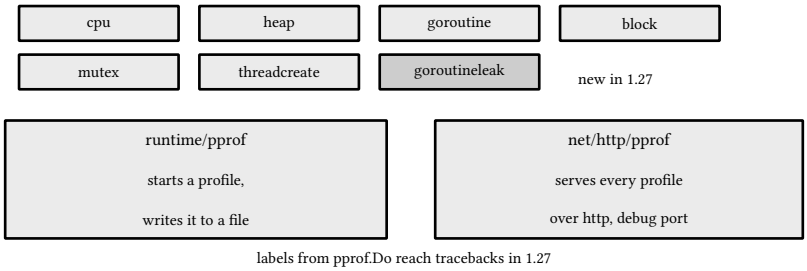


figure 45: the profile set, one registry, two ways in

The test asserts the profile exists, which is also a version check: on 1.26 this lookup returns nil and the test fails. Tracebacks gained goroutine labels in 1.27, so pprof labels set with `pprof.Do` appear in stack dumps, and the `GODEBUG` mechanism gates compatibility switches per module from the `go.mod` `godebug` directive. The remaining 1.27 runtime note for application code: the asynctimerchan compatibility period ended, timer channels from `time.After` and friends are unbuffered with no opt out, which the concurrency chapter's select patterns already assume.

9.4. the goroutine lifecycle

A goroutine starts with `go`, runs until its function returns, and nothing observes that return. Lifecycle management is explicit: `WaitGroup` for completion, channels for results, `context` for cancellation, and `runtime.NumGoroutine` plus the goroutine profile for leaks after the fact. The leak profile automates what teams did by diffing goroutine dumps, blocked forever on a send nobody receives, a mutex nobody releases, and it flags exactly those goroutines unreachable from any live work.

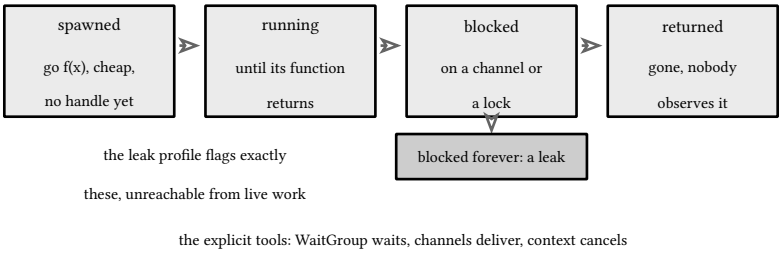


figure 46: spawned to returned with nobody watching, the leak is the blocked-forever case

sources: go.dev/doc/diagnostics, pkg.go.dev/runtime and runtime/debug for the `debug.SetMaxStack` 1 gib default, and the go 1.27 release notes runtime section, accessed 2026-09-08. Verified by `go/samples/ch09` tests on go1.27.0, including the goroutineleak profile presence check.

10. allocation: mspans, size classes, escape analysis, sync.Pool

Every go allocation funnels through one function, `mallocgc`, and almost all of them resolve without a lock. This chapter walks the path a 17-byte allocation actually takes on go 1.27, quotes the runtime's own size class table, makes escape analysis print its verdicts, demonstrates the pool layer the runtime sanctions on top, and closes on the corpus particle kernel in both memory layouts, measured on this machine.

10.1. the fast path

`mallocgc` sorts every request into tiny, small, or large before anything else: noscan objects under 16 bytes go to the tiny allocator, everything up to 32 KiB takes the size class path through the per-P `mcache`, and everything larger goes straight to `mheap` in whole pages. The 1.27 runtime compiles the small path into 81 size-specialized routines (`runtime/malloc_tables_generated.go`, `mallocScanTable`), the change behind the up to 30 percent faster sub-80-byte allocations the runtime chapter cites, and ordinary code can watch the decisions it makes:

```
// classBucket allocates one n-byte noscan object through a package
// sink so the compiler cannot elide it, then returns the BySize bucket
// whose Mallocs count moved. BySize[N] holds the class with
// Size == the bucket's size, the same classes as the runtime table.
// A neighbor goroutine's allocation can move a second bucket inside
// the window, so the reading retries until it is self-consistent: no
// bucket moved (the allocation went past the last reported class), or
// exactly one bucket able to hold n moved.
func classBucket(n int) (size uint32) {
  for range 25 {
    var before, after runtime.MemStats
    runtime.ReadMemStats(&before)
    bigEnoughToEscape = make([]byte, n)
    runtime.ReadMemStats(&after)
    moved := -1
    for i := range after.BySize {
      if after.BySize[i].Mallocs > before.BySize[i].Mallocs {
        if moved ≥ 0 {
          moved = -2
          break
        }
        moved = i
      }
    }
    if moved == -1 {
      return 0
    }
    if moved ≥ 0 && after.BySize[moved].Size ≥ uint32(n) {
      return after.BySize[moved].Size
    }
  }
```

listing 56: one allocation, then read the bucket that moved (source: go/samples/ch10/classes.go, lines 8-38)

`ReadMemStats` stops the world, so the `BySize` histogram it returns is exact: bucket N counts the mallocs the runtime charged to that size class. The test built on this helper feeds it 8, 16, 17, 24, 25, 32, 33, 48, 64, and 80 byte requests and reads back 16, 16, 24, 24, 32, 32, 48, 48, 64, 80, the table's rounding decisions observed live rather than trusted from a book.

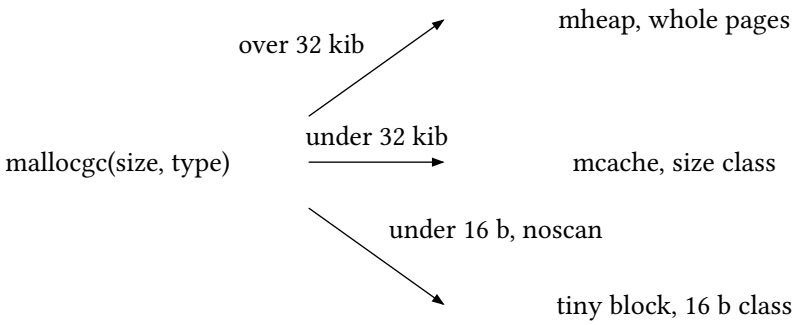


figure 47: every request sorted at the door, three doors out

The sorting is why allocation cost in go is roughly constant per size class, not per byte: a 40-byte and a 48-byte allocation cost the same work, both are class 5.

10.2. the mspan

The unit underneath every small allocation is the mspan, a run of 8 KiB pages (`PageShift` is 13) carved into same-size objects. Each span carries `startAddr` and `npages`, an `allocBits` bitmap with one bit per object slot, and a `freeindex` cursor so the next allocation scans forward from the last hit instead of from the start. Classes cap at `MaxObjsPerSpan` 1024 objects and `MaxSizeClassNPages` 10 pages, which is why the smallest classes fill 8192-byte spans with 1024 objects of 8 bytes and the largest small class, 32768 bytes, is a span of one object. Every span also carries a scan or noscan flavor, 68 classes times 2, because pointer-free spans sweep and scan differently.

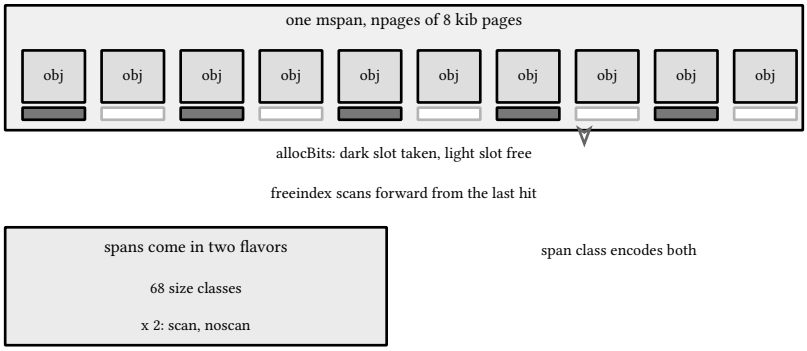


figure 48: one mspan: pages, the object bitmap, and the free cursor

A span's objects are contiguous, so freeing never compacts, it only clears bits, and the mark-sweep collector from the runtime chapter walks the same bitmaps when it decides what survives.

10.3. mcache, mcentral, mheap

Three levels, one rule: the level that answers most allocations takes no lock. Each P owns an `mcache` with one span per span class, so a small allocation on the fast path is a bump inside a span the running thread already owns. When a cache empties it refills from the `mcentral` for that class, which holds partial and full span sets split by swept and unswept, and that refill takes the class's lock, amortized over a whole span of objects. `mheap` sits under everything: the page allocator over arenas, and the direct source for large objects.

```
// largeObjectBytes allocates one n-byte object through the sink and
// returns the TotalAlloc delta: exactly n for a small class hit, the
// page-rounded size for an mheap direct allocation. Foreign bytes
// from a neighbor goroutine land in the same window, so the reading
// retries until the delta is exactly n or a whole number of 8 KiB
// pages, the two shapes the runtime itself can produce.
func largeObjectBytes(n int) uint64 {
  best := ^uint64(0)
  for range 25 {
    var before, after runtime.MemStats
    runtime.ReadMemStats(&before)
    bigEnoughToEscape = make([]byte, n)
    runtime.ReadMemStats(&after)
    d := after.TotalAlloc - before.TotalAlloc
    if d < best {
      best = d
    }
    if d == uint64(n) || (d > uint64(n) && d%8192 == 0) {
      return d
    }
  }
  return best
```

listing 57: totalalloc tells large objects from class hits (source: go/samples/ch10/classes.go, lines 60-81)

The test around this helper measures the boundary from above: a 32768-byte allocation is accounted as exactly 32768 bytes, still the top size class, while 32769 is accounted as 40960, five 8 KiB pages from `mheap`, measured on this machine, 2026-09-13. There is no class table entry in between, the large path rounds to whole pages. Both readers retry until their reading is self-consistent, exactly one bucket moved or exactly the allocation and nothing else, so a neighbor goroutine's allocation landing inside the window cannot inflate or misroute the count.

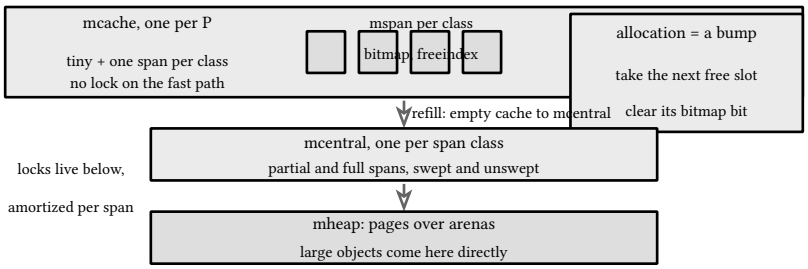


figure 49: the hierarchy: no lock, one lock, the global lock

10.4. the size class table

Go 1.27 moved the table out of `runtime/sizeclasses.go` into `internal/runtime/gc/sizeclasses.go`, generated by `runtime/_mkmalloc`, 67 real classes numbered 1 to 67, class 0 reserved for large objects. The excerpt below is a verbatim copy from this machine's go 1.27.0 source, and every row the chapter leans on is re-asserted against the live runtime by the tests:

```
//     1          8        8192     1024           0     87.50%          8
//     2         16        8192      512           0     43.75%         16
//     3         24        8192      341           8     29.24%          8
//     4         32        8192      256           0     21.88%         32
//     5         48        8192      170          32     31.52%         16
//     6         64        8192      128           0     23.44%         64
//     7         80        8192      102          32     19.07%         16
//     8         96        8192       85          32     15.95%         32
//     9        112        8192       73          16     13.56%         16
//    10        128        8192       64           0     11.72%        128
//    11        144        8192       56         128     11.82%         16
//    12        160        8192       51          32      9.73%         32
//    13        176        8192       46          96      9.59%         16
//    14        192        8192       42         128      9.25%         64
//    15        208        8192       39          80      8.12%         16
//    16        224        8192       36         128      8.15%         32
//    17        240        8192       34          32      6.62%         16
//    18        256        8192       32           0      5.86%        256
//    19        288        8192       28         128     12.16%         32
//    20        320        8192       25         192     11.80%         64
//    21        352        8192       23          96      9.88%         32
//    22        384        8192       21         128      9.51%        128
//    23        416        8192       19         288     10.71%         32
//    24        448        8192       18         128      8.37%         64
```

listing 58: classes 1 through 24, verbatim from the go 1.27 runtime source (source: go/samples/ch10/sizeclasses-go127.txt, lines 7-30)

Read the columns as: class number, bytes per object, bytes per span, objects per span, tail waste, max waste at the worst request size, and minimum alignment. Class 3 is where a 17-byte allocation lands, 24 bytes per object, 341 objects in an 8192-byte span with 8 bytes of tail waste. The constants close the system:

```
const (
  MinHeapAlign       = 8
  MaxSmallSize       = 32768
  SmallSizeDiv       = 8
  SmallSizeMax       = 1024
  LargeSizeDiv       = 128
  NumSizeClasses     = 68
  PageShift          = 13
  MaxObjsPerSpan     = 1024
  MaxSizeClassNPages = 10
  TinySize           = 16
  TinySizeClass      = 2
)
```

listing 59: the boundaries: 32 kib small, 16 byte tiny, 8 kib pages (source: go/samples/ch10/sizeclasses-go127.txt, lines 76-88)

```
// TestSeventeenByteObjectsCostTwentyFour measures the heap slope of
// 65536 retained 17-byte objects and asserts the class 3 cost, 24
// bytes each, within one byte of slack for accounting edges.
func TestSeventeenByteObjectsCostTwentyFour(t *testing.T) {
  slope := heapSlope(17, 1<<16)
  if slope < 23 || slope > 25 {
    t.Fatalf("17-byte slope %.3f, want 24.000 +/- 1", slope)
  }
}
```

listing 60: the 17-byte cost asserted as a measured heap slope (source: go/samples/ch10/classes_test.go, lines 27-35)

That test retained 65536 objects of 17 bytes and measured the heap grow by 24.000 bytes per object, 1572864 bytes total, the class 3 row holding up under measurement on this machine, 2026-09-13.

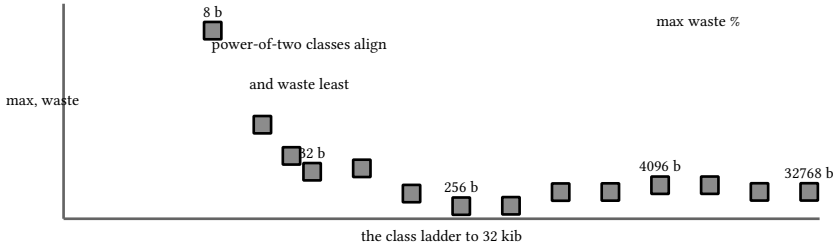


figure 50: class size against max waste, the spacing is the trade

The alignment column encodes the same fact: class 1, 4, 6, 10, 18, 26 and friends are the power-of-two sizes, minimal waste, special alignment, and go 1.27's specialized routines are densest exactly there.

10.5. tiny and large

Under 16 bytes and pointer-free, the tiny allocator packs multiple objects into one 16-byte block, with a worst case of 2x waste when all but one sub-object dies, a bound the runtime chose because 8 would waste nothing but combine nothing and 32 could waste 4x. The measured consequence on this machine:

```
// TestTinyPackingHalvesTheCost measures 8-byte noscan objects: the
// tiny allocator fits two per 16-byte block, slope 8, while a race
// detector build disables the tiny allocator and pays the full 16.
func TestTinyPackingHalvesTheCost(t *testing.T) {
  slope := heapSlope(8, 1<<16)
  if slope < tinySlopeWant-1 || slope > tinySlopeWant+1 {
    t.Fatalf("8-byte slope %.3f, want %.3f +/- 1", slope, tinySlopeWant)
  }
}
```

listing 61: two 8-byte objects per 16-byte block, asserted as a slope (source: go/samples/ch10/classes_test.go, lines 37-45)

65536 retained 8-byte objects grew the heap by exactly 8.000 bytes per object, 524288 bytes, half the class 2 cost, the packing observed. The `race_on.go` companion file flips the expectation to 16 under `go test -race`, because shadow memory needs one shadow per object and the race detector disables the tiny allocator entirely, a build mode visible in allocation counts. Above 32 KiB the size classes end and pages begin:

```
// TestLargeObjectsPageRound goes straight to mheap with no size
// class: 32 KiB still rounds to itself, 32 KiB + 1 rounds up to five
// 8 KiB pages.
func TestLargeObjectsPageRound(t *testing.T) {
  if got := largeObjectBytes(1 << 15); got != 1<<15 {
    t.Errorf("32768-byte allocation accounted %d, want 32768", got)
  }
  if got := largeObjectBytes(1<<15 + 1); got != 5*8192 {
    t.Errorf("32769-byte allocation accounted %d, want 40960", got)
  }
}
```

listing 62: the boundary: 32768 stays, 32769 pays five pages (source: go/samples/ch10/classes_test.go, lines 55-65)

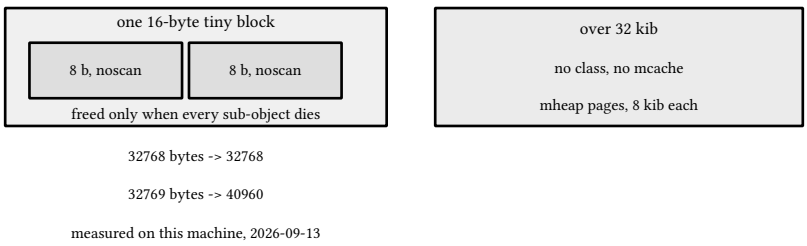


figure 51: tiny packing below, page rounding above

10.6. escape analysis

Allocation avoidance is decided before the runtime ever sees a request: escape analysis in the compiler assigns every value to a stack frame or to the heap. The tool is `go build -gcflags=-m`, and its verdicts for this chapter's own sample file were captured verbatim and committed:

```
// lives on the heap: "make([]int, 0) escapes to heap".
func Grown(n int) []int {
  s := make([]int, 0)
  for i := 0; i < n; i++ {
    s = append(s, i)
  }
  return s
}

// AsAny boxes a value into an interface: "v escapes to heap".
func AsAny(v int) any {
  return v
}

// Sum keeps everything in registers and on the stack: the compile
// says nothing about it, the quiet case.
func Sum(a, b int) int {
  total := a + b
  return total
}
```

listing 63: the quiet function allocates nothing, its neighbors say why (source: go/samples/ch10/escape.go, lines 33-52)

```
./escape.go:14:2: moved to heap: p
./escape.go:21:13: ... argument does not escape
./escape.go:21:15: p.x escapes to heap
./escape.go:21:20: p.y escapes to heap
./escape.go:26:11: dst does not escape
./escape.go:35:11: make([]int, 0) escapes to heap
./escape.go:37:13: append escapes to heap
./escape.go:44:9: v escapes to heap
```

listing 64: the compiler's verdicts for exactly those lines, captured 2026-09-13 (source: go/samples/ch10/escape-m.txt, lines 73-80)

`moved to heap` is the returned pointer, `escapes to heap` marks values boxed into `fmt`'s interface arguments or an `any` return, and `does not escape` is the callee borrowing caller memory, the verdict every hot function wants. `Sum` appears nowhere in the output: nothing it touches leaves the frame, and the test asserts `testing.AllocsPerRun` of 0 on it. A second test re-runs the compiler with the same flag and asserts the four pinned verdicts still appear, so the capture cannot rot into fiction as the toolchain moves.

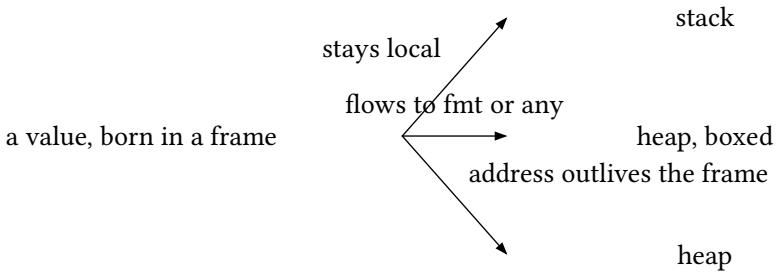


figure 52: where a value lives, decided at compile time

PITFALL: the interface tax is silent

Converting a value to `any` or passing it to a `fmt` verb boxes it, and the boxing allocates even when the value itself is two integers. The escape lines for `p.x` and `p.y` above are that tax, printed. In hot paths pass values, not interfaces.

10.7. sync.Pool

When allocation is unavoidable but objects are interchangeable, `sync.Pool` is the layer the runtime blesses: each P keeps a private slot and a shared list, `Get` checks the private slot, then the shared head, then steals from other Ps, then falls back to `New`. Collection interacts with pools through a two-generation victim cache: one GC moves pool contents aside where `Get` can still reach them, the next drops them, so a pool can never pin memory across two collections.

```
// least two objects per P guarantees the shared lists hold at least
// puts minus GOMAXPROCS objects, because each P parks at most one in
// its private slot: after one GC every one of those still serves
// from the victim cache, after a second GC nothing does.
func VictimCensus() (freshAfterOne, freshAfterTwo int64) {
  puts := 2*runtime.GOMAXPROCS(0) + 4
  runtime.GC()
  runtime.GC()
  base := poolNews.Load()
  for i := 0; i < puts; i++ {
    PutBuffer(new(buf512))
  }
  runtime.GC()
  for i := 0; i < puts; i++ {
    GetBuffer()
  }
  freshAfterOne = poolNews.Load() - base

  for i := 0; i < puts; i++ {
    PutBuffer(new(buf512))
  }
  runtime.GC()
  runtime.GC()
  for i := 0; i < puts; i++ {
    GetBuffer()
  }
  freshAfterTwo = poolNews.Load() - base - freshAfterOne
  return freshAfterOne, freshAfterTwo
}
```

listing 65: a census: fill, collect once, drain, collect twice, drain (source: go/samples/ch10/pool.go, lines 36-64)

Measured on this machine, 2026-09-13, with 44 objects over a 20-P machine: after one GC at most GOMAXPROCS of them came back from the constructor, the shared lists served the rest from the victim cache, and after the second GC every one of the 44 Gets allocated fresh. The test asserts both bounds, and the bounds are provable, not sampled: each P parks at most one object in its private slot, everything else rides shared lists.

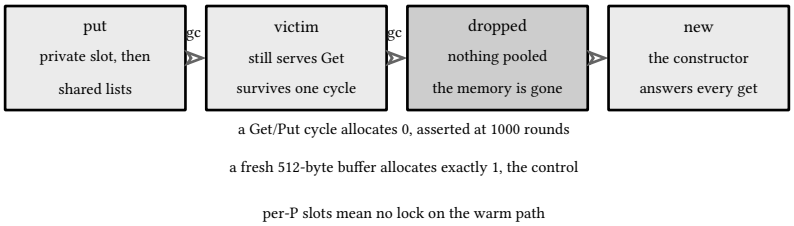


figure 53: the pool across two collections

10.8. the particle kernel

The corpus particle kernel is N particles times 8 float64 fields, a pure arithmetic step with an analytic force, and go holds it two ways: [8][]float64, eight columns, or []particle, rows of one 64-byte struct, which lands on exactly class 6 when it escapes:

```
// particle is the AoS row: 8 float64 fields, 64 bytes noscan, which
// lands exactly on size class 6 when one escapes.
type particle struct {
  x, y, z    float64
  vx, vy, vz float64
  m, q       float64
}

// soaKernel is [8][]float64, one column per field.
type soaKernel [nField][]float64

func newRand() *rand.Rand {
  return rand.New(rand.NewPCG(0x5eed2026, 0x10000003))
}
```

listing 66: the row type, and the seeded pcg every book shares (source: go/samples/ch10/particle.go, lines 33-46)

```
// StepSoa advances every particle once: pos += vel*dt + a*dt*dt,
// vel += a*dt, computing all three accelerations from the current
// state before writing, allocating nothing.
func StepSoa(k *soaKernel) {
  n := len(k[ix])
  dt2 := dt * dt
  for i := 0; i < n; i++ {
    ax, ay, az := accel(k[ix][i], k[iy][i], k[iz][i], k[im][i], k[iq][i])
    k[ix][i] += k[ivx][i]*dt + ax*dt2
    k[iy][i] += k[ivy][i]*dt + ay*dt2
    k[iz][i] += k[ivz][i]*dt + az*dt2
    k[ivx][i] += ax * dt
    k[ivy][i] += ay * dt
    k[ivz][i] += az * dt
  }
}
```

listing 67: the SoA step: read five columns, write six, allocate nothing (source: go/samples/ch10/particle.go, lines 95-110)

The same arithmetic in row order is `StepAoS` four lines shorter, and both step functions assert `AllocsPerRun` of 0 at fixed input, so representation is a locality question, not an allocation one. The smoke test runs 10000 particles through both forms from the same seed and requires bit-identical fields afterward, which holds because per-particle arithmetic runs in the same order either way.

Measured on this machine, 2026-09-13, go1.27.0, 12th gen i7-12700F, 20 hardware threads, one goroutine, 16 steps over 1m particles (`go/samples/ch10/bench-1m.txt`, summarized with benchstat): `[]particle` 136.8 ms per K-step, 8.2 ns per particle per step, within a 7 percent band, `[8][]float64` 518.3 ms, 30.9 ns, within 12 percent. The row form wins here because the kernel touches all eight fields of every particle each step, one cache line per particle in one stream, while the column form walks eight streams the hardware prefetcher tracks worse. Column layout pays off when a pass touches a subset of fields, not when every pass touches everything.

The same day's runs bracket the column number honestly: a first bench session of the same binary measured 187 ms before sustained load, and the 100m one-off below measured 8.9 ns per step in a single pass. Bandwidth-bound loops swing with machine state, so the pinned numbers are the dated capture's, with the spread recorded rather than hidden.

The 100m one-off ran alone and sequentially with `GOGC=off`, so the 6.4 GB working set could not double under the default heap goal: 5.237 s to allocate and seed eight 800 MiB columns, 14.236 s for the 16 steps, 8.898 ns per particle per step, 5.96 GiB live, checksum 25003226.600833, then the memory was released and the scratch program deleted. One billion particles is 64 GB of columns and stays arithmetic on this machine, not a measurement.

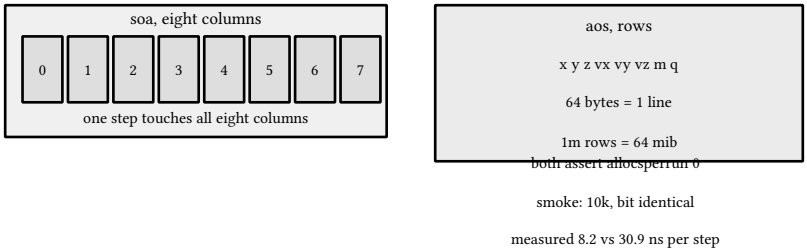


figure 54: one particle, two layouts, one cache line either way

sources: go.dev/src runtime and internal/runtime/gc sources on this machine (`malloc.go`, `malloc_tables_generated.go`, `mheap.go`, `mcache.go`, `mcentral.go`, `sizeclasses.go`, `sync/pool.go` from go1.27.0), pkg.go.dev/runtime for MemStats and testing, and pkg.go.dev/sync for Pool, accessed 2026-09-13. All measured numbers are from runs on this machine, 2026-09-13, pinned in `go/samples/ch10/escape-m.txt`, `sizeclasses-go127.txt`, and `bench-1m.txt`, re-asserted by the 20 tests of `go/samples/ch10`.

11. profiling: pprof, the tracer, benchstat, p99

The runtime chapter listed the profile set. This chapter uses it: capture a profile from a real workload, read it with the tools, make one measured change, prove it with benchstat, and state latencies as percentiles instead of averages. Every capture in this chapter was taken on this machine and committed with its date, and the suite re-asserts the load-bearing parts of each one.

11.1. the profile set

One workload feeds every capture below: a deterministic chunk loop, sha256 over seeded bytes, cheap enough for the suite and real enough to profile:

```
func ProcessChunk(i int) uint64 {
  r := rand.New(rand.NewPCG(uint64(i), 0x5eed2026))
  buf := make([]byte, ChunkBytes)
  for j := 0; j < len(buf); j += 8 {
    v := r.Uint64()
    for k := 0; k < 8 && j+k < len(buf); k++ {
      buf[j+k] = byte(v >> (8 * k))
    }
  }
  sum := sha256.Sum256(buf)
  var first uint64
  for k := 0; k < 8; k++ {
    first = first<<8 | uint64(sum[k])
  }
  return first
}

// ServiceLoop runs every chunk and records each one's latency in
// milliseconds into lat, returning the sum of all checksums so a
```

listing 68: one chunk: pcg-seeded bytes, hashed, checksummed (source: go/samples/ch11/service.go, lines 23-41)

The six captures: cpu profiles sample the instruction pointer, heap profiles carry allocation sites, goroutine dumps hold every stack, block and mutex profiles time contention, the 1.27 `goroutineleak` detector flags unreachable blocked goroutines, and the execution tracer records scheduler events on a timeline. Cpu, heap, and trace answer why is it slow, the contention pair answers why is it waiting, the goroutine set answers what is alive.

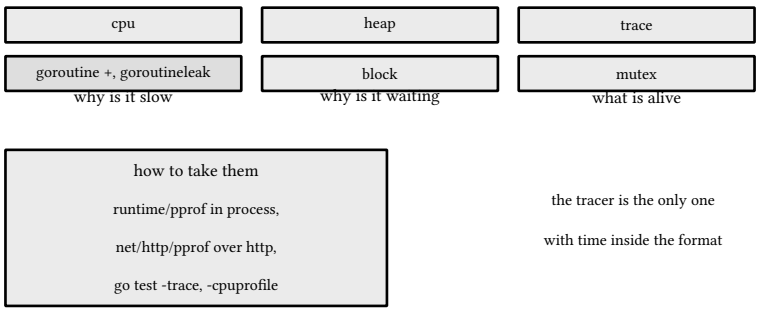


figure 55: six captures, three questions

11.2. pprof, read as text

`go tool pprof` needs no browser: `-top` prints the flat and cumulative table. The capture below came from `go test -cpuprofile` over the chunk loop's own test, then `go tool pprof -top` over the profile file:

```
Showing nodes accounting for 40ms, 100% of 40ms total
      flat  flat%   sum%        cum   cum%
      30ms 75.00% 75.00%       30ms 75.00%  crypto/internal/fips140/sha256.blockSHANI
      10ms 25.00%  100%        40ms  100%  gobook/ch11.ProcessChunk
         0     0%  100%        30ms 75.00%  crypto/internal/fips140/sha256.(*Digest).Write
         0     0%  100%        30ms 75.00%  crypto/internal/fips140/sha256.block
         0     0%  100%        30ms 75.00%  crypto/sha256.Sum256
         0     0%  100%        40ms  100%  gobook/ch11.ServiceLoop
         0     0%  100%        40ms  100%  gobook/ch11.TestServiceLoopP99RunsTheChunks
```

listing 69: the whole cost of 1024 hashed chunks, flat table in nine lines (source: go/samples/ch11/pprof-top.txt, lines 12-20)

Flat is time inside a frame, cum is time inside the subtree, and the sample math is visible: 40 ms of samples over a 42.74 ms profile window, 93.59 percent of it. The story is two rows. 75 percent is `blockSHANI`, the hardware sha extension inner loop, and 25 percent is the kernel's own `ProcessChunk`, the pcg fill loop. The in-suite test that mirrors this capture asserts the file starts with the gzip magic `1f 8b` (a pprof profile is gzipped protobuf) and holds real bytes, so the artifact cannot rot.

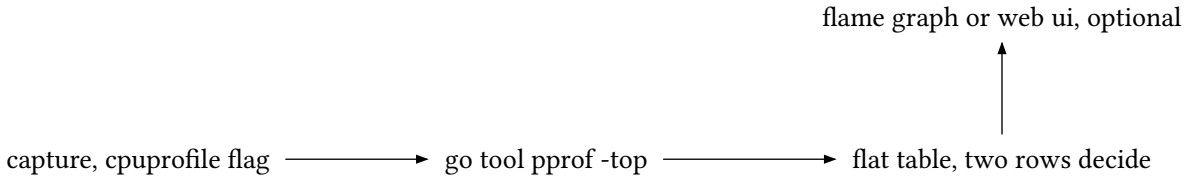


figure 56: profile to answer

11.3. the execution tracer

Cpu profiles say where cycles went, the tracer says what the scheduler did while they went there: goroutine park and resume, channel operations, network polls, gc phases, all on one timeline. `go test -trace trace.out` captures a whole test run, `trace.Start` scopes it, and `go tool trace trace.out` opens the viewer over the same events. The suite captures one loop and checks the format itself:

```go
// TestTraceCaptures: the execution tracer writes a real file: header
// magic for the go 1.25+ format, and a size that grew with the work.
func TestTraceCaptures(t *testing.T) {
  f, err := os.CreateTemp("", "ch11-trace-*")
  if err ≠ nil {
    t.Fatal(err)
  }
  defer os.Remove(f.Name())
  defer f.Close()

  if err := trace.Start(f); err ≠ nil {
    t.Fatal(err)
  }
  lat := &Latencies{}
  ServiceLoop(8, lat)
  trace.Stop()
  info, err := os.Stat(f.Name())
  if err ≠ nil {
    t.Fatal(err)
  }
  head := make([]byte, 14)
  if _, err := f.ReadAt(head, 0); err ≠ nil {
    t.Fatal(err)
  }
  if string(head[:5]) ≠ "go 1." {
    t.Fatalf("trace header %q, want go 1.x magic", string(head))
  }
  if info.Size() < 500 {
    t.Fatalf("trace file %d bytes, want ≥ 500", info.Size())
  }
```

listing 70: trace start, work, stop, then assert the format (source: go/samples/ch11/metrics_test.go, lines 96-125)

The header the test asserts is literally `go 1.26 trace` on a go 1.27 runtime, the format is versioned independently of the compiler, and the file grows with the events recorded, so a few hundred bytes minimum is the floor that says events landed. When a latency question is about coordination rather than computation, a cpu profile shows nothing and this file shows everything.

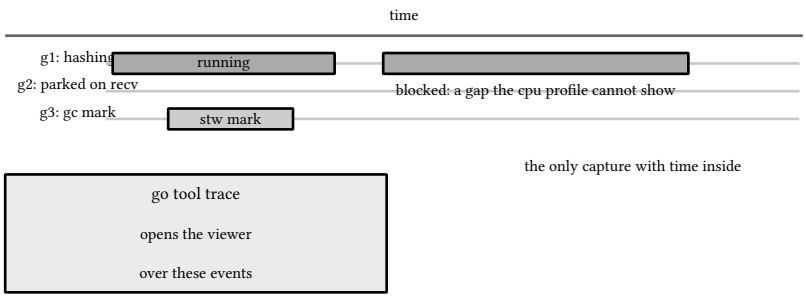


figure 57: one timeline, every actor on it

11.4. benchstat, honest comparison

A single benchmark number is weather, not climate. The discipline: run both sides ten times, let `benchstat` (golang.org/x/perf) compute the statistics, and quote the table it prints. The pair lives in the samples: the loop concatenation every language has warned about since its first manual, and the builder:

```
func SlowBuild(n int) string {
  s := ""
  for i := 0; i < n; i++ {
    s += "station"
  }
  return s
}

// FastBuild appends into one strings.Builder whose buffer grows
// amortized, the measured change.
func FastBuild(n int) string {
  var b strings.Builder
  for i := 0; i < n; i++ {
    b.WriteString("station")
  }
  return b.String()
}
```

listing 71: the A side and the B side, same output (source: go/samples/ch11/build.go, lines 14-31)

```
            │    sec/op     │     sec/op      vs base    │
SlowBuild-20   5.868µ ± 21%
FastBuild-20                   554.0n ± 15%
geomean        5.868µ          554.0n         ? ¹ ²
¹ benchmark set differs from baseline; geomeans may not be comparable
```

listing 72: what benchstat printed over the ten-run capture (source: go/samples/ch11/benchstat-ab.txt, lines 7-11)

The table carries three verdicts at once: 5.868 microseconds against 554 nanoseconds, about ten times, 15240 bytes against 1016, and 63 allocations against 7, both allocation columns flat enough that the delta is structural, not noise. The capture underneath (`bench-ab.txt`) holds all twenty raw runs. Note what the tool does not let you do: with +-21 percent on the A side, a 10 percent improvement claim from one run each would be unprovable, and benchstat would show overlapping intervals instead of the clean column above.

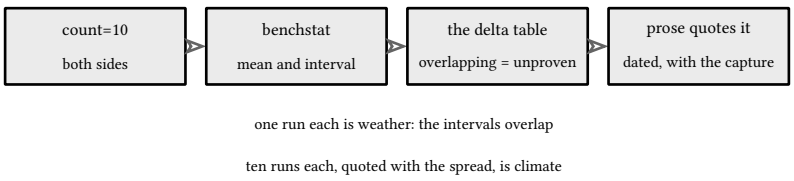


figure 58: the flow that earns a number

11.5. runtime/metrics

`GODEBUG` output is for humans. Programs that report on themselves read `runtime/metrics`: named values, one `Sample` slice, one `Read` call, versions discoverable at runtime so a wrong name is a loud panic, never a silent zero:

```go
// Gauge reads one runtime/metrics value by name and returns it as a
// float64, the shape every gauge kind converts to.
func Gauge(name string) float64 {
  sample := []metrics.Sample{{Name: name}}
  metrics.Read(sample)
  if sample[0].Value.Kind() == metrics.KindBad {
    panic("metrics: unknown gauge " + name)
  }
  v := sample[0].Value
  switch v.Kind() {
  case metrics.KindUint64:
    return float64(v.Uint64())
  case metrics.KindFloat64:
    return v.Float64()
  case metrics.KindFloat64Histogram:
    // a histogram gauge has no single number: report the median
    // bucket boundary, the honest scalar summary available
    h := v.Float64Histogram()
    return h.Buckets[len(h.Buckets)/2]
  }
  panic("metrics: gauge " + name + " is not a scalar")
}
```

listing 73: one gauge, by name, every scalar kind covered (source: go/samples/ch11/metrics.go, lines 7-28)

The panel the chapter reads covers allocation bytes, gc cycles, gc cpu share, mutex wait seconds, live goroutines, and the heap goal, and the tests use it the way production code would: the allocation gauge moves at least a mebibyte when a mebibyte is allocated, the cycle gauge counts two forced collections as two. Histogram-valued metrics come back as `Float64Histogram` with explicit bucket boundaries, the honest form of a latency gauge, and the helper's fallback of a median bucket is marked as exactly that.

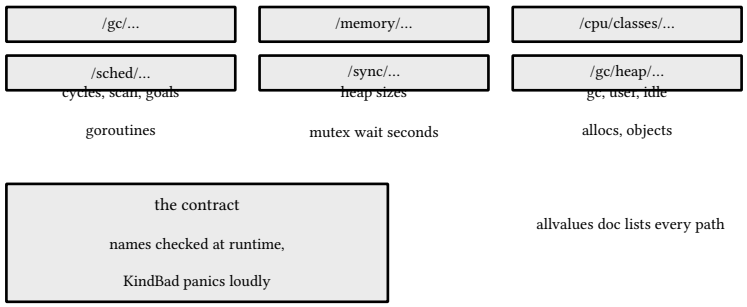


figure 59: the gauge board, by path

11.6. gctrace, the raw feed

`GODEBUG=gctrace=1` prints one line per collection to stderr, ugly and indispensable in a pinch. A capture of this chapter's own suite run is committed, and the test parses it structurally rather than trusting the prose:

```
gc 1 @0.026s 0%: 0+0.49+0 ms clock, 0+0/1.4/0+0 ms cpu, 3→3→1 MB, 4 MB goal, 0 MB stacks, 0 MB globals,
20 P
gc 2 @0.031s 0%: 0+1.0+0 ms clock, 0+0/3.0/0+0 ms cpu, 3→4→1 MB, 4 MB goal, 0 MB stacks, 0 MB globals, 20
P
gc 3 @0.032s 1%: 0+0.49+0 ms clock, 0+0.99/1.9/0+0 ms cpu, 3→5→1 MB, 4 MB goal, 0 MB stacks, 0 MB
globals, 20 P
gc 4 @0.033s 2%: 0+1.5+0 ms clock, 0+2.0/5.0/0+0 ms cpu, 3→5→2 MB, 4 MB goal, 0 MB stacks, 0 MB globals,
20 P
gc 5 @0.034s 2%: 0+1.6+0 ms clock, 0+0/3.3/0+0 ms cpu, 4→6→2 MB, 5 MB goal, 0 MB stacks, 0 MB globals, 20
P
```

listing 74: five of the 38 lines, 2026-09-13, this machine (source: go/samples/ch11/gctrace.txt, lines 8-12)

Reading one line: cycle 4 started 33 ms into the process, gc has cost 2 percent of cpu so far, the three clock numbers are sweep termination, concurrent mark, and mark termination, the heap triple is 3->5->2, mb at cycle start, after mark, and live after sweep, then the 4 mb goal the pacer set for the next cycle, stack and global bytes, and the P count. Two details the capture documents: `(forced)` marks `runtime.GC` calls, and cycle numbers restart mid-file because `GODEBUG` propagates to the go tool's child processes, so the compile step's collections appear too. The parser in the samples turns each line into those numbers and the test asserts cycles count up within each process and the goal never sits under the live set.

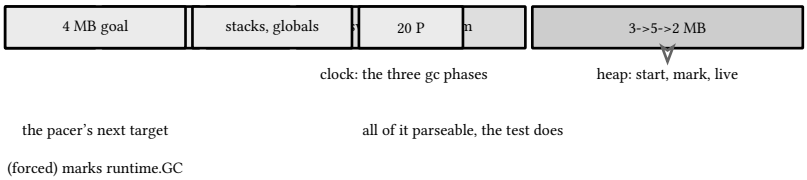


figure 60: one gctrace line, annotated

11.7. p99, the honest latency

Averages hide the tail, and the tail is what users feel. The corpus percentile definition is nearest-rank: sort the chunk latencies, take the ceil(p*n)-th. The harness is deliberately small:

```go
type Latencies struct {
  ms []float64
}

// Record adds one chunk's duration in milliseconds.
func (l *Latencies) Record(ms float64) { l.ms = append(l.ms, ms) }

// Len reports how many chunks were recorded.
func (l *Latencies) Len() int { return len(l.ms) }

// At returns the nearest-rank percentile for p in (0, 1]: sort, then
// take the ceil(p*n)-th smallest.
func (l *Latencies) At(p float64) float64 {
  if len(l.ms) == 0 {
    return 0
  }
  sorted := append([]float64(nil), l.ms...)
  sort.Float64s(sorted)
  rank := int(math.Ceil(p * float64(len(sorted))))
  if rank < 1 {
    rank = 1
  }
  return sorted[rank-1]
}

// Summary returns the four percentiles the corpus table carries.
func (l *Latencies) Summary() Percentiles {
  return Percentiles{P50: l.At(0.50), P95: l.At(0.95), P99: l.At(0.99), P999: l.At(0.999)}
}
```

listing 75: record, sort, index: the whole definition (source: go/samples/ch11/p99.go, lines 17-45)

One measured gotcha belongs here because the suite hit it: this machine's monotonic clock advances in roughly 0.5 ms steps (40 distinct readings in 20 ms, measured 2026-09-13), so timing a 20 microsecond chunk with `time.Now` mostly returns 0, and 193 of 200 timings of a 32 KiB hash read exactly that. Sub-tick work must be summed over many iterations, which is precisely why the benchmark harness runs `b.N` or `b.Loop` repetitions instead of one timed call. The suite's 1024-chunk run therefore asserts the harness mechanics, monotone percentiles and a non-zero tail, while the percentile numbers worth quoting come from chunks big enough to dwarf the tick, in the row kernel below.

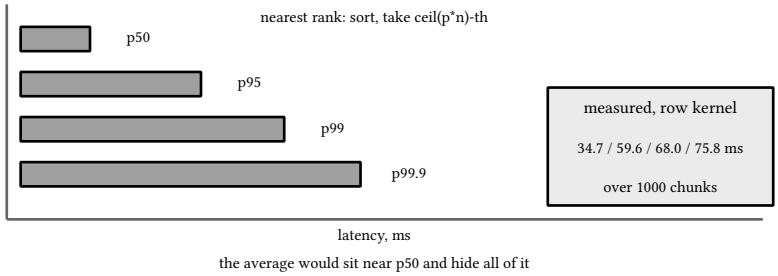


figure 61: the tail ladder over 1000 chunks

11.8. the row kernel, ten million rows

The corpus row kernel gives go a real database to profile: one `readings` table, station plus 8 REAL columns, 412 stations, seeded rows written 10000 at a time through a prepared statement inside a transaction, pure go sqlite underneath:

```
func (k *RowKernel) InsertBatch(ctx context.Context, batch, rows int) error {
  r := rand.New(rand.NewPCG(uint64(batch), 0x10000003))
  tx, err := k.db.BeginTx(ctx, nil)
  if err ≠ nil {
    return err
  }
  defer tx.Rollback()
  stmt, err := tx.PrepareContext(ctx,
    `INSERT INTO readings (station, v0, v1, v2, v3, v4, v5, v6, v7)
     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`)
  if err ≠ nil {
    return err
  }
  defer stmt.Close()
  for i := 0; i < rows; i++ {
    st := fmt.Sprintf("st%03d", r.IntN(NumStations))
    if _, err := stmt.ExecContext(ctx,
      st, r.Float64(), r.Float64(), r.Float64(), r.Float64(),
      r.Float64(), r.Float64(), r.Float64(), r.Float64()); err ≠ nil {
      return err
    }
  }
  return tx.Commit()
}

// SeedRows inserts rows rows in transactions of up to BatchRows,
```

listing 76: one 10k-row chunk: begin, prepare, insert, commit (source: go/samples/ch11/rowkernel.go, lines 54-79)

Measured on this machine, 2026-09-13, modernc.org/sqlite v1.58.0, one goroutine, `go/samples/ch11/row-10m.txt`: 10,000,000 rows inserted in 37.321 s, 267942 rows per second, over exactly 1000 chunks whose latencies were p50 34.688 ms, p95 59.607 ms, p99 67.957 ms, p99.9 75.824 ms, into a 0.81 GiB database. The SQL aggregation pass, `GROUP BY` with min, max, and avg over all eight columns, returned 412 station rows scanning 10m rows in 30.805 s, 324614 rows per second, and the manual cursor walk over the same data folded sums in go at 610720 rows per second. The walk beats the aggregate because sqlite computes 24 values per group while the walk adds 8 per row, and the pass choice is exactly the kind of decision a profile should drive, not a hunch.

The aggregation pass both ways:

```
func (k *RowKernel) AggregateSQL(ctx context.Context) ([]StationStat, float64, error) {
  start := time.Now()
  rows, err := k.db.QueryContext(ctx, `SELECT station,
    min(v0), max(v0), avg(v0), min(v1), max(v1), avg(v1),
    min(v2), max(v2), avg(v2), min(v3), max(v3), avg(v3),
    min(v4), max(v4), avg(v4), min(v5), max(v5), avg(v5),
    min(v6), max(v6), avg(v6), min(v7), max(v7), avg(v7)
    FROM readings GROUP BY station ORDER BY station`)
  if err ≠ nil {
    return nil, 0, err
  }
  defer rows.Close()
  var stats []StationStat
  count := 0
  for rows.Next() {
    var s StationStat
    dest := make([]any, 0, 25)
```

listing 77: the group by over all eight columns (source: go/samples/ch11/rowkernel.go, lines 109-125)

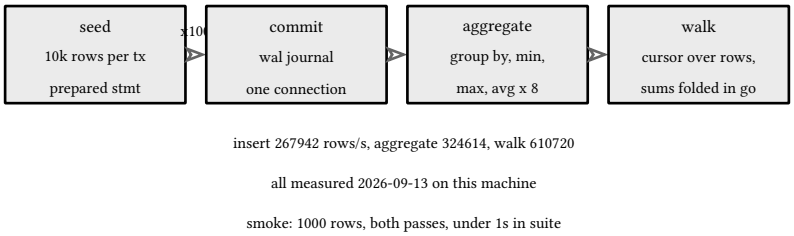


figure 62: the kernel: seed, commit, aggregate, walk

NOTE: scale ladder honesty

10m rows is the quoted in-prose run. The 100m one-off ran alone and sequentially the same day, `row-100m.txt`: 100,000,000 rows in 6m45.78s, 246439 rows per second, 8 percent under the 10m rate so the extrapolation held, into an 8.14 GiB database deleted after the run, chunk percentiles p50 38.530, p99 66.462, p99.9 71.699 ms over 10000 chunks, sql aggregate 225924 rows per second, walk 613804. One billion rows is arithmetic, roughly 81 GiB at this schema, not a measurement on this machine.

11.8.1. the duckdb lane, same kernel

The infrastructure book runs the same row kernel on duckdb, and its engine chapter book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine) is where the engine questions live. This book's lane exists to drive it from go: the twin keeps the shape and swaps the plumbing, rows entering through the driver's appender (`sql.Conn.Raw` down to the `*duckdb.Conn`, `NewAppenderFromConn`, vectors of 2048 rows, one Flush per 10000-row batch), the group by running unchanged SQL over vectorized aggregation, and the walk analog scanning the result stream chunk by chunk with sums folded in go. sqlite's REAL is a float64, duckdb's is a float32, so the twin table declares DOUBLE, and the whole lane rides the duckdb_use_lib build tag because the driver links the pinned official duckdb.dll.

Measured 2026-09-20, duckdb 1.5.5, one connection over the engine's own thread pool, `go/samples/ch11/row-10m-duckdb.txt`: 10,000,000 rows appended in 12.913 s, 774437 rows per second, p99 68.56 ms per 10k-row batch, the group by returning 412 stations at 124918491 rows per second, and the chunk-scan walk at 4864899 rows per second. The aggregate is not a like-for-like cell against the sqlite row: the engine parallelizes the scan across its thread pool while every other row in the table is one thread, so that column compares engine architecture, not language.

11.9. which tool when

the question	the tool
which function is hot	pprof -top over a cpu profile
where do allocations come from	heap profile, alloc_space
why is it blocked	block and mutex profiles
what leaks	goroutine + goroutineleak dumps
when did scheduling cost time	the tracer, go tool trace
did the change work	bench -count=10, then benchstat
how bad is the tail	nearest-rank percentiles, this chapter

figure 63: one question, one tool

The undercurrent is the same everywhere: capture from the real path, quote the tool's output, keep the capture beside the claim, and let a test re-derive what a test can re-derive. The capstone chapter applies all of it to the crawler.

11.10. across the six books

The same two kernels ran in all six language books under one contract, N particles of 8 float64 fields stepped 16 times and 10 million readings rows of one station key plus 8 real columns, every number below quoted from the chapter that measured it on this machine, 2026-09-13. The first two columns are the particle step at 1m in each language's row-of-objects repr and its flat column repr, and they disagree on purpose: the flat repr wins 1.4x in c, 2.6x in javascript, and 21x in python, the object repr wins 2.3x in c# and 3.8x in go because the step touches all eight fields of every particle and one cache line per particle beats eight streams in flight, and lua's ffi cdata wins 11.5x over boxed tables. The last four columns are the row kernel at 10m rows under the shared contract, insert through 10k-row batches (committed transactions on sqlite, appender flushes on duckdb), the sql group by, the manual scan (cursor walk on sqlite, chunk stream on duckdb), and the p99 of one 10k-row batch. Lua's p99 cell sits empty on purpose: its os.clock ticks at 1 ms and the chapter refuses to quote a percentile it cannot read, the boundary every book states in its own clock. The duckdb row is the engine lane this chapter measured: same kernel over the pinned 1.5.5 dll, its group by parallelized across the engine's thread pool. Its particle columns sit empty on purpose: the engine is the subject there, not a language repr, so there is no object or flat step to time.

book	objects ns	flat ns	insert/s	group/s	walk/s	p99 ms
c, clang -O2	6.15 to 6.75	4.53 to 4.91	468,803	584,696	2,242,885	28.3
c#, .net 11	5.7	13.1	339,091	723,248	501,820	48.5
go 1.27	8.2	30.9	267,942	324,614	610,720	67.957
javascript, node 26	12.4	4.72	530.42k	851k	562.3k	17.914
python 3.14	345.7	16.18	327,703	536,378	307,199	39.60
lua 5.5 + luajit	114.1	9.9	504,765	573,915	580,353	
duckdb 1.5.5			774,437	124.92m	4,864,899	68.56

Table 1: the two kernels across the six language books plus the duckdb engine lane: particle step ns at 1m in each book's object and flat repr, then the 10m row kernel with insert, group by, and walk in rows per second and p99 in milliseconds per 10k-row batch, every cell quoted from the chapter that measured it

The ladder above the table stays in its own book: the 100m particle one-off measured 5.768 ns flat and 7.229 ns object in c, 21.815 ns flat in c# (memory bound, soa only), 8.898 ns flat in go, 4.63 ns flat in javascript, 16.88 ns flat in python, and 8.2 ns ffi in lua, with the boxed 100m priced at 13.9 GB and refused. The 100m row one-off held its 10m insert rate within 5 percent in c, javascript, and lua, gained 4 percent in python, and paid 8 percent in go and 21 percent in c#. One billion is arithmetic in every book, 64 GB of particle state and roughly 81 to 87 GB of rows, not measured on this machine.

sources: go.dev/doc/diagnostics (profiling, tracing, runtime metrics, gdb), pkg.go.dev/runtime/pprof, runtime/trace, runtime/metrics, and testing, the go.dev/blog tracer and execution tracer posts, plus pkg.go.dev/modernc.org/sqlite and pkg.go.dev/github.com/duckdb/duckdb-go/v2, accessed 2026-09-13. Measured numbers are from runs on this machine, 2026-09-13, pinned in `go/samples/ch11/pprof-top.txt`, `bench-ab.txt`, `benchstat-ab.txt`, `gctrace.txt`, `row-10m.txt`, and `row-100m.txt`, the duckdb lane 2026-09-20 in `row-10m-duckdb.txt`, re-asserted by the 14 tests of `go/samples/ch11` plus the duckdb_use_lib-gated parity gate.

12. stdlib tour 1: fmt, errors, slices, maps, cmp, time, slog

Half of go's day to day value is the standard library, and the bet it makes is different from C#'s: fewer packages, each a default you keep. This tour covers the text and data plumbing every program touches. The sample package for this chapter is deliberately test only, its files call the stdlib directly, so every listing is a passing test.

12.1. fmt

Formatting is one function family, `Sprintf`, `Fprintf`, `Errorf`, sharing a verb language:

```
func TestFmtVerbs(t *testing.T) {
  cases := []struct{ got, want string }{
    {fmt.Sprintf("%d", 42), "42"},
    {fmt.Sprintf("%q", "hi"), `"hi"`},
    {fmt.Sprintf("%.2f", 3.14159), "3.14"},
    {fmt.Sprintf("%T", 3.14), "float64"},
    {fmt.Sprintf("%[2]s %[1]s", "a", "b"), "b a"},
    {fmt.Sprintf("%v", []int{1, 2}), "[1 2]"},
  }
  for _, c := range cases {
    if c.got ≠ c.want {
      t.Errorf("Sprintf = %q, want %q", c.got, c.want)
    }
  }
```

listing 78: verbs, precision, index arguments (source: go/samples/ch12/stdlib_test.go, lines 14-27)

`%v` is the value, `%+v` adds struct fields, `%#v` go syntax, `%q` quotes strings and runes, `%T` prints the type, and indexed arguments `%[1]s` reorder. The same verbs drive `log` and `fmt.Fscan` in reverse.

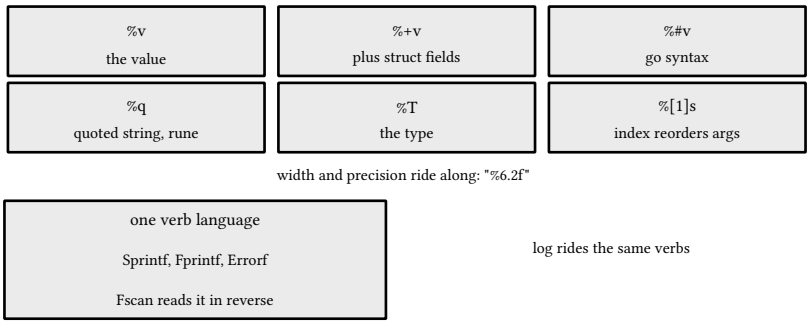


figure 64: the verb grid, one language driving the whole fmt family

12.2. slices and maps

Since go 1.21 the `slices` and `maps` packages replaced almost every hand written loop, and since 1.23 they return iterators, functions composable before anything materializes:

```go
func TestSlicesToolkit(t *testing.T) {
  xs := []int{3, 1, 2}
  slices.Sort(xs)
  if !slices.Equal(xs, []int{1, 2, 3}) {
    t.Fatalf("Sort = %v", xs)
  }
  if !slices.Contains(xs, 2) || slices.Contains(xs, 9) {
    t.Fatal("Contains misbehaves")
  }
  if slices.Index(xs, 3) != 2 || slices.Index(xs, 9) != -1 {
    t.Fatal("Index misbehaves")
  }

  // iterator based: Sorted materializes any sequence, Backward and
  // Chunk stream positions and fixed size windows
  words := []string{"pear", "fig", "date"}
  if got := slices.Sorted(slices.Values(words)); got[0] != "date" {
    t.Fatalf("Sorted = %v", got)
  }
  var visited []string
  for _, w := range slices.Backward(words) {
    visited = append(visited, w)
  }
  if fmt.Sprint(visited) != "[date fig pear]" {
    t.Fatalf("Backward visited %v, want [date fig pear]", visited)
  }
  var lens []int
  for chunk := range slices.Chunk(words, 2) {
    lens = append(lens, len(chunk))
  }
  if fmt.Sprint(lens) != "[2 1]" {
    t.Fatalf("Chunk sizes = %v, want [2 1]", lens)
  }
}
```

listing 79: the toolkit plus the iterator trio (source: go/samples/ch12/stdlib_test.go, lines 30-63)

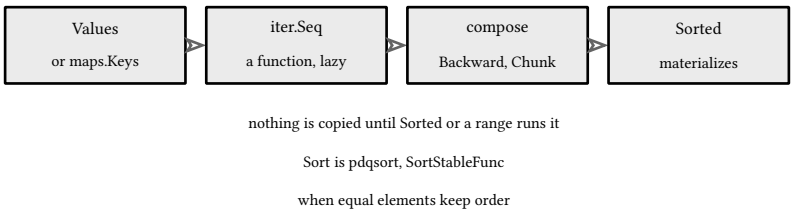


figure 65: iterators compose before anything materializes

`Values` yields the elements as a lazy sequence, `Backward` yields index and value from the end, `Chunk` streams fixed size windows, and `maps.Keys` returns `iter.Seq`, so `for k := range maps.Keys(m)` walks a map without copying its keys. Sorting is `slices.Sort` for ordered elements and `SortFunc` with `cmp.Compare` for the rest, both pattern defeating quicksort and not stable, and `slices.SortStableFunc` takes the same comparison function when equal elements must keep order:

```
func TestMapsToolkit(t *testing.T) {
  m := map[string]int{"b": 2, "a": 1}
  if got := slices.Sorted(maps.Keys(m)); fmt.Sprint(got) ≠ "[a b]" {
    t.Fatalf("Sorted(maps.Keys) = %v", got)
  }

  clone := maps.Clone(m)
  clone["c"] = 3
  maps.DeleteFunc(clone, func(k string, v int) bool { return v > 2 })
  if len(clone) ≠ 2 {
    t.Fatalf("DeleteFunc left %d entries", len(clone))
  }
  if len(m) ≠ 2 {
    t.Fatal("Clone isolated the original map")
  }
```

listing 80: clone isolates, keys iterates, deletefunc filters (source: go/samples/ch12/stdlib_test.go, lines 65-79)

`maps.Clone` is the shallow copy, `maps.Copy` merges, and `maps.DeleteFunc` filters in place. The `cmp` package holds two functions, `Compare` for three way ordering and `Or` for first nonzero default, the most used one liner in modern go:

```
func TestCmpOrPicksFirstNonZero(t *testing.T) {
  if got := cmp.Or("", "fallback"); got ≠ "fallback" {
    t.Fatalf(`cmp.Or("", "fallback") = %q`, got)
  }
  if got := cmp.Or("first", "second"); got ≠ "first" {
    t.Fatalf(`cmp.Or("first","second") = %q`, got)
  }
```

listing 81: cmp.Or replaces the ternary go refuses to grow (source: go/samples/ch12/stdlib_test.go, lines 82-88)

12.3. time

`time.Time` is a value with a location, `time.Duration` is an int64 of nanoseconds, and the constants `time.Second` and friends make the arithmetic readable. Parsing and formatting go through layout strings, and the two canonical ones are constants:

```
func TestTimeFormatParseRoundTrip(t *testing.T) {
  stamp := "2026-09-08T12:30:00Z"
  parsed, err := time.Parse(time.RFC3339, stamp)
  if err ≠ nil {
    t.Fatalf("Parse: %v", err)
  }
  if parsed.Format(time.RFC3339) ≠ stamp {
    t.Fatalf("round trip = %q", parsed.Format(time.RFC3339))
  }
  later := parsed.Add(90 * time.Minute)
  if later.Sub(parsed) ≠ 90*time.Minute {
    t.Fatal("Add/Sub disagree")
  }
  if parsed.Compare(later) ≠ -1 {
    t.Fatal("Compare says parsed is not before later")
  }
```

listing 82: parse, format, add, sub, compare (source: go/samples/ch12/stdlib_test.go, lines 91-106)

The reference layout is `2006-01-02 15:04:05`, monotonic counting from a zero time, and `time.Since(t)` reads the monotonic clock so duration math never jumps when the wall clock adjusts. `Timer` channels are unbuffered since go 1.23 with no compat flag remaining in 1.27, and `CutLast`, new in `strings` and `bytes`, splits around the final separator.

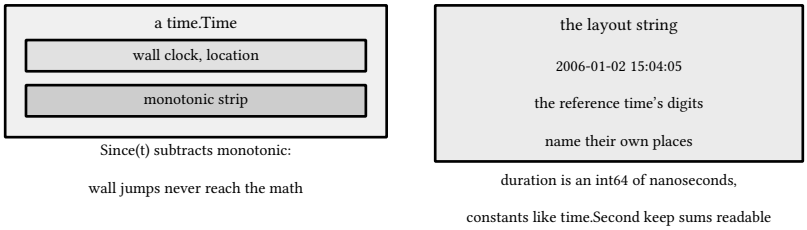


figure 66: two clocks in one value: wall for people, monotonic for math

12.4. slog

Structured logging arrived in go 1.22 and ended the logging library wars by being good enough to keep. A `Logger` wraps a `Handler`, handlers choose the format, and attributes are key value pairs:

```
func TestSlogStructuredOutput(t *testing.T) {
  var buf bytes.Buffer
  logger := slog.New(slog.NewTextHandler(&buf, &slog.HandlerOptions{Level: slog.LevelDebug}))

  logger.Info("user logged in", "id", 42, "roles", []string{"dev"})
  logger.Warn("slow query", slog.Duration("took", 250*time.Millisecond))
  with := logger.With("service", "api")
  with.Error("boom")

  // TextHandler stamps every line with time=<rfc3339>; skip that one
  // field and each rendered line compares exactly
  got := bytes.Split(bytes.TrimSuffix(buf.Bytes(), []byte("\n")), []byte("\n"))
  want := [][]byte{
    []byte(`level=INFO msg="user logged in" id=42 roles=[dev]`),
    []byte(`level=WARN msg="slow query" took=250ms`),
    []byte(`level=ERROR msg=boom service=api`),
  }
  if len(got) ≠ len(want) {
    t.Fatalf("wrote %d lines:\n%s", len(got), buf.String())
  }
  for i, w := range want {
    if j := bytes.IndexByte(got[i], ' '); j ≥ 0 {
      got[i] = got[i][j+1:]
    }
    if !bytes.Equal(got[i], w) {
      t.Errorf("line %d = %q, want %q", i, got[i], w)
    }
  }
}
```

listing 83: text handler, typed attrs, with, levels (source: go/samples/ch12/stdlib_test.go, lines 109-137)

`TextHandler` writes `key=value` lines, `JSONHandler` writes json, levels gate at debug, info, warn, error, and the test asserts the rendered lines, the `250ms` duration included. `Logger.With` returns a child carrying bound attributes, the replacement for context fields in logging libraries, and `slog.SetDefault` wires the package level `slog.Info` calls to the same handler. The handler interface is public, so one custom handler serves a whole service, and the `slog.HandlerOptions` level and source toggles cover the rest.

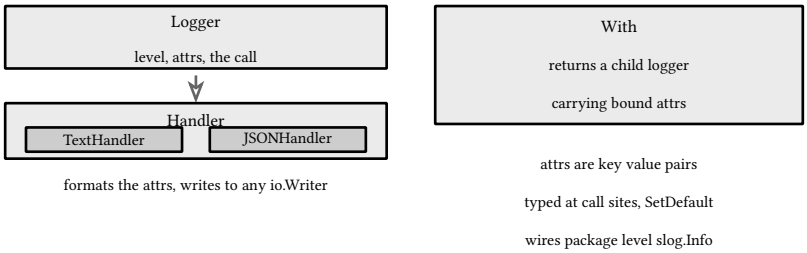


figure 67: logger wraps handler, with binds a child, attrs are the payload

sources: pkg.go.dev for fmt, slices, maps, cmp, time, log/slog, and the go 1.27 release notes for CutLast, accessed 2026-09-08. All behavior verified by `go/samples/ch12` tests, 7 of them.

13. stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io

The second tour covers what arrived newest and what serves the network. The headline is `encoding/json/v2`, stable in go 1.27 with the v1 package now running on top of it, and the tour also covers the `uuid` package, both new enough that most go code in the wild predates them.

13.1. json v2

The import is `encoding/json/v2`, the package name is still `json`, and every entry point takes variadic options:

```
func Marshal(in any, opts ...Options) ([]byte, error)
func Unmarshal(in []byte, out any, opts ...Options) error
// plus MarshalWrite, MarshalEncode, UnmarshalRead, UnmarshalDecode
```

fragment, illustrative, not executed

The defaults are stricter than v1, invalid utf-8 and duplicate object names are rejected rather than repaired, and names match case sensitively. The struct tag language grew one tag that matters:

```
type profile struct {
  Name   string   `json:"name"`
  City   string   `json:"city,omitempty"`
  Notes  string   `json:"notes,omitzero"`
  Tags   []string `json:"tags,omitzero"`
  Items  []string `json:"items,omitempty"`
  Logins int      `json:"logins"`
}
```

listing 84: one struct, three tag behaviors (source: go/samples/ch13/stdlib2_test.go, lines 20-27)

`omitzero` and `omitempty` sound interchangeable and are not, the distinction the test pins from both sides:

```go
func TestOmitZeroVersusOmitEmpty(t *testing.T) {
  // omitzero keys off Go zero values: nil slices vanish, but a
  // non-nil empty slice is not zero and stays, encoded as []
  b, err := json.Marshal(profile{Name: "x", Tags: []string{}})
  if err ≠ nil {
    t.Fatal(err)
  }
  if string(b) ≠ `{"name":"x","tags":[],"logins":0}` {
    t.Fatalf("omitzero non-nil empty slice dropped: %s", b)
  }

  // omitempty keys off the JSON encoding: anything that encodes as
  // empty, null, "", {}, [], vanishes regardless of nilness
  b, err = json.Marshal(profile{Name: "x", Items: []string{}})
  if err ≠ nil {
    t.Fatal(err)
  }
  if string(b) ≠ `{"name":"x","logins":0}` {
    t.Fatalf("omitempty non-nil empty slice kept: %s", b)
  }
}
```

listing 85: go zero values versus empty json encodings (source: go/samples/ch13/stdlib2_test.go, lines 49-69)

`omitzero` consults the Go type system, a nil slice is zero and vanishes, a non-nil empty slice is not zero and stays as `[]`. `omitempty` consults the JSON encoding, anything that would encode as null, empty string, {}, or `[]` vanishes whatever its Go value. The v2 documentation's advice: prefer `omitzero` when either would do, it is the one whose semantics a reader can reason about locally.

	omitzero	omitempty
nil slice	omits	omits, null
[]int{} non nil	keeps: []	omits, encodes empty
"" string	omits	omits
empty map	keeps: {}	omits

one asks the go zero value, the other the json encoding

prefer omitzero: its meaning reads locally

v2 defaults: invalid utf-8 and duplicates rejected

figure 68: omitzero asks the go type, omitempty asks the json encoding

Options compose at call sites, and the useful three:

```
func TestDeterministicSortsMapKeys(t *testing.T) {
  b, err := json.Marshal(map[string]int{"z": 1, "a": 2}, json.Deterministic(true))
  if err ≠ nil {
    t.Fatal(err)
  }
  if string(b) ≠ `{"a":2,"z":1}` {
    t.Fatalf("Deterministic = %s", b)
  }
}

func TestRejectUnknownMembers(t *testing.T) {
  var p profile
  err := json.Unmarshal([]byte(`{"name":"x","ghost":1}`), &p, json.RejectUnknownMembers(true))
  if err == nil {
    t.Fatal("unknown member accepted")
  }
  if err := json.Unmarshal([]byte(`{"name":"x"}`), &p, json.RejectUnknownMembers(true)); err ≠ nil {
    t.Fatalf("known members rejected: %v", err)
  }
}

func TestStringifyNumbers(t *testing.T) {
  b, err := json.Marshal(profile{Name: "x", Logins: 7}, json.StringifyNumbers(true))
  if err ≠ nil {
    t.Fatal(err)
  }
  if !strings.Contains(string(b), `"logins":"7"`) {
    t.Fatalf("StringifyNumbers = %s", b)
  }
}
```

listing 86: deterministic maps, strict input, quoted numbers (source: go/samples/ch13/stdlib2_test.go, lines 71-100)

`Deterministic(true)` sorts map keys, `RejectUnknownMembers(true)` turns typos into errors on the way in, `StringifyNumbers(true)` quotes numbers both directions for apis that demand string ids. On strictness, unknown members are ignored by default like v1, invalid utf-8 is not repaired, and the escapes are minimal, all reversible through jsontext options when a legacy peer needs the old behavior.

13.2. jsontext

Under v2 sits a token layer, `encoding/json/jsontext`, for code that must see json as syntax rather than as Go values, streaming protocols, validators, transformers:

```
func TestJsontextIndentAndPeek(t *testing.T) {
  b, _ := json.Marshal(map[string]int{"a": 1})
  v := jsontext.Value(b)
  if err := v.Indent(jsontext.WithIndent("  ")); err ≠ nil {
    t.Fatal(err)
  }
  if v.String() ≠ "{\n  \"a\": 1\n}" {
    t.Fatalf("Indent = %q", v.String())
  }

  dec := jsontext.NewDecoder(strings.NewReader(`[1,2]`))
  if kind := dec.PeekKind(); kind ≠ '[' {
    t.Fatalf("PeekKind = %v, want '['", kind)
  }
}
```

listing 87: indent a value, peek the next kind (source: go/samples/ch13/stdlib2_test.go, lines 102-116)

`Value` holds pre-encoded json and reformats in place, `Indent` taking options like `WithIndent`, `Canonicalize` applies RFC 8785 canonical form, and the `Decoder` `PeekKind` looks ahead without consuming, the first step of every hand rolled dispatch on mixed streams. `MarshalEncode` and `UnmarshalDecode` bridge the layers, writing go values into an encoder mid stream.

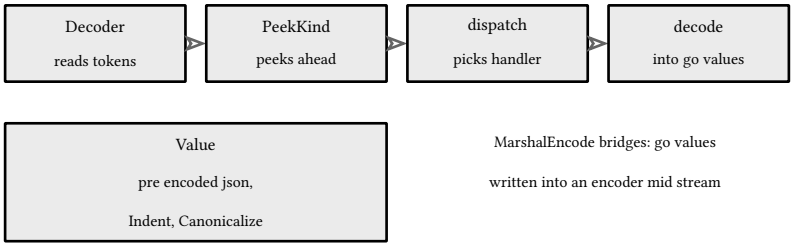


figure 69: the token layer: peek without consuming, reformat in place, bridge mid stream

13.3. uuid

Go 1.27 also standardized `uuid`, RFC 9562, `[16]byte` with text marshaling wired in:

```go
func TestUuidBasics(t *testing.T) {
  a := uuid.New()
  if a == uuid.Nil() {
    t.Fatal("New produced the nil uuid")
  }
  if v := uuid.NewV7(); v.Compare(uuid.NewV7()) ≥ 0 {
    t.Fatal("two sequential NewV7 values are not ordered")
  }

  parsed, err := uuid.Parse(a.String())
  if err ≠ nil {
    t.Fatal(err)
  }
  if parsed ≠ a {
    t.Fatalf("Parse(String()) round trip changed %v", a)
  }
  if _, err := uuid.Parse("not a uuid"); err == nil {
    t.Fatal("Parse accepted garbage")
  }
}

func TestMustParsePanics(t *testing.T) {
  defer func() {
    if recover() == nil {
      t.Fatal("MustParse did not panic on garbage")
    }
  }()
  uuid.MustParse("nope")
}
```

listing 88: new, newv7 ordering, parse round trip, must panic (source: go/samples/ch13/stdlib2_test.go, lines 118-146)

`New` is currently v4, 122 random bits, and `NewV7` is the one to reach for in databases, 48 bits of unix millisecond timestamp plus random, so ids sort by creation time. `Parse` accepts hyphenated, braced, urn, and bare hex spellings, and the type is comparable and json serializable through its text methods.

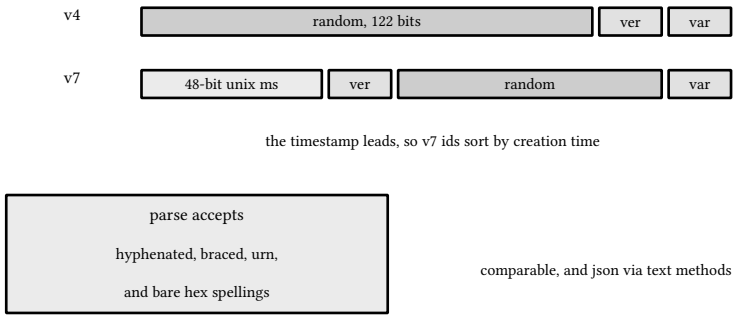


figure 70: sixteen bytes, two layouts: v4 all random, v7 timestamp first

13.4. net/http

The server is still the standard library's crown: a handler is a function, and `httptest.NewServer` gives it a real local listener for tests. Go 1.27's `httptest.NewTestServer` goes further, an in-memory fake network with no sockets at all, designed to pair with `synctest` bubbles for deterministic concurrency tests, and production servers gained `MaxHeaderValueCount` and RFC 9218 http/2 priority honoring:

```
func TestHttpRoundTrip(t *testing.T) {
  srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, `{"status":"ok"}`)
  }))
  defer srv.Close()

  resp, err := http.Get(srv.URL)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer resp.Body.Close()
  if resp.StatusCode ≠ http.StatusOK {
    t.Fatalf("status = %d", resp.StatusCode)
  }
  body, err := io.ReadAll(resp.Body)
  if err ≠ nil {
    t.Fatal(err)
  }
  var payload struct {
    Status string `json:"status"`
  }
  if err := json.Unmarshal(body, &payload); err ≠ nil {
    t.Fatal(err)
  }
  if payload.Status ≠ "ok" {
    t.Fatalf("payload = %+v", payload)
  }
}
```

listing 89: serve, get, read, decode (source: go/samples/ch13/stdlib2_test.go, lines 148-175)

The client needs no configuration for the ordinary case, `http.Get`, read the body, close it, and since 1.27 the http/1 client drains small unread bodies on close so keep alive connections survive.

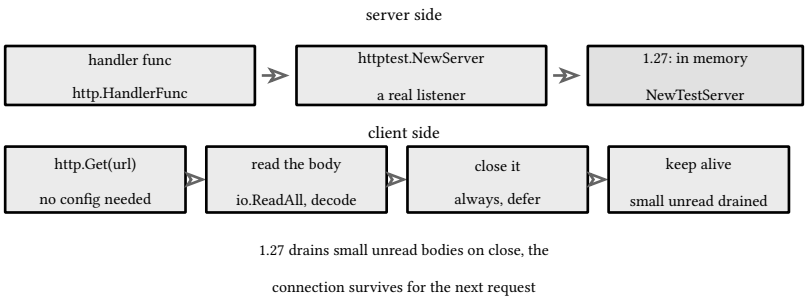


figure 71: the round trip: handler is a function, client is get, read, close

13.5. crypto and simd

Hashes are `Sum256` and friends over byte slices, and the known vector test pins the digest of `"hello"` to its published value. The post quantum story is the 1.27 headline: `crypto/mldsa` implements ML-DSA signatures from FIPS 204, TLS 1.3 negotiates the ML-DSA signature schemes, and ML-KEM hybrid key exchange ships on by default. `crypto/rand` remains the randomness source, never `math/rand`, which exists for simulation.

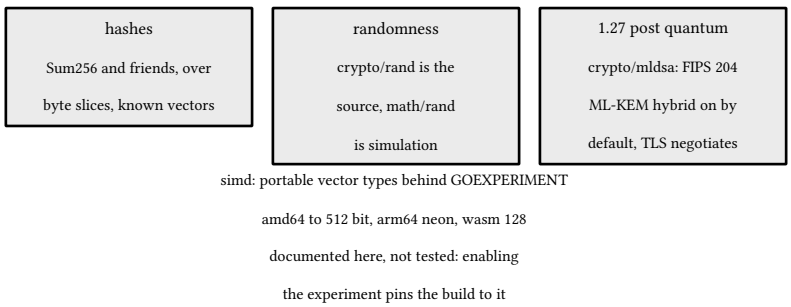


figure 72: the crypto and simd corner of the tour

NOTE: simd is experimental and off by default

The `simd` and `simd/archsimd` packages give portable, vector width agnostic types like `simd.Int8s` and `simd.Float32s`, amd64 up to 512 bit, arm64 neon, and wasm 128 bit. In go 1.27 they sit behind `GOEXPERIMENT=simd` with an unstable api, so this book documents them without testing them: enabling the experiment in a module pins the build to toolchains honoring it.

13.6. os and io

File io is `os.Open`, `os.Create`, `os.WriteFile`, with `t.TempDir` giving tests a directory the framework cleans:

```
func TestOsAndIoRoundTrip(t *testing.T) {
  dir := t.TempDir()
  path := filepath.Join(dir, "note.txt")
  if err := os.WriteFile(path, []byte("line one\nline two\n"), 0o644); err ≠ nil {
    t.Fatal(err)
  }
  f, err := os.Open(path)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer f.Close()

  var sb strings.Builder
  n, err := io.Copy(&sb, f)
  if err ≠ nil || n ≠ 18 {
    t.Fatalf("io.Copy = %d %v, want 18 nil", n, err)
  }
  if !strings.Contains(sb.String(), "line two") {
    t.Fatalf("copied %q", sb.String())
  }
}
```

listing 90: write, copy into a builder, assert the bytes (source: go/samples/ch13/stdlib2_test.go, lines 186-206)

The `io` package is the core three interfaces, `Reader`, `Writer`, `ReadWriter`, plus the combinators over them, `Copy`, `ReadAll`, `MultiReader`, `Pipe`, and `strings.Builder` as the in memory writer. Everything streams, `io.Copy` moves bytes through a 32 kb buffer without loading either end, and the interfaces compose with every type in this chapter, hash writers, http bodies, file handles, json encoders.

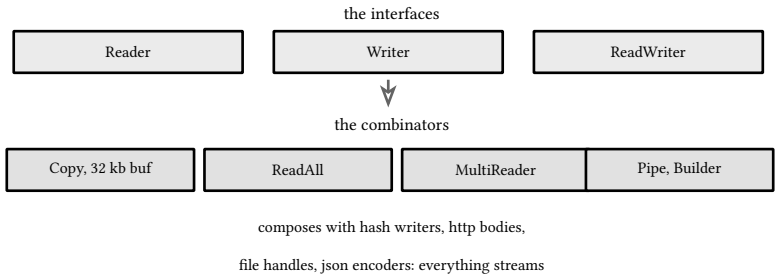


figure 73: three interfaces, combinators over them, everything composes

sources: pkg.go.dev for encoding/json/v2, encoding/json/jsontext, uuid, net/http, crypto, os, io, accessed 2026-09-08, and the go 1.27 release notes for the simd, mldsa, and httptest changes. Verified by `go/samples/ch13` tests, 11 of them.

14. testing

Testing is a language feature in go the same way formatting is: one command, one file convention, no framework choice to make. This entire book's go verification is `go test ./...` plus `go vet`, and this chapter's sample folder is a working demonstration of every test kind the toolchain runs.

14.1. the test function

A test is a function named `TestXxx` taking `*testing.T` in a `_test.go` file. Table driven is the house style, a slice of cases looped once, and subtests give each case a name:

```
func TestSubtestsRunWithParallel(t *testing.T) {
  words := []string{"alpha", "beta", "gamma"}
  for i, w := range words {
    // each subtest owns its assertion: parallel subtests run after
    // the parent body returns, so the parent cannot check their
    // results from inside its own loop
    t.Run(w, func(t *testing.T) {
      t.Parallel()
      if words[i] ≠ w {
        t.Errorf("loop capture broke: %q vs %q", words[i], w)
      }
    })
  }
}
```

listing 91: subtests with parallel bodies (source: go/samples/ch14/testing_test.go, lines 22-35)

The subtlety the test documents: `t.Parallel` pauses a subtest until the parent's function body returns, so the parent cannot check subtest results after its loop, each subtest asserts its own outcome. Parallel tests share process cores through the same scheduler as everything else, `go test -parallel` caps it.

Failures are `t.Error` to mark and continue, `t.Fatal` to stop the test, both taking format strings, and `t.Helper` marks assertion utilities so line numbers point at the caller.

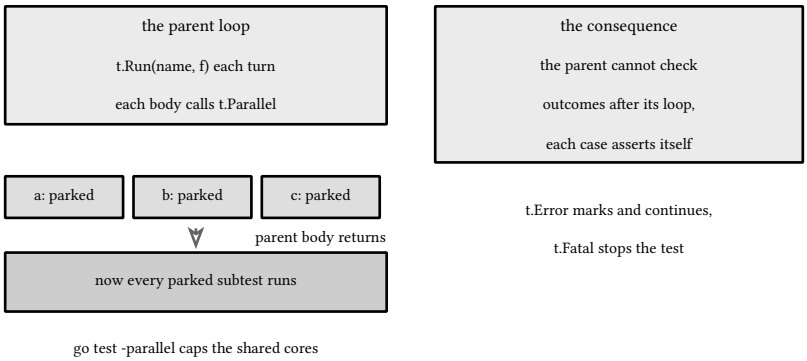


figure 74: t.Parallel parks each subtest until the parent body returns

14.2. examples are tests

A function named `ExampleXxx` with an `// Output:` comment is a test that compares stdout:

```
func ExampleGreet() {
  fmt.Println(Greet("world"))
  // Output:
  // hello, world
}
```

listing 92: the example doubles as documentation (source: go/samples/ch14/testing_test.go, lines 16-20)

Examples fail when output drifts, which keeps documentation executable, and godoc renders them beside the function they demonstrate. `go doc -ex` lists them since 1.27.

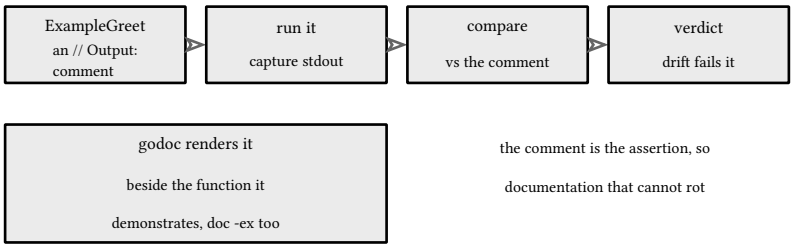


figure 75: the example with an output comment is a stdout-comparing test

14.3. fuzzing

`FuzzXxx` functions declare a fuzz target with seed corpus entries, `f.Add`:

```
func FuzzReverse(f *testing.F) {
  f.Add("hello")
  f.Add("héllo")
  f.Add("")
  f.Fuzz(func(t *testing.T, s string) {
    r := Reverse(s)
    if len(r) ≠ len(s) {
      t.Fatalf("Reverse changed length of %q", s)
    }
    if Reverse(r) ≠ s {
      t.Fatalf("double reverse changed %q", s)
    }
  })
}
```

listing 93: seed corpus plus the property (source: go/samples/ch14/testing_test.go, lines 68-81)

Plain `go test` runs the seeds, so the corpus is an ordinary table test. `go test -fuzz=FuzzReverse` then generates inputs hunting for the property to break, length preservation and double reverse identity here, and failing discoveries land in the module's `testdata/fuzz` corpus, committed so they become permanent regression cases.

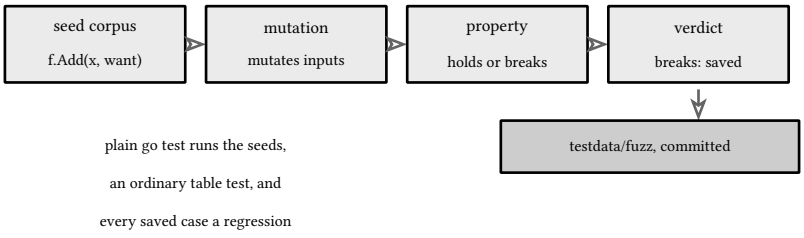


figure 76: seeds to mutation to property, failures become permanent cases

14.4. benchmarks

`BenchmarkXxx` with `*testing.B` measures, `b.Loop()` (go 1.24 onward) runs the body until the measurement is stable and resets allocs per iteration, replacing the manual `for i := 0; i < b.N; i++`:

```
func BenchmarkReverse(b *testing.B) {
  for b.Loop() {
    Reverse("the quick brown fox")
  }
}
```

listing 94: b.Loop, the modern harness (source: go/samples/ch14/testing_test.go, lines 83-87)

`go test -bench=. -benchmem` reports throughput and allocations per operation, and `pprof` profiles from chapter 9 attach with `-cpuprofile` and `-memprofile` for the follow up question.

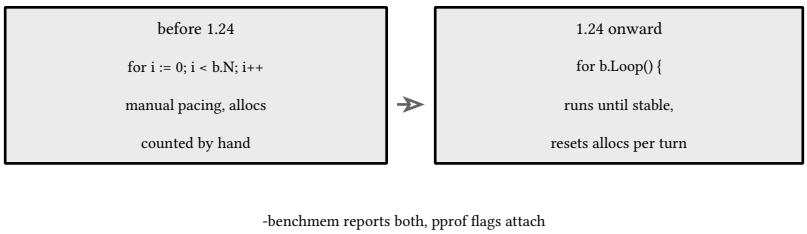


figure 77: b.Loop, the 1.24 harness that replaced the b.N loop

14.5. testing/synctest

Concurrency tests that sleep are flaky or slow. Go 1.25 stabilized `testing/synctest`, a bubble of fake time where sleeps complete the moment every goroutine in the bubble is durably blocked:

```
func TestSynctestBubbleFastForwardsTime(t *testing.T) {
  synctest.Test(t, func(t *testing.T) {
    start := time.Now()
    time.Sleep(3 * time.Hour)
    // the bubble runs in microseconds of real time, but the
    // clock inside advanced the full three hours
    if elapsed := time.Since(start); elapsed < 3*time.Hour {
      t.Fatalf("fake clock advanced only %v", elapsed)
    }
  })
}
```

listing 95: three hours pass in microseconds of real time (source: go/samples/ch14/testing_test.go, lines 37-47)

`synctest.Test` opens the bubble, `synctest.Wait` blocks until others park, and go 1.27 adds `synctest.Sleep`, the combination as one call:

```go
func TestSynctestSleepCombinesSleepAndWait(t *testing.T) {
  synctest.Test(t, func(t *testing.T) {
    start := time.Now()
    // new in go 1.27: sleep and durably wait in one call
    synctest.Sleep(time.Hour)
    if elapsed := time.Since(start); elapsed < time.Hour {
      t.Fatalf("synctest.Sleep advanced only %v", elapsed)
    }
  })
}
```

listing 96: sleep plus wait, new in 1.27 (source: go/samples/ch14/testing_test.go, lines 49-58)

Inside a bubble, timers, channels, and goroutines interact on deterministic fake time, which is how the two tests above pass four hours of sleeping, three plus one, in microseconds of real time. Pair it with `httptest.NewTestServer`, 1.27's socket free server, and a full http pipeline tests deterministically in memory.

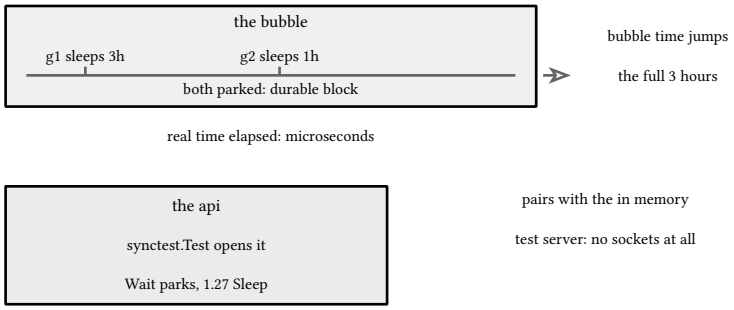


figure 78: the synctest bubble: fake time advances only when everyone is parked

NOTE: the whole verification chain

`go test ./...` runs tests, examples, and seed corpora. `go vet ./...` runs the analyzers. `gofmt -l .` checks formatting. `go test -race ./...` finds data races at run time. This repo's `make verify-go` runs the first two for every go module it contains, and the race detector is one flag away, worth a local run on any package that touches chapter 6.

sources: pkg.go.dev/testing and testing/synctest, go.dev/blog on the b.Loop harness and fuzzing, accessed 2026-09-08. Verified by `go/samples/ch14`, 5 tests, 1 example, 1 fuzz target with 3 seeds, 1 benchmark, zero sleeps of real time.

15. idioms

Go's designers say the language is done and the idioms are not. This chapter is the set of conventions working go code converges on, each one small, all of them load bearing in the capstone that follows.

15.1. accept interfaces, return structs

Functions take the narrowest interface they need and constructors return concrete types. The caller converts down, the callee never converts up, and the interface lives beside its consumer, not beside implementations:

```
// types that implement it: interfaces belong to consumers.
type Namer interface {
  Name() string
}

type user struct{ name string }

func (u user) Name() string { return u.name }

func GreetFrom(n Namer) string { return "hello, " + n.Name() }
```

listing 97: the interface belongs to greet, not to user (source: go/samples/ch15/idioms.go, lines 35-44)

`Namer` exists because `GreetFrom` needs one name. If another type grows a `Name` method it satisfies the interface without asking, and the test's `user` never imports or mentions the abstraction. Package `io` is this pattern at standard library scale, `io.Reader` defined where it is consumed, implemented everywhere.

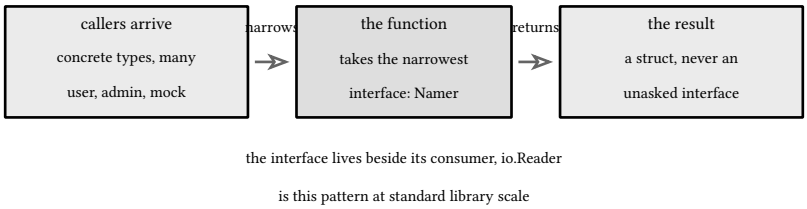


figure 79: the conversion directions: narrowest in, concrete out

15.2. functional options

Constructors with many optional parameters become unreadable, and go has neither overloading nor default arguments. The idiomatic answer is variadic option functions over a struct with usable defaults:

```
type Server struct {
  timeout time.Duration
  retries int
}

type Option func(*Server)

func WithTimeout(d time.Duration) Option { return func(s *Server) { s.timeout = d } }

func WithRetries(n int) Option { return func(s *Server) { s.retries = n } }

func NewServer(opts ...Option) *Server {
  s := &Server{timeout: 5 * time.Second, retries: 1}
  for _, opt := range opts {
    opt(s)
  }
  return s
}
```

listing 98: defaults in the constructor, overrides in options (source: go/samples/ch15/idioms.go, lines 15-33)

`NewServer()` gives the defaults, `NewServer(WithTimeout(d))` overrides one, and adding a knob means adding an option without touching any call site. The heavy alternative, a config struct, suits knobs that callers construct dynamically.

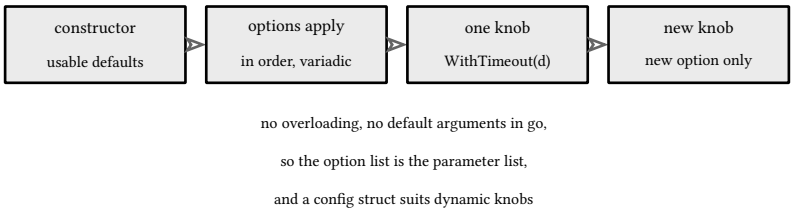


figure 80: defaults in the constructor, options override in order

15.3. zero values that work

Every type should aim for a usable zero value, because go declares zero values everywhere: struct fields, map values, new variables. `sync.Mutex`, `bytes.Buffer`, `strings.Builder`, and the capstone's accumulator below all work without constructors:

```
// Sum accumulates from zero: the zero value needs no constructor,
// it is already a valid empty sum.
func Sum(ns ...int) int {
  total := 0
  for _, n := range ns {
    total += n
  }
  return total
}
```

listing 99: sum from zero, no init required (source: go/samples/ch15/idioms.go, lines 46-54)

The test drives a `strings.Builder` from `var b strings.Builder` the same way. When a zero value cannot be valid, a constructor returns the type in a good state and unexported fields keep it there, which is the whole encapsulation story of go: export the type, hide the fields, offer functions.

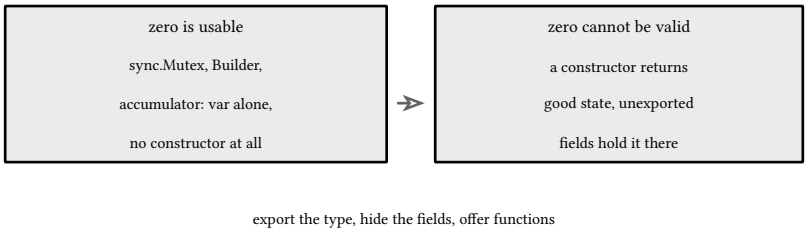


figure 81: aim at a usable zero, and when zero cannot work, construct

15.4. errors are values, no hierarchies

Go programs do not build exception taxonomies. A function returns one error carrying the situation in its message, wraps with `%w` when context accumulates, and exports sentinels only for conditions callers must distinguish:

```go
// Load wraps an error with context instead of defining a hierarchy:
// one error type, the message carries the situation.
func Load(path string) ([]byte, error) {
  b, err := os.ReadFile(path)
  if err ≠ nil {
    return nil, fmt.Errorf("load %s: %w", path, err)
  }
  return b, nil
}

// ErrUnavailable is the one sentinel the package exports.
var ErrUnavailable = errors.New("unavailable")
```

listing 100: wrap with context, one exported sentinel (source: go/samples/ch15/idioms.go, lines 56-67)

`Load` adds the filename, `ErrUnavailable` exists because some caller branches on it, and the package stops there. The test asserts the message carries the filename, which is the property that keeps errors debuggable in logs.

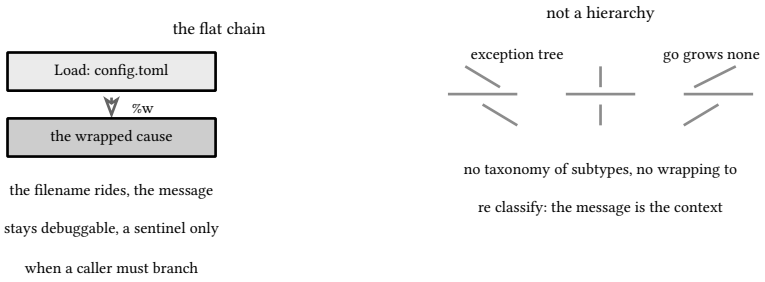


figure 82: flat chain, not a taxonomy: one error, wrapped context, rare sentinels

NOTE: the smaller vocabulary

Naming: mixedCase, no underscores, initialisms capitalized, `URL`, `ID`. Errors: lowercase messages, no punctuation. One file per concern, package names singular and short, `strconv` not `stringutilities`. Tests beside the code, table driven, no framework. `gofmt` is the reviewer of style and the argument is over.

sources: go.dev/doc/effective_go and the go proverbs talk, plus pkg.go.dev/go for the standard library expressions of these idioms, accessed 2026-09-08. Verified by `go/samples/ch15` tests.

16. htmx 4: html over the wire

The last two chapters leave a gap: go serves html beautifully and tests it over httptest, but the browser side of interactivity is still hand written javascript. htmx closes that gap from the html side. Ordinary elements gain attributes that issue ajax requests, the server answers with html fragments, and the response lands in the page without a line of client render code. This chapter builds one fragment server, `go/samples/ch16`, 12 tests, every path driven through httptest, zero browser. htmx 4.0.0 is the version taught, vendored into the package because npm's `latest` tag still points at 2.0.10 while the 4.x documentation lives at four.htmx.org, and the vendored file is pinned by a digest the test recomputes.

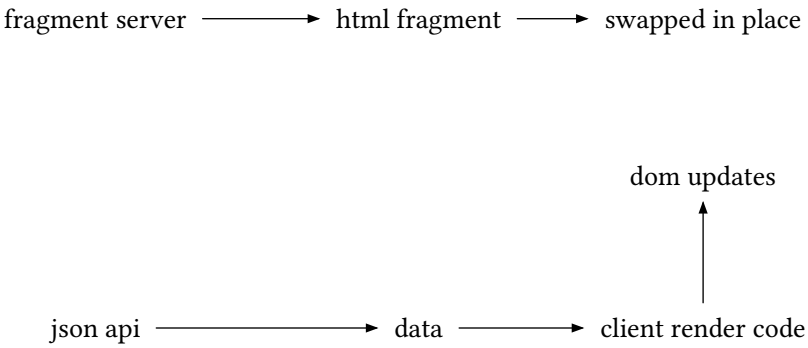


figure 83: two ways to update a page

16.1. fragments, not json

A json api and a client render layer move two jobs into the browser: fetching data and turning data into dom. htmx keeps both on the server. The element says where a request goes and where its answer lands, the handler answers html, and the same handler often serves both faces of one url, the full document for a normal navigation and the bare fragment when htmx asks. The `HX-Request` header is how the server tells the two apart, and `Vary` keeps a cache from serving one face for the other:

```
// page answers both faces of one url: the whole document for a
// normal navigation, the bare list when htmx asks. Vary: HX-Request
// keeps any cache from serving one face for the other.
func page(n *Notes) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    w.Header().Add("Vary", "HX-Request")
    if r.Header.Get("HX-Request") == "true" {
      writeHTML(w, http.StatusOK, notesList(n.all()))
      return
    }
    writeHTML(w, http.StatusOK, indexHTML(n))
  }
}
```

listing 101: one route, two faces, the vary header keeping them apart (source: go/samples/ch16/htmx.go, lines 89-101)

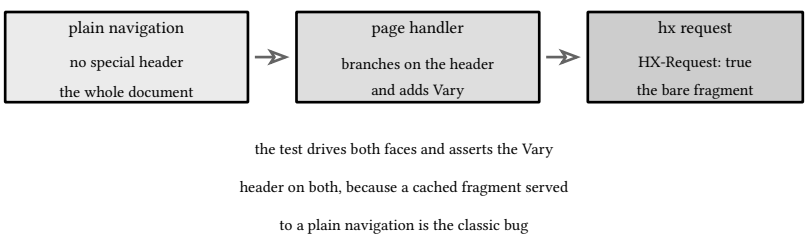


figure 84: the same bytes asked two ways, distinguished by one request header

The fragment face is ordinary html, escaped exactly like a full page, so the same rendering discipline covers both.

16.2. what changed in 4.0

Version 4 keeps the idea and rewires the middle. Five changes matter to a go author, each from the migration notes at four.htmx.org, accessed 2026-09-21. The core now rides the fetch api, and the new `hx-config` attribute maps fetch options per element, `timeout`, `credentials`, `cache`, `redirect`, `integrity`, with one deliberate omission: `mode` is always reset to `htmx.config.mode`, `same-origin` by default, so injected markup cannot widen a request's scope. Attribute inheritance flipped from implicit to explicit, a child now opts in with `hx-target:inherited` and combines with `:append`, restorable to 2.x behavior through `htmx.config.implicitInheritance`. Error responses are swapped by default, where 2.x dropped 400 and 500 bodies, revertible with `htmx.config.noSwap` or per element `hx-status`. Event names were standardized and rationalized, with the `htmx-2-compat` extension bridging old names. And history no longer snapshots the dom into session storage, it re-issues a request to the server, which is why fragment routes should stay answerable forever.

```
<div hx-config='"timeout": 4000, "redirect": "error"'
     hx-get="/fragment/notes">list</div>
<!— mode is never honored here: same-origin is pinned —>
```

fragment, illustrative, not executed

what	2.x	4.0
transport	xmlhttprequest	fetch, hx-config options
inheritance	implicit	explicit, opt in per attribute
400 and 500	not swapped	swapped by default
event names	accumulated	standardized, compat ext
history	dom snapshot cache	a fresh server request

also new: built in innerMorph and outerMorph swaps

all five rows cited to four.htmx.org migration

figure 85: 2.x habits against 4.0 defaults, five rows from the migration notes

16.3. the core attributes

Five attributes carry the whole library. `hx-get` and `hx-post` issue the request, `hx-target` picks the css selector the answer lands in, `hx-swap` picks how it lands, and `hx-trigger` picks what fires it. The demo's mux routes them with go 1.22 method patterns, so a wrong verb is a 405 from the mux, never from handler code:

```
// NewMux routes the demo with method patterns: a wrong verb is a 405
// from the mux itself, never handler code.
func NewMux(n *Notes) http.Handler {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /{$}", page(n))
  mux.HandleFunc("GET /fragment/notes", func(w http.ResponseWriter, r *http.Request) {
    writeHTML(w, http.StatusOK, notesList(n.search(r.URL.Query().Get("q"))))
  })
  mux.HandleFunc("GET /fragment/search", func(w http.ResponseWriter, r *http.Request) {
    writeHTML(w, http.StatusOK, strings.Join(rowList(n.search(r.URL.Query().Get("q"))), ""))
  })
  mux.HandleFunc("GET /fragment/time", clock)
  mux.HandleFunc("GET /fragment/tip", func(w http.ResponseWriter, _ *http.Request) {
    writeHTML(w, http.StatusOK, "<p>htmx 4 swaps error responses by default</p>\n")
  })
  mux.HandleFunc("POST /notes", addNote(n))
  mux.HandleFunc("POST /notes/clear", clearNotes(n))
  mux.HandleFunc("POST /logout", logout)
  mux.HandleFunc("GET /events", events)
  mux.Handle("GET /static/", http.FileServer(http.FS(staticFS)))
  return mux
}
```

listing 102: the demo's whole route table, method patterns on every entry (source: go/samples/ch16/htmx.go, lines 66-87)

The served page carries each attribute on a real element, and one test walks the whole catalog against the served bytes:

```
<input type="search" name="q"
  hx-get="/fragment/search" hx-trigger="keyup changed delay:500ms"
  hx-target="#results" hx-swap="innerHTML">
```

fragment, illustrative, not executed

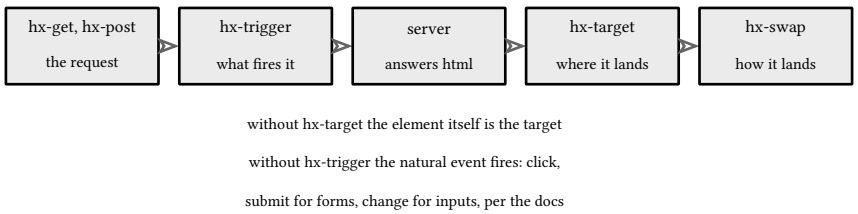


figure 86: five attributes, five decisions, one request cycle

16.4. triggers and their modifiers

`hx-trigger` replaces the natural event and its modifiers carry real logic. `every 2s` polls on an interval. `changed` fires only when the value differs. `delay:500ms` waits, resetting the countdown when the event refires, the debounce. `throttle:1s` fires immediately then holds, queueing the last event for the end, the rate limit. `once` fires a single time. `from:` listens on another element, `load` fires when the element first appears, and `revealed` when it scrolls into view. The demo needs three of them for real, and the page markup shows the wiring:

```
b.WriteString("<input type=\"search\" name=\"q\" placeholder=\"search\"\n" +
  "  hx-get=\"/fragment/search\" hx-trigger=\"keyup changed delay:500ms\"\n" +
  "  hx-target=\"#results\" hx-swap=\"innerHTML\">\n")
fmt.Fprintf(&b, "<ul id=\"results\"><li>%s</li></ul>\n", esc("type to filter"))
fmt.Fprintf(&b, "<ul id=\"notes\" hx-get=\"/fragment/notes\" hx-trigger=\"every 2s\"\n"+
  "  hx-swap=\"outerHTML\">%s</ul>\n", strings.Join(rowList(n.all()), ""))
fmt.Fprintf(&b, "<p><span id=\"count\">%d notes</span>\n", n.count())
b.WriteString("<form hx-post=\"/notes\" hx-target=\"#notes\" hx-swap=\"outerHTML\">\n" +
  "  <input type=\"text\" name=\"body\" placeholder=\"new note\" autocomplete=\"off\">\n" +
  "  <button>add</button>\n</form>\n")
b.WriteString("<button hx-post=\"/notes/clear\" hx-swap=\"none\">clear</button>\n")
b.WriteString("<button hx-post=\"/logout\" hx-swap=\"none\">logout</button>\n")
b.WriteString("<span id=\"clock\" hx-get=\"/fragment/time\" hx-trigger=\"every 5s\"" +
  " hx-swap=\"innerHTML\">clock</span>\n")
b.WriteString("<div id=\"tip\" hx-get=\"/fragment/tip\" hx-trigger=\"load once\">loading</div>\n")
b.WriteString("<div hx-ext=\"sse\" sse-connect=\"/events\">" +
  "<span sse-swap=\"tick\">sse: 0</span></div>\n")
b.WriteString("</body>\n</html>\n")
```

listing 103: the served body: debounce search, two pollers, a one shot load (source: go/samples/ch16/htmx.go, lines 213-230)

```
<div hx-get="/fragment/tip" hx-trigger="revealed once">tip</div>
<body> <input hx-get="/fragment/search"
  hx-trigger="keyup changed throttle:1s from:body"> </body>
```

fragment, illustrative, not executed

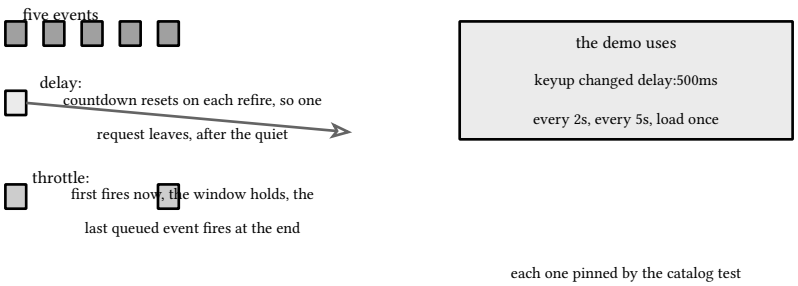


figure 87: delay versus throttle under a burst of five events

16.5. swaps and out of band

`hx-swap` defaults to `innerHTML`. The demo also uses `outerHTML`, where the fragment carries its own id and replaces itself whole, keeping the id stable across polls, and `none`, where the element fires a request and ignores the body. One response can update more than one place: elements marked `hx-swap-oob="true"` are pulled out of the response and swapped into their own id matches instead of the target, so the note post returns the list and the counter in the same bytes:

```
// oobCount is the out of band payload: an element htmx swaps into its
// id match instead of the target, so one response refreshes the list
// and the count beside it.
func oobCount(count int) string {
  return fmt.Sprintf("<span id=\"count\" hx-swap-oob=\"true\">%d notes</span>\n", count)
}
```

listing 104: the oob counter, one response, two destinations (source: go/samples/ch16/htmx.go, lines 165-170)

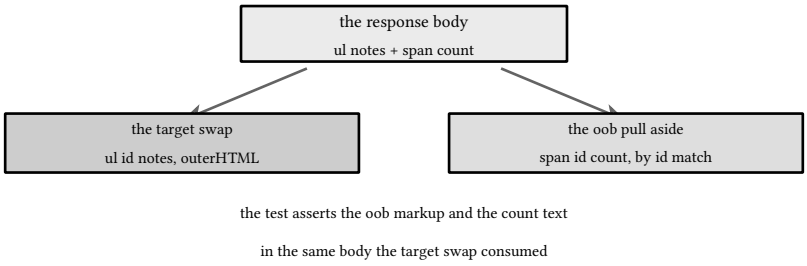


figure 88: the response body split at the target and at the id match

16.6. the response protocol

Headers are the server's channel into the client loop. `HX-Request` rides every ajax request and is the branching input. `HX-Trigger` fires a client side event from the server, the honest way to tell listeners a mutation happened without making them parse html. `HX-Redirect` hands the client a location to load, `HX-Refresh` reloads, `HX-Push-Url` and `HX-Replace-Url` edit the address bar, and `HX-Retarget`, `HX-Reswap`, `HX-Reselect` rewrite the swap contract on the way out. Status codes participate too: 204 and 304 sit in the default `noSwap` list, so a no-content answer costs nothing, which suits actions whose visible effect arrives on the next poll:

```
// clearNotes answers 204: htmx ignores the body and swaps nothing,
// the button carries hx-swap="none", and HX-Trigger tells any
// listener to refresh what it owns.
func clearNotes(n *Notes) http.HandlerFunc {
  return func(w http.ResponseWriter, _ *http.Request) {
    n.clear()
    w.Header().Set("HX-Trigger", "notesCleared")
    w.WriteHeader(http.StatusNoContent)
  }
}

// logout answers 204 with HX-Redirect: htmx sets window.location to
// the header value, a redirect the client performs without a round
// trip of its own.
func logout(w http.ResponseWriter, _ *http.Request) {
  w.Header().Set("HX-Redirect", "/")
  w.WriteHeader(http.StatusNoContent)
}
```

listing 105: 204 answers: swap nothing, say why through the trigger header (source: go/samples/ch16/htmx.go, lines 122-139)

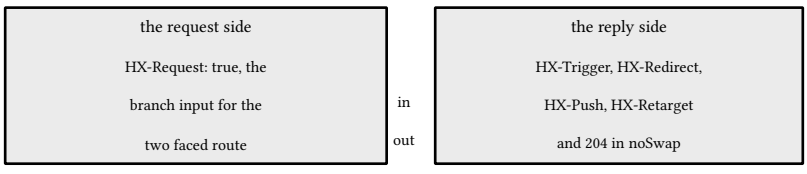


figure 89: the header vocabulary as one channel per intent

16.7. polling and server sent events

`hx-trigger="every 2s"` is the two attribute dashboard: one element, one interval, one fragment route. Its cost is arithmetic, open tabs times frequency, so the fragment must be cheap and uncached, which the demo's clock handler states outright with `Cache-Control: no-store`. When the server owns the timing instead, server sent events flip the direction: one response held open

on the text/event-stream wire, each event an event line, a data line, a blank line, and a flush. The browser half rides htmx's bundled sse extension, hx-ext="sse" with sse-connect and sse-swap, and the go half is the whole of this section:

```
// clock serves the polled time fragment. Polling answers change with
// requests, so the fragment must never be cached.
func clock(w http.ResponseWriter, _ *http.Request) {
  w.Header().Set("Cache-Control", "no-store")
  writeHTML(w, http.StatusOK,
    "<span id=\"clock\">"+time.Now().UTC().Format("15:04:05")+" utc</span>\n")
}

// events is the server half of server sent events: one response on
// the text/event-stream wire, each event an event line, a data line,
// a blank line, and a flush so the bytes leave immediately.
func events(w http.ResponseWriter, _ *http.Request) {
  fl, ok := w.(http.Flusher)
  if !ok {
    http.Error(w, "streaming unsupported", http.StatusInternalServerError)
    return
  }
  w.Header().Set("Content-Type", "text/event-stream")
  for i := 1; i <= 3; i++ {
    fmt.Fprintf(w, "event: tick\ndata: %d\n\n", i)
    fl.Flush()
  }
}
```

listing 106: the no-store clock and the sse stream, flush between events (source: go/samples/ch16/htmx.go, lines 141-163)

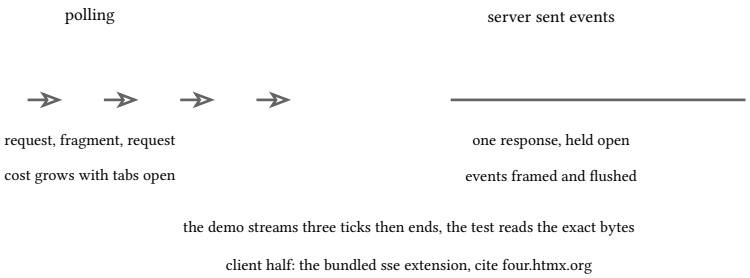


figure 90: polling asks on a timer, sse holds one response open

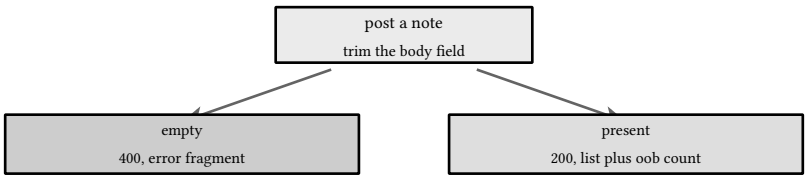
16.8. validation and errors

Validation lives where the data lands, on the server, and in 4.0 the error path is a fragment path: a 400 whose body is the markup that explains the problem, swapped into the form's target exactly like a success, where 2.x dropped the body and needed configuration to do better. The demo's empty note returns one:

```
// addNote validates the form. An empty body is a 400 whose body is an
// error fragment, which htmx 4 swaps by default and 2.x did not. A
// valid note returns the whole updated list plus an out of band
// counter, and announces itself through HX-Trigger so client
// listeners can react without parsing the body.
func addNote(n *Notes) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    body := strings.TrimSpace(r.PostFormValue("body"))
    if body == "" {
      writeHTML(w, http.StatusBadRequest,
        "<p class=\"error\">a note needs a body</p>\n")
      return
    }
    count := n.add(body)
    w.Header().Set("HX-Trigger", "notesSaved")
    writeHTML(w, http.StatusOK, notesList(n.all())+oobCount(count))
  }
}
```

listing 107: validate, answer a fragment either way, announce the save (source: go/samples/ch16/htmx.go, lines 103-120)

Bodies are hostile input and the fragment renderers escape them with `html.EscapeString` before they touch markup, the same discipline a full page owes. The test posts whitespace, asserts the 400, and asserts the error paragraph is in the body it can read.



2.x dropped the 400 body: 4.0 swaps it, says the migration

notes page, and reverting is a config line

figure 91: one handler, two fragment outcomes, both swapped in 4.0

16.9. serving and pinning htmx

The script is an asset like any other, so it is vendored into the package and served by the standard library, `go:embed` folding the bytes into the binary and `http.FileServer` over `http.FS` serving them. The pin is the digest: a SHA-384 over the exact bytes, embedded in the `integrity` attribute so the browser verifies the script it runs, and recomputed by the test over the bytes the server actually serves, so the vendored file cannot drift silently. The digest below matches the published artifact byte for byte:

```
//go:embed static/htmx.min.js
var staticFS embed.FS

// HtmxSri pins the vendored htmx.min.js, sha384 over the exact bytes.
// The file cannot change under the digest, and the test recomputes it
// from the served bytes, so a swap cannot pass silently.
const HtmxSri = "sha384-BvJpBiO8Kh31EqtJe5DRIeWrHWnCGkwytKs9NKFi86Hhw96dEqdEMzZDeK9iEGTc"
```

listing 108: embed the bytes, pin them with the sri constant (source: go/samples/ch16/htmx.go, lines 19-25)

```
func TestServedHtmxDigestMatchesSri(t *testing.T) {
  ts := newServer(t)
  code, header, body := get(t, ts.URL+"/static/htmx.min.js", nil)
  if code ≠ http.StatusOK {
    t.Fatalf("status = %d", code)
  }
  if ct := header.Get("Content-Type"); ct == "" {
    t.Fatal("no content type on the static asset")
  }
  sum := sha512.Sum384([]byte(body))
  got := "sha384-" + base64.StdEncoding.EncodeToString(sum[:])
  if got ≠ HtmxSri {
    t.Fatalf("served digest = %s, want %s", got, HtmxSri)
  }
}
```

listing 109: the test recomputes the digest from the served bytes (source: go/samples/ch16/htmx_test.go, lines 70-84)

NOTE: why vendored instead of installed

The npm registry's `latest` tag still points at 2.0.10 while 4.0.0 rides the `next` tag, so `npm install htmx` buys the wrong major version. The 4.0.0 file here is copied byte-identical from this repo's vendored copy, 36716 bytes, its SHA-384 digest matching the published integrity value, and the documentation consulted is four.htmx.org, accessed 2026-09-21. A build step would add a toolchain to avoid a file copy.

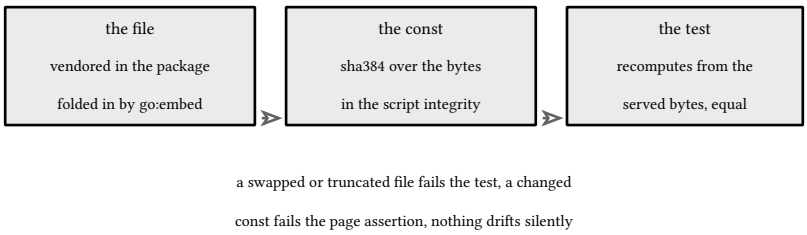


figure 92: the pin: one file, one digest, three places it is checked

sources: four.htmx.org for the htmx 4.0.0 reference and migration notes, accessed 2026-09-21, and pkg.go.dev for net/http, embed, and crypto/sha512. Verified by `go/samples/ch16` tests, 12 of them.

17. templ: compiled html components

Chapter 16 built fragments with `strings.Builder` and hand escaped strings, which is honest work and scales poorly. templ is the compiled alternative: components are declared in a `.templ` file as typed functions, a generator turns them into ordinary go source, and the fragment a handler serves becomes a function call. The version is v0.3.1020, pinned in both of this book's modules, and its generated files are committed so the builds need no templ cli. This chapter's samples live in `go/samples/ch17`, 10 tests, and the capstone that follows builds its web ui on exactly what is taught here.

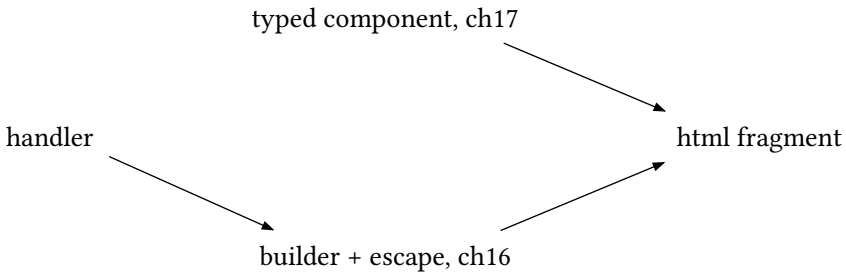


figure 93: the same fragment, two ways

17.1. the component is a function

A templ component looks like markup and is a function. The `templ` keyword declares it, the parameters are typed go values, `{ name }` interpolates, and calling `Hello("gabriel")` returns a `templ.Component`, one method, render to a writer:

```
// Hello is the whole idea: a component is a function, { name }
// interpolates a go value, and templ compiles the file to go code.
templ Hello(name string) {
  <p class="greeting">hello, { name }</p>
}
```

listing 110: markup that is a typed function (source: go/samples/ch17/components.templ, lines 3-7)

```
func TestHelloRendersExactly(t *testing.T) {
  if got := render(t, Hello("gabriel")); got ≠ `<p class="greeting">hello, gabriel</p>` {
    t.Fatalf("Hello = %s", got)
  }
}
```

listing 111: the exact bytes, asserted whole (source: go/samples/ch17/serve_test.go, lines 24-28)

The output is deterministic and whitespace free because the generator emits only the markup it was given, so the test can pin the whole string rather than a substring. The render helper every later section uses is four lines:

```
// Render renders one component to a string, the shape an htmx swap
// consumes and the bridge between a component and a handler.
func Render(ctx context.Context, c templ.Component) (string, error) {
  var b strings.Builder
  if err := c.Render(ctx, &b); err ≠ nil {
    return "", err
  }
  return b.String(), nil
}
```

listing 112: a component to a string, the swap shaped bridge (source: go/samples/ch17/serve.go, lines 20-28)

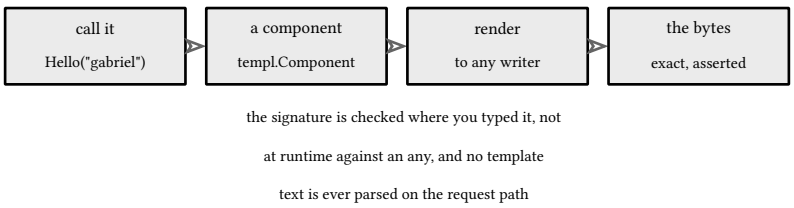


figure 94: component as function: typed in, bytes out, no parse step

17.2. codegen, with the artifact committed

The `.templ` file is not read at runtime. `templ generate` compiles it to `components_templ.go`, ordinary go that imports the templ runtime, and this repo commits that artifact, so `make verify-go` needs no templ cli installed: the module builds from source alone. The directive documents regeneration, pinned to the same version as the runtime dependency:

```
// The generated components_templ.go is committed so this module
// builds and tests with no templ CLI installed; it is generated
// code and exempt from the 400 line source ceiling.
//go:generate go run github.com/a-h/templ/cmd/templ@v0.3.1020 generate
```

listing 113: regeneration is one command, version pinned (source: go/samples/ch17/serve.go, lines 6-9)

```
    templ_7745c5c3_Err = templruntime.WriteString(templ_7745c5c3_Buffer, 1, "<p class=\"greeting\">hello,
")
    if templ_7745c5c3_Err ≠ nil {
      return templ_7745c5c3_Err
    }
    var templ_7745c5c3_Var2 string
    templ_7745c5c3_Var2, templ_7745c5c3_Err = templ.JoinStringErrs(name)
    if templ_7745c5c3_Err ≠ nil {
      return templ.Error{Err: templ_7745c5c3_Err, FileName: `ch17/components.templ`, Line: 6, Col: 34}
    }
    _, templ_7745c5c3_Err = templ_7745c5c3_Buffer.WriteString(templ.EscapeString(templ_7745c5c3_Var2))
    if templ_7745c5c3_Err ≠ nil {
      return templ_7745c5c3_Err
    }
    templ_7745c5c3_Err = templruntime.WriteString(templ_7745c5c3_Buffer, 2, "</p>")
```

listing 114: what the generator emitted for hello (source: go/samples/ch17/components_templ.go, lines 34-47)

The emitted code is readable: a static write of the literal, a `JoinStringErrs` for the interpolated value, `EscapeString` before it touches the buffer, the error threaded through. Committed generated code has precedent in chapter 1's `greeting_gen.go`, and the `_templ.go` files are exempt from the repo's 400 line source ceiling because their length is the generator's business, measured at 302 lines for 54 lines of source.

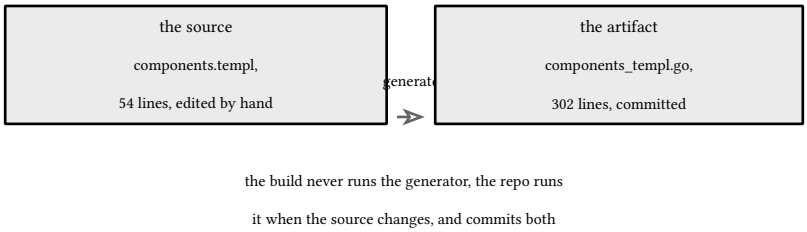


figure 95: the .templ file is a source language, the .go file is the artifact

17.3. composition and children

Components nest two ways: `@Hello(...)` places another component, and a parameter of type `templ.Component` becomes a slot the caller fills with content between the call's braces. `Card` is the small slot, `Page` the document shell, and both are ordinary values, so the shell of a page is built by function composition rather than a layout inheritance scheme:

```
// Card is the slot: children arrives as a component, @ places it.
templ Card(title string, children templ.Component) {
  <section class="card">
    <h2>{ title }</h2>
    @children
  </section>
}
```

listing 115: the slot: children arrive as a component, @ places it (source: go/samples/ch17/components.templ, lines 9-15)

```
// Page is the same slot idea at document scale, the shell every full
// response wraps.
templ Page(title string, children templ.Component) {
  <!doctype html>
  <html>
    <head>
      <meta charset="utf-8"/>
      <title>{ title }</title>
    </head>
    <body>
      @children
    </body>
  </html>
}
```

listing 116: the shell, the same slot at document scale (source: go/samples/ch17/components.templ, lines 17-30)

The composition test pins the whole tree in one string, doctype to closing html, which is the property that matters: what nests is what renders.

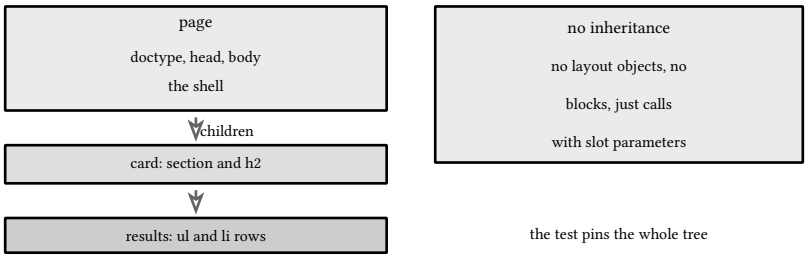


figure 96: composition is call structure: page around card around results

17.4. contextual escaping, honestly against html/template

Values are escaped by where they land. The same string in a text context and an attribute context comes out differently escaped, url attributes go through scheme sanitization, and the whole contract is generated per expression rather than applied by a runtime escaper. The standard library's `html/template` is also context aware, so the honest comparison is side by side, same markup, same hostile value:

```
// Hostile is the escaping demo's payload: values an attacker chose.
type Hostile struct {
  Name string
  Path string
}

// Escapes renders the same value twice, once in a text context and
// once in an attribute context, plus a url attribute, so the test
// can see each escape behave differently.
templ Escapes(h Hostile) {
  <p data-name={ h.Name }>{ h.Name }</p>
  <a href={ h.Path } id="esc-link">link</a>
}
```

listing 117: one value, three contexts: text, attribute, url (source: go/samples/ch17/components.templ, lines 32-44)

```
// stdlibEscapes is the same markup through html/template, the
// standard library's context aware escaper, there for the honest
// side by side.
var stdlibEscapes = template.Must(template.New("esc").Parse(
  `<p data-name="{{.Name}}">{{.Name}}</p>` + "\n" +
    `<a href="{{.Path}}" id="esc-link">link</a>` + "\n"))
```

listing 118: the identical markup through html/template (source: go/samples/ch17/serve.go, lines 30-35)

Both engines escape <b> to <b> in both contexts, and both refuse an unsafe url scheme, templ answering about:invalid#TemplFailedSanitizationURL where html/template answers #ZgotmplZ. The difference is when the checking happens: templ rejects a misspelled component or a wrong typed argument at compile time and emits plain writes, html/template parses its text at startup and resolves {{.Name}} through reflection on an any. The security property is a tie, the type safety is not:

```
func TestURLSanitization(t *testing.T) {
  got := render(t, Escapes(Hostile{Name: "n", Path: "javascript:alert(1)"}))
  if !strings.Contains(got, `href="about:invalid#TemplFailedSanitizationURL"`) {
    t.Fatalf("unsafe url survived: %s", got)
  }
  safe := render(t, Escapes(Hostile{Name: "n", Path: "/p?a=1&b=2"}))
  if !strings.Contains(safe, `href="/p?a=1&amp;b=2"`) {
    t.Fatalf("safe url mangled: %s", safe)
  }
  var out strings.Builder
  if err := stdlibEscapes.Execute(&out, Hostile{Name: "n", Path: "javascript:alert(1)"}); err ≠ nil {
    t.Fatal(err)
  }
  if !strings.Contains(out.String(), `href="#ZgotmplZ"`) {
    t.Fatalf("stdlib let the scheme through: %s", out.String())
  }
}
```

listing 119: both sanitizations pinned, templ's and the stdlib's (source: go/samples/ch17/serve_test.go, lines 67-83)

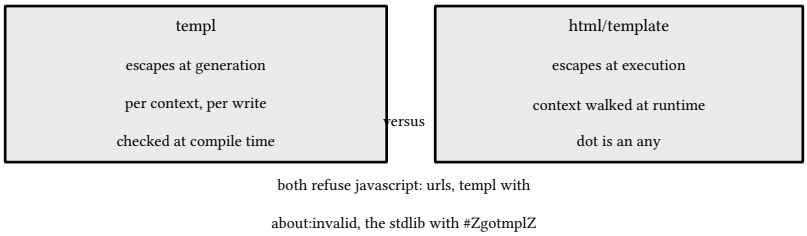


figure 97: same contract, different sentinels, different checking time

17.5. rendering to strings for htmx swaps

Chapter 16's contract was never about strings.Builder, it was that a handler answers html a swap can consume. A templ component renders to exactly that, so the fragment route becomes one call, and the component carries its own id the way the poll fragment there did:

```
// Results is the fragment an htmx swap consumes: its own id, bare
// rows, ready for an outerHTML swap into itself.
templ Results(q string, hits []string) {
  <ul id="results" data-q={ q }>
    for _, hit := range hits {
      <li>{ hit }</li>
    }
  </ul>
}
```

listing 120: the fragment as a component: own id, bare rows, a loop (source: go/samples/ch17/components.templ, lines 46-54)

```
  mux.HandleFunc("GET /fragment/results", func(w http.ResponseWriter, r *http.Request) {
    q := r.URL.Query().Get("q")
    body, err := Render(r.Context(), Results(q, search(corpus, q)))
    if err ≠ nil {
      http.Error(w, err.Error(), http.StatusInternalServerError)
      return
    }
    writeHTML(w, http.StatusOK, body)
  })
```

listing 121: the fragment route: filter, render, answer (source: go/samples/ch17/serve.go, lines 62-70)

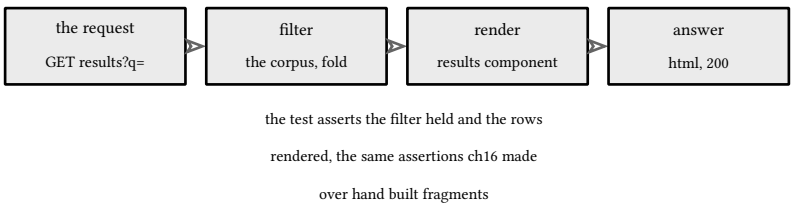


figure 98: the swap consumes a component render, same bytes as ch16

17.6. wiring into net/http

The mux is unchanged from the last chapter, method patterns and all, because components only replace the inside of handlers. The page route composes the whole tree in one expression, shell around card around results, and the wrong verb still answers 405 from the mux itself:

```
// NewMux serves the demo with method patterns: the page as a
// composed component tree, the fragment as a bare component render.
func NewMux(corpus []string) http.Handler {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /{$}", func(w http.ResponseWriter, r *http.Request) {
    page := Page("corpus", Card("search", Results("", search(corpus, ""))))
    body, err := Render(r.Context(), page)
    if err ≠ nil {
      http.Error(w, err.Error(), http.StatusInternalServerError)
      return
    }
    writeHTML(w, http.StatusOK, body)
  })
  mux.HandleFunc("GET /fragment/results", func(w http.ResponseWriter, r *http.Request) {
    q := r.URL.Query().Get("q")
    body, err := Render(r.Context(), Results(q, search(corpus, q)))
    if err ≠ nil {
      http.Error(w, err.Error(), http.StatusInternalServerError)
      return
    }
    writeHTML(w, http.StatusOK, body)
  })
  return mux
}
```

listing 122: two routes, one composed page, one bare fragment (source: go/samples/ch17/serve.go, lines 49-72)

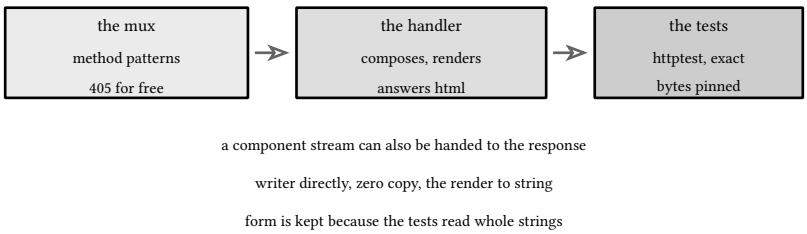


figure 99: components inside handlers, the mux untouched

17.7. when not to

The standard library already serves html, and `html/template` in particular is dependency free, parsed once, context aware. templ buys typed parameters, compile time checking, and component composition, and charges a dependency plus a generation step whose artifact must be committed or installed on every machine. For one or two pages of mostly static markup the stdlib wins and no generator is worth its footprint. For a surface of many fragments, an htmx app especially, where every route answers a component and the fragments nest, the typing pays for itself, and the capstone's web ui is exactly that shape. templ is also html only: json answers stay with the json encoder, and the component file is the wrong place for anything that is not markup.

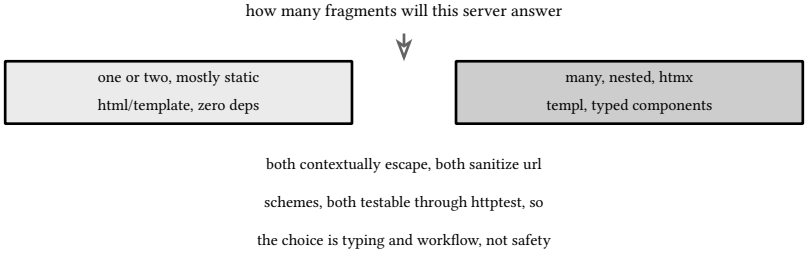


figure 100: the decision: how much fragment surface, that is the whole question

sources: templ.guide for the language, component, and security references of templ v0.3.1020, accessed 2026-09-21, and pkg.go.dev for html/template and net/http. Verified by `go/samples/ch17` tests, 10 of them.

18. http kernel

Everything else in this service part sits on one small package: the http kernel. This chapter builds it in `goapi/internal/httpx`: the handler contract every route shares, the ServeMux route table with method patterns, the constant error envelope every non-2xx response answers with, the json/v2 read and write helpers, the httptest discipline that tests all of it without a socket, and the app skeleton that later chapters wire their families into.

18.1. the handler contract

A go handler is one function of a writer and a request, and that shape is the whole server: `http.Handler` is an interface with one method, `http.HandlerFunc` adapts a plain function to it, and anything that implements it can serve. The kernel keeps the shape and adds one convention on top, the error-returning adapter. A kernel handler returns an error instead of writing a failure, and the adapter turns the error into the contract envelope, so no route formats a failure body by hand and no route can answer a 500 with the wrong shape.

The adapter's rule is two branches. An `*httpx.E`, built by `Fail`, renders as its own code, message, and status. Anything else is an unknown failure, and unknown failures are answered as `internal` with the detail kept for the logs, because a leaked `sql: no rows` tells a client more than it should know. The two-line happy path is the point: handlers read like functions, and every failure in the part flows through one `if`:

```
// Handler is the kernel's handler adapter: return an error rather than
// writing one. An *E renders as its own envelope; any other error is an
// internal 500 whose detail stays out of the response.
type Handler func(w http.ResponseWriter, r *http.Request) error

func (h Handler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
  if err := h(w, r); err ≠ nil {
    e, ok := errors.AsType[*E](err)
    if !ok {
      e = &E{Status: StatusOf(Internal), Code: Internal, Message: "internal error"}
    }
    Error(w, r, e)
  }
}
```

listing 123: the adapter: handlers return errors, the kernel renders the failure envelope (source: go/api/internal/httpx/kernel.go, lines 103-116)

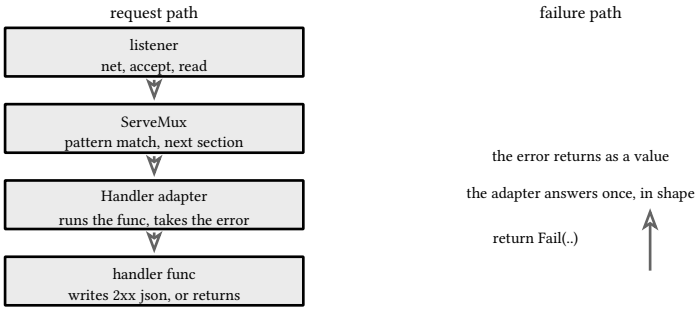


figure 101: the request climbs down the stack, the failure climbs back up as a value

18.2. ServeMux as the route table

Since go 1.22 the standard mux is a real router. A pattern is `METHOD HOST/PATH` with every part optional, wildcards are whole segments like `{id}` and trailing `{path...}`, `{$}` matches only the end, and a `GET` pattern also serves `HEAD`. Precedence falls out of one sentence: when two patterns match, the more specific wins, where more specific

means matching a strict subset of the other's requests, and two patterns that tie panic at registration time, so an ambiguous table never ships. The kernel's table lives in `New` as method patterns, `GET /healthz` and `GET /readyz` first, one `Register` call per family appended as the part grows.

Two of the mux's answers violate the contract, so the kernel wraps it. An unknown path gets plain text `404 page not found`, a known path with the wrong method gets plain text `405` plus an `Allow` header, and the contract demands the envelope on both. `Fallback` asks the mux which pattern won: a non-empty pattern means a real route, and an empty pattern means the default writer is about to answer, so the kernel learns which status the default writer would pick, keeps the mux's `Allow` header, and answers with the envelope itself:

```
func Fallback(mux *http.ServeMux) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    h, pattern := mux.Handler(r)
    if pattern ≠ "" {
      mux.ServeHTTP(w, r)
      return
    }
    p := &probe{h: make(http.Header)}
    h.ServeHTTP(p, r)
    if allow := p.h.Get("Allow"); allow ≠ "" {
      w.Header().Set("Allow", allow)
    }
    if p.status == http.StatusMethodNotAllowed {
      Error(w, r, &E{Status: p.status, Code: NotFound, Message: "method not allowed"})
      return
    }
    Error(w, r, &E{Status: StatusOf(NotFound), Code: NotFound, Message: "route not found"})
  })
}
```

listing 124: the fallback owns the two answers the default mux would get wrong (source: go/api/internal/httpx/mux.go, lines 15-33)

NOTE: mux.Handler does not fill in wildcards

The wrapped dispatch serves matched requests through `mux.ServeHTTP`, not through the handler `mux.Handler` returns. The documented contract of `mux.Handler` is that it does not modify the request, so `r.PathValue` still returns the empty string and `r.Pattern` is unset. Only the mux's own `ServeHTTP` fills both in. The kernel's precedence test caught exactly this on its first run.

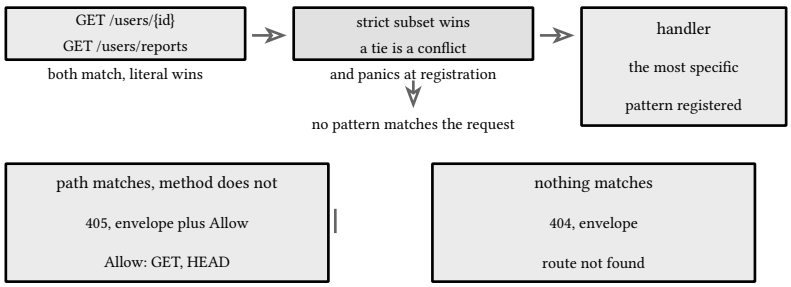


figure 102: pattern precedence: specific beats general, ties panic, leftovers split into 405 and 404

18.3. the error envelope

The contract freezes one failure shape for the whole api, and the shape is smaller than most: an error object carrying a code from a closed set of eleven, one human sentence, optional `details` on `validation_failed` only, and the request id. Every non-2xx rides it, including the 404 and 405 the fallback owns, the 422 the user chapter adds, and the 429 and 503 the rate and load chapters add. A constant envelope beats bare status codes because a client

writes one error path instead of twelve, the statuses the wire actually serves, a grep for `code:"conflict"` works across every endpoint, and the day a status needs to move the code does not.

The codes are declared once, as a `Code` string type with eleven constants, and a single table maps each code to the status it carries everywhere: `invalid_json` to 400, `validation_failed` to 422, `payload_too_large` to 413, `unauthorized` 401, `forbidden` 403, `not_found` 404, `conflict` 409, `precondition_failed` 412, `rate_limited` 429, `overload` 503, `internal` 500. `invalid_json` and `not_found` are the split codes, 400 for a malformed body and 415 for a wrong media type on one, 404 and 405 through the fallback on the other, and each split site passes its status explicitly. The struct is two levels deep so the wire shape stays nested the way the vectors freeze it, and `omitzero` keeps `details` absent everywhere except validation:

```
// Envelope is the constant shape of every non-2xx body the API emits,
// including 404 on unknown routes and 405 on method mismatch.
type Envelope struct {
  Error ErrorBody `json:"error"`
}

// ErrorBody is the error member of the envelope. details ride along on
// validation_failed only; omitzero keeps them absent otherwise.
type ErrorBody struct {
  Code      Code   `json:"code"`
  Message   string `json:"message"`
  Details   []any  `json:"details,omitzero"`
  RequestID string `json:"request_id"`
}
```

listing 125: the envelope: one shape, closed codes, omitzero on details (source: go/api/internal/httpx/kernel.go, lines 71-84)

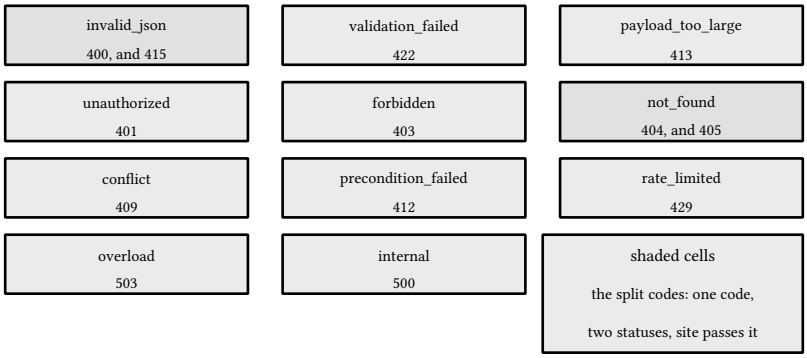


figure 103: eleven codes, ten statuses in the table, twelve on the wire

18.4. writing json responses with json/v2

Bodies go out through `encoding/json/v2`, the stable v2 the tour chapter covered. `WriteJSON` is four lines: set the content type, commit the status, `MarshalWrite` the value straight into the writer. The ordering is a real tradeoff, not an oversight: the header commits before the body serializes, which is fine for the kernel's types, structs that marshal by construction, and wrong for anything that can fail to marshal, where a half-committed response is the worst outcome. `Error` is the same four lines pointed at the envelope, and it is the only place a failure body is written, so the wire shape has one author.

Reading is stricter than writing, because request bodies are input. `Bind` is the kernel's one decode path, generic over the target type so each handler decodes into its own struct, and it enforces three rules in a fixed order: the media type must parse as `application/json` with `mime.ParseMediaType`, so a charset parameter still passes, and anything else is 415 `invalid_json`. The body is then wrapped in `http.MaxBytesReader` at the contract's 1 MiB cap, and a read past it surfaces as `payload_too_large` 413 through `errors.AsType`. What survives goes to `json.UnmarshalRead`, and a

malformed body is 400 `invalid_json`. The options parameter passes json/v2 options through, which is how the user chapter layers `RejectUnknownMembers` on without the kernel knowing about it:

```
func Bind[T any](r *http.Request, opts ...json.Options) (T, error)
// 415 wrong media type, 413 past 1 MiB, 400 malformed
```

fragment, illustrative, not executed

```
// WriteJSON writes v as the response body at the given status. The
// header commits before the body serializes, which is safe for the
// kernel's marshal-by-construction types and wrong for anything that
// can fail to marshal.
func WriteJSON(w http.ResponseWriter, status int, v any) error {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  return json.MarshalWrite(w, v)
}

// Error renders e as the contract envelope at e's status. It is the
// only place a failure body is written.
func Error(w http.ResponseWriter, r *http.Request, e *E) {
  WriteJSON(w, e.Status, Envelope{Error: ErrorBody{
    Code:      e.Code,
    Message:   e.Message,
    Details:   e.Details,
    RequestID: RequestID(r),
  }})
}
```

listing 126: the two write sites: WriteJSON for values, Error for failures (source: go/api/internal/httpx/kernel.go, lines 118-137)

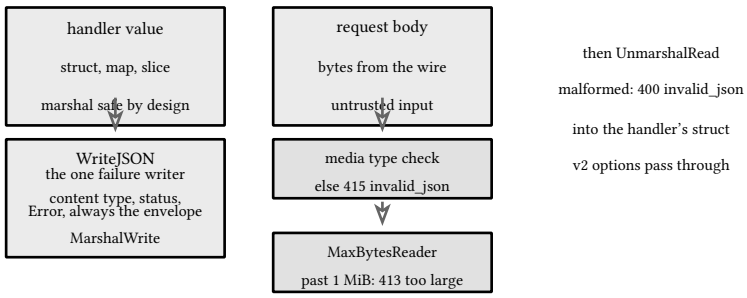


figure 104: two write sites, one decode path, four contract statuses

18.5. httptest the serverless server

`net/http/httptest` gives the kernel its whole test story without a socket. `httptest.NewRequest` builds a request from a method, a target, and an optional body, `httptest.NewRecorder` stands in for the writer and records the status code, the header map, and the body bytes, and a test serves the handler into the recorder and asserts on exact values. Nothing listens, so there is no port to collide, no firewall prompt, and no timing: the round trip is a function call.

The pattern the whole module copies is three lines, build, serve, assert, and the helper every test in the package shares decodes the recorded body back into an `Envelope` first, so a body that is not the contract shape fails before the assertion it was heading for. Exact bytes are asserted where the contract freezes them, the health body for instance, and structural assertions elsewhere:

```
func TestEnvelopeOnUnknownRoute(t *testing.T) {
  app := New(Config{})
  rec := httptest.NewRecorder()
  app.Handler().ServeHTTP(rec, httptest.NewRequest(http.MethodGet, "/nope", nil))
  if rec.Code ≠ http.StatusNotFound {
    t.Fatalf("status %d, want 404", rec.Code)
  }
  if env := decode(t, rec); env.Error.Code ≠ NotFound {
    t.Fatalf("code %q, want not_found", env.Error.Code)
  }
}
```

listing 127: the recorder round trip: serve the root handler, read everything back (source: go/api/internal/httpx/kernel_test.go, lines 29-39)

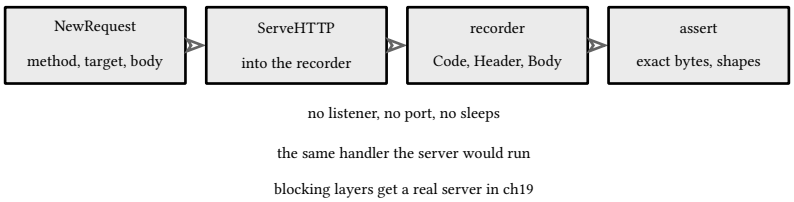


figure 105: request to handler to recorder to assertions, all one function call

18.6. the app skeleton and its wiring table

The kernel also ships the program skeleton, `httpx.App`, and the single file that wires it. `Config` carries the listen address, the build version, and a readiness probe, and `ConfigFromEnv` reads `GOAPI_ADDR` and `GOAPI_VERSION` so one binary runs anywhere. `New` builds the app and registers the kernel's two routes through the adapter, `Mux` is exported because every family registers on it, and `Use` appends a middleware layer, first used outermost, which is the seam the next chapter plugs its stack into. `Handler` folds the layers over the fallback-wrapped mux, and `ListenAndServe` is deliberately plain: graceful drain is a load-chapter concern that will land as a new file in this same package, not a change to this one.

The two kernel routes are the deployment contract. `GET /healthz` answers liveness, 200 with `{"status":"ok","version":".."}`, where the version comes from the environment so a deployment can prove what it is running. `GET /readyz` answers readiness, 200 while the probe says ready and the envelope's `overload` code at 503 the moment it does not, which is the hook the drain wiring flips. `cmd/api/main.go` stays twenty lines for the whole part, pinned here once and never again: each later chapter's registration is two to five lines, a construction and a `Register` call, and the integrator applies them inside `wire.go`, the append-only wiring table the single `wire` call enters:

```go
// Command api runs the goapi service. This file stays twenty lines for
// the whole part: the ch29 health dispatch must answer before any
// wiring runs, and everything else is the single call into wire.go,
// the append-only wiring table where every later family lands below
// the last.
package main

import (
  "os"

  "goapi/internal/httpx"
)

func main() {
  if len(os.Args) > 1 && os.Args[1] == "-health" {
    healthMain()
    return
  }
  wire(httpx.New(httpx.ConfigFromEnv()))
}
```

listing 128: main.go, pinned once for the part: the ch29 health dispatch before the single wiring call (source: go/api/cmd/api/main.go, lines 1-20)

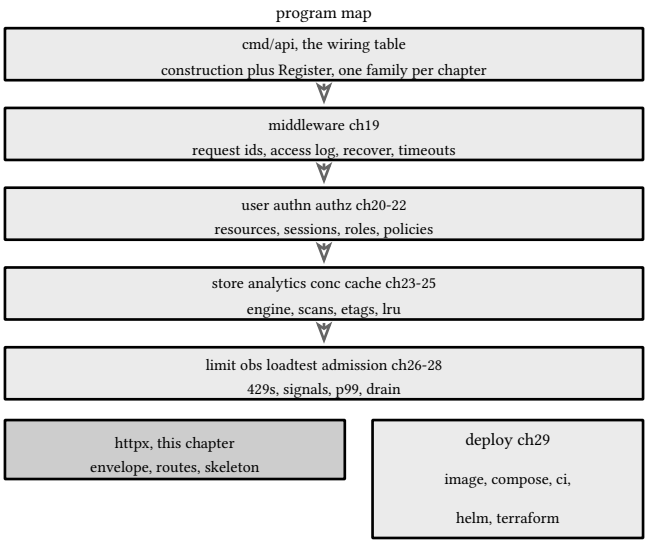


figure 106: the part's twelve packages stack on the kernel, the wiring table owns the order

18.7. what the kernel proves

The kernel is small on purpose, three files and a test file, and what it proves is that the part's shape is already fixed. The envelope exists, so every later failure is a `Fail` call and never a handbuilt body. The decode ladder exists, so every write route inherits the 415, the 413, and the 400 without repeating them. The route table exists with method patterns and a fallback that keeps the contract on the two answers the default mux gets wrong. The skeleton exists, with a public mux for registrations and a middleware seam for layers, and the roadmap from here is additive:

```go
// New builds the app and registers the kernel routes. The handlers are
// the kernel's Handler adapter, the pattern every later family copies.
func New(cfg Config) *App {
  app := &App{Mux: http.NewServeMux(), cfg: cfg}
  app.Mux.Handle("GET /healthz", Handler(app.healthz))
  app.Mux.Handle("GET /readyz", Handler(app.readyz))
  return app
}

// Use appends one middleware layer. The first layer used is the
// outermost at request time.
func (a *App) Use(m func(http.Handler) http.Handler) {
  a.mw = append(a.mw, m)
}
```

listing 129: New registers the kernel routes, Use takes the next chapter's layers (source: go/api/internal/httpx/server.go, lines 45-58)

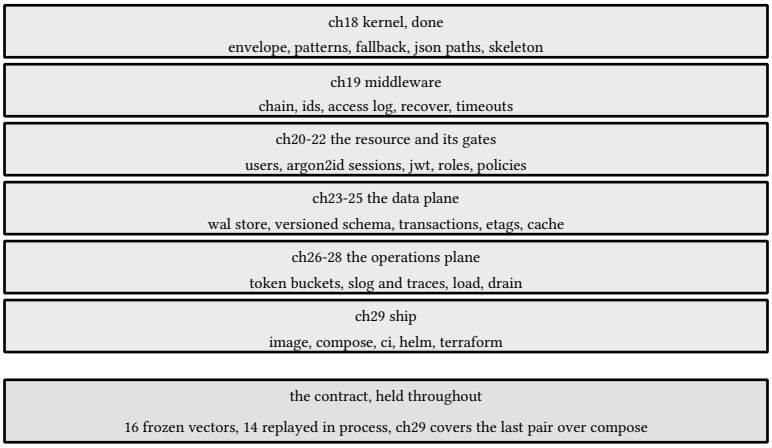


figure 107: the part as seven rows: every chapter adds routes or layers, none rewrites the kernel

sources: pkg.go.dev for net/http, encoding/json/v2, net/http/httptest, mime, and uuid, accessed 2026-09-25, the ServeMux pattern and precedence rules quoted from the go 1.27 source in GOROOT, and `mux.Handler`'s wildcard caveat read from the same source. Verified by `go/api/internal/httpx` tests, 13 of them, plus the package gates: gofmt, go vet, and go test over the whole goapi module.

19. middleware

Cross-cutting behavior does not belong inside the handlers, and go does not make anyone put it there. This chapter composes the service's middleware stack over the previous chapter's kernel, in `goapi/internal/middleware`: the chain type that folds layers into one handler, panic recovery into the contract envelope, request ids born once and quoted everywhere, the access log with its status-capturing writer, timeouts at two layers, and the wiring order that makes the whole stack correct rather than merely present.

19.1. the chain type

One layer is one function: given the next handler, return the handler that wraps it. That is the whole `Func` type, and it is the same shape the standard library's own middleware uses, so anything written against it composes with anything else. `Chain` folds a list of layers into one by wrapping right to left, so the first argument ends up outermost: it sees the request first and the response last, which is the onion picture every middleware chapter draws because it is the true one. A chain is itself a `Func`, so chains compose into bigger chains, and the kernel's `App.Use` appends onto the same fold, first used outermost.

The loop runs backwards for a reason worth saying out loud: folding left to right would make the last layer outermost, and the wiring order in this chapter's sixth section is a contract, not an accident. A test pins it, asserting the pass order `outer,middle,inner,handler` from a recorder:

```
// Func is one middleware layer: given the next handler, return the
// handler that wraps it.
type Func func(next http.Handler) http.Handler

// Chain composes layers into one Func. Chain(a, b, c)(h) serves a,
// then b, then c, then h, so the first argument is the outermost layer
// and sees the request first and the response last.
func Chain(ms ...Func) Func {
  return func(next http.Handler) http.Handler {
    for i := len(ms) - 1; i ≥ 0; i-- {
      next = ms[i](next)
    }
    return next
  }
}
```

listing 130: Func is one layer, Chain folds them outermost first (source: go/api/internal/middleware/chain.go, lines 11-25)

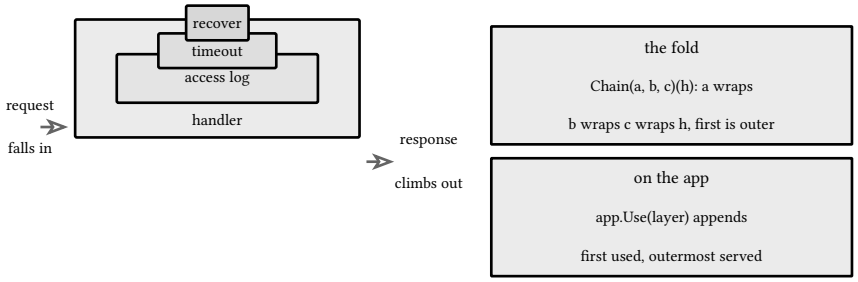


figure 108: the onion: first argument outermost, request in, response out

19.2. recover

A panicking handler must not take the process down, and the client must still get the contract envelope, and a bare `http.Handler` has no way to promise either on its own. The recover layer installs one deferred function: if the handler returns normally the deferred call sees a nil recover and costs nothing, if it panics the layer logs the value

and the stack with the request id and answers 500 `internal` through the kernel's one failure writer, so the shape on the wire is the envelope even for the failures nobody planned.

One panic is deliberately let through. `http.ErrAbortHandler` is the server's own signal, the sentinel a handler panics with to make the connection drop, and a middleware that converts it into a 500 changes what the server was told to do. The layer rethrows it verbatim, and the test for that asserts the exact sentinel crosses the boundary. The other rule is placement: a `recover` only sees panics on its own goroutine, and this stack's timeout layer moves the handler onto another goroutine, so recover sits inside it, innermost, a restriction the sixth section turns into the wiring order:

```
func Recover(log *slog.Logger) Func {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      defer func() {
        v := recover()
        if v == nil {
          return
        }
        if v == http.ErrAbortHandler {
          panic(v)
        }
        log.Error("panic recovered",
          "request_id", httpx.RequestID(r),
          "panic", fmt.Sprint(v),
          "stack", string(debug.Stack()))
        httpx.Error(w, r, httpx.Fail(httpx.Internal, "internal error"))
      }()
      next.ServeHTTP(w, r)
    })
  }
}
```

listing 131: recover: nil costs nothing, abort rethrows, the rest becomes the envelope (source: go/api/internal/middleware/recover.go, lines 18-38)

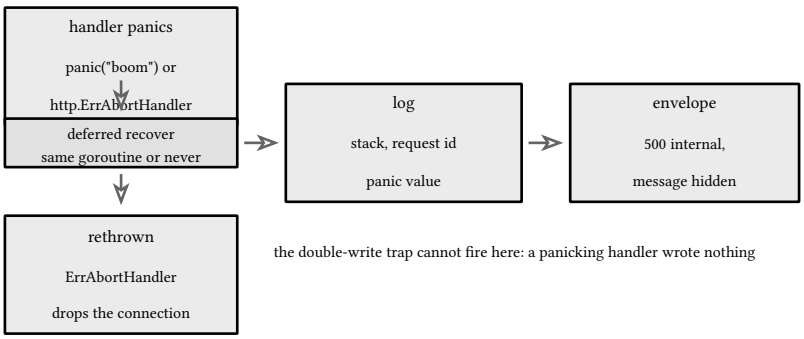


figure 109: panic unwind: one path to the log, one to the envelope, one rethrown

19.3. request-id

Every request needs one id that every artifact agrees on: the access log line, the error envelope, the panic log, and later the trace. The layer resolves it exactly once, at the edge, using the kernel's `RequestID` order: the context value if some outer layer already set one, else the client's `X-Request-ID` when it parses as a uuid, else a fresh uuid v7. Parsing is not politeness, it is contract: the envelope promises `request_id` is a uuid, and a client-supplied free-text header echoed into logs is log injection. Garbage is replaced, not echoed, and the v7 choice keeps ids time-ordered the way the store chapter's ids are.

The layer then does two small things with the value: it sets the response header, so a client can correlate its own logs with the server's, and it installs the id in the request context with the kernel's `WithRequestID`, so everything downstream reads one value without touching headers again. The test for it checks all three births: a client uuid echoes, garbage does not, and the envelope written far downstream carries the id the edge resolved:

```
// RequestID resolves one id per request: the client's X-Request-ID when
// it parses as a uuid, else a fresh uuid v7. It echoes the id on the
// response and stores it in the request context, so the envelope, the
// access log, and the handler all quote the same value.
func RequestID() Func {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      id := httpx.RequestID(r)
      w.Header().Set(httpx.HeaderRequestID, id)
      next.ServeHTTP(w, httpx.WithRequestID(r, id))
    })
  }
}
```

listing 132: resolve once at the edge, echo it, put it in the context (source: go/api/internal/middleware/requestid.go, lines 9-21)

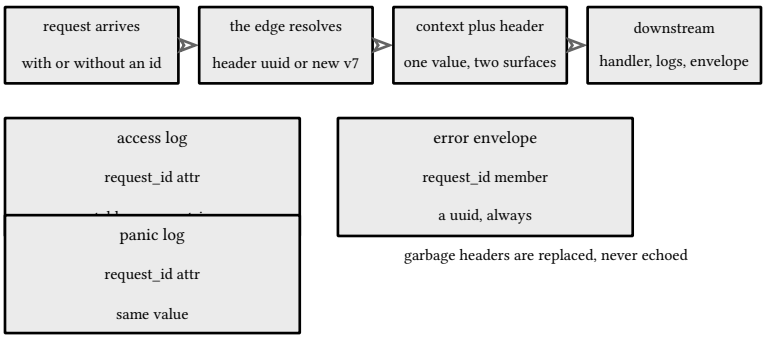


figure 110: one id, born at the edge, quoted by everything downstream

19.4. the access log

Handlers write a status and never return it, so a middleware that wants to log what actually went out has to watch it go. The access log wraps the `ResponseWriter` in a `statusWriter` that records the first status and counts body bytes, then writes one `slog` line after the handler returns: method, path, status, bytes, duration, request id. First status is the load-bearing phrase, because the double `WriteHeader` trap is real: a handler that writes 201 then 500 has already committed 201, the server logs a superfluous-write warning, and a logger that recorded the second call would report a status the client never saw. The wrapper applies the same first-wins rule the wire does, and a body write with no explicit status counts as an implicit 200, exactly as the server itself treats it.

Wrapping a `ResponseWriter` has a second, quieter cost: the wrapper is a new type, and the type assertions that power the optional interfaces now fail. `Flush` is forwarded by hand and `Unwrap` lets `http.NewResponseController` walk to the real writer, which is the modern fix. `Hijacker` and `io.ReaderFrom` are not forwarded here, and prose says so honestly: a stack that needs websockets or sendfile must forward them or use `ResponseController` throughout:

```
type statusWriter struct {
  http.ResponseWriter
  status int
  bytes  int
}

func (w *statusWriter) WriteHeader(status int) {
  if w.status == 0 {
    w.status = status
  }
  w.ResponseWriter.WriteHeader(status)
}

func (w *statusWriter) Write(p []byte) (int, error) {
  if w.status == 0 {
    w.status = http.StatusOK
  }
  n, err := w.ResponseWriter.Write(p)
  w.bytes += n
  return n, err
}
```

listing 133: the status-capturing writer: first status wins, bytes counted (source: go/api/internal/middleware/accesslog.go, lines 38-58)

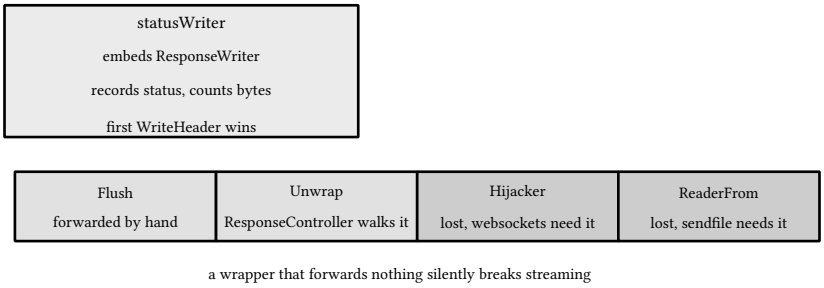


figure 111: the wrapper forwards what it names, loses what it does not

19.5. timeouts at two layers

A service needs timeouts at two layers because the two layers fail differently. The transport layer is the `http.Server`'s own clock: `ReadHeaderTimeout` bounds how long a client may take to finish sending headers, `ReadTimeout` the request including its body, `WriteTimeout` the response, `IdleTimeout` a kept-alive connection. These are coarse, they protect the process from slow clients, and when one fires the connection just dies, plain by design. The handler layer is finer: one request's handler time, after which the client deserves the contract's `overload` envelope rather than a dropped connection, and that is a middleware.

The standard `http.TimeoutHandler` already bounds handler time and is the right answer when plain text is acceptable, but its expiry body is a bare message string, and this api's every non-2xx is the envelope, so the layer is hand-rolled on the same skeleton the standard one uses. The handler runs on a new goroutine with a deadline context and a `detachedWriter`: its own header map, status, and body buffer, with a mutex so a handler that outlives its deadline races against nothing. The outer select waits on the handler's return or the context: on return the captured response commits to the real writer untouched, on deadline expiry the envelope answers and the abandoned handler's buffer is dropped unread, and on a canceled parent, a client that left, nothing is written at all because there is no one to read it:

```
func Timeout(d time.Duration) Func {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      ctx, cancel := context.WithTimeout(r.Context(), d)
      defer cancel()

      tw := &detachedWriter{hdr: make(http.Header)}
      done := make(chan struct{})
      go func() {
        defer close(done)
        next.ServeHTTP(tw, r.WithContext(ctx))
      }()

      select {
      case <-done:
        tw.commit(w)
      case <-ctx.Done():
        // expiry answers overload; a canceled parent left no one to answer
        if errors.Is(ctx.Err(), context.DeadlineExceeded) {
          httpx.Error(w, r, httpx.Fail(httpx.Overload, "request timed out"))
        }
      }
    })
  }
}
```

listing 134: the handler deadline: detached writer, envelope on expiry, silence on cancel (source: go/api/internal/middleware/timeout.go, lines 20-44)

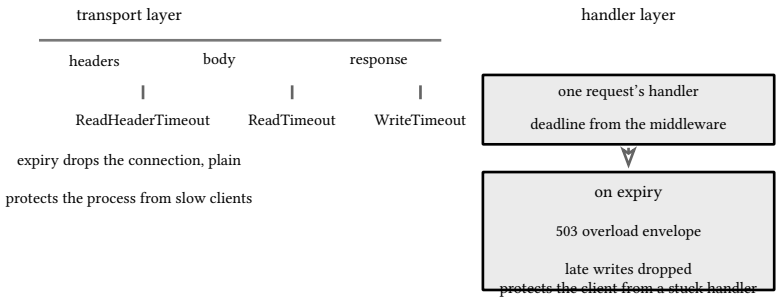


figure 112: two clocks: transport deadlines kill connections, the handler deadline answers

19.6. wiring order and why

The order of the four layers is contract, and each position is a sentence. Request id outermost, because every other artifact quotes it: the access log attribute, the panic log, the envelope, all of it. Access log second, outside recovery, so a recovered 500 is logged as the 500 the client saw rather than the 200 a wrapper never observed. Timeout third. Recover innermost, and this one is mechanical rather than aesthetic: the timeout layer runs the handler on another goroutine, and a panic never crosses a goroutine boundary, so a recover placed outside the timeout would not catch the handler's panic, the process would die, and the whole point of the layer would be gone. The wiring applied by the integrator lives in the program's append-only table, `cmd/api/wire.go`, and reads top to bottom as the request flows:

```
log := slog.New(slog.NewJSONHandler(os.Stdout, nil))
app.Use(middleware.RequestID())
app.Use(middleware.AccessLog(log))
```

```
app.Use(middleware.Timeout(5 * time.Second))
app.Use(middleware.Recover(log))
```

fragment, illustrative, not executed

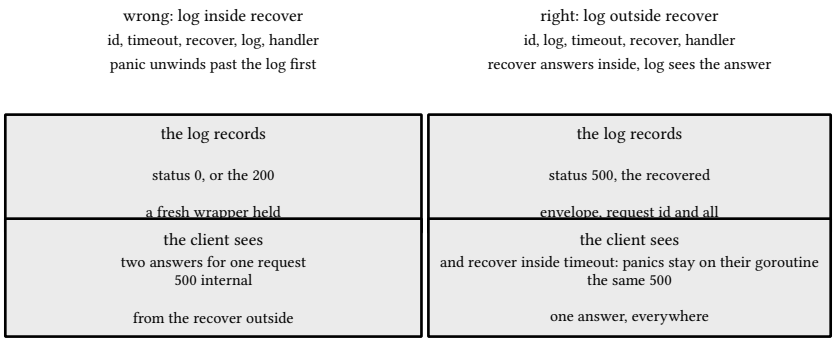


figure 113: wrong order versus right: where the log sits decides what it sees

19.7. testing middleware

Middleware is tested like handlers, mostly. The fast paths go through the recorder exactly as the kernel chapter's tests do: serve a layer wrapped around a tiny handler, read back the status, the headers, the body, and for the logging layers the captured `slog` text, one buffer per assertion. The blocking paths are the addition: a layer whose handler never returns cannot be tested through a function call that never returns, so the timeout tests use a real `httptest.NewServer` with a channel-gated handler, the gate held closed while the client blocks on its request, closed after the assertions so the abandoned goroutine can finish and join the test through a second channel. The client-gone path needs no server at all: a request built with an already-canceled context distinguishes a client leaving from a deadline expiring without any timing dependence.

Nothing sleeps. The deadline fires because the test waits on the response, not on the clock, the race detector runs over the whole package because this is the module's first genuinely concurrent code, and the stack test at the end is the one that fails when anyone reorders the wiring, which is the property the whole chapter exists to keep:

```
func TestStackOrderLogSeesRecovered500(t *testing.T) {
  log, buf := testLog()
  stack := Chain(
    RequestID(),
    AccessLog(log),
    Timeout(10*time.Second),
    Recover(log),
  )(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    panic("boom")
  }))
  rec := httptest.NewRecorder()
  r := httptest.NewRequest(http.MethodGet, "/x", nil)
  r.Header.Set(httpx.HeaderRequestID, "01a0f5d5-6c80-7ca2-8d7f-652c7031c304")
  stack.ServeHTTP(rec, r)
  if rec.Code ≠ http.StatusInternalServerError {
    t.Fatalf("status %d, want the recovered 500", rec.Code)
  }
  var env httpx.Envelope
  if err := json.UnmarshalRead(rec.Body, &env); err ≠ nil || env.Error.RequestID ≠
"01a0f5d5-6c80-7ca2-8d7f-652c7031c304" {
    t.Fatalf("body %q err %v, want the envelope carrying the request id", rec.Body.String(), err)
  }
  if !strings.Contains(buf.String(), "status=500") {
    t.Fatalf("access log %q, want the 500 the client saw: recover must sit inside timeout, log outside
recover", buf.String())
  }
}
```

listing 135: the stack test: wrong order fails here, log and envelope agree on the 500 (source: go/api/internal/middleware/middleware_test.go, lines 245-269)

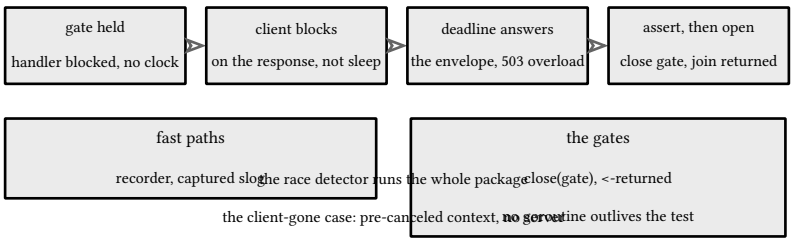


figure 114: the harness: gates hold handlers closed, channels join them back

sources: pkg.go.dev for net/http, TimeoutHandler, ResponseController, log/slog, and net/http/httptest, accessed 2026-09-25, with the transport timeout semantics and the goroutine-local rule for recover read from the go 1.27 source in GOROOT. Verified by `go/api/internal/middleware` tests, 12 of them, under `go test -race`, plus gofmt and go vet over the module.

20. the user resource

The service's first real resource is the user: a record with an id, an email, a display name, roles, a creation time, and a version. This chapter builds the whole resource layer on the chapter 18 kernel: the representation and its two projections, strict json/v2 decoding, validation from first principles, merge-patch semantics for updates, keyset pagination for lists, the status contract that binds failures to codes, and the golden handler tests that pin all of it against the frozen contract vectors.

20.1. the representation

A resource is three things at once: the bytes on the wire, the fields each reader may see, and the strong etag that names one exact version of the body. Go structs express all three cheaply. The stored record and the full wire representation are one type, because json/v2 tags map every field to its member and a dash tag keeps the password hash out of every marshal without a second struct to drift. The projection is a second struct, and one method chooses between them:

```
// User is the stored record. PasswordHash rides along for the authn
// family but never serializes, so the wire representation and the
// stored record stay one type without leaking the hash.
type User struct {
  ID           string    `json:"id"`
  Email        string    `json:"email"`
  DisplayName  string    `json:"display_name"`
  Roles        []string  `json:"roles"`
  CreatedAt    time.Time `json:"created_at"`
  Version      int       `json:"version"`
  PasswordHash string    `json:"-"`
}

// Public is the list projection: email and roles are admin-only
// fields, so non-admin readers see exactly these four members.
type Public struct {
  ID          string    `json:"id"`
  DisplayName string    `json:"display_name"`
  CreatedAt   time.Time `json:"created_at"`
  Version     int       `json:"version"`
}
```

listing 136: the record, its wire tags, and the public projection (source: go/api/internal/user/user.go, lines 25-45)

The projection is where field-level authorization lives. Email and roles are admin-only data, so the list handler never decides what to hide, it calls one method and writes whatever comes back. That is the whole trick: a rule enforced in one function cannot be forgotten in a second handler. The version member is the quiet third actor. It bumps exactly once per successful write, and the chapter 24 concurrency material will promote it into the compare-and-swap substrate.

The etag is not stored anywhere, it is computed: marshal the full representation once, sha256 those exact bytes, quote the hex. Because the etag is a function of the body, the two can never disagree, and a client that saves the etag has effectively saved the body's fingerprint:

```
func ETagBody(body []byte) string {
  sum := sha256.Sum256(body)
```

```
  return `"` + hex.EncodeToString(sum[:]) + `"`
}
```

fragment, illustrative, not executed

go field	json member
ID string	id
Email string	email
DisplayName string	display_name
Roles []string	roles
CreatedAt time.Time	created_at
Version int	version
PasswordHash string	dash, never sent

figure 115: the struct fields against the wire members, one field kept off the wire

20.2. decoding with strictness

Input is hostile until proven otherwise, and the kernel's decode ladder answers three questions in a fixed order: is the media type json at all, is the body parseable inside the 1 MiB cap, and does it carry only members the contract knows. The register route runs the whole ladder in one call, because the kernel's bind helper already owns the first two steps and json/v2 options ride through as the third:

```
func (h *handlers) register(w http.ResponseWriter, r *http.Request) error {
  req, err := httpx.Bind[registerRequest](r, json.RejectUnknownMembers(true))
  if err ≠ nil {
    return err
  }
  if details := ValidateRegister(req.Email, req.Password, req.DisplayName); len(details) > 0 {
    return httpx.Fail(httpx.ValidationFailed, "validation failed", toAny(details)...)
  }
  hash, err := h.d.HashPassword(req.Password)
  if err ≠ nil {
    return err
  }
  u := User{
    ID:           h.d.NewID(),
    Email:        NormalizeEmail(req.Email),
    DisplayName:  req.DisplayName,
    CreatedAt:    h.d.Now(),
    PasswordHash: hash,
  }
  if err := h.d.Store.Create(r.Context(), &u); err ≠ nil {
    if errors.Is(err, ErrConflict) {
      return httpx.Fail(httpx.Conflict, "email already registered")
    }
    return err
  }
```

listing 137: strict bind, rule validation, then the store, in that order (source: go/api/internal/user/handlers.go, lines 49-73)

RejectUnknownMembers is the strictness that pays rent. A client that sends a pet member against a contract with no pet gets a 400, not a silently ignored field, so a typo'd member name surfaces at the border instead of corrupting data six layers in. The typed struct makes the accepted set explicit in its tags, which is why the unknown member failure belongs to the decode step rather than validation.

The order matters for the client's sake: the 415 fires before a body byte is read, the 400 at parse time, and only then does validation answer 422 with one detail per broken rule. A client fixing its request learns everything wrong with it in one round trip, and nothing downstream, hashing, ids, the store, runs on input that already failed.

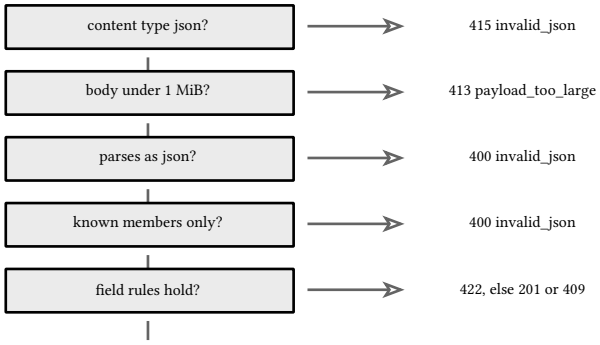


figure 116: the decode ladder: each rung rejects a class of bad input

20.3. validation from first principles

Validation rules come from the contract, not from a library: passwords at least 12 characters and at most 128, one email shape, display names 1 to 64 characters. Stating them as plain code keeps every number greppable and every message owned. The function returns one detail per violated rule rather than failing on the first, so the 422 body's details array tells the whole story:

```
func ValidateRegister(email, password, displayName string) []string {
  var details []string
  switch {
  case email == "":
    details = append(details, "email is required")
  case !ValidEmail(email):
    details = append(details, "email is not a valid address")
  }
  switch {
  case password == "":
    details = append(details, "password is required")
  case len(password) < MinPasswordLength:
    details = append(details, "password must be at least 12 characters")
  case len(password) > MaxPasswordLength:
    details = append(details, "password must be at most 128 characters")
  }
  switch {
  case displayName == "":
    details = append(details, "display_name is required")
  case len(displayName) > MaxDisplayName:
    details = append(details, "display_name must be at most 64 characters")
  }
  return details
}
```

listing 138: one detail per broken rule, in field order (source: go/api/internal/user/validate.go, lines 23-46)

Three classes of rule run here. Presence rules catch missing members. Shape rules catch an email that parses as nothing, where the standard library's address parser plus a dotted-domain check approximates the real grammar without rejecting addresses people actually hold. Range rules catch lengths. The bounds live in named constants next to the rules, so the handler, the tests, and this chapter quote one set of numbers.

The 422 and the 409 are different animals and the boundary is worth stating. A rule violation is about the request alone, checkable without touching state, so it is 422 validation. The email conflict is about the store's current

contents, so it is 409 conflict, and it surfaces only after validation passed and the create call answered its conflict error. Same request shape, two different truths.

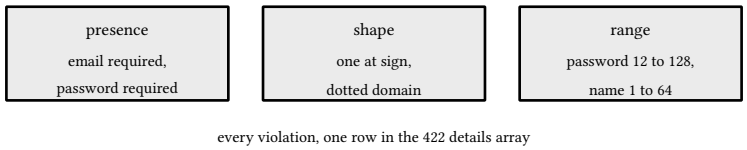


figure 117: three rule classes, one details array

20.4. PATCH merge semantics

RFC 7386 merge-patch gives updates three-way semantics a typed struct cannot express: an absent member leaves the field alone, null deletes it, a value replaces it. So the patch decodes into a member map, not a struct, and the handler walks the members in sorted order so details are deterministic:

```
for _, key := range sortedKeys(patch) {
  v := patch[key]
  switch key {
  case "display_name":
    if v == nil {
      details = append(details, "display_name cannot be removed")
      continue
    }
    s, isStr := v.(string)
    if !isStr {
      details = append(details, "display_name must be a string")
      continue
    }
    if bad := ValidateDisplayName(s); bad != nil {
      details = append(details, bad...)
      continue
    }
    name = s
  case "id", "email", "created_at", "version", "roles":
    details = append(details, key+" is immutable")
  default:
    details = append(details, "unknown field "+key)
  }
}
```

listing 139: the member walk: immutable members refuse, null on the display name too, the rest validate (source: go/api/internal/user/handlers.go, lines 188-211)

The contract makes one member writable, the display name, and the walk shows the full policy in one switch. A null on it answers 422, because deleting the display name would leave the record invalid, so the delete semantics run and the validation rules answer. The immutable members, id, email, created_at, version, roles, refuse the moment they appear, null or not: patching them is not a validation typo, it is an attempt to write a field the contract closed. Unknown members refuse too, the same strictness the typed register decode enforced with an option, enforced here by the member walk because a map cannot carry that option.

After the walk, the precondition and the write run in order. The client's If-Match must equal the etag of the current body, else 412, and the update itself carries the expected version into the store so a write that lost a race answers 412 as well rather than silently clobbering. The no-op patch, an empty object, is legal and answers 200 with the current representation, nothing bumped.

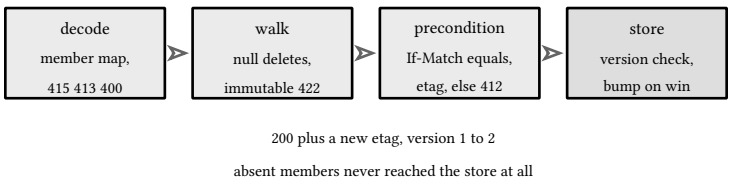


figure 118: a patch request walks the members, then the precondition, then the store

20.5. keyset pagination

Offset pagination asks for row 86041 onward, and it fails twice. The database still walks and discards every skipped row, so page 4000 costs what a table scan costs. And offsets are positions in a moving file: one registration between requests and a row appears twice or vanishes. Keyset pagination fixes both by remembering where it stopped rather than counting from the front. The order is the pair (created_at, id), the cursor encodes the last row's position, and the next page is everything strictly after it:

```
func EncodeCursor(k CursorKey) (string, error) {
  b, err := json.Marshal(cursorJSON{T: k.T, I: k.I})
  if err ≠ nil {
    return "", err
  }
  return base64.RawURLEncoding.EncodeToString(b), nil
}

// DecodeCursor parses a client cursor. Any error means 422: a cursor
// is either one we issued or garbage, never a 404.
func DecodeCursor(s string) (CursorKey, error) {
  b, err := base64.RawURLEncoding.DecodeString(s)
  if err ≠ nil {
    return CursorKey{}, fmt.Errorf("cursor is not base64url: %w", err)
  }
  var c cursorJSON
  if err := json.Unmarshal(b, &c); err ≠ nil {
    return CursorKey{}, fmt.Errorf("cursor is not valid json: %w", err)
  }
  if c.T == 0 || c.I == "" {
    return CursorKey{}, fmt.Errorf("cursor is empty")
  }
  return CursorKey{T: c.T, I: c.I}, nil
}
```

listing 140: the cursor: created_at milliseconds plus the id tiebreaker, base64url unpadded (source: go/api/internal/user/pagination.go, lines 28-51)

The payload is deliberately tiny, two members, because the cursor is a position, not a page number. Time is milliseconds since the epoch, the id breaks ties when two users share a creation millisecond, and uuid v7 ids sort by their own embedded time, so the composite order is total. The encoding is base64url without padding: opaque to clients, trivially decodable in a test, and the frozen vectors carry its exact bytes. A cursor the service never issued is garbage, not a missing page, so decode failures answer 422, and the limit must be an integer in 1..100, 422 otherwise, with a default of 20.

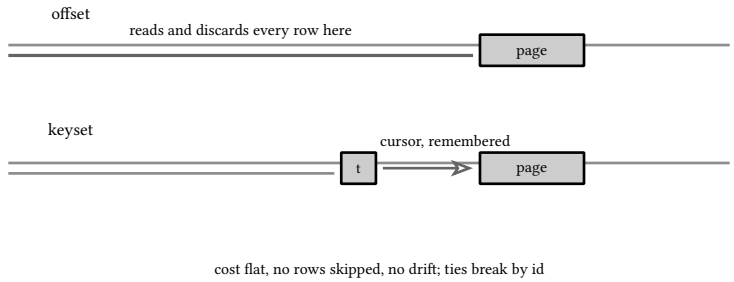


figure 119: offset walks the whole prefix, keyset jumps to the remembered position

20.6. the status contract

Every failure this layer can produce already has a code in the chapter 18 table, so the resource layer states outcomes and the table states statuses. The get handler is the whole ladder in one screen: 401 without an actor, 404 for an unknown id, 403 for a stranger touching someone else's record, then the etag work:

```
func (h *handlers) get(w http.ResponseWriter, r *http.Request) error {
  actor, ok := ActorFrom(r)
  if !ok {
    return httpx.Fail(httpx.Unauthorized, "authentication required")
  }
  id := r.PathValue("id")
  u, err := h.d.Store.Get(r.Context(), id)
  if err ≠ nil {
    return notFound(err)
  }
  // The full record carries email and roles, so reading one is
  // owner or admin business: a stranger guessing ids gets the same
  // 403 the object routes below answer, the IDOR discipline chapter
  // 22 tests exhaustively.
  if actor.UserID ≠ id && !actor.Admin() {
    return httpx.Fail(httpx.Forbidden, "not the owner")
  }
```

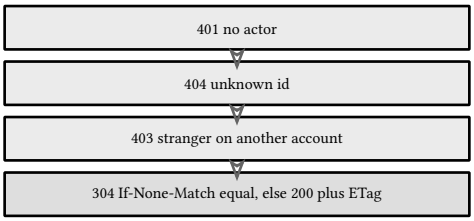
listing 141: the failure ladder before any body is written (source: go/api/internal/user/handlers.go, lines 84-100)

After the ladder comes the one status that is not a failure, 304. If the client's If-None-Match equals the computed etag, the body it already holds is current, and the service answers the header and nothing else:

```
etag := ETagBody(body)
if r.Header.Get("If-None-Match") == etag {
  w.Header().Set("ETag", etag)
  w.WriteHeader(http.StatusNotModified)
  return nil
}
w.Header().Set("ETag", etag)
writeBody(w, http.StatusOK, body)
```

fragment, illustrative, not executed

The order inside the ladder is a policy choice this chapter owns: the 404 answers before the 403, so an unknown id and a stranger's id are distinguishable. APIs that prefer to hide existence answer 404 for both, and the chapter 22 audit walks that tradeoff explicitly. What the contract forbids is a third path: no route here invents a status, a message family, or an envelope shape, because every failure goes through the one Fail call the kernel renders.



patch adds 412 above the ladder, one envelope for every failure

figure 120: one request climbs the ladder, each rung one contract code

20.7. golden handler tests

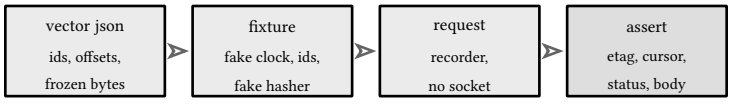
The frozen vectors under `contract/testdata` are the fixture source of truth, and the tests read them as data: ids, timestamps, expected etag strings, expected cursor bytes all come from the JSON files, so a test never re-derives a constant the contract already froze. The walk test shows the shape, seed three users at the vector's offsets, ask for a page as a non-admin, and demand the exact next cursor:

```
func TestPaginationWalk(t *testing.T) {
  v := loadVector(t, "13-list-walk.json")
  f := newFixture(t)
  _, graceID, _ := seedWalkUsers(t, f)
  // The walk's requester is grace, the second user, a non-admin.
  grace := &Actor{UserID: graceID, SessionID: "s-grace", Roles: []string{RoleUser}}
  rec := f.do(t, grace, "GET", v.Request.Path, "", nil)
  if rec.Code != 200 {
    t.Fatalf("page 1 status = %d, body %s", rec.Code, rec.Body.String())
  }
  page := bodyJSON(t, rec).(map[string]any)
  want := v.Response.Body.(map[string]any)
  if got, ok := page["next_cursor"].(string); !ok || got != want["next_cursor"] {
    t.Errorf("next_cursor = %v, want the frozen bytes %v", page["next_cursor"], want["next_cursor"])
  }
  if !reflect.DeepEqual(page["items"], want["items"]) {
    t.Errorf("page 1 items = %v, want %v", page["items"], want["items"])
  }
```

listing 142: the vector walk: cursor bytes exact, items exact (source: go/api/internal/user/user_test.go, lines 474-491)

Exactness is the point. The etag assertion proves the body bytes byte for byte, because the etag is the sha256 of those bytes, and the cursor assertion proves the encoding down to the missing padding. The harness around it injects everything time-dependent: a fake clock advanced by hand, an id source that hands out the vector's uuids, and a fake hasher standing in for the argon2 the authn chapter brings. Nothing sleeps, nothing dials, every response comes out of a recorder.

The same discipline covers the rest of the slice: register 201 and 409, the 422 details array, unknown and immutable patch members, the stale If-Match 412, the 25-user default-limit walk, and the ownership grid. When a vector changes the code is wrong; when the code changes an output byte a vector fails. Drift has nowhere to hide.



drift means the code is wrong, never the vector

figure 121: a golden test: the vector file drives the fixture and judges the recorder

sources: RFC 7386, JSON Merge Patch, and RFC 9110, HTTP Semantics sections 8.8.3 and 13.1.1 on etags and conditional requests, both at rfc-editor.org; pkg.go.dev/encoding/json/v2 for RejectUnknownMembers and

omitzero in go 1.27; the keyset-versus-offset argument follows use-the-index-luke.com on offset paging. Accessed 2026-09-25. Verified by `go/api/internal/user` tests under `go test` and `go vet`, race detector on.

21. authn

Authentication answers one question, who is calling, and this chapter builds every piece of the answer: argon2id password storage with parameters carried in the hash string, a login path flattened so unknown emails and wrong passwords cost the same, server-side sessions behind a store port, the cookie that carries the refresh token, HS256 and EdDSA json web tokens built from cryptographic primitives rather than a library, rotation with reuse detection on the refresh family, and the injected-clock test discipline that keeps all of it under a second.

21.1. password storage with argon2id

Passwords are never stored, only proofs that cost an attacker real time and memory to reproduce. Argon2id is the memory-hard answer: it fills and re-reads a block of memory a set number of times, so a gpu farm cannot parallelize guesses the way it parallelizes sha256, and the parameter set travels inside the encoded string. The tiers are named values, not magic numbers:

```
type Params struct {
  Time    uint32 // passes over memory
  Memory  uint32 // KiB of memory each pass touches
  Threads uint8  // parallel lanes
  KeyLen  uint32 // derived key bytes
}

// DefaultParams is the issue tier, the OWASP-recommended argon2id
// configuration for interactive logins: 19 MiB, two passes, one lane.
var DefaultParams = Params{Time: 2, Memory: 19456, Threads: 1, KeyLen: 32}

// TestParams is the reduced test tier, a named constant so tests stay
// honest about trading cost for speed: 8 MiB and one pass keep each
// hash in the tens of milliseconds while exercising the same code
// path. Production tier is asserted structurally, never run in tests.
var TestParams = Params{Time: 1, Memory: 8 * 1024, Threads: 1, KeyLen: 32}

// PasswordHasher is the hashing port: the user package's register
// route takes it as a function field, this package provides the
// argon2id implementation, and tests provide fakes.
type PasswordHasher interface {
  Hash(password string) (string, error)
  Verify(encoded, password string) bool
}
```

listing 143: the parameter tiers and the hashing port, one type for both (source: go/api/internal/authn/password.go, lines 22-45)

The issue tier follows the OWASP recommendation for interactive logins, 19 MiB and two passes. The test tier is deliberately named `TestParams`, a confession: 8 MiB and one pass keep every hash in the suite in the tens of milliseconds while exercising the identical code path. A structural test pins the production floor, memory at least 19456 KiB and time at least two passes, without ever running it, so the suite stays fast and the tier cannot silently decay.

Hashing mints a fresh 16-byte salt and renders one PHC-style string, version, parameters, salt, derived key, so verification replays the exact parameters the hash was written with, never today's defaults. Upgrading the tier re-hashes opportunistically on the next login, because the string says which tier wrote it:

```
func (a *Argon2) Hash(password string) (string, error) {
  salt := make([]byte, 16)
  if _, err := rand.Read(salt); err ≠ nil {
    return "", fmt.Errorf("authn: salt: %w", err)
  }
  key := argon2.IDKey([]byte(password), salt, a.Params.Time,
    a.Params.Memory, a.Params.Threads, a.Params.KeyLen)
  return encode(a.Params, salt, key), nil
}
```

fragment, illustrative, not executed

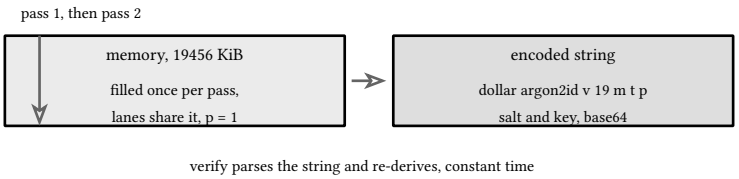


figure 122: argon2id: lanes fill a memory block, passes re-read it, the salt and params ride along

Argon2id is worth naming as a construction, because it is a schedule over a primitive, not a new primitive. Everything it hashes, it hashes with blake2b: the first block is the compression of the parameters, the salt, and the output length, each lane then fills its strip of the memory array by compressing the previous block with a referenced block from an earlier lane, and each additional pass re-reads and re-compresses the whole array in a different order. The memory hardness is the schedule's property, an attacker who shrinks the array cannot answer the compression chain, and the gpu resistance is the lane structure forcing the same memory traffic a cpu pays. The x/crypto function is therefore a pure function of stdlib-shaped parts, and its `IDKey` call is the entire contract the chapter's hashing port wraps.

21.2. login and timing flattening

A login endpoint that answers fast for unknown emails and slowly for wrong passwords hands out an account census to anyone with a stopwatch. The flattening fix sits in the unknown-email branch: run the same argon2 verification the wrong-password path runs, against a dummy hash computed once at service construction, discard the result, and answer the identical bytes:

```
func (s *Service) login(w http.ResponseWriter, r *http.Request) error {
  req, err := httpx.Bind[loginRequest](r, json.RejectUnknownMembers(true))
  if err ≠ nil {
    return err
  }
  if req.Email == "" || req.Password == "" {
    return httpx.Fail(httpx.ValidationFailed, "validation failed", "email is required", "password is
required")
  }
  u, err := s.d.Users.GetByEmail(r.Context(), user.NormalizeEmail(req.Email))
  if err ≠ nil {
    // Unknown email: verify against the dummy hash so the work,
    // and therefore the latency profile, matches the wrong
    // password path, then answer the identical bytes.
    s.d.Hasher.Verify(s.dummyHash, req.Password)
    return httpx.Fail(httpx.Unauthorized, "invalid credentials")
  }
  if !s.d.Hasher.Verify(u.PasswordHash, req.Password) {
    return httpx.Fail(httpx.Unauthorized, "invalid credentials")
  }
  return s.issue(w, r, u.ID)
}
```

listing 144: both failure branches answer the same 401 after the same work (source: go/api/internal/authn/handlers.go, lines 146-166)

Both branches return the same envelope, code, message, and no distinguishing header, and the rate limiter of chapter 26 will add the same headers to both. The only observable difference left is network jitter. The success path runs on through `issue`, which mints the session, the refresh token, the cookie, and the access token in one place, so login stays readable as two facts: prove, then issue.

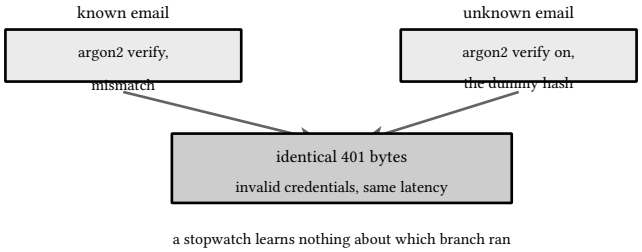


figure 123: the timing-equalized login: unknown email pays what a wrong password pays

21.3. server-side sessions

A session is the server's own record of one login: an id, the user, the expiry, a revoked flag, and the sha256 of the current refresh token. The token itself is never stored, only its hash, so a stolen store dump cannot be replayed against the live service. The store port mirrors the user package's: an interface frozen here, the in-memory implementation in this chapter, the engine implementation in chapter 23:

```go
// SessionStore is the session persistence port, the counterpart of
// the user package's Store. Chapter 21 ships the in-memory
// implementation below; chapter 23 implements it over the engine. The
// interface is frozen here for both.
type SessionStore interface {
  // Create stores a new session with its initial refresh hash.
  Create(ctx context.Context, s Session) error
  // Get fetches by id, ErrNoSuchSession otherwise.
  Get(ctx context.Context, id string) (Session, error)
  // GetByRefresh resolves the session a currently-valid refresh
  // token belongs to. Superseded and unknown tokens answer
  // ErrUnknownRefresh; whether the session is still alive is the
  // caller's time policy.
  GetByRefresh(ctx context.Context, hash [32]byte) (Session, error)
  // Rotate swaps the session's current refresh token for next and
  // extends its expiry. Presenting a superseded token revokes the
  // whole family and answers ErrSessionRevoked: the rotated token
  // dies with the replayed one. Unknown tokens answer
  // ErrUnknownRefresh.
  Rotate(ctx context.Context, current, next [32]byte, expiresAt time.Time) (Session, error)
  // Revoke kills a session and its whole refresh family.
  Revoke(ctx context.Context, id string) error
}
```

listing 145: the session store port: five verbs, three errors, frozen for both stores (source: go/api/internal/authn/session.go, lines 47-69)

The lifetime rules stay with the caller, on purpose. The store knows structural state, which token is current, which session is revoked; the service knows the time and applies the policy, a session counts as alive while not revoked and not expired. That split keeps the engine implementation free of clock reads and makes every expiry test pure arithmetic on an injected clock:

```go
type Session struct {
  ID          string
  UserID      string
  RefreshHash [32]byte
  ExpiresAt   time.Time
  Revoked     bool
}

func HashToken(token string) [32]byte {
  return sha256.Sum256([]byte(token))
}
```

fragment, illustrative, not executed

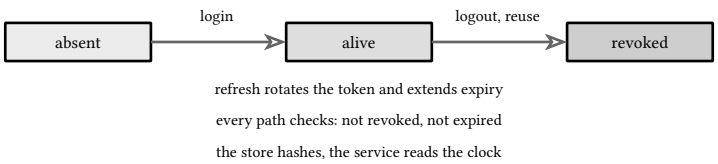


figure 124: the session lifecycle: login creates, refresh extends, logout or reuse revokes

21.4. cookie mechanics

The browser carries the refresh token in a cookie, and the cookie's attributes are its entire protection. Path scopes it to the api tree so it never rides on requests to other origins hosted beside the service. HttpOnly keeps scripts

from reading it, so an injected snippet cannot exfiltrate what it cannot see. Secure keeps it off plain http. SameSite Lax blocks the cookie on cross-site posts while allowing ordinary top-level navigation, except a top-level post within two minutes of the cookie's creation, the 6265bis lax-allowing-unsafe window:

```
func SetSessionCookie(w http.ResponseWriter, token string) {
  http.SetCookie(w, &http.Cookie{
    Name:     CookieName,
    Value:    token,
    Path:     "/api",
    HttpOnly: true,
    Secure:   true,
    SameSite: http.SameSiteLaxMode,
  })
}

// ClearSessionCookie expires the cookie: Max-Age=0 rendered exactly,
```

listing 146: the session cookie: four attributes, one line of go (source: go/api/internal/authn/cookie.go, lines 16-27)

Logout clears it with Max-Age 0, rendered exactly. Go's cookie writer emits a delete only for a negative Max-Age, which serializes as Max-Age 0, the byte form every browser honors, and the attributes stay set so the deletion overwrites the exact cookie it replaces. The tests assert the whole Set-Cookie line byte for byte against the frozen vector, attributes, order, and semicolons included:

```
func ClearSessionCookie(w http.ResponseWriter) {
  http.SetCookie(w, &http.Cookie{
    Name: CookieName, Value: "",
    Path: "/api", MaxAge: -1,
    HttpOnly: true, Secure: true,
    SameSite: http.SameSiteLaxMode,
  })
}
```

fragment, illustrative, not executed

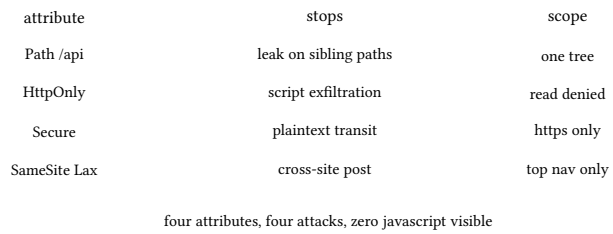


figure 125: the cookie attribute matrix: what each one stops

21.5. JWT from scratch with HMAC

A json web token is three base64url parts, header, claims, signature, and the whole format fits in two functions of cryptographic primitives. HMAC-SHA256 signs the first two segments with a shared secret, verification recomputes and compares in constant time, and the claim rules run after the signature, never before:

```
func SignHS256(secret []byte, c Claims) (string, error) {
  hs, ps, err := segments(header{Alg: algHS256, Typ: typJWT}, c)
  if err != nil {
    return "", err
  }
  mac := hmac.New(sha256.New, secret)
  mac.Write([]byte(hs + "." + ps))
  sig := base64.RawURLEncoding.EncodeToString(mac.Sum(nil))
  return hs + "." + ps + "." + sig, nil
}

// ParseHS256 checks signature, algorithm, issuer, and expiry against
// now, and answers the claims. The signature comparison is constant
// time.
func ParseHS256(token string, secret []byte, now time.Time) (Claims, error) {
  c, err := parse(token, algHS256, func(msg []byte, sig []byte) bool {
    mac := hmac.New(sha256.New, secret)
    mac.Write(msg)
    return subtle.ConstantTimeCompare(mac.Sum(nil), sig) == 1
  })
  if err != nil {
    return Claims{}, err
  }
  return check(c, now)
}
```

listing 147: sign and verify HS256, no library, two primitives (source: go/api/internal/authn/jwt.go, lines 81-105)

The claims struct is small and alphabetical by declaration, exp, iat, iss, sid, sub, which makes the serialized form deterministic, the same bytes for the same facts, so the golden test reproduces the frozen vector's token exactly, signature included. There is no roles claim on purpose: authorization reads the store, and the sid claim makes every outstanding token revocable by killing one session row. The parser refuses what the classics try: an alg none header never reaches the payload, a tampered claim fails the constant-time compare, the wrong secret fails it too, and an expired or foreign-iss token fails the claim checks with its own errors.

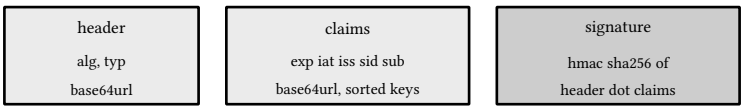


figure 126: three segments, the signature covers exactly the first two

21.6. JWT with EdDSA

HMAC has one secret and both sides hold it, so whoever can verify tokens can also mint them. In one process that is fine, and it is why the issued lane stays HS256: one signer, one verifier, one binary. The moment verification spreads, a public-key scheme earns its complexity. EdDSA splits the pair, the private key signs, the public key verifies, and the verification side provably cannot forge:

```
// SignEdDSA signs claims with an Ed25519 private key, the asymmetric
// variant: verifiers hold only the public key, so the party checking
// tokens can never mint them.
func SignEdDSA(priv ed25519.PrivateKey, c Claims) (string, error) {
  hs, ps, err := segments(header{Alg: algEdDSA, Typ: typJWT}, c)
  if err != nil {
    return "", err
  }
  sig := ed25519.Sign(priv, []byte(hs+"."+ps))
  return hs + "." + ps + "." + base64.RawURLEncoding.EncodeToString(sig), nil
}

// ParseEdDSA verifies an EdDSA token against the public key and the
// same claim rules as ParseHS256.
func ParseEdDSA(token string, pub ed25519.PublicKey, now time.Time) (Claims, error) {
  c, err := parse(token, algEdDSA, func(msg []byte, sig []byte) bool {
    return ed25519.Verify(pub, msg, sig)
  })
  if err != nil {
    return Claims{}, err
  }
  return check(c, now)
}
```

listing 148: the same envelope, an asymmetric signature (source: go/api/internal/authn/jwt.go, lines 107-129)

The two parsers cross-check each other in the tests: an HS256 parser refuses the EdDSA token on the alg pin, the EdDSA parser refuses the HS256 token the same way, and the alg confusion attack, present a token signed with the algorithm the attacker prefers, dies at the header. The claim checks are shared, so issuer, expiry, and the constant-time habits do not fork between algorithms.

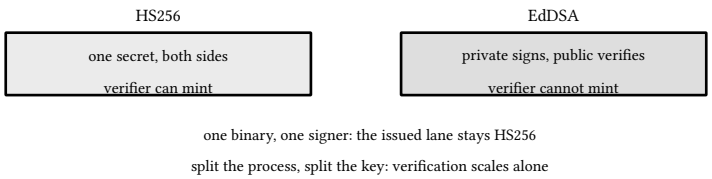


figure 127: shared secret versus split key: who can mint, who can check

21.7. the refresh rotation and reuse detection

Access tokens live fifteen minutes, the refresh token is the long-lived credential, and it rotates on every use. Rotation means the store remembers the token it just retired, marked superseded, because presenting a superseded token is the strongest theft signal the design gets: the legitimate holder moved on, so someone else is replaying. The answer is family revocation, the replayed token and its successor die together:

```
func (m *MemSessions) Rotate(_ context.Context, current, next [32]byte, expiresAt time.Time) (Session,
error) {
  m.mu.Lock()
  defer m.mu.Unlock()
  rec, ok := m.tokens[current]
  if !ok {
    return Session{}, ErrUnknownRefresh
  }
  s := m.sessions[rec.sid]
  if rec.superseded || s.Revoked {
    s.Revoked = true
    return *s, ErrSessionRevoked
  }
  m.tokens[current] = tokenRec{sid: s.ID, superseded: true}
  m.tokens[next] = tokenRec{sid: s.ID}
  s.RefreshHash = next
  s.ExpiresAt = expiresAt
  return *s, nil
}
```

listing 149: rotate: superseded tokens revoke the family, unknown ones miss (source: go/api/internal/authn/session.go, lines 125-142)

The vector pair pins the whole story. Refresh at plus twenty returns a new cookie and a new access token. Replay the old cookie at plus twenty-one and both die: the replay answers 401 session revoked, and the successor, still perfectly formed, answers 401 session revoked one second later. An attacker who stole the token before the holder refreshed gets one use, then loses the family along with the victim.

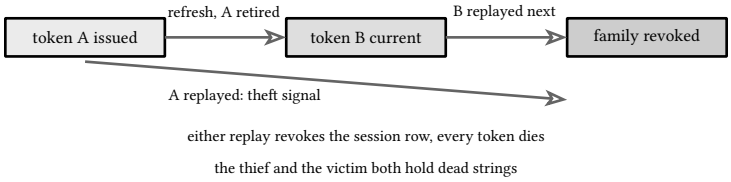


figure 128: the refresh family: rotation forks, reuse revokes everything

21.8. testing auth without walls

Every time-dependent behavior here is tested as math. The clock is a struct advanced by hand, so token expiry is one addition. The id source hands out the vector's uuids, so the sid claim lands exact. The random reader is a fixed byte buffer, so the refresh token and the Set-Cookie line reproduce the frozen vector byte for byte, and a counting hasher proves the dummy verify ran on the unknown-email path:

```
  // Unknown email: same request id, identical bytes, and the argon2
  // dummy verify still ran.
  before := f.hasher.verifyCalls
  unknown := f.do(t, "POST", "/api/auth/login", `{"email":"nobody@example.org","password":"whatever
twelve"}`, map[string]string{
    "Content-Type": "application/json",
    "X-Request-ID": "01a0f5d5-9778-7448-898e-3562a3eee849",
  })
  // The wrong-password call above also had a request id injected to
  // make the two bodies comparable: redo it with the same id.
  wrong2 := f.do(t, "POST", "/api/auth/login", vecBody(t, v), map[string]string{
    "Content-Type": "application/json",
    "X-Request-ID": "01a0f5d5-9778-7448-898e-3562a3eee849",
  })
  if unknown.Code ≠ 401 || unknown.Body.String() ≠ wrong2.Body.String() {
    t.Errorf("unknown email body %q differs from wrong password %q", unknown.Body.String(),
wrong2.Body.String())
  }
  if f.hasher.verifyCalls ≠ before+2 {
    t.Errorf("verify calls = %d after two failures, want %d (dummy flatten on the unknown email)",
f.hasher.verifyCalls, before+2)
  }
```

listing 150: identical bytes asserted, the flatten count asserted (source: go/api/internal/authn/authn_test.go, lines 313-331)

No test sleeps, no test opens a socket, and the argon2 tier keeps the whole suite under a second. The concurrency test rotates one token from eight goroutines and asserts exactly one winner, which the race detector reads as one clean critical section. When the vectors change the code is wrong, and nothing in this chapter has a second source of truth to drift toward.

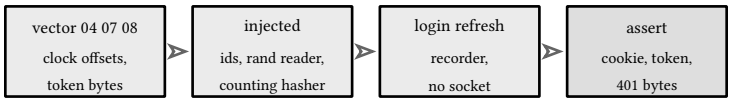


figure 129: an auth golden test: fixed inputs in, frozen bytes out

sources: RFC 9106, Argon2, for the memory-hard function and parameter format, and RFC 7519 plus RFC 8032 for jwt structure and EdDSA at rfc-editor.org; cheatsheetseries.owasp.com, Password Storage Cheat Sheet, for the argon2id tier recommendation; RFC 6265bis for cookie attributes and Max-Age semantics; pkg.go.dev/golang.org/x/crypto/ argon2. Accessed 2026-09-25. Verified by `go/api/internal/authn` tests under `go test` and `go vet`, race detector on.

22. authz and rbac

Authentication settled who is calling; authorization decides what that caller may do, and this chapter builds the whole answer: the distinction between the two questions, roles as stored data with a grant and revoke surface, policies as plain functions, the 401 versus 403 discipline, object-level authorization on the routes that carry ids, a deny-by-default guard with a route audit that keeps every pattern honest, and the role by endpoint matrix test that pins the whole grid in one table.

22.1. authentication versus authorization

Two questions, two layers, two failure codes. Authentication runs first and installs an actor or does not; authorization reads that actor against the route and the object. The service's wiring order makes the sequence physical: the identity middleware wraps the guard, the guard wraps the route table, so by the time a policy runs, the actor question is settled and a policy never parses a token, checks a signature, or reads a cookie:

```
// Anyone is the open route policy: registration and login, the routes
// whose whole job is to take credentials from strangers.
func Anyone(*http.Request, string) error { return nil }

// Authenticated requires a resolved actor and nothing more: the
// caller proved who they are, and the route holds no object.
func Authenticated(r *http.Request, _ string) error {
  if _, ok := user.ActorFrom(r); !ok {
    return httpx.Fail(httpx.Unauthorized, "authentication required")
  }
  return nil
}
```

listing 151: the two cheapest policies: open, and any-actor (source: go/api/internal/authz/policy.go, lines 22-33)

Conflating the two questions produces the classic broken answer, a 403 for an anonymous request. A caller that never proved anything has no authorization to deny, only an authentication demand, and the distinction is observable: 401 means try again with credentials, 403 means do not bother, this identity will never pass. Every layer in this chapter keeps that boundary, and the wiring order enforces it before any handler runs.

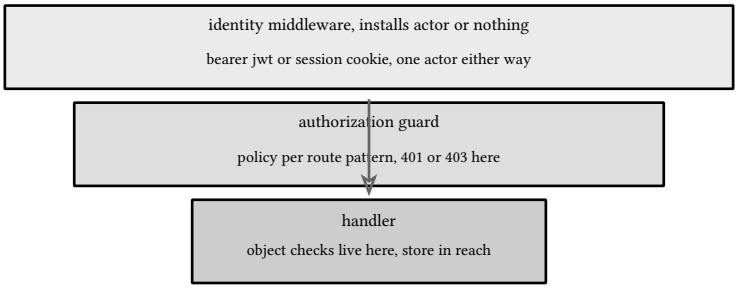


figure 130: two layers in wiring order: identity outside, guard inside, handler last

22.2. roles and the grant table

Roles are rows, not magic. Each account carries a set, the bootstrap rule grants the first registration admin, and the admin routes add and remove one role at a time, answering the resulting set. The invariant that matters is never zero admins, and it lives in the store where both mutating verbs run it, grant and revoke alike:

```
// GrantRole implements Store.
func (m *MemStore) GrantRole(_ context.Context, id, role string) ([]string, error) {
  m.mu.Lock()
  defer m.mu.Unlock()
  u, ok := m.byID[id]
  if !ok {
    return nil, ErrNotFound
  }
  if !hasRole(u.Roles, role) {
    u.Roles = append(u.Roles, role)
    sort.Strings(u.Roles)
  }
  if err := m.adminsRemain(u); err ≠ nil {
    return nil, err
  }
  m.byID[id] = u
  return append([]string(nil), u.Roles...), nil
}
```

listing 152: grant adds a role, the admin invariant gates both verbs (source: go/api/internal/user/store.go, lines 208-225)

A revoke computes the post state, checks that some admin still stands across the whole table, and only then persists, so the rejected demotion leaves no trace. The frozen vector walks the story: demoting the only admin answers 409 conflict, last admin, granting admin to a second user succeeds, and revoking that second admin then succeeds too, because the first one is standing again. The bootstrap rule makes the lockout unreachable in practice, the invariant makes it unreachable in theory.

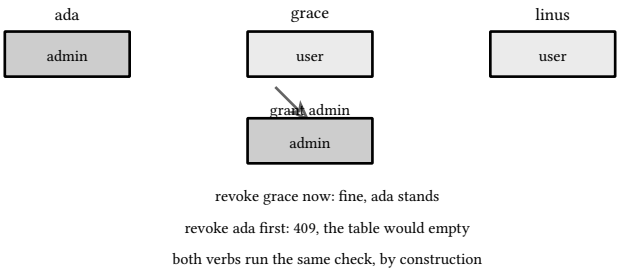


figure 131: the grant table: one column per user, the admin row must never empty

22.3. policies as functions

A policy is one function from request plus pattern to verdict: nil allows, a contract error denies. The whole surface composes from four of them, open, any-actor, a required role, an owner-or-admin check, and each is short enough to read in one breath:

```
func RequireOwner(param string) Policy {
  return func(r *http.Request, pattern string) error {
    a, ok := user.ActorFrom(r)
    if !ok {
      return httpx.Fail(httpx.Unauthorized, "authentication required")
    }
    id, found := PathParam(r.URL.Path, pattern, param)
    if !found || (a.UserID ≠ id && !a.Admin()) {
      return httpx.Fail(httpx.Forbidden, "not the owner")
    }
    return nil
  }
}
```

listing 153: the object policy: own the id or hold admin (source: go/api/internal/authz/policy.go, lines 54-66)

The pattern rides along as an argument because of one subtlety: the guard wrapper runs before the mux dispatches, and the request's path values are filled only at dispatch. So the policy extracts the wildcard itself, walking path and pattern in lockstep, the same segment match the mux performs. Plain functions beat a rules dsl or annotation mining at this size because the compiler type-checks the table, the audit test can walk it, and a new rule is a function next to its three siblings.

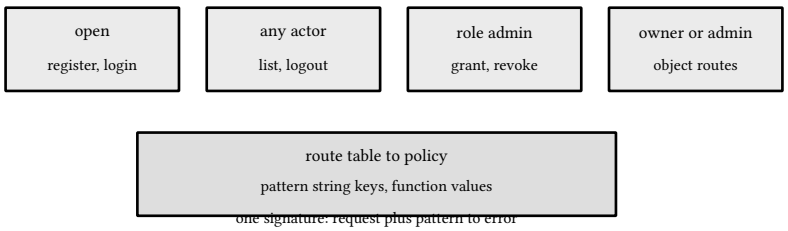


figure 132: four policies, one signature, the table maps routes onto them

22.4. 401 versus 403

The boundary lives in one place per policy: the actor check answers 401, the rule check answers 403, and nothing in between exists:

```
// RequireRole builds the policy that demands one role. The 401 versus
// 403 discipline lives here: no actor is an authentication failure, a
// present actor without the role is an authorization failure.
func RequireRole(role string) Policy {
  return func(r *http.Request, _ string) error {
    a, ok := user.ActorFrom(r)
    if !ok {
      return httpx.Fail(httpx.Unauthorized, "authentication required")
    }
    if !a.HasRole(role) {
      return httpx.Fail(httpx.Forbidden, "insufficient role")
    }
    return nil
  }
}
```

listing 154: the discipline: no actor is 401, wrong actor is 403 (source: go/api/internal/authz/policy.go, lines 35-49)

Read it as a state machine. Anonymous gets 401 with the same message every time, authentication required, no hint about the route's existence or its rules. A present actor that fails the rule gets 403 with the rule's own words, insufficient role or not the owner, which is a statement about the actor, never about the resource. The one

deliberate blur is existential: this service answers 404 for an unknown id before 403 for a stranger's id, which reveals that an id exists to anyone authenticated. Hiding it instead means answering 404 for both, and the tradeoff is real, disclosure against debuggability. What is never acceptable is mixing the codes by accident, a 403 for a missing Authorization header, a 401 for a demoted admin.

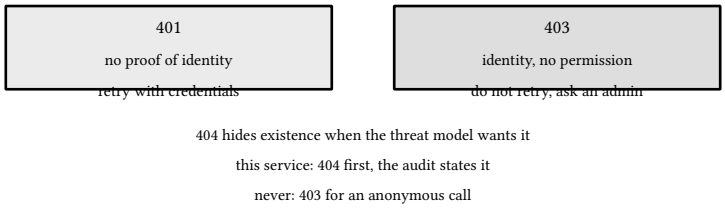


figure 133: the two failure codes, and the one deliberate blur

22.5. object-level authorization

Route rules cannot guard a route whose danger lives in the path parameter. `GET /api/users/` followed by someone else's id is the IDOR class, insecure direct object reference, and it survives every route-level check because the route is the same for everyone. The check must happen after the fetch, where the code knows whether the object exists and who owns it, which is why the object routes carry only the any-actor policy in the table and the ownership rule runs in the handler:

```
func TestIDORObjectRoutes(t *testing.T) {
  f := newFixture(t)
  seed(t, f)
  adaPath := "/api/users/" + seeded["ada"]
  etag := map[string]string{"If-Match": etagOf(t, f, seeded["ada"])}
  for _, tc := range []struct {
    method, body string
    hdr          map[string]string
  }{
    {"GET", "", nil},
    {"PATCH", `{"display_name":"linus hack"}`, etag},
    {"DELETE", "", nil},
  } {
    rec := f.do(t, "linus", tc.method, adaPath, tc.body, tc.hdr)
    if rec.Code ≠ 403 {
      t.Errorf("stranger %s on ada: status %d, want 403", tc.method, rec.Code)
    }
  }
  rec := f.do(t, "linus", "GET", "/api/users/01a0f5d5-ffff-7000-8000-000000000fff", "", nil)
  if rec.Code ≠ 404 {
    t.Errorf("unknown id status = %d, want 404", rec.Code)
  }
}
```

listing 155: the IDOR test: stranger on every object verb, unknown id 404 (source: go/api/internal/authz/authz_test.go, lines 239-261)

The test pins both edges. Linus touching ada's id gets 403 on read, write, and delete, whatever etag he presents, and an unknown id gets 404 for everyone, the ordering the last section discussed. Putting the owner check in the guard instead would flatten unknown and forbidden into one 403, leak existence, and duplicate the store read the handler needs anyway, so the layering is not style, it is the only arrangement that answers all three questions correctly.

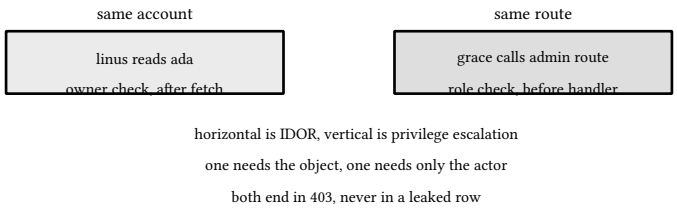


figure 134: horizontal versus vertical: two axes of privilege, two different checks

22.6. deny by default with a route audit

One table maps every registered pattern to its policy. A guard wrapper sits in the middleware stack, resolves the pattern the mux would match, and refuses any pattern missing from the table before a handler byte runs:

```
func Guard(mux *http.ServeMux, table map[string]Policy) func(http.Handler) http.Handler {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      _, pattern := mux.Handler(r)
      if pattern == "" {
        next.ServeHTTP(w, r)
        return
      }
      policy, ok := table[pattern]
      if !ok {
        httpx.Error(w, r, httpx.Fail(httpx.Forbidden, "no policy for route"))
        return
      }
      deny(w, r, policy(r, pattern), next)
    })
  }
}
```

listing 156: the guard: no table row, no response (source: go/api/internal/authz/guard.go, lines 21-37)

The audit test closes the loop from the other side: every table row must resolve to a registered route with exactly that pattern, and a hand-written inventory of the contract's routes must be present, twelve here and fourteen once chapter 23 appends the reports pair, so a route cannot exist unaudited, a row cannot point at nothing, and a new route that forgets its policy becomes instantly, loudly unreachable instead of quietly open. Requests that match no pattern at all fall through to the kernel fallback, because 404 and 405 are routing answers, not authorization verdicts:

```
for pattern := range RoutePolicies {
  method, path, _ := strings.Cut(pattern, " ")
  req := httptest.NewRequest(method, wildcardFree(path), nil)
  _, matched := f.mux.Handler(req)
  if matched ≠ pattern {
    t.Errorf("table row %q resolves to %q", pattern, matched)
  }
}
```

fragment, illustrative, not executed

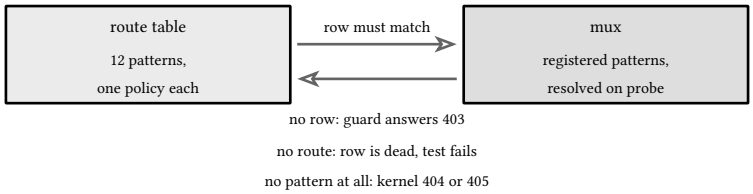


figure 135: the audit: table and mux face each other, both directions checked

22.7. the rbac matrix test

The grid is the proof: every route times every actor kind, anonymous, regular user, admin, stranger on someone else's object, one expected status per cell, asserted in one table-driven test through the full guarded stack, the same path production traffic takes:

```
cases := []struct {
  name   string
  actor  string
  method string
  path   string
  body   string
  hdr    map[string]string
  want   int
}{
  {"anon list", "", "GET", "/api/users", "", nil, 401},
  {"user list", "grace", "GET", "/api/users", "", nil, 200},
  {"admin list", "admin", "GET", "/api/users", "", nil, 200},
  {"anon get", "", "GET", gracePath, "", nil, 401},
  {"stranger get", "linus", "GET", gracePath, "", nil, 403},
  {"owner get", "grace", "GET", gracePath, "", nil, 200},
  {"admin get", "admin", "GET", gracePath, "", nil, 200},
  {"stranger patch", "linus", "PATCH", gracePath, `{"display_name":"x"}`, etagHdr, 403},
  {"owner patch missing if-match", "grace", "PATCH", gracePath, `{"display_name":"grace m"}`, nil, 412},
  {"owner patch", "grace", "PATCH", gracePath, `{"display_name":"grace m"}`, etagHdr, 200},
  {"anon delete", "", "DELETE", gracePath, "", nil, 401},
  {"stranger delete", "linus", "DELETE", gracePath, "", nil, 403},
  {"owner delete", "grace", "DELETE", gracePath, "", nil, 204},
```

listing 157: the matrix rows: actor times route, one status each (source: go/api/internal/authz/authz_test.go, lines 186-207)

Each row is a cell with a name that fails loudly. The grid runs the vector's last-admin chain alongside, so the 409 and the two 200s around it are pinned by frozen bytes, not just by intent. Adding a route costs three lines, one table row per interesting actor, and the audit plus the matrix together answer the two questions reviewers actually ask: is every route covered, and does every covered route behave for every kind of caller.

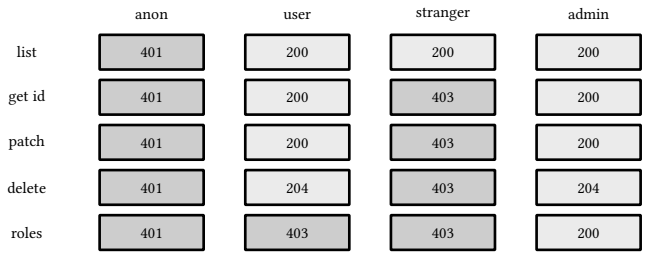


figure 136: the matrix: routes one way, actors the other, statuses in the cells

sources: OWASP, Insecure Direct Object Reference Prevention Cheat Sheet and Access Control Cheat Sheet, at cheatsheetseries.owasp.com, for the IDOR class and the deny-by-default guidance; RFC 9110 sections 15.5.1 to 15.5.4 on 401, 403, and 404 semantics; the go 1.27 net/http ServeMux documentation for pattern matching and

request-scoped path values. Accessed 2026-09-25. Verified by `go/api/internal/authz` tests under `go test` and `go vet`, race detector on.

23. store

The service has run so far on in-memory maps, which tests love and restarts hate. The earlier answer was a translated sqlite driver, the module's one community dependency, and the dependency ruling retired it: whatever the platform lacks gets built from the platform's own primitives. This chapter owns the machinery outright. A write-ahead log of CRC32C framed records, replay that recovers state and tolerates a torn tail, transaction markers that make groups atomic, snapshot compaction whose body carries an append-only version list, and a hand-rolled database/sql driver, goapiwal, taught as the bridge to the standard interface while the service paths call the engine directly, a split the chapter states where it matters. Above the engine sit the documented key scheme, keyset scans over sorted keys, analytics aggregated in plain loops, and the isolation lessons restated over locks the module built itself.

23.1. database/sql from first principles

database/sql is a pool and a protocol. It is not itself a database: a program opens a driver by name, gets a pool of connections it does not schedule itself, and executes statements whose arguments are values, never string concatenation. The surface an implementer owes is small, driver.Driver handing out driver.Conn, each conn preparing driver.Stmt values, and anything that satisfies that shape can sit under sql.DB. The engine satisfies it through a driver registered under the name goapiwal, sync.Once guarding the double registration a second import could cause, and its DSN is a plain file path. The driver's tests go through database/sql itself, sql.Open, prepared statements, a BeginTx through both exits under the deferred rollback, two pools on one engine.

The driver is the teaching bridge, and the split is stated plainly. The service's store surfaces call the engine directly and skip database/sql, because a pool adds nothing when the write side already serializes and every conn would borrow the same process-wide engine for the path. The driver exists because the standard interface is the ecosystem's common tongue, and because implementing it is the proof that the engine is a database by the definition go programs use. Its statement set is fixed, four shapes over the op vocabulary, and every argument is a value:

```
// the whole dialect the driver speaks
// "GET ?"      one key
// "PUT ? ?"    key and value
// "DEL ?"      one key
// "SCAN ? ?"  range bounds
```

fragment, illustrative, not executed

```
// Prepare compiles one of the four op strings. Anything else is a
// programming error and fails loud.
func (c *walConn) Prepare(query string) (driver.Stmt, error) {
  switch query {
  case qGet:
    return &walStmt{conn: c, query: query, args: 1, kind: stmtGet}, nil
  case qPut:
    return &walStmt{conn: c, query: query, args: 2, kind: stmtPut}, nil
  case qDel:
    return &walStmt{conn: c, query: query, args: 1, kind: stmtDel}, nil
  case qScan:
    return &walStmt{conn: c, query: query, args: 2, kind: stmtScan}, nil
  }
  return nil, fmt.Errorf("goapiwal: unknown query %q", query)
}
```

listing 158: prepare recognizes the four op strings and nothing else parses (source: go/api/internal/store/engine/driver.go, lines 103-117)

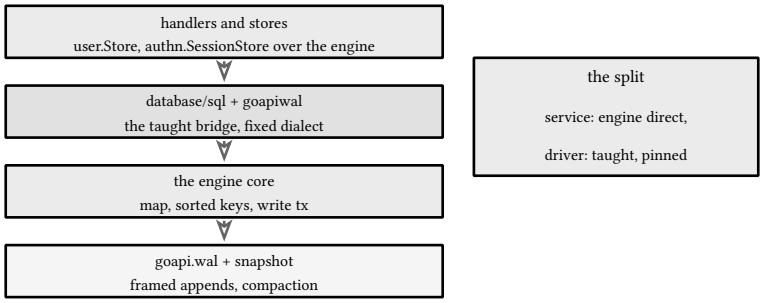


figure 137: the stack the store sits on: standard interface, owned engine, wal plus snapshot, no cgo

23.2. migrations test driven

Schema changes are code changes, so they ship as records, and the history is append-only by contract: a shipped version is frozen, a correction is a new numbered version, and no record is ever edited. The registry is no longer a table the database keeps about itself. It is the version list at the head of the snapshot body, and appending a version is a compaction: the registry and the state it describes land together in one snapshot file, written to a temporary, fsynced, and renamed into place. Version records never ride the WAL, they live only in the snapshot body, and the fsync the commit path skips is spent here, at rotation. A snapshot that arrives torn despite the fsync is not trusted: the open sequence falls back to WAL-only recovery and the pre-compaction state is gone, the one loss the design accepts and the torn-snapshot test documents. Migrate applies each pending version at most once, so a version that fails halfway leaves nothing durable and can be retried: the in-memory registry takes the label only once the snapshot holds it. There is no sql to embed anymore: the binary carries its history as code and appends it to the store on migration.

This wave ships three versions. The first marks the base key scheme, users, grants, sessions with their refresh token history, idempotency keys, events, reports, one record for the whole vocabulary. The second marks the version field inside user values, and stands as its own record because it is the substrate the next chapter's optimistic concurrency stands on. The third marks the idempotency header keys the concurrency chapter fills. Each demonstrates the same discipline on its own day: a new fact is a new record. The test drives the runner from an empty directory, proves all three records applied in order, re-runs, and proves nothing applies twice, then closes and reopens the store to prove the registry came back with it.

```go
// Migrate applies every pending registry label exactly once. Each
// label is appended as a version record and compacts, so the registry
// and the state it describes land in one snapshot file and a crashed
// append leaves either the old file or the new one, never half.
func (d *DB) Migrate(ctx context.Context) error {
  _ = ctx
  applied := d.Eng.Versions()
  for _, name := range migrationNames {
    if slices.Contains(applied, name) {
      continue
    }
    if err := d.Eng.AppendVersion(name); err ≠ nil {
      return fmt.Errorf("store: migrate %s: %w", name, err)
    }
  }
  return nil
}
```

listing 159: one version, one compaction, one append to the record list, applied at most once (source: go/api/internal/store/store.go, lines 55-71)

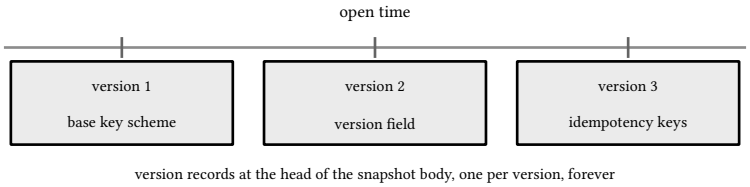


figure 138: the version list only grows right

23.3. queries with context every time

Every method takes a context, because the ports froze that shape and the request's own context is what every handler passes down. What the engine does with it is stated in the code, and the statement is the design: nothing. The surface accepts the context and hands it to the callback, and its own steps go without, the comment at the wrapper naming the trade, the engine has no cancellation point, transactions run short, and open says the same about recovery, synchronous and uncancellable by design. A dead context therefore never tears a frame mid-write, because there is no interruption point to cancel, and the work a canceled request started still lands, bounded by how short every transaction is kept. The timeout middleware keeps the other half of the bargain, stopping the response path while the store finishes a step that was always going to be quick.

The adapters implement the two ports the earlier chapters froze, user Store and authn SessionStore, error for error, so no handler can tell the engine from the in-memory maps the tests grew up on. A read runs inside a read tx: take the read lock, clone the key space into an immutable snapshot, release the lock, then every lookup is a map hit on a key the scheme names and a decode of deterministic json/v2 bytes into a domain value that carries no engine types. The clone is the stated price of the design: the engine copies the map and the sorted key slice per read, shares the value slices because puts replace and never mutate them, and at book scale, a map of a few hundred entries cloned per read, that is cheap enough to keep the engine plain. The scheme is documentation the keys enforce, user records under u:id: with the created_at inside as sign-flipped fixed-width hex milliseconds beside the id, a u:byid: index beside the u:email: uniqueness claim, grants under r:grant:, sessions under s:id: with their whole token history under t:hash:, events under e: with day and id inside the key so day order is key order, idempotency and report rows under idem: and rep:. The fixed-width pair inside the key is exactly the (created_at, id) pair the pagination cursor carries, so storage and keyset order share one notion of position. The one translation the contract cares about survives as an explicit existence check: the email key already taken maps to ErrConflict and nothing else, because that is the only store error a client ever caused.

```
// Get implements user.Store. The record and its roles come from one
// read snapshot: two snapshots could answer a user from one instant
// wearing a role set from another.
func (s *UserStore) Get(ctx context.Context, id string) (user.User, error) {
  var u user.User
  err := s.DB.WithReadTx(ctx, func(_ context.Context, snap *engine.Snapshot) error {
    var err error
    u, _, err = getUser(snap, id)
    if err ≠ nil {
      return err
    }
    u.Roles = rolesOf(snap, u.ID)
    return nil
  })
  if err ≠ nil {
    return user.User{}, err
  }
  return u, nil
}
```

listing 160: a read through the store surface: ctx in, one snapshot, key lookup, decode, roles (source: go/api/internal/store/engine_user.go, lines 186-204)

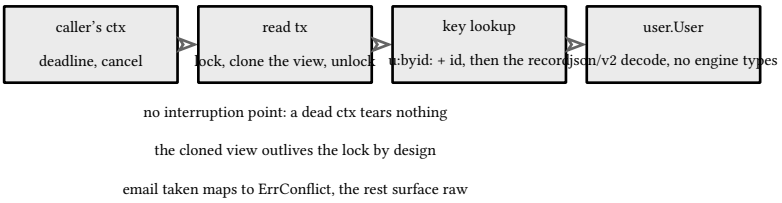


figure 139: one read's path from caller to view to domain value

23.4. the transaction idiom

One shape covers every write in the module: take the transaction, run, commit, and let the error exit discard. The engine's write tx gives the shape new bones. The write side takes the full lock, so exactly one writer holds the group at a time, and the function's ops accumulate into a pending group memory has never seen. Commit is append then mutate: the group's frames, the begin marker, the ops, the commit marker, are encoded into one buffer and written in one call, and only then does memory take the group's puts and deletes. The write reaches the OS but is not fsynced, the line sqlite's synchronous NORMAL draws, durable across application crashes, relaxed about power loss, with Sync as the operator's stronger call. Discard is the rollback: the wrapper defers it, so fn's error and a panic unwinding through both roll back, and after commit the same call is a no-op, the idiom the sqlite era taught, with nothing left to undo because nothing was appended until commit. The suite proves the unwind path with a test that fires a panicking fn and then writes again, a leaked lock would hang it forever. The engine has no cancellation point that could add a third exit, ctx is discarded on purpose, and every fn the module writes is short, straight-line staging over the locked engine.

The order is the guarantee. Frames first, memory second, means the log is always at least as current as memory, so a crash between the append and the mutation recovers to the committed state, and the inverse order would leave memory holding a state the log cannot rebuild, the exact failure recovery exists to prevent. The store wraps the shape twice, WithTx over the write tx and WithReadTx over the read view, and the session store's Rotate is still the hardest caller: on a superseded or revoked presentation the family revocation commits even though Rotate answers with an error, the revocation runs inside the write tx and the error is returned after it, so the family cannot be rolled back to life. The plain read keeps the older discipline: GetByRefresh answers a superseded hash exactly as an unknown one, ErrUnknownRefresh, because which of the two it is only matters to Rotate, which reacts by revoking.

```
func (t *WriteTx) Commit() error {
  if t.fatal {
    return errors.New("engine: transaction already finished")
  }
  defer t.finish()
  frames := make([]Record, 0, len(t.ops)+2)
  frames = append(frames, Record{Op: opTxBegin, Seq: t.seq})
  frames = append(frames, t.ops...)
  frames = append(frames, Record{Op: opTxCommit, Seq: t.seq})
  if err := t.e.wal.Append(frames...); err != nil {
    return err
  }
  for _, rec := range t.ops {
    switch rec.Op {
    case opPut:
      t.e.set(string(rec.Key), append([]byte(nil), rec.Value...))
    case opTombstone:
      t.e.unset(string(rec.Key))
    }
  }
  t.e.seq = t.seq
  return nil
}
```

listing 161: the commit order: the group's frames append in one write, then memory mutates (source: go/api/internal/store/engine/engine.go, lines 170-192)

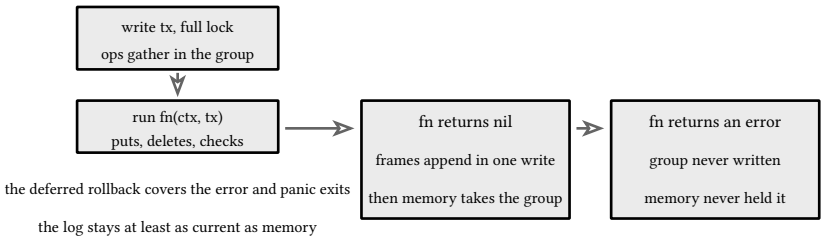


figure 140: a transaction's exits, every one of them clean

23.5. isolation levels and what the engine actually gives you

The textbook menu, read uncommitted through serializable, is a menu of guarantees a general engine picks among, and none of its items is what this engine serves. A read tx clones the key space under the read lock and keeps the clone for its whole life. No dirty reads, because memory only mutates after the group is logged and a reader's view is a state that was committed. No non-repeatable reads within one transaction, because the view never moves. Write skew stays possible under the weak discipline, reading a snapshot outside the write tx and committing after: two serialized writers can act on disjoint stale reads, each commit clean, the combined fact wrong. Writers serialize, one at a time, on the full lock. That is snapshot isolation, and this module built it, so the boundary is a fact about its own locks stated where the code states it, with no configuration file standing between the claim and the mechanism.

The demo stays deterministic and single threaded, which is the whole trick. The reader opens a read tx and reads version 1, capturing its view. A second writer commits version 2. The reader reads again inside its tx and still sees 1, and a fresh read outside sees 2. No goroutine, no barrier, no sleep, the interleave is the test's own statement order, so the demonstration cannot flake:

```
// TestSnapshotPin is the isolation lesson re-expressed at the engine:
// a snapshot is a private clone taken under the read lock, so a writer
// that commits afterward changes the engine but never the snapshot a
// reader already holds.
func TestSnapshotPin(t *testing.T) {
  e := newEngine(t)
  commit(t, e, "k", "version 1")
  pinned := e.Read()
  tx := e.Begin()
  tx.Put("k", []byte("version 2"))
  if err := tx.Commit(); err ≠ nil {
    t.Fatalf("commit: %v", err)
  }
  if got := mustGet(t, pinned, "k"); got ≠ "version 1" {
    t.Fatalf("pinned snapshot reads %q, want version 1", got)
  }
  if got := mustGet(t, e.Read(), "k"); got ≠ "version 2" {
    t.Fatalf("fresh read = %q, want version 2", got)
  }
}
```

listing 162: pin a view, commit past it, read again: the interleave is the statement order (source: go/api/internal/store/engine/engine_test.go, lines 116-135)

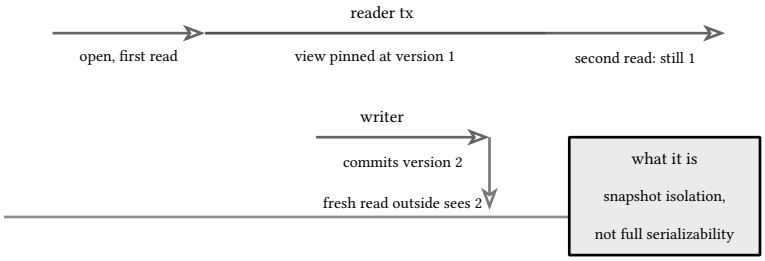


figure 141: one timeline: the reader's view holds

23.6. the lost update demo and the CAS answer

Two writers read the same user, both compute, both write, and one update silently vanishes. The engine reproduces it with two blind writes: both succeed, the second name is all that survives, and the version field never moved, so nothing anywhere records that anything was lost. Serialization does not fix this, and owning the machinery makes the reason visible: the full lock guarantees each write lands whole, and says nothing about the stale read the write was computed from, so the serialized blind writers merely lose the update in a tidy order. That invisibility is what makes the lost update the classic concurrency bug, the writer that caused it gets a success answer.

The version field is the detector and compare-and-swap is the fix, the strong discipline in the engine's vocabulary. The whole exchange, read the current value under the write lock, refuse anything but the expected version, bump and write, runs inside one write tx, so no second writer can slip between the read and the write, and the lock holding the window shut is the same lock the blind demo ran under, the difference is entirely the check. A mismatch answers ErrVersion. Of two racers holding version 1 exactly one wins, and the loser gets ErrVersion, a fact it must handle, which over http is exactly the 412 the next chapter's ETag duel stages with two released PATCHers:

```
func (s *UserStore) UpdateDisplayName(ctx context.Context, id string, expectVersion int, displayName
string) (user.User, error) {
  var out user.User
  err := s.DB.WithTx(ctx, func(ctx context.Context, tx *engine.WriteTx) error {
    u, suffix, err := getUser(tx, id)
    if err ≠ nil {
      return err
    }
    if u.Version ≠ expectVersion {
      return user.ErrVersion
    }
    u.DisplayName = displayName
    u.Version++
    if err := putUser(tx, u, suffix); err ≠ nil {
      return err
    }
    u.Roles = rolesOf(tx, u.ID)
    out = u
    return nil
  })
  if err ≠ nil {
    return user.User{}, err
  }
  return out, nil
}
```

listing 163: the CAS: check and bump share one write tx, one winner (source: go/api/internal/store/engine_user.go, lines 261-284)

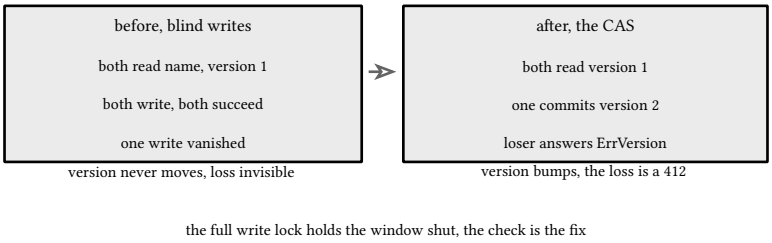


figure 142: same race, two outcomes: silent loss versus a refused writer

23.7. the analytics query as an in-Go walk

Reports were sql's home turf, and window functions kept them one query. Owning the store means owning the iteration too, so the report is now a walk over the events range, and the invariants the windows encoded are loops with names. The scan reads the e: prefix in day order inside one read tx, so one report always reads one snapshot of the events keys, never a torn view of an insert in flight. Presence collapses events to one entry per user per day, because actives count users, not actions. A user's first day inside the window is the newcomer, which is what ROW_NUMBER's nth = 1 used to mark. The delta is today's active minus yesterday's, the first day 0, LAG's arithmetic without the self-join. Leaders rank by count descending then user id ascending, a total order, so a replayed report cannot differ run to run.

The report routes keep their shape over the walk. Create validates the window, hashes the exact request bytes, and answers 202 pending or replays an earlier submission of the same key, refusing the same key under different bytes with 422 and details, the replay check running inside the write tx that stores the row. Get computes the series on first read, flips the row to done, and answers the frozen golden body, which is why the replay in the contract's reports pair upgrades to 200 with identical bytes.

```
// presence is one (day, user) fact, the collapse the SQL era's
// DISTINCT performed: actives count users, not actions.
type presence struct {
  day  string
  user string
}

// walkEvents scans the window's events in key order and hands every
// (day, user) presence to fn. The scan is sorted, so one day's keys
// arrive contiguously and in ascending day order.
func walkEvents(snap *engine.Snapshot, from, to string, fn func(p presence)) {
  lo, hi := eventBounds(from, to)
  for _, pair := range snap.Scan(lo, hi) {
    day := pair.Key[len("e:") : len("e:")+len("2006-01-02")]
    user := pair.Value
    if i := bytes.IndexByte(user, 0); i ≥ 0 {
      user = user[:i]
    }
    fn(presence{day: day, user: string(user)})
  }
}
```

listing 164: the walk: the e: range scan in day order, each event one (day, user) presence (source: go/api/internal/analytics/analytics.go, lines 37-57)

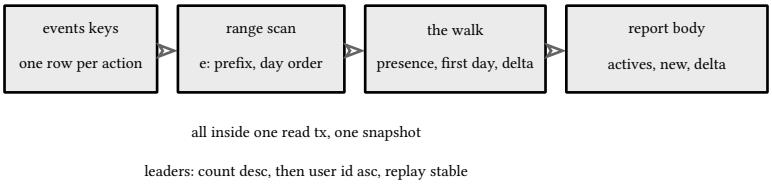


figure 143: keys to walk to the report body

23.8. store tests without docker

Every test in the package starts the same way: open an engine in a fresh temp directory, which loads the snapshot if one survived and replays the WAL over it, migrate it, run, close. No container, no shared server, no ordering between tests, and nothing in the package sleeps, because the file facts the engine adds are deterministic by construction: corruption flips every byte of the log one at a time, truncation cuts at every length, and a torn tail is geometry, not timing. The walk that reads a log stops at the first frame that does not hold: a remaining tail shorter than the eight-byte header ends it, a payload under two bytes is refused, the version byte and the op byte being the floor, and a length that overruns, a checksum that lies, or a payload this build cannot parse stop it the same way.

The recovery property rides a fuzz target, FuzzWALReplayPrefix, whose invariant is named in prose: for any framed log and any truncation of it, replaying the decodable prefix answers exactly the state of the ops whose transactions committed inside that prefix. Its seeds, among them a multi transaction log, a torn mid-frame tail, and a frame whose checksum lies, run as plain table cases under go test, and the mutation engine stays in the opt-in lane, the suite chapter's discipline applied to recovery. The concurrency tests, the readers racing a writer, meet on channels, and the CAS duel and the snapshot demo need no goroutine at all because the statement order is the schedule. The clock stays injected, the analytics fixture seeds exactly the three events the frozen vector's setup names, and the golden report body the tests assert is the contract's own bytes:

```
// newTestDBAt opens a database at path and migrates it.
func newTestDBAt(t *testing.T, path string) *DB {
  t.Helper()
  db, err := Open(context.Background(), path)
  if err ≠ nil {
    t.Fatalf("open: %v", err)
  }
  if err := db.Migrate(context.Background()); err ≠ nil {
    t.Fatalf("migrate: %v", err)
  }
  return db
}

// newTestDB opens a database in a fresh temp directory and migrates
// it. Every test in the package starts here, so no test shares state
// with another and nothing needs a container: the store's whole test
// story is a temp file.
func newTestDB(t *testing.T) *DB {
  t.Helper()
  return newTestDBAt(t, filepath.Join(t.TempDir(), "goapi.db"))
}
```

listing 165: the whole test story: a temp dir, an open that recovers, a migrate, per test (source: go/api/internal/store/store_test.go, lines 19-39)

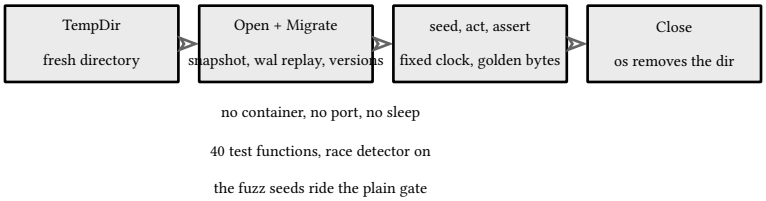


figure 144: a store test's lifecycle, nothing shared between runs

sources: pkg.go.dev for hash/crc32, database/sql, database/sql/driver, os, and testing, accessed 2026-09-26, the Castagnoli polynomial and MakeTable construction read from the hash/crc32 documentation, the Rename replace semantics read from the os documentation, and the driver surface, Driver, Conn, Stmt, and the sql.DB pooling contract, read from the database/sql and database/sql/driver pages. Verified by `go/api/internal/store`, `go/api/internal/store/engine`, and `go/api/internal/analytics` tests, 40 of them, with FuzzWALReplayPrefix's seeds riding the same plain `go test`, and the module gates: go vet, go test, and the race detector pass over the whole goapi module in `make verify-go`.

24. concurrency control

The store chapter's engine gave one writer the lock: one writer wins, the loser learns the fact. This chapter moves that fact to the http edge and builds everything around it: If-Match preconditions end to end with the ETag duel two released PATCHers settle, If-None-Match and the 304 that saves the body, idempotency keys with stored response snapshots so a retried register replays instead of double-creating, singleflight collapsing a herd into one fill, the synchronization primitives revisited as shapes with jobs, errgroup with bounded fan-out and first-error cancel, and the race canary that documents what the detector actually proves.

24.1. the lost update prevented

Two clients read the same user, both patch, and without protection one write silently vanishes. The stack answers in three layers. The precondition wrapper fails fast: an If-Match that is missing, the contract chose 412 over the 428 the RFC suggests, or stale against the current representation refuses the request before the handler burns its decode. The handler still checks the etag against the body it fetched, because a route can exist without the wrapper. And the store's compare-and-swap inside one write transaction is the guarantee itself, the only layer that is atomic. Fast fail, invariant, atomicity, in that order.

The duel stages the whole chain over the real routes: the store, the user handlers, the wrapper, two PATCHers carrying the same valid etag, released together by one close of a barrier. Both pass the wrapper, both pass the handler's check, and the CAS decides. Exactly one 200, exactly one 412, version 2 standing after, every run:

```
// duel releases two PATCHers carrying the same valid If-Match together
// through one close of a barrier and answers their two status codes.
// Both pass the wrapper's fast check, both pass the handler's body
// check, and the store's CAS decides.
func duel(t *testing.T, root http.Handler, id, etag string) []int {
  t.Helper()
  barrier := make(chan struct{})
  codes := make(chan int, 2)
  for _, name := range []string{"duel one", "duel two"} {
    go func(name string) {
      r := patchReq(t, id, etag, `{"display_name":"`+name+`"}`)
      <-barrier
      w := httptest.NewRecorder()
      root.ServeHTTP(w, r)
      codes <- w.Code
    }(name)
  }
  close(barrier)
  return []int{<-codes, <-codes}
}
```

listing 166: the duel: one barrier releases both writers, the store's CAS crowns one (source: go/api/internal/conc/conc_test.go, lines 88-107)

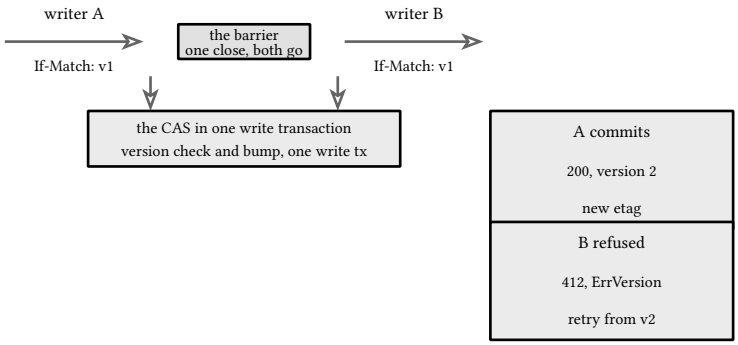


figure 145: two PATCHers, one barrier, one winner

24.2. If-None-Match and 304

The read side of the same coin costs nothing when nothing changed. A GET carrying an If-None-Match that matches the current representation answers 304 Not Modified, and the wrapper answers it before the handler runs: the store read and the marshal are both skipped, and the client, which already holds the body the etag names, keeps it. The response carries the etag and nothing else, because a 304 with a body would make the client cache the wrong thing for the wrong reason, the body it was told it already has.

The check is four decisions in order: no header means an ordinary read; a missing resource passes through so the handler's own 404 answers, the wrapper never invents statuses; a header that does not match the current etag is a normal read too, the client's copy is stale, so it gets a fresh one; only the match answers 304:

```
// revalidate answers the conditional GET: an If-None-Match matching
// the current representation writes 304 with the etag stamped and an
// empty body, skipping the handler's store read and marshal entirely,
// and answers true. A missing or stale header answers false and the
// request runs normally.
func revalidate(w http.ResponseWriter, r *http.Request, etags ETagger, id string) bool {
  inm := r.Header.Get("If-None-Match")
  if inm == "" {
    return false
  }
  cur, ok := etags.ETag(r.Context(), id)
  if !ok || inm ≠ cur {
    return false
  }
  w.Header().Set("ETag", cur)
  w.WriteHeader(http.StatusNotModified)
  return true
}
```

listing 167: the conditional GET: one comparison, an empty body, the etag stamped (source: go/api/internal/conc/etag.go, lines 80-97)

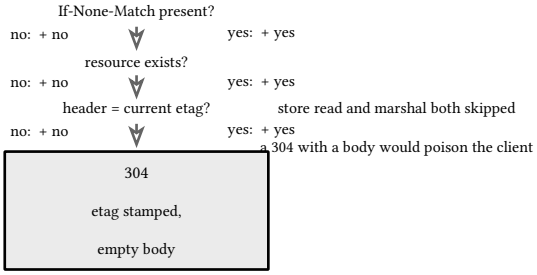


figure 146: the conditional GET decision

24.3. idempotency keys

A client that times out retries, and a register route that recreates on retry is a double-charge machine. The contract's answer is the Idempotency-Key header. The exact request bytes are hashed under sha256, whitespace drift is a different request, and the key is scoped by route and caller identity, so two clients may use the same key freely. The first successful run is stored as a snapshot, status, full header set, body, and a later request with the same key and the same hash replays those bytes at the stored status: the retried register returns the original 201, Location and ETag included, byte for byte.

The two failure paths are as deliberate. The same key under a different hash is a 422 with details, the client is misusing the contract, and the stored snapshot is untouched. And a failing first run is never stored: a retry after a validation error deserves a fresh run, not a replayed 422, so only outcomes under 400 become snapshots, and no 3xx exists on this route today. The store side is one engine keyspace under the idem: prefix with a 24 hour expiry, lazily swept on read, and the reports route is deliberately not on this guard, because its replay upgrades a stored 202 to the completed report, a sanctioned divergence that is the handler's own logic:

```
func IdempotencyGuard(mux *http.ServeMux, keys KeyStore) func(http.Handler) http.Handler {
  return func(next http.Handler) http.Handler {
    live := &flights{waiting: map[flightKey]chan struct{}{}}
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      _, pattern := mux.Handler(r)
      key := r.Header.Get("Idempotency-Key")
      if !idempotent[pattern] || key == "" {
        next.ServeHTTP(w, r)
        return
      }
      runKeyed(w, r, live, keys, next, key, pattern)
    })
  }
}
```

listing 168: the guard: route and key resolved, everything keyed delegated (source: go/api/internal/conc/idempotency.go, lines 71-84)

The missing half was concurrency. A lookup that misses, a handler that runs, a save that lands only after the handler returns: two callers under the same key inside that window both miss, both run, and the loser answers the handler's own 409 for an operation that succeeded and is recorded under its key. The adversarial pass opened exactly this window by parking one register inside the injected hash and letting a second run to completion. The guard closes it with an in-flight table keyed by the same route, identity, and key triple the store rows live under: the first caller claims the slot and answers a release on the way in, a later caller waits on its channel and then reads the settled row, so the concurrent double submit replays the 201 the first caller stored:

```
func settle(w http.ResponseWriter, r *http.Request, live *flights, keys KeyStore, key, route, identity
string, hash []byte) (release func(), run bool) {
  wait, rel := live.claim(flightKey{route, identity, key})
  if wait == nil {
    return rel, true
  }
  select {
  case <-wait:
  case <-r.Context().Done():
    return nil, false
  }
  if replayStored(w, r, keys, key, route, identity, hash) {
    return nil, false
  }
  return func() {}, true
}
```

listing 169: settle: first caller runs and releases, later callers wait and re-read the settled row (source: go/api/internal/conc/idempotency.go, lines 150-164)

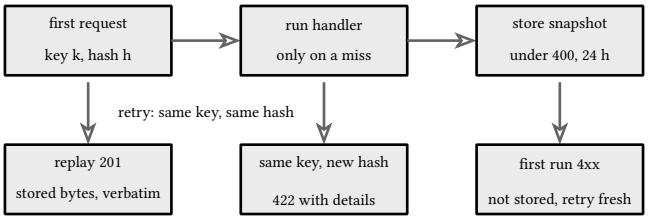


figure 147: one key's life: first run, snapshot, replay

24.4. singleflight

When a hot key expires, every request for it computes the same expensive thing at once, the thundering herd, and the origin pays for N identical reads. singleflight collapses the herd: the first caller runs the function, every concurrent caller with the same key waits on that one run and receives its result, and the shared flag tells callers their answer was shared. The entry is forgotten the moment the call completes, which is the property that keeps this honest: the herd that arrives after the lull fills again. A collapse tool, not a cache, and the distinction is a design decision, not a limitation, the next chapter puts a cache in front of this loader and gets both behaviors explicitly.

The loader wraps the group with two doors. Do is the plain form, synchronous, the caller blocks until the fill lands. DoChan registers the caller synchronously and answers through a channel, and that synchronous registration is what the tests lean on: once DoChan returns, the caller has provably joined an in-flight fill or started the only one, so the collapse assertion, exactly one fill for three registered callers, has no scheduling window to flake through:

```
type Loader struct {
  group singleflight.Group
}

// Result is one DoChan outcome: the value, the error, and whether that
// value was shared with other callers.
type Result struct {
  Value  any
  Err    error
  Shared bool
}

// NewLoader returns an empty loader.
func NewLoader() *Loader { return &Loader{} }

// Do runs fn for key, at most once across concurrent callers. fn runs
// uncanceled by design: one caller giving up must not kill the shared
// computation every other caller is waiting on. The bool answer
// reports whether the result was shared with other callers.
func (l *Loader) Do(key string, fn func() (any, error)) (any, bool, error) {
  v, err, shared := l.group.Do(key, fn)
  return v, shared, err
}
```

listing 170: the loader: one door synchronous, one door provably registered (source: go/api/internal/conc/singleflight.go, lines 12-34)

The x/sync group the loader wraps is the same honesty. Its source is one `sync.Mutex` over a lazily built `map[string]*call`, each call carrying its own `sync.WaitGroup`, the value and error written once before `Done`, a duplicate count, and the channel list `DoChan` appends to. The first caller for a key creates the entry under the mutex and runs the function, a later caller finds the entry, bumps the duplicate count, unlocks, and parks on that call's `WaitGroup`, and the runner wakes the waiters and deletes the map entry inside one locked defer, which is the forget-on-completion property the next chapter's cache leans on. The provable registration the tests rely on is simply the map lookup and the channel append sharing one critical section, so joining an in-flight fill has no scheduling window.

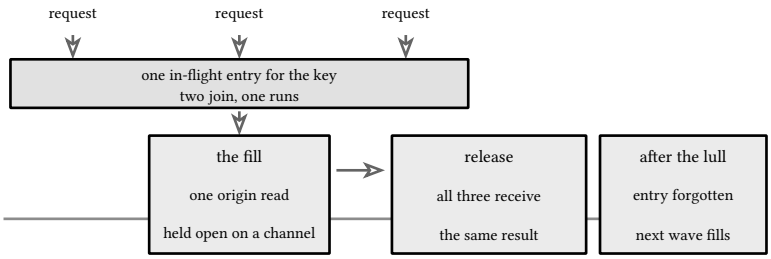


figure 148: three requests arrive together, one origin read happens

24.5. the primitives revisited

The mutex family is small and each shape has one job. Counter is the whole mutex story in two methods: shared mutable state, one lock, nothing clever between them. Stats is the RWMutex case, and the rule is measured, not assumed, readers must dominate and the sections must be long enough to pay for the heavier bookkeeping, otherwise a plain mutex wins. The buffered channel is a semaphore for free, capacity tokens, acquire takes, release gives, with a context exit added because a plain channel cannot be canceled out of. The Once family memoizes one expensive computation, OnceFunc for the void case, OnceValue when the error or value must be carried, OnceValues for pairs, and wg.Go removes the loop-carried Add that leaked waiters for a decade.

Under all of them the race detector, and what it proves is narrower than folklore: it verifies happens-before, that every pair of accesses to shared state is ordered by some synchronization edge, a lock, a channel, a WaitGroup, an atomic. It does not prove your locking policy correct, a data-race-free program still loses updates happily if the lock is held at the wrong altitude, and it does not find races in interleavings a run never took. That is why the gate runs it on every commit and why the canary at the end of this chapter exists:

```
type Counter struct {
  mu sync.Mutex
  n  int64
}

// Add applies delta under the lock and answers the new value.
func (c *Counter) Add(delta int64) int64 {
  c.mu.Lock()
  defer c.mu.Unlock()
  c.n += delta
  return c.n
}

// Load answers the current value under the lock.
func (c *Counter) Load() int64 {
  c.mu.Lock()
  defer c.mu.Unlock()
  return c.n
}
```

listing 171: the mutex story in two methods: shared state, one lock (source: go/api/internal/conc/locks.go, lines 12-30)

```
sem := make(Semaphore, 2)  // capacity is the concurrency bound
sem.Acquire(ctx)           // or ctx.Done()
sem.Release()              // give the token back
```

fragment, illustrative, not executed

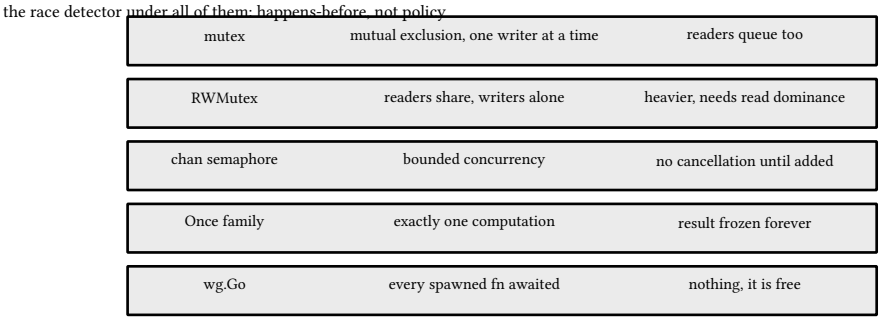


figure 149: each primitive, the guarantee it buys

24.6. errgroup with bounded fan-out

Spawn a goroutine per item and the size of your input becomes the size of your database connection pool's worst day: the queue you built to protect the origin becomes the stampede you meant to prevent. errgroup with SetLimit is the valve. The helper runs one task per index with at most limit in flight, and the group's context is canceled the moment any task errors, so tasks still queued or blocked on their own context see Done and stop early, and Wait answers with that first error, the one worth acting on, not the last cancel noise to land.

Because the valve is an x/sync type, it earns its keep here by being nothing but this chapter's primitives in a named shape. The v0.22.0 source declares the whole mechanism in five fields: a `sync.WaitGroup` every `Go` adds to and every exit releases, a `sem chan token` whose buffer is the SetLimit bound, one token taken on the way in and given back on the way out, an `errOnce sync.Once` guarding the `err` slot it latches, so the first error is stored once and later ones cannot overwrite it, and a cancel saved from `context.WithCancelCause`, fired on that first error or at Wait. The cancellation traveling left through the context is the ch06 close, the bound is the buffered-channel semaphore from the primitives section, and the first-error discipline is the Once family. A dependency that decomposes into primitives you already own is cheap to trust, because reading its source beats reading its documentation.

The test pins both promises deterministically. The bound holds, two tasks observed in flight together while the rest queue, the peak exactly the limit. And the first error cancels, the one failing task returns the error, the other nine hold on the group's context and leave when it closes, no work between them:

```
// FanOut runs task for 0..n-1 under one errgroup with at most limit
// running at once. The group's context is canceled the moment any task
// returns an error, so the tasks still queued or waiting on their own
// context see Done and stop early, and FanOut answers with that first
// error. The bound is the whole point: unbounded fan-out converts one
// hot key into one thousand simultaneous database queries, and SetLimit
// is the valve that keeps the load the pool can actually serve.
func FanOut(ctx context.Context, limit, n int, task func(ctx context.Context, i int) error) error {
  g, ctx := errgroup.WithContext(ctx)
  g.SetLimit(limit)
  for i := 0; i < n; i++ {
    g.Go(func() error {
      return task(ctx, i)
    })
  }
  return g.Wait()
}
```

listing 172: bounded fan-out: SetLimit is the valve, the group ctx is the alarm (source: go/api/internal/conc/errgroup.go, lines 9-25)

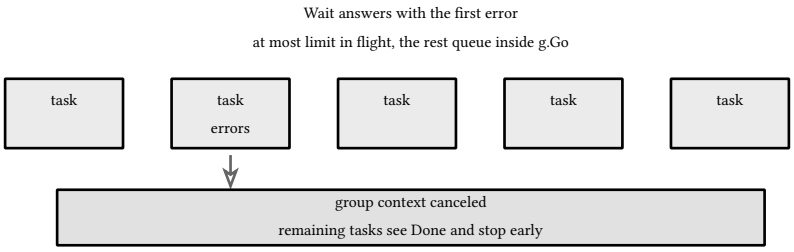


figure 150: the first error travels left through the context

24.7. the race canary

The canary is a test whose subject is the tool. Its broken half is a counter with no lock, two goroutines hammering one variable with nothing between them, pure shared state. Its guarded twin is the same hammering through the Counter from this chapter, clean by construction. The suite runs the guarded twin in process, always green, and executes the broken twin as a child process, then asserts what the build mode proves: compiled with the detector, the child must fail, because unsynchronized shared state is a detected fact, and compiled without it, the child must pass, silently losing increments, which is the entire argument for the gate.

The child-process construction is what keeps the suite green while the canary stays live. A broken test run in process under `-race` fails the build, which is the detector doing its job at the worst possible altitude, in the gate instead of the finding. Spawned behind an environment key, the broken path runs only when the parent asks for it, and the parent turns the child's exit code into the assertion. Delete the lock in the guarded twin and the in-

process run fails under -race with a report naming the exact accesses and the goroutines that raced, which is the demo the canary exists to keep honest:

```
const canaryChildEnv = "GOAPI_CANARY_UNGUARDED"

// unguardedCounter is deliberately broken shared state: two goroutines
// increment one variable with nothing between them, no lock, no
// atomic, no channel. Under the race detector this is a report the
// moment the accesses overlap. Without it, it is silent, which is the
// entire lesson.
func unguardedCounter(times int) {
  var n int64
  start := make(chan struct{})
  done := make(chan int64, 2)
  worker := func() {
    <-start
    for i := 0; i < times; i++ {
      n++
    }
    done <- n
  }
  go worker()
  go worker()
  close(start)
  <-done
  <-done
}
```

listing 173: the deliberately broken half: two goroutines, one variable, no lock (source: go/api/internal/conc/race_canary_test.go, lines 10-33)

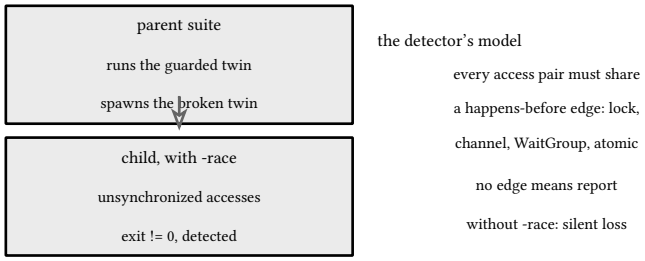


figure 151: the canary's two verdicts, one mechanism

sources: pkg.go.dev for sync, sync/atomic, testing/synctest, and golang.org/x/sync, accessed 2026-09-25, the Once family and wg.Go signatures quoted from the go 1.27 source in GOROOT, singleflight's Do and DoChan registration semantics read from the x/sync v0.22.0 source in the module cache, and the happens-before model stated as the race detector's own documentation does. Verified by go/api/internal/conc tests, 14 of them, the canary pair and the flight pair included, under go vet, go test, and the module-wide race leg, where the canary's child fails on cue and the guarded twin stays clean.

25. caching

A cache is a promise to answer from a copy, and every promise has a price. This chapter prices it honestly: what caching promises and what it silently trades away, the http semantics a private api owes its clients, the LRU from first principles over container/list and a map, TTL beside LRU so age evicts as well as crowding, the stampede guard that rides the previous chapter's singleflight, invalidation by delete after write, and the discipline of what never gets cached at all.

25.1. what caching promises

A cache answers from a copy to save the origin the work, and both sides of the bargain matter: the copy must be cheap to find, and the answer must be one the origin would still give. Every caching decision is a bet that the read will be asked again before it changes, and the three terms of the bet are capacity, how many copies, lifetime, how long each stays, and invalidation, what a write owes the copies it made wrong. Miss any term and the cache stops being an optimization and becomes a second source of truth, which is the failure mode every caching chapter exists to prevent.

This service's copy lives in one layer, a bounded store of responses in front of the route table, constructed with its three terms up front and an injected clock so every one of them is testable as arithmetic rather than waiting:

```
type Layer struct {
  cache  *Cache[snapshot]
  loader *conc.Loader
}

// NewLayer returns a layer holding at most max entries for ttl each,
// reading the clock from now.
func NewLayer(max int, ttl time.Duration, now func() time.Time) *Layer {
  return &Layer{cache: NewCache[snapshot](max, ttl, now), loader: conc.NewLoader()}
}
```

listing 174: the layer: capacity, lifetime, and a clock, stated at construction (source: go/api/internal/cache/httpsem.go, lines 50-59)

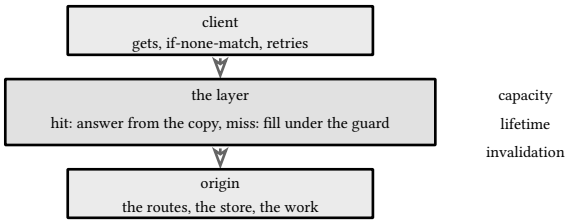


figure 152: the layer sits between client and origin, answering what it can

25.2. HTTP caching semantics for private APIs

A private api's responses are personalized and credentialed, so the shared-cache machinery of the web mostly does not apply, but the headers still owe clients a contract. Cache-Control states what may be stored and for how long, and private keeps shared proxies out of it, these bodies vary by who is asking. Vary names the request headers the representation depends on, here the bearer token and the session cookie, which is precisely the pair the cache key carries so two identities never share an entry. no-store means never, not even for a moment, and it is the default.

The reads are revalidation business. max-age=0 with must-revalidate tells the client nothing is fresh for long, every reuse round-trips the etag through the 304 lane the previous chapter built, and the layer's own copy is short-lived and keyed by the vary values, a latency optimization behind the protocol, never a replacement for it. The

table is deny by default: a route is cacheable only when a row says so, and no other file in the service can make one cacheable by accident:

```
var routePolicy = map[string]policy{
  "GET /api/users":      {control: "private, max-age=0, must-revalidate", vary: "Authorization, Cookie",
cacheable: true},
  "GET /api/users/{id}": {control: "private, max-age=0, must-revalidate", vary: "Authorization, Cookie",
cacheable: true},
}

// noStore is the default posture for every route the table does not
// name: a private api caches nothing unless a route says otherwise.
const noStore = "private, no-store"
```

listing 175: the per-route policy: two cacheable reads, everything else no-store by default (source: go/api/internal/cache/httpsem.go, lines 26-33)

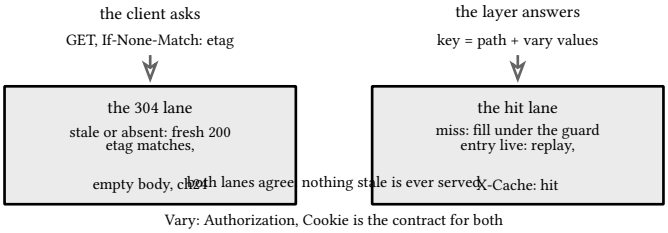


figure 153: two freshness decisions: the client's and the layer's

25.3. LRU from first principles

Least recently used is one map and one list. The map finds any key in constant time, the list carries the recency order with the hottest key at the front, and the two agree through pointers: every entry lives in both structures, the map pointing at the list node and the node carrying the key so eviction can delete from the map without walking it. A read moves its node to the front, a write inserts at the front, and when capacity is exceeded the node at the back is the eviction, found in constant time, removed from both structures in constant time.

The proof is the scramble test: fill the cache to capacity, touch a key from the middle of the order, and the next insert evicts the true least recently used entry, not the oldest insert. A plain map evicts nothing and grows forever, a plain list finds in linear time, and the pairing is the textbook answer to why both exist:

```
type Cache[V any] struct {
  mu    sync.Mutex
  max   int
  ttl   time.Duration
  now   func() time.Time
  order *list.List
  items map[string]*list.Element
}

// entry is one cached value: its key for O(1) eviction from the map,
// the value, and the instant it stops being fresh.
type entry[V any] struct {
  key string
  val V
  exp time.Time
}
```

listing 176: the pairing: the map finds, the list orders, the entry carries its key and expiry (source: go/api/internal/cache/cache.go, lines 21-36)

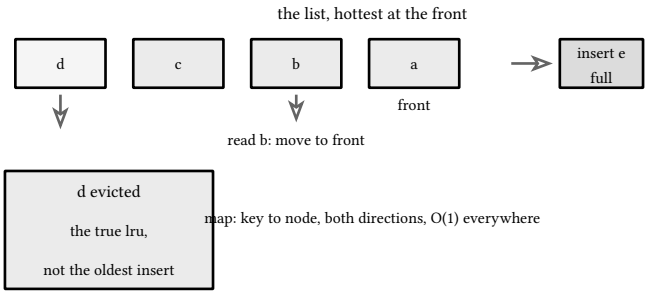


figure 154: a read moves its node to the front, an insert evicts the back

25.4. TTL beside LRU

LRU answers crowding, not age. An entry nobody reads can be least recently used forever and still hold its bytes, and an entry everybody reads can be hot long after the origin moved on. The TTL gives every entry an expiry stamped from the injected clock at write time, and reads check it lazily: an expired entry evicts itself on contact, so every read is its own eviction decision and no sweep is needed for correctness.

The sweep is for memory, not correctness. A janitor goroutine wakes on a ticker and removes whatever aged out unread, and because it is nothing but a ticker and a clock, it tests whole hours inside a synctest bubble, the fake clock advancing only when every goroutine in the bubble is parked, the assertion after the sleep running in microseconds of real time:

```go
// Get returns the live value for key and marks it most recently used.
// An expired entry is removed on the spot and answers a miss: expiry
// is lazy, every read is its own eviction decision.
func (c *Cache[V]) Get(key string) (V, bool) {
  c.mu.Lock()
  defer c.mu.Unlock()
  el, ok := c.items[key]
  if !ok {
    var zero V
    return zero, false
  }
  e := el.Value.(*entry[V])
  if !c.now().Before(e.exp) {
    c.order.Remove(el)
    delete(c.items, key)
    var zero V
    return zero, false
  }
  c.order.MoveToFront(el)
  return e.val, true
}
```

listing 177: the read: expiry is lazy, the hit moves to the front, the miss cleans up (source: go/api/internal/cache/cache.go, lines 50-70)

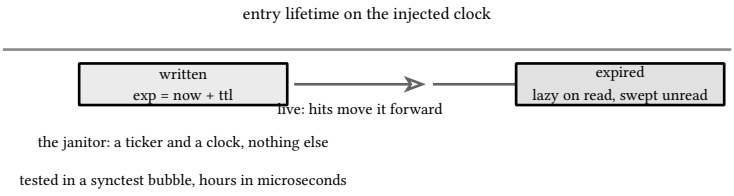


figure 155: two clocks push entries out: age and crowding

25.5. the stampede and its guard

Every entry expires eventually, and the moment a hot one does, the herd arrives together: a hundred concurrent misses, a hundred identical origin reads, the exact load spike the cache existed to prevent, arriving at the worst moment, the cold one. The guard is the previous chapter's loader worn one layer down: a miss fills through singleflight, the herd collapses into one origin read, every waiter receives the same result, and that result is the only thing that ever writes the cache entry.

Fill rides the loader's DoChan door on purpose. Registration is synchronous, so once Fill returns the caller has provably joined an in-flight fill or started the only one, and the collapse test holds that property exactly: three callers registered while the origin read is held open on a channel, one fill, three identical answers, no scheduling window. And errors do not land in the cache, a failed fill misses again, so an origin outage cannot be memoized into a sticky failure:

```
// Fill registers this caller on the loader's key synchronously and
// runs fill in its own goroutine, answering through a channel.
// Concurrent Fill calls for one key run fill exactly once and share
// its result: the thundering herd becomes one origin read. The
// synchronous registration is the property the deterministic collapse
// test leans on, and it is why Fill rides singleflight's DoChan rather
// than Do: once Fill returns, the caller has provably joined an
// in-flight fill or started the only one.
func Fill[V any](l *conc.Loader, key string, fill func() (V, error)) <-chan FillResult[V] {
  ch := l.DoChan(key, func() (any, error) { return fill() })
  out := make(chan FillResult[V], 1)
  go func() {
    r := <-ch
    res := FillResult[V]{Err: r.Err}
    if r.Value != nil {
      res.Value = r.Value.(V)
    }
    out <- res
  }()
  return out
}
```

listing 178: the fill: registered synchronously, collapsed to one origin read (source: go/api/internal/cache/stampede.go, lines 11-31)

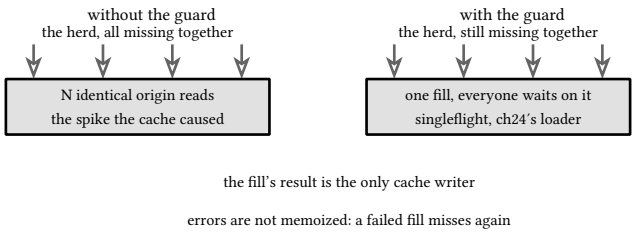


figure 156: the same expiry, with and without the guard

25.6. invalidation on write

A write changes the origin, and the copies of what it changed are now wrong. The discipline is delete after write, never update: the writer drops the affected entries and lets the next read refill them through the guard. Deleting is idempotent and always safe, and the refill is the single writer the cache already trusts. Updating instead, pushing the writer's view into the entry, sounds cheaper, one write instead of a miss, but the writer does not know what a concurrent reader cached mid-flight, and an update that lands just before a stale read's own set leaves the wrong value with a fresh timestamp, the worst of both worlds.

The rule is path plus collection: a write to a row invalidates the row and the list that reads it, a write to the collection invalidates the list and any rows under it. The staged test runs two identical caches through the same write, one updated, one deleted, and only the deleted one refills the origin's new truth:

```
func (l *Layer) Invalidate(path string) {
  parent := ""
  if i := lastSlash(path); i > 0 {
    parent = path[:i]
  }
  l.cache.DeleteWhere(func(key string) bool {
    _, keyPath, _ := splitKey(key)
    affected := keyPath == path || keyPath == parent || strings.HasPrefix(keyPath, path+"/")
    return !affected
  })
}
```

listing 179: delete after write: the path, its collection, its children, nothing else (source: go/api/internal/cache/invalidation.go, lines 14-25)

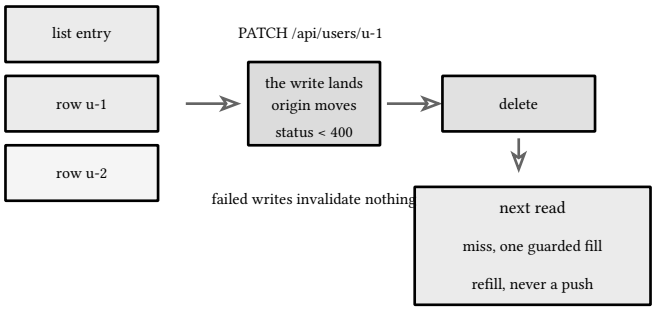


figure 157: write to a row: the row and the list go, the refill is the only writer

25.7. what not to cache

The last decision is the one that prevents the incidents. Credential routes, login and refresh, carry secrets and must never be replayable from a copy. Rate-limited responses, the 429 lane, must always reach the client, because a cached budget answer is a lie about money. Personalized bodies without a vary key to separate identities are a cross-user leak waiting for traffic. Anything the contract marks no-store is a legal promise, not a hint. The layer's policy table is deny by default, cacheable only where a row says so, which makes the exclusion list the empty set of accidents: a new route opts in or it does not get cached, and no change elsewhere in the service can flip that.

The test pins the posture end to end: a no-store route asked three times reaches the origin three times, answers from itself every time, and never once says hit. The cheap check is the one worth keeping in the gate, because the day someone adds a cacheable row to the wrong route, this is the test that says which lane they put it in:

```
func TestNoStoreNeverCaches(t *testing.T) {
  mux := http.NewServeMux()
  var reads atomic.Int64
  mux.HandleFunc("GET /api/auth/login", func(w http.ResponseWriter, _ *http.Request) {
    reads.Add(1)
    w.Write([]byte("credentials lane"))
  })
  root := NewLayer(8, time.Minute, frozen).Wrap(mux)(mux)
  for i := 0; i < 3; i++ {
    r := httptest.NewRequest(http.MethodGet, "/api/auth/login", nil)
    w := httptest.NewRecorder()
    root.ServeHTTP(w, r)
    if w.Header().Get(XCache) == "hit" {
      t.Fatal("a no-store route answered from the cache")
    }
    if cc := w.Header().Get("Cache-Control"); cc ≠ noStore {
      t.Fatalf("cache-control = %q, want %q", cc, noStore)
    }
  }
  if reads.Load() ≠ 3 {
    t.Fatalf("origin reads = %d, want 3: no-store routes always reach the origin", reads.Load())
  }
}
```

listing 180: the exclusion test: no-store routes always pay the origin (source: go/api/internal/cache/cache_test.go, lines 274-296)

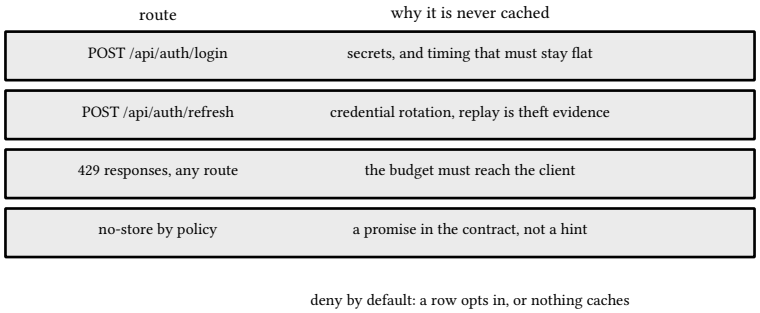


figure 158: the exclusion list, each row with its reason

sources: pkg.go.dev for container/list, net/http, and testing/synctest, accessed 2026-09-25, the Cache-Control, Vary, and no-store semantics quoted from RFC 9111 section 5.2 and RFC 9110 section 12.5.5 at rfc-editor.org, the synctest bubble rules read from the go 1.27 source and its documentation, and singleflight's DoChan registration semantics from the x/sync v0.22.0 source in the module cache, pinned by the previous chapter. Verified by `go/api/internal/cache` tests, 13 of them, under gofmt, go vet, go test, and the scoped race leg, with the janitor's whole life exercised inside one synctest bubble.

26. rate limiting

A service that answers every request it receives eventually answers none of them: password guessing arrives as a flood of logins, a runaway client loop arrives as the same request ten thousand times, a retry storm arrives the moment a deploy slows one route down. This chapter builds the service's rate limiting in `goapi/internal/limit`: the token bucket derived from first principles with an injected clock, the production gate on `golang.org/x/time/rate` with one limiter per key and idle sweeping, the 429 contract with its headers and envelope, what to key a limit on and what never to trust, and the test discipline that keeps every assertion off the wall clock.

26.1. why limits exist

Capacity is finite and failure is nonlinear. Past some offered load a service does not degrade gently, it falls off a cliff: queues grow, latency climbs past every client timeout, memory fills with requests waiting on answers that can no longer arrive, and the health check itself starts timing out. A limit is the refusal to stand on that cliff. It converts an overload that would take the whole service down into a fast, honest rejection of the excess, and the client that honors it gets a working service at one request per second instead of a dead one at zero.

The two limited routes carry numbers chosen for what they protect. Login allows a burst of five per address, then one per second, because every login attempt pays the argon2 verification budget and a credential guesser does not need more than five tries to be dangerous. Register allows ten per client ip for the same reason plus its idempotency bookkeeping. Both refill one token per second, so a client that trips the limit is back to full capacity five and ten seconds later, which is the difference between a limit and a lockout:

```
// The contract's two limited routes, verbatim numbers: login bursts 5
// per lower(email), register bursts 10 per client ip, both refill one
// token per second.
const (
  LoginBurst    = 5
  RegisterBurst = 10
  RefillPerSec  = 1.0
  LoginMsg      = "too many login attempts"
  RegisterMsg   = "too many registration attempts"
  // DefaultIdleTTL retires a key's limiter after this much disuse.
  DefaultIdleTTL = 10 * time.Minute
)
```

listing 181: the contract's two rules as one constant block, numbers and messages (source: go/api/internal/limit/handler.go, lines 30-41)

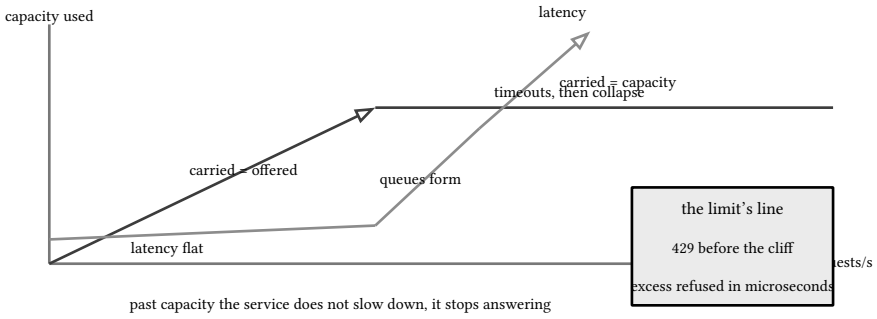


figure 159: offered load versus what the service can carry, the cliff between them

26.2. the token bucket from first principles

A token bucket holds a float, a capacity, and a refill rate. It starts full, every allowed request removes one token, and time adds tokens continuously at the configured rate until the capacity caps them. That is the entire algorithm,

and its two knobs split the two jobs cleanly: capacity is the burst allowance, how much may arrive at once, and the refill rate is the sustained allowance, how much may arrive per second forever. The contract's login rule, capacity five at one per second, permits five requests in the first instant, a sixth at the one second mark, and then one per second for as long as the client cares to wait.

The arithmetic of a denial is the part worth deriving rather than reciting. After the sixth same-second draw the level is exactly zero, so the next token exists one full second in the future, and that instant is both the `Reset` header and the `Retry-After`. Half a second later the level is 0.5: still not spendable, and the next whole token is now only half a second away. A whole level always waits a full period, a fraction waits its complement, which is the `nextTokenIn` rule: the wait is `1 - frac(level)` over the refill rate, and a denial never spends anything:

```
func (b *Bucket) Allow(now time.Time) Decision {
  b.refill(now)
  d := Decision{Limit: int(b.capacity)}
  if b.level ≥ 1 {
    b.level--
    d.Allowed = true
  }
  d.Remaining = int(b.level)
  wait := b.nextTokenIn()
  d.ResetAt = now.Add(wait)
  if !d.Allowed {
    d.RetryAfter = wait
  }
  return d
}

// nextTokenIn is the time until the level reaches the next whole
// token: a whole fraction of the current level waits a full period, an
// empty bucket waits one period, and a fraction waits its complement.
func (b *Bucket) nextTokenIn() time.Duration {
  frac := b.level - float64(int(b.level))
  return time.Duration((1 - frac) / b.perSecond * float64(time.Second))
}
```

listing 182: Allow: one draw, one decision, and the wait computed from the fractional level (source: go/api/internal/limit/bucket.go, lines 64-86)

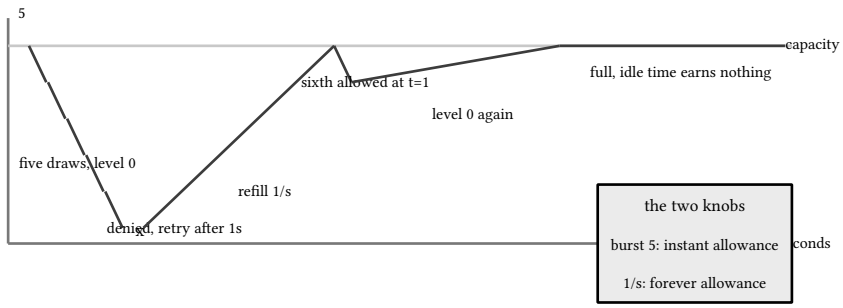


figure 160: bucket level over one draw timeline, the refill climbing between bursts

26.3. x/time/rate in production

The teaching bucket is inspectable and stays in the chapter, the shipped gate runs on `golang.org/x/time/rate`, which implements the same bucket with reservations and a `Wait` mode behind one mutex. One limiter per key lives in a map, created on first use, and the map is the piece that needs a policy: every distinct key is a goroutine-cheap but unbounded allocation, so an idle ttl retires keys nobody has touched. The sweep is amortized into the request path,

every `idleTTL` of clock time a request sweeps inside the lock it already holds, which deletes the janitor goroutine entirely, no ticker, no stop method, no leak to chase in the load chapter.

Read as primitives, the shipped limiter is the teaching bucket with the refill moved into the clock. The v0.15.0 struct is one `sync.Mutex`, the limit, the burst, and the token level stamped with the time it was last read: there is no goroutine and no ticker anywhere in the file, `advance` converts elapsed time into tokens lazily on every call, so the passage of time is the refill. Reservations are the one move the teaching bucket does not make: a grant for a future token lets the level go negative, and `Wait` is that reservation plus a timer armed at the delay, cancellable by context. One mutex, one float, one clock read, and a timer carry the whole production gate, which is why this chapter could build a faithful one first.

The third admission mode deserves its sentence. `Allow` rejects, `Wait` blocks until a token exists or the context dies, and a queue that wants to shed load rejects while a background worker that must not drop work waits. `Wait` is not used on this api's routes because an http handler that blocks on a limiter holds its connection and its goroutine, which is precisely the resource the limit exists to protect. The gate wiring is three lines in the program's table, and its position in the onion is deliberate, between the access log and the timeout, so every 429 is logged with its status and id and no denial ever burns handler deadline budget:

```
limiter := limit.NewLimiter(nil, 0)
limiter.Rule("/api/auth/login", limit.LoginRule())
limiter.Rule("/api/users", limit.RegisterRule())
app.Use(limiter.Middleware)
```

fragment, illustrative, not executed

```
func (l *Limiter) limiterFor(rule Rule, key string, now time.Time) *rate.Limiter {
  l.mu.Lock()
  defer l.mu.Unlock()
  if now.Sub(l.lastSweep) ≥ l.idleTTL {
    l.sweepLocked(now)
  }
  e, ok := l.buckets[key]
  if !ok {
    e = &entry{lim: rate.NewLimiter(rate.Limit(rule.PerSecond), rule.Burst)}
    l.buckets[key] = e
  }
  e.last = now
  return e.lim
}

// Sweep retires every key idle longer than the ttl and returns how
// many went. Exported for tests and for operators probing memory.
func (l *Limiter) Sweep(now time.Time) int {
  l.mu.Lock()
  defer l.mu.Unlock()
  return l.sweepLocked(now)
}
```

listing 183: per-key limiters, created on first use, swept on the same lock (source: go/api/internal/limit/handler.go, lines 191-212)

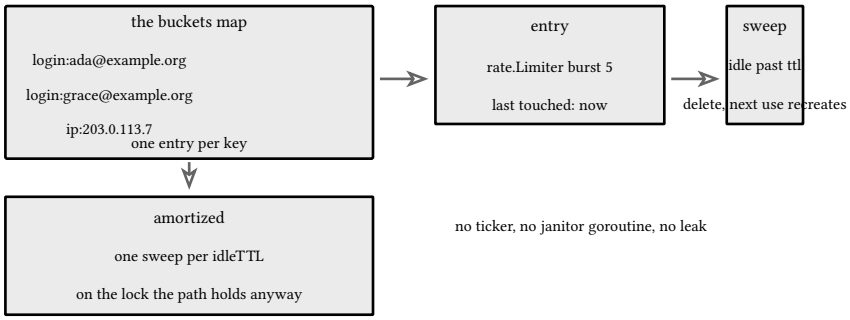


figure 161: key to limiter map, first use creates, idle age retires

26.4. the 429 contract

The wire shape is pinned by the contract and the frozen vector, and the middleware writes it in one place. Every response from a limited route carries three headers: `X-RateLimit-Limit` is the burst capacity, `X-RateLimit-Remaining` is whole tokens left after this draw, `X-RateLimit-Reset` is the unix second at which the next token arrives. A denial adds `Retry-After` in seconds, the ceiling of the wait, and answers the standard envelope with code `rate_limited`, never a bare status or a bespoke error body.

The sixth login vector pins every byte. The clock sits at the frozen `t0` plus twelve seconds, five earlier logins at the same instant left the bucket reporting remaining 4, 3, 2, 1, 0, and the sixth draw finds an empty bucket: status 429, `Limit` 5, `Remaining` 0, `Reset` 1790830813 which is exactly `t0` plus thirteen seconds, `Retry-After` 1, and the envelope's message the frozen `too many login attempts`. The middleware writes the three headers before the handler runs, so they ride on successes too, and the denial path writes the envelope through the kernel's one failure writer, which is why the request id in the body matches the one the access log just recorded:

```
func (l *Limiter) Middleware(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    l.mu.Lock()
    rule, limited := l.rules[r.URL.Path]
    l.mu.Unlock()
    if !limited {
      next.ServeHTTP(w, r)
      return
    }
    d := l.allow(rule, rule.Key(r))
    w.Header().Set(HeaderLimit, strconv.Itoa(d.Limit))
    w.Header().Set(HeaderRemaining, strconv.Itoa(d.Remaining))
    w.Header().Set(HeaderReset, strconv.FormatInt(d.ResetAt.Unix(), 10))
    if !d.Allowed {
      w.Header().Set(HeaderRetryAfter, strconv.Itoa(int(math.Ceil(d.RetryAfter.Seconds()))))
      httpx.Error(w, r, httpx.Fail(httpx.RateLimited, rule.Message))
      return
    }
    next.ServeHTTP(w, r)
  })
}
```

listing 184: the middleware: three headers always, Retry-After and the envelope on denial (source: go/api/internal/limit/handler.go, lines 138-158)

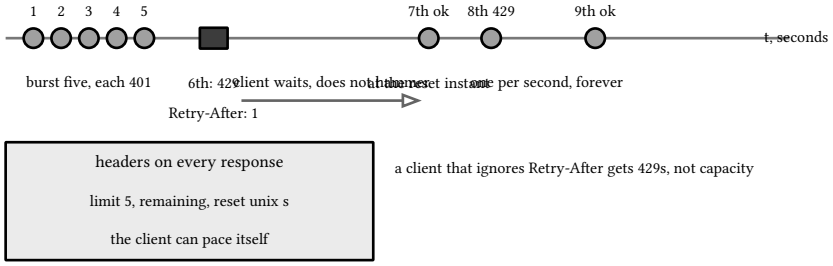


figure 162: a client that honors Retry-After: five through, one refused, the next at the reset instant

26.5. what to key on

A limit is only as good as its key, and the two routes key on different things for opposite reasons. Login keys on `lower(email)`: the attack the rule exists for is guessing one account's password, so the bucket belongs to the account, and the casing normalization means `ADA@Example.ORG` and `ada@example.org` are one bucket. The key lives in the request body, so the middleware reads the body once, bounded by the kernel's one mebibyte cap, and hands it back unchanged for the handler to decode again, a re-read that costs a copy of a body that is by construction tiny. Register keys on the client ip from the remote address, and the proxy headers stay unread on purpose: `X-Forwarded-For` is a header any client can write, so a limiter that keys on it verbatim sells a fresh bucket of ten registrations per forged address, one per request, forever.

Two shared buckets close the dodge holes. A body the email key cannot parse lands in one `login:unparsed` bucket, so garbage cannot orbit the limit by being invalid, and a remote address that fails to split lands in its own fallback. The test for the spoof is the contract said out loud: eleven register posts from one address with a different forged `X-Forwarded-For` on the last one, and the eleventh is still the 429:

```
// KeyEmail keys the login route on lower(email). The key lives in the
// body, so the body is read once, bounded, and handed back unchanged
// for the handler to decode again. A body without a parseable email
// shares one bucket: garbage must not sidestep the limit.
func KeyEmail(r *http.Request) string {
  body, err := io.ReadAll(io.LimitReader(r.Body, httpx.MaxBodyBytes))
  r.Body = io.NopCloser(bytes.NewReader(body))
  if err ≠ nil {
    return "login:unreadable"
  }
  var b struct {
    Email string `json:"email"`
  }
  if json.Unmarshal(body, &b) ≠ nil || b.Email == "" {
    return "login:unparsed"
  }
  return "login:" + user.NormalizeEmail(b.Email)
}
```

listing 185: KeyEmail: a bounded read, the body handed back, garbage into one shared bucket (source: go/api/internal/limit/handler.go, lines 62-79)

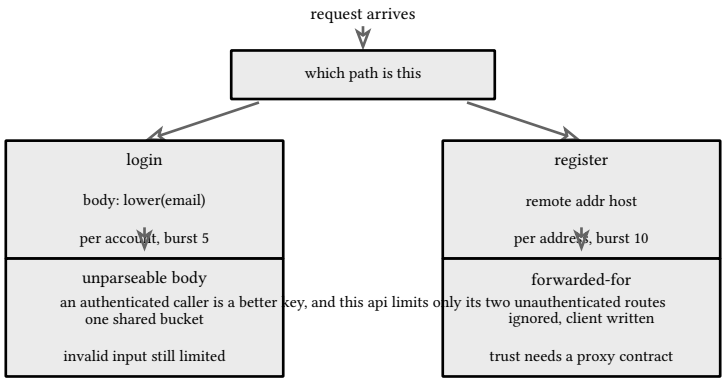


figure 163: key choice: the route decides what the bucket belongs to

26.6. limits under test

Every time-dependent behavior in this chapter is arithmetic on a fixed clock, because a rate limiter tested against the wall clock is tested against a scheduler. The injected clock moves in explicit offsets, the fixed-draw vectors walk the bucket through its whole grammar, burst, cap, sustained rate, half-token denial, and the vector replay runs the production middleware against the frozen file: five setup draws assert the remaining sequence 4 through 0, the sixth asserts the vector's status, every header byte, and the envelope's code and message, with the request id checked for uuid v7 shape only since this layer mints it fresh. The `Wait` path runs inside a `testing/synctest` bubble, where the fake clock jumps the moment every goroutine is durably blocked, so the limiter's one-second wait costs no real time and the test still asserts the bubble advanced by a full refill period. The race detector runs over the package because the buckets map is concurrent state, and no test anywhere in it sleeps:

```
c := &clock{now: t0.Add(12 * time.Second)}
l := NewLimiter(c.Now, 0)
l.Rule("/api/auth/login", LoginRule())
h := l.Middleware(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
  w.WriteHeader(http.StatusUnauthorized)
}))

// the vector's setup: five logins at this offset, remaining 4..0
for want := 4; want ≥ 0; want-- {
  rec := httptest.NewRecorder()
  h.ServeHTTP(rec, loginBody("ada@example.org"))
  if rec.Code ≠ http.StatusUnauthorized {
    t.Fatalf("setup draw status %d, want 401", rec.Code)
  }
  if got := rec.Header().Get(HeaderRemaining); got ≠ strconv.Itoa(want) {
    t.Fatalf("setup remaining %s, want %d", got, want)
  }
}
```

listing 186: the vector replay: frozen clock at t0+12s, five draws, then the frozen sixth (source: go/api/internal/limit/limit_test.go, lines 125-142)

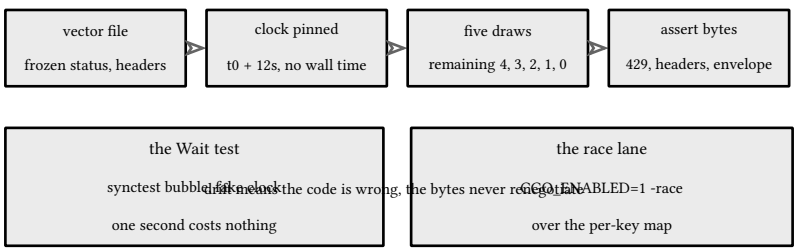


figure 164: the fixed-vector pipeline: file in, draws through the gate, bytes out

26.7. admission preview

A rate limit refuses requests over time, an admission gate refuses them over concurrency, and the two compose because they bound different axes of the same cliff. Little's law states the trade in one line: the average number of requests inside the system equals the arrival rate times the time each spends there, `L = lambda W`. Offered load raises the right side two ways, more arrivals or slower service, and past the point where every arriving request finds all workers busy, latency rises steeply while throughput stops moving, the hockey stick the load chapter measures. A rate limit caps lambda at the edge. An admission gate caps L directly with an in-flight semaphore, holds one slot per admitted request, and answers the contract's `overload` envelope the instant the slots run out, which converts a queue of waiting requests, each holding memory and a goroutine, into a cheap refusal. The semaphore and its 503 are the load chapter's to build, along with the drain it protects:

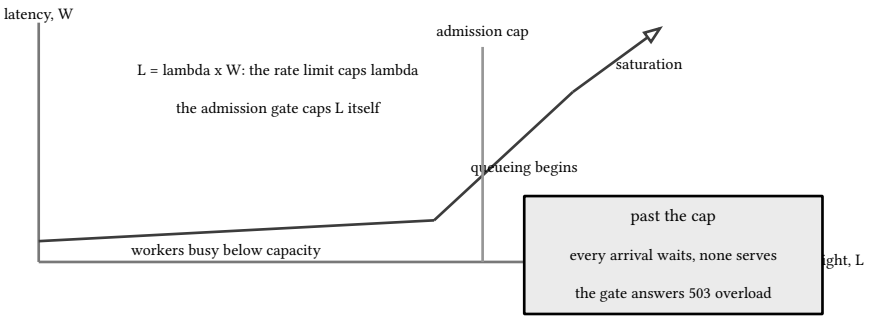


figure 165: latency versus concurrency: the hockey stick the admission cap cuts off

sources: pkg.go.dev for golang.org/x/time/rate, read against the go 1.27 module cache source for Allow, Wait, and TokensAt, plus the testing/synctest package doc for the bubble and durable blocking rules, and RFC 6585 for status 429 with RFC 9110 for the Retry-After field semantics, all accessed 2026-09-25. Verified by `go/api/internal/limit` tests, 11 of them, under `go test -race`, plus gofmt and go vet over the module, with the sixth-login vector replayed byte for byte from `contract/testdata/06-login-429.json`.

27. observability

A service no one can see is a service no one can fix, and a service that logs prose no machine can query is seen only by whoever is reading along live. This chapter builds the service's observability in `goapi/internal/obs`: the real slog setup with json lines, source locations, context loggers, and redaction of secret-bearing fields, the W3C traceparent header parsed and formatted from scratch with spans held in context and rendered as a tree, the metrics registry and its prometheus text exposition with the counter and latency histogram the contract pins, p99 done honestly by nearest rank with the bucket approximation labeled as the upper bound it is, pprof mounted on the internal listener, and one login request followed through every signal at once.

27.1. slog the real setup

The production logger is one constructor, not a pattern study: json lines to a writer, a level, source locations on demand, and one `ReplaceAttr` hook that does two jobs. The first job is redaction, installed once at the top so every line the service ever writes passes through it, top-level attrs and group members alike: a closed set of keys, folded to lower case, string values swapped for `[redacted]`, because a login handler that logs its request body verbatim logs a password, and grep never forgets. The second job is the fixed clock: when the option's `Now` is set, the hook replaces the time attr with it, which is how the golden test pins bytes without touching the wall, and how a log line can be replayed in a test the way a response body can:

```
// LogOptions configures NewLogger. Now, when set, replaces the
// handler's time attribute with a fixed value, which is how the
// golden test pins bytes without a clock.
type LogOptions struct {
  Writer io.Writer
  Level  slog.Level
  // Source records the file and line of each log call.
  Source bool
  // Now fixes the time attribute, tests only.
  Now func() time.Time
}

// NewLogger builds the production logger: json lines, a level, source
// locations when asked, redaction always.
func NewLogger(o LogOptions) *slog.Logger {
  return slog.New(slog.NewJSONHandler(o.Writer, &slog.HandlerOptions{
    Level:       o.Level,
    AddSource:   o.Source,
    ReplaceAttr: replaceAttr(o.Now),
  }))
}
```

listing 187: one constructor: json, level, source, and the hook that redacts and fixes time (source: go/api/internal/obs/logger.go, lines 42-62)

The golden test pins the whole line, and the line earns a sentence of reading: the time member is the injected clock, the level and msg come from the call, attrs land in call order, and the password attr is the masked value, never the secret. With source on, the same constructor adds a `source` member carrying the file and function of the log call, which is why the level exists at all: debug lines are compiled in but silent until the level drops.

```
{"time":"2026-10-01T05:00:00Z","level":"INFO","msg":"login
failed","email":"ada@example.org","password":"[redacted]","attempts":3}
```

fragment, illustrative, not executed

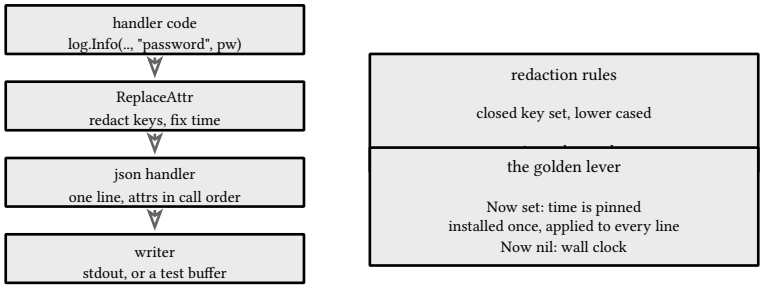


figure 166: the logger's own stack: call site through handler to writer

27.2. structured beats interpolated

An interpolated message packs data into prose, and prose is lossy: `login failed for ada, attempt 3` cannot be grouped by email, counted by attempt, or joined to its request id without a regex that breaks the day the message rewords itself. The structured line carries the same information as fields, and every field keeps its name, its type, and its place in an index. The service's rule is the one the access log already follows: messages name the event, attrs carry the data, and no handler interpolates a value it could pass as an attr.

Handlers never touch a global to do it. The context carries the configured logger, `FromContext` resolves it with a fallback to `slog.Default` so a missing logger can never panic a request, and `RequestLogger` attaches the request id the kernel minted at the edge, the same value the access log line and any error envelope carry. One id, one line of grep, every artifact of one request:

```
// FromContext resolves the context's logger, falling back to
// slog.Default so a missing logger never panics a request.
func FromContext(ctx context.Context) *slog.Logger {
  if log, ok := ctx.Value(loggerKey{}).(*slog.Logger); ok {
    return log
  }
  return slog.Default()
}

// RequestLogger returns the logger for one request: the context
// logger with the request id attached, so handler logs, the access
// log, and any error envelope quote one value.
func RequestLogger(r *http.Request) *slog.Logger {
  return FromContext(r.Context()).With("request_id", httpx.RequestID(r))
}
```

listing 188: the context logger and the request logger that attaches the id (source: go/api/internal/obs/logger.go, lines 81-95)

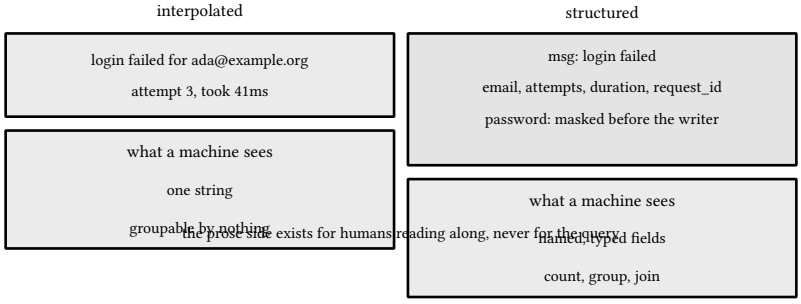


figure 167: the same failure, line-oriented and structured

27.3. tracing from scratch

A trace is the shape of one request's time: nested spans, each with a name, an id, a parent, and a start and end, joined across services by one shared header. The W3C trace context format is deliberately small, four lowercase hex fields, `version-traceid-spanid-flags`, 2, 32, 16, and 2 digits, and small enough to parse and format by hand, which is what this package does: no sdk, no collector, no spans leave the process, the point is to know the format and the tree, because every vendor's tracing story is this header plus a place to send the spans.

The parser is strict where the spec allows a choice. Four fields of the exact lengths, version `ff` reserved and rejected, all-zero trace or span ids rejected, uppercase hex rejected because lowercase is the canonical rendering and this parser takes the strict side, and only the sampled bit of the flags byte kept, the bit the spec says propagates, everything else masked to zero exactly as the wire writes it back. On the server side one call continues an inbound trace: parse, then a root span whose trace id is the inbound trace and whose parent is the remote span, sampling bit inherited. A header that is absent or malformed starts a fresh trace and reports false, because one bad header must never kill a request:

```
func ParseTraceparent(v string) (TraceID, SpanID, byte, error) {
  var traceID TraceID
  var spanID SpanID
  parts := strings.Split(v, "-")
  if len(parts) ≠ 4 || len(parts[0]) ≠ 2 || len(parts[1]) ≠ 32 || len(parts[2]) ≠ 16 || len(parts[3]) !
= 2 {
    return traceID, spanID, 0, ErrMalformedTraceparent
  }
  if parts[0] == "ff" {
    return traceID, spanID, 0, ErrMalformedTraceparent
  }
  for _, p := range parts {
    if strings.IndexFunc(p, func(r rune) bool { return r ≥ 'A' && r <= 'Z' }) ≥ 0 {
      return traceID, spanID, 0, ErrMalformedTraceparent
    }
  }
```

listing 189: parse: four fields, exact lengths, reserved version and case rejected (source: go/api/internal/obs/trace.go, lines 45-59)

```
// ContinueFrom opens a root span joined to an inbound traceparent:
// same trace, parent is the remote span, sampling flag carried. A
// header that is absent or malformed starts a fresh trace and
// reports false, so one bad header never kills a request.
func (tr *Tracer) ContinueFrom(ctx context.Context, name, header string) (context.Context, *Span, bool) {
  if header == "" {
    ctx, sp := tr.Start(ctx, name)
    return ctx, sp, false
  }
  traceID, parentID, flags, err := ParseTraceparent(header)
  if err ≠ nil {
    ctx, sp := tr.Start(ctx, name)
    return ctx, sp, false
  }
  ctx, sp := tr.open(ctx, name, nil)
  sp.TraceID = traceID
  sp.ParentID = parentID
  sp.Sampled = flags&0x01 == 0x01
  return ctx, sp, true
}
```

listing 190: continue an inbound trace, or root a fresh one when the header fails (source: go/api/internal/obs/trace.go, lines 148-168)

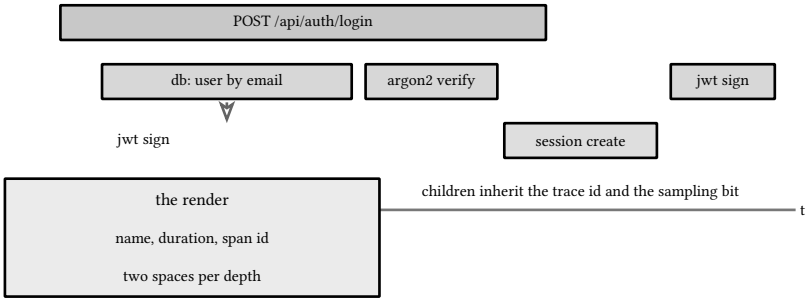


figure 168: one login request's spans over time, the tree the render prints

27.4. metrics and the prometheus text format

Two series carry the load story, and both are registry types, not libraries. The counter is one series per route and status code, `goapi_requests_total{route,code}`, incremented once per request by the middleware. The histogram is one per route, `goapi_request_duration_seconds`, with the contract's bucket ladder tight at the low end where service latencies live and geometric through the tail: one, two and a half, five, ten, twenty five, fifty milliseconds, then the hundreds, seconds, and `+Inf`. Each observation increments every bucket whose bound it fits under, which is what makes the counts cumulative, plus the `+Inf` bucket, the sum, and the count, the four members a scraper needs.

The exposition writer renders the prometheus text format directly, help and type lines, series sorted by label tuple, bucket lines ascending with `+Inf` last, and the frozen metrics vector pins it byte for byte: one slow `/healthz` observation renders exactly the eight lines the contract froze, because the writer elides zero buckets, the one deliberate divergence from `client_golang`, which prints every finite bucket at zero as well. The route label is the matched pattern minus its method, so the label reads `/healthz`, not `GET /healthz`, and the middleware asks the mux for the match itself, the same `mux.Handler` call the authz guard and the preconditions layers use, because reading `r.Pattern` after the inner handler returns is a trap: the timeout layer beneath binds its deadline with `r.WithContext`, which hands the mux a copy of the request, and the pattern the mux fills in on that copy never travels back out to the layers above, so the label would read empty on every series over the real wiring. Each recorded observation then lands in the route's histogram:

```
func (m *Metrics) Observe(route string, d time.Duration) {
  m.mu.Lock()
  defer m.mu.Unlock()
  h := m.hists[route]
  if h == nil {
    h = &histogram{cum: make([]uint64, len(DurationBuckets)+1)}
    m.hists[route] = h
  }
  secs := d.Seconds()
  for i, bound := range DurationBuckets {
    if secs <= bound {
      h.cum[i]++
    }
  }
  h.cum[len(h.cum)-1]++
  h.sum += secs
  h.n++
}
```

listing 191: one observation: cumulative buckets, sum, count (source: go/api/internal/obs/metrics.go, lines 66-83)

```
# HELP goapi_requests_total requests by route and code.
# TYPE goapi_requests_total counter
goapi_requests_total{route="/healthz",code="200"} 1
# HELP goapi_request_duration_seconds request latency.
# TYPE goapi_request_duration_seconds histogram
goapi_request_duration_seconds_bucket{route="/healthz",le="+Inf"} 1
goapi_request_duration_seconds_sum{route="/healthz"} <observed>
goapi_request_duration_seconds_count{route="/healthz"} 1
```

listing 192: the frozen vector the exposition is checked against, byte for byte (source: go/api/contract/testdata/15-metrics.txt, lines 1-8)

The endpoint itself binds on the internal listener in production, next to the pprof surface of the sixth section, and for two reasons: the contract says the metrics route is unauthenticated, and the authz guard wraps the public mux deny by default, so a route with no policy row belongs on the mux the guard never touches.

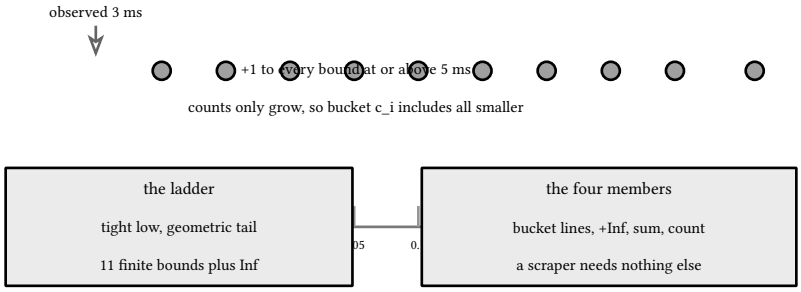


figure 169: cumulative buckets: each observation fills every bound above it

27.5. p99 done honestly

The percentile definition is nearest rank: sort the latencies, take the ceil of p over 100 times n as the rank, answer that rank's value. One generic function over any ordered type is the whole definition, the same one the profiling chapter pinned and the load chapter reports from, and its honesty is in the details it refuses to hide: p99 of one hundred values is the ninety-ninth smallest, not an interpolation between the ninety-ninth and hundredth, and p99.9 of ten samples clamps to the largest, because a percentile past the data is a bound, not a measurement.

The bucket approximation is the second half of honesty. A scraped histogram cannot answer nearest rank, it can only say which bucket the rank lands in, so `FromCumulative` answers the smallest bound whose cumulative count

reaches the rank, which is an upper bound on the true value, never the value itself, and answers not-knowable when the rank falls into `+Inf`. The test pins the contrast with fixed vectors: ninety-nine samples at 0.9 milliseconds and one at 1.1 gives an exact p99 of 0.9 and a bucket p99 of 1.0, the bound. One caveat carries forward from the profiling chapter: this machine's monotonic clock steps in roughly half milliseconds, so timing sub-tick work with `time.Now` mostly returns zero, and the latency numbers worth quoting are either sums over many iterations or observations bigger than the tick:

```
func NearestRank[T cmp.Ordered](sorted []T, p float64) T {
  if len(sorted) == 0 {
    var zero T
    return zero
  }
  rank := int(math.Ceil(p / 100 * float64(len(sorted))))
  if rank < 1 {
    rank = 1
  }
  if rank > len(sorted) {
    rank = len(sorted)
  }
  return sorted[rank-1]
}
```

listing 193: nearest rank: ceil the rank, clamp the ends, no interpolation anywhere (source: go/api/internal/obs/quantile.go, lines 15-28)

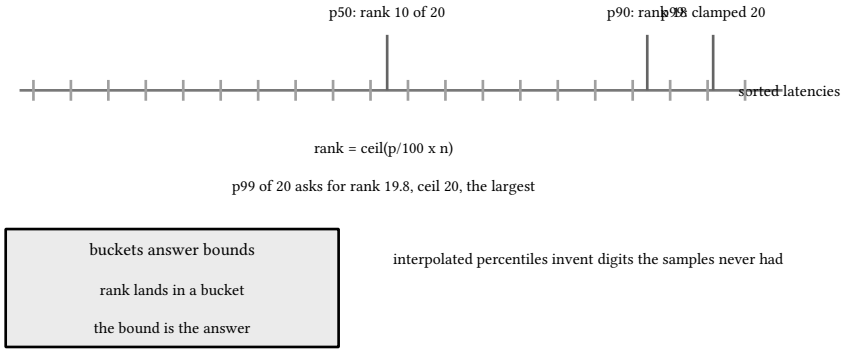


figure 170: the tail ladder: where p50, p90, p99, p99.9 sit in one sorted sample

27.6. pprof on the live server

The profiling surface ships with the binary, mounted by one call, and where it is mounted is the whole security story. The index serves every named profile, `/debug/pprof/heap`, `goroutine`, `block`, `mutex`, `threadcreate`, and `allocs`, and the four fixed endpoints cover the special ones: `profile` takes a cpu sample for the seconds you ask, `trace` records an execution trace, `symbol` and `cmdline` serve the tooling. A profile is an oracle: `cmdline` reads the process arguments, the heap reads object sizes, a cpu profile reads where the work happens. So the mux it lands on is the internal listener, the one bound next to the public port and reachable only from inside the deployment, never the public one, and the wiring is one goroutine with its own address:

```
// RegisterDebug mounts the standard profiling surface under
// /debug/pprof/ on the mux it is given: the index, which also serves
// every named profile, plus the four fixed endpoints. Production
// binds it on the internal listener only, never the public one,
// because a profile is an oracle: cmdline leaks the process, heap
// leaks sizes, cpu leaks where the work is.
func RegisterDebug(mux *http.ServeMux) {
  mux.HandleFunc("GET /debug/pprof/", pprof.Index)
  mux.HandleFunc("GET /debug/pprof/cmdline", pprof.Cmdline)
  mux.HandleFunc("GET /debug/pprof/profile", pprof.Profile)
  mux.HandleFunc("GET /debug/pprof/symbol", pprof.Symbol)
  mux.HandleFunc("GET /debug/pprof/trace", pprof.Trace)
}
```

listing 194: one call mounts the whole standard surface, listener placement is the policy (source: go/api/internal/obs/pprof.go, lines 7-20)

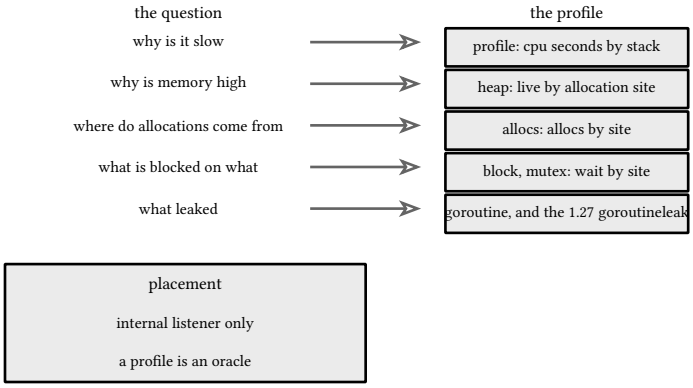


figure 171: which profile answers which question

27.7. one request fully observed

Wire the signals in the order the request meets them and one login tells the whole story. The middleware stack is the previous chapters' onion with one layer added: the access log already records method, path, status, bytes, duration, and the request id, and the metrics middleware wraps the same stack one layer out, so the counter ticks at the status the writer really wrote and the histogram at the duration the access log also printed. The span wraps the handler, the log line and the span agree on start and end because they are the same request, and the metrics route label agrees with the access log's path because both read the same matched request. Reading it back is what the tests do, separately and without ceremony: the middleware test drives a real mux and asserts the labels and counts that come out, the logger test asserts the id on the line, the tracer test pins the rendered tree bytes:

```
func (m *Metrics) Middleware(mux *http.ServeMux) func(http.Handler) http.Handler {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      start := time.Now()
      route := RouteLabel(mux, r)
      sw := &codeWriter{ResponseWriter: w}
      next.ServeHTTP(sw, r)
      m.CountRequest(route, sw.status())
      m.Observe(route, time.Since(start))
    })
  }
}
```

listing 195: the metrics layer: pattern asked of the mux, status captured, both series ticked (source: go/api/internal/obs/metrics.go, lines 167-178)

```
metrics := obs.NewMetrics()
app.Use(metrics.Middleware(app.Mux))
debugMux := http.NewServeMux()
debugMux.HandleFunc("GET /metrics", func(w http.ResponseWriter, r *http.Request) {
  metrics.WriteExposition(w)
})
obs.RegisterDebug(debugMux)
go func() {
  log.Error("debug listener exited", "err", http.ListenAndServe(debugAddr, debugMux))
}()
```

fragment, illustrative, not executed

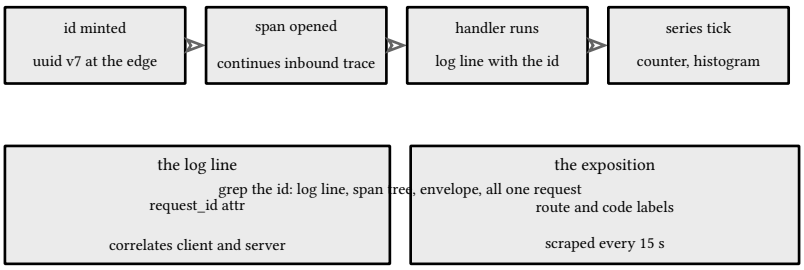


figure 172: one request id through every signal

sources: pkg.go.dev for log/slog's HandlerOptions, ReplaceAttr, and Attr kinds, net/http/pprof, and runtime/pprof including the go 1.27 goroutineleak profile, the W3C trace context format at w3c.github.io/trace-context for the traceparent grammar and the sampled flag, and the prometheus exposition format documentation with the client_golang histogram behavior read from its source, all accessed 2026-09-25. Verified by `go/api/internal/obs` tests, 17 of them, under `go test -race`, plus gofmt and go vet over the module, with the metrics exposition replayed line for line from `contract/testdata/15-metrics.txt`.

28. load and p99

A service that has never been loaded is a service whose p99 is a guess, and a deploy that has never been drained is a deploy that drops requests every time it ships. This chapter closes the service part with the tools that make both claims checkable: the open-loop and closed-loop harness in `goapi/internal/loadtest`, the report table and its honest percentiles, a benchstat comparison over captures committed with the code, graceful shutdown as a new file in the kernel's package, the admission gate that answers overload at the cap, and the go 1.27 goroutine leak check that holds it all honest after a run.

28.1. the load harness

The two loops differ in what they hold constant. The closed loop holds concurrency constant: `workers` goroutines, each firing one request at a time and starting the next only after the previous response lands, so offered load is whatever the server can keep up with and never exceeds the worker count. It measures a system at a chosen concurrency. The open loop holds arrival rate constant: request `i` is scheduled at `i` over rate seconds past the start, fired whether or not anything has come back, so when the server slows, requests pile up exactly the way they do in production, and the cliff becomes visible. Little's law from the rate limiting chapter predicts the pile: in-flight equals arrival rate times service time, and an open loop at fixed arrival rate turns rising latency directly into rising in-flight count.

Both loops share one harness with three injected seams: the client, so tests drive deterministic responses through a `RoundTripper` function with no socket, the clock, so latency is arithmetic on ticks, and the sleeper, so the open loop's pacing is a function the test can call instantly while still asserting the schedule. Warmup runs first on every loop and never touches the sink, because the first request pays for the dial, the dns lookup, and the cold caches, and none of that is the number the run exists to measure:

```
func (h *Harness) Closed(ctx context.Context, workers, perWorker, warmup int) (Result, error) {
  s := &sink{res: newResult()}
  for i := 0; i < warmup; i++ {
    if err := h.fire(ctx, nil, true); err != nil {
      return s.res, err
    }
  }
  start := h.Now()
  var wg sync.WaitGroup
  for w := 0; w < workers; w++ {
    wg.Go(func() {
      for i := 0; i < perWorker; i++ {
        // a canceled context ends the worker; the recorded
        // count says how far it got
        _ = h.fire(ctx, s, false)
      }
    })
  }
  wg.Wait()
  s.res.Elapsed = h.Now().Sub(start)
  return s.res, nil
}
```

listing 196: the closed loop: warmup first, one request at a time per worker, elapsed from the injected clock (source: go/api/internal/loadtest/harness.go, lines 93-114)

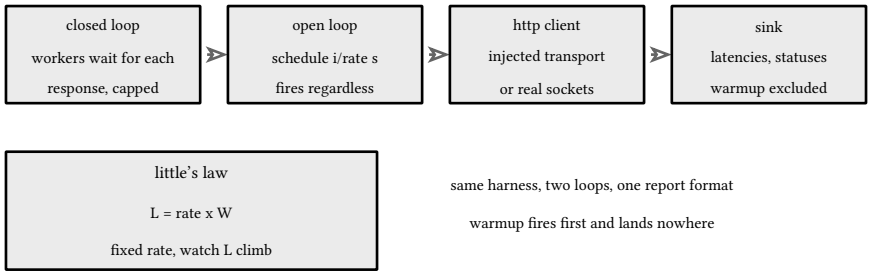


figure 173: workers, server, latency sink: one harness, two clocks

28.2. the report

One run folds into one row. The percentiles are nearest rank, the corpus definition, computed by the observability package's helper over the sorted latencies, and the achieved rate is the recorded count over the elapsed span the injected clock measured. Rendering goes through `tabwriter`, one header row and one row per run with columns that name their unit, because a latency column that does not say milliseconds is a unit error waiting for a reader.

The golden test pins the bytes from a fixed latency set, and it pins them twice. The first row summarizes the raw samples. The second summarizes the same samples as a scraper would see them, cumulative histogram counts, exact count and sum, percentiles read out of the buckets, and the row says so in its source column: six latencies of 1, 1, 2, 3, 5, and 8 milliseconds give an exact p50 of 2 and p99 of 8, while the bucketed row reads p50 as 2.5 and p99 as 10, the bounds the rank lands in. Two rows, same run, the difference is the resolution a histogram throws away, stated in the table instead of hidden in a footnote:

```
func Render(w io.Writer, reps ...Report) error {
  tw := tabwriter.NewWriter(w, 0, 4, 2, ' ', 0)
  fmt.Fprintln(tw, "name\tn\terrors\trps\tp50 ms\tp90 ms\tp99 ms\tmax ms\tsrc")
  for _, r := range reps {
    if _, err := fmt.Fprintf(tw, "%s\t%d\t%d\t%.1f\t%.3f\t%.3f\t%.3f\t%.3f\t%s\n",
      r.Name, r.N, r.Errors, r.RPS,
      ms(r.P50), ms(r.P90), ms(r.P99), ms(r.Max), r.Source); err ≠ nil {
      return err
    }
  }
  return tw.Flush()
}
```

listing 197: the table: tabwriter aligned, units in the header, source column labeling the approximation (source: go/api/internal/loadtest/report.go, lines 100-111)

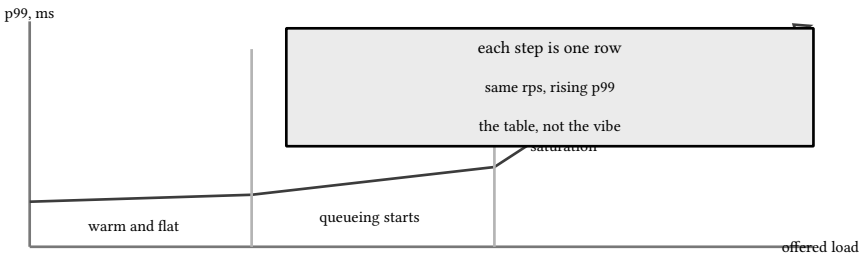


figure 174: p99 under offered-load steps: where the cliff shows in the table

28.3. benchstat over committed captures

A single benchmark number is weather, the profiling chapter already said so, and the discipline is the same one it taught: run both sides ten times, let `benchstat` compute the statistics, quote the table, commit the capture. The two sides here are the route bare, one recorder round trip through the kernel's json write path, and the same route under the full cross-cutting stack the shipped binary runs, request id, access log, metrics with the histogram tick,

the admission gate, and recover. The capture lives with the code in captures/, dated, twenty raw runs, and the summary is what the chapter quotes:

```
goos: windows
goarch: amd64
pkg: goapi/internal/loadtest
cpu: 12th Gen Intel(R) Core(TM) i7-12700F
            │ internal/loadtest/captures/2026-09-25-stack.txt │
            │                      sec/op                     │
Bare-20                                          606.8n ± 2%
Observed-20                                      2.002µ ± 9%
geomean                                          1.102µ
```

listing 198: benchstat over the committed capture, 2026-09-25, this machine (source: go/api/internal/loadtest/captures/2026-09-25-benchstat.txt, lines 1-9)

Reading it: the bare route answers in 606.8 nanoseconds with a two percent spread, the observed route in 2.002 microseconds with a nine percent spread, about 3.3 times, which is the per-request price of every cross-cutting chapter combined, and the intervals do not overlap, so the ratio is structural rather than noise. Two microseconds is also the right scale check: the stack costs about two microseconds against a login that costs tens of milliseconds of argon2, and knowing which costs dominate is the whole skill.

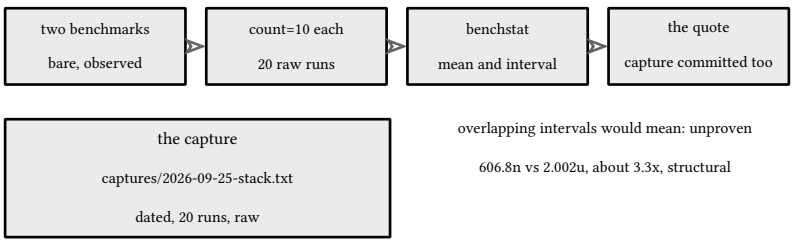


figure 175: the bench flow: code, capture, statistics, quote

28.4. graceful shutdown

Shutdown has an order, and the order is the contract. Readiness flips first: the drainer's state moves to draining, /readyz answers its 503 overload envelope, and every load balancer watching the service stops routing new requests here while the old ones are still being answered well. Then the listener closes, so new connections are refused at the tcp level. Then Server.Shutdown waits for in-flight requests to finish under a grace deadline, and returns the deadline error when grace expires with work still inside, so the caller decides whether that is fatal instead of the helper pretending the drain succeeded.

The serve helper is one new file in the kernel's package, shutdown.go, next to the code it drains, touching none of the kernel's pinned files. It takes the listener rather than an address, which is what makes it testable: the test owns the port, the helper owns the lifecycle, and the drainer is shared state the readiness route already reads through the config hook. The wiring in the program's table replaces the plain serve tail: build the drainer, install it as the app's readiness probe, serve on a signal context, drain with thirty seconds of grace. The frozen main.go prefix keeps its byte-stable tail and simply never reaches it, which is the one sanctioned way this part adds a serve path without editing a pinned listing:

```
func Serve(ctx context.Context, ln net.Listener, srv *http.Server, dr *Drainer, grace time.Duration) error
{
  serveErr := make(chan error, 1)
  go func() { serveErr <- srv.Serve(ln) }()
  select {
  case err := <-serveErr:
    dr.close()
    return err
  case <-ctx.Done():
    dr.BeginDrain()
    dctx, cancel := context.WithTimeout(context.Background(), grace)
    defer cancel()
    err := srv.Shutdown(dctx)
    dr.close()
    return err
  }
}
```

listing 199: Serve: serve until ctx dies, flip readiness, drain under grace, report the deadline (source: go/api/internal/httpx/shutdown.go, lines 83-98)

```
dr := httpx.NewDrainer()
app.SetReady(dr.Ready)
srv := &http.Server{Handler: app.Handler(), ReadHeaderTimeout: 5 * time.Second}
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
ln, err := net.Listen("tcp", httpx.ConfigFromEnv().Addr)
if err != nil { log.Fatal(err) }
if err := httpx.Serve(ctx, ln, srv, dr, 30*time.Second); err != nil { log.Fatal(err) }
```

fragment, illustrative, not executed

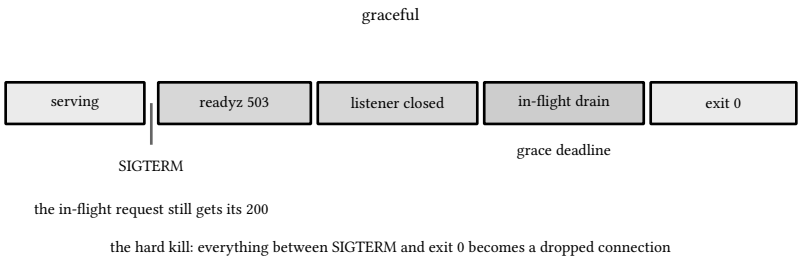


figure 176: SIGTERM to drain to exit, against the hard kill

28.5. testing shutdown without sleeps

The drain test is four observations on one gated request, and none of them wait on the clock. The handler blocks on a channel the test holds. Cancel the context. First, readiness flips on the same tick: the test yields until the drainer's state moves, `runtime.Gosched` in a loop, no sleep anywhere. Second, fresh dials are refused: the test probes with a client that disables keep-alive, so every probe is a new connection, and once the listener closes every probe fails, which is the poll of real state, not of a timer. Third, the gated request still completes: release the channel, the in-flight response arrives a 200, and the serve helper returns nil. Fourth, in the sibling test, an expired grace is reported as the deadline error rather than success. The listener-failure path has its own test, because a helper that swallows the listen error would hide a misconfigured port behind a clean exit:

```
cancel()
// the flip happens in the Serve goroutine the moment it observes
// the canceled context: yield until it does, no sleeps
for i := 0; dr.Ready(); i++ {
  if i > 1<<22 {
    t.Fatal("readiness never flipped after cancel")
  }
  runtime.Gosched()
}

// new connections are refused once the listener closes; each probe
// is a fresh dial, and the loop polls real state, no sleeps
refused := false
for i := 0; i < 10000; i++ {
  resp, err := client.Get("http://" + addr + "/")
  if err != nil {
    refused = true
    break
  }
  resp.Body.Close()
}
if !refused {
  t.Fatal("10k fresh dials during drain all connected, the listener never closed")
}
```

listing 200: cancel, yield until the flip, then poll fresh dials until the listener refuses (source: go/api/internal/loadtest/shutdown_test.go, lines 99-122)

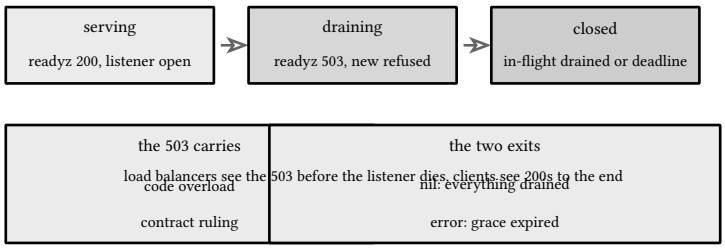


figure 177: the drain lifecycle: serving, draining, closed

28.6. admission control

The rate limit bounds arrivals over time. The admission gate bounds requests inside the service at once, and the mechanism is a counting semaphore: one buffered channel of capacity `n`, a send to take a slot, the receive riding a `defer` to give it back, so a handler that panics still releases, the recover layer beside it does the rest. The nonblocking select is the whole policy: a slot is taken when one is free, and when the gate is full the answer is immediate, the contract's overload envelope at 503, never a queue, because a queued request holds a goroutine and memory the service just proved it cannot spend. The cap is the deployment's honest concurrency, worker and memory budget, not what a client would like.

The barrier test pins exactly who gets in. A gate of four, four concurrent requests each blocked on a barrier after entering, so the test observes `InFlight` at exactly the cap, then four synchronous requests that all answer 503 with the envelope, then the barrier opens and the holders finish. Little's law closes the loop: at a fixed arrival rate, the gate caps `L`, and capped `L` with rising service time means the excess is refused at the door instead of queueing inside:

```
func (g *Gate) Middleware(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    select {
    case g.slots <- struct{}{}:
    default:
      httpx.Error(w, r, httpx.Fail(httpx.Overload, OverloadMessage))
      return
    }
    defer func() { <-g.slots }()
    next.ServeHTTP(w, r)
  })
}

// InFlight reports how many slots are held right now, the number the
// little's law arithmetic in the load chapter is about.
```

listing 201: the gate: nonblocking take, defer release, overload envelope at the cap (source: go/api/internal/admission/admission.go, lines 35-49)

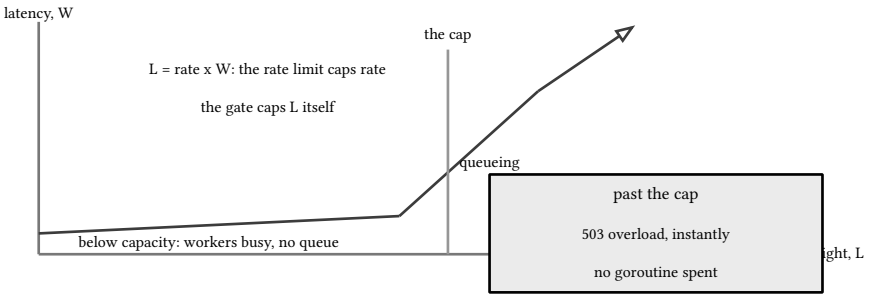


figure 178: latency versus in-flight: the hockey stick the gate cuts

28.7. the goroutine leak check

A load tool that leaks goroutines per run silently slows every later run and lies about the service it measures. Go 1.27 ships the answer in the runtime: the `goroutineleak` profile, whose write runs a detection GC and reports the goroutines that are blocked and unreachable from anything live, which is the definition of a leak. The check is four lines at the end of the harness test: run the closed loop against a real `httptest` server, close the server, close the client's idle connections, look the profile up, write it once to trigger detection, and assert the count is zero. On failure the test dumps the leaked stacks at verbosity 2, because a leak assertion that says only a number sends the reader hunting blind:

```
func TestHarnessRunLeaksNoGoroutines(t *testing.T) {
  ts := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
  }))
  h := New()
  h.BaseURL = ts.URL
  clock := &tickClock{now: t0}
  h.Now = clock.Now

  if _, err := h.Closed(context.Background(), 4, 5, 2); err ≠ nil {
    t.Fatalf("harness run: %v", err)
  }
  ts.Close()
  h.Client.CloseIdleConnections()

  prof := pprof.Lookup("goroutineleak")
  if prof == nil {
    t.Fatal("this runtime carries no goroutineleak profile")
  }
  if err := prof.WriteTo(io.Discard, 0); err ≠ nil {
    t.Fatalf("leak profile: %v", err)
  }
```

listing 202: run, close everything, then ask the runtime what is still alive (source: go/api/internal/loadtest/leak_test.go, lines 19-40)

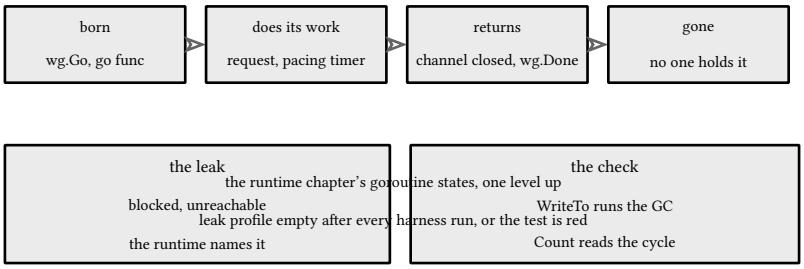


figure 179: a goroutine lifecycle: born, joined, or leaked

sources: pkg.go.dev for net/http Server.Shutdown semantics, the ListenAndServe and Serve contract, testing/synctest is absent here but runtime/pprof's goroutineleak profile is read from the go 1.27 source in GOROOT including its detection GC, golang.org/x/perf benchstat for the summary statistics, and the little's law statement as used across the corpus, accessed 2026-09-25. Verified by `goapi/internal/loadtest` and `goapi/internal/admission` tests, 15 of them, under `go test -race`, plus the committed capture `2026-09-25-stack.txt` and the benchstat summary over it, quoted above, measured on this machine, a 12th gen i7-12700F under windows/amd64.

29. ship it

The module ends as a directory of tests unless it also ships. This chapter turns the service into an artifact and a lane: a multi-stage image that runs anywhere, one compose command that brings it up healthy, a make gate that replays the contract over that stack, a CI workflow that runs the same legs on every push, and the two envelopes real platforms ask for, a helm chart and a terraform slice. Everything leans on machinery the earlier chapters already built: the health and ready routes chapter 18 owns, the strong ETags chapter 20 computes, the drain state chapter 28 wires, the metrics chapter 27 exposes. One rule disciplines the whole chapter: the compose lane is the only thing that executes, and the chart, the terraform, and the workflow ship as taught, documentation-verified copies.

29.1. the image

Two stages, one boundary. The builder stage is the stock golang image, which exists only to run the compiler, and the runtime stage is scratch, which exists only to run the binary. The copy across the boundary is a single file, and every property of the service module conspired to make that legal: the store engine is pure go, so CGO_ENABLED=0 builds a static binary with no c runtime to carry, the schema versions live in the store's own snapshot header, so the history ships inside the store files, and the configuration is environment variables, so no file needs mounting to start:

```
# multi-stage keeps the go toolchain out of the shipped image: the
# builder compiles once, the runtime copies one file. the store engine
# is pure go, so CGO_ENABLED=0 holds and no c runtime ships.
FROM golang:1.27 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -o /out/api ./cmd/api \
    && mkdir /data && chown 65532:65532 /data

# scratch runtime: the static binary, an empty /data the non-root user
# owns so the store volume mounts writable on first run, nothing else.
FROM scratch
ARG VERSION=dev
ENV GOAPI_VERSION=${VERSION}
COPY --from=build --chown=65532:65532 /out/api /api
COPY --from=build --chown=65532:65532 /data /data
USER 65532:65532
EXPOSE 8080
ENTRYPOINT ["/api"]
```

listing 203: the whole image: builder stage, static binary, scratch runtime, non-root user (source: go/api/Dockerfile, lines 1-21)

Four decisions in that file earn a second look. The go.mod copy precedes the source copy so the dependency download layer caches independently of code edits, the common multi-stage economy. The /data directory is created in the builder and copied with an owner, because a named volume inherits the image content's ownership on first mount, and a volume mounting onto a root-owned directory would deny the container user its database. The user is numeric, 65532, since scratch carries no /etc/passwd to resolve a name. And the build keeps its symbols: -trimpath strips local paths from the binary but -s -w, the size-shaving pair, is deliberately absent, because chapter 27 teaches profiling the live server and a stripped binary answers those profiles with addresses instead of function names.

The healthcheck has the same shape problem in reverse: scratch has no shell, no wget, no curl, so nothing exists in the image to probe with. The answer is the one executable that does exist, the service itself. The binary accepts

a -health argument, probes /healthz on its configured address over the loopback interface, and exits 0 or 1, so the container orchestrator's probe command is the same binary, one flag apart from the server.

The build context is the whole module, tests and frozen vectors included, because the compiler only reaches what the package graph names and the rest rides the context without landing in the image. A .dockerignore would trim that upload and speed the build marginally, at the cost of one more file whose exclusions can silently drift from the module's shape, so the file here is none and the trade is stated instead of paid.

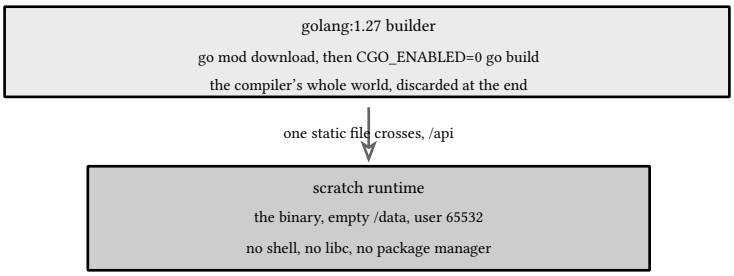


figure 180: two stages, one file crosses: the builder dies with the build, scratch runs the binary

Distroless is the other honest choice: a minimal base with ca certificates, tzdata, and a non-root user baked in, at a few megabytes more. This service needs none of that, so scratch wins, but the moment the binary dials tls or formats local times, distroless is the smaller of the correct options rather than the larger of the fast ones.

29.2. compose one command

The compose file is one service and its volume, and the contract it enforces is the return value of a single command. `docker compose up -d --build --wait` builds the image, creates the container, starts it, and then blocks until the healthcheck passes, so a green up means the image compiled, the migrations ran against a fresh volume, and the listener answers probes. `down -v` removes the volume in the same breath, so every lane run starts from migration zero, which is what keeps the suite behind it deterministic:

```yaml
x-hc: &hc
  interval: 2s
  timeout: 2s
  retries: 30
  start_period: 3s

services:
  api:
    build: .
    environment:
      GOAPI_JWT_SECRET: dev-only-not-a-secret
      GOAPI_DB: /data/goapi.db
      # the metrics and pprof listener, bound inside the container and
      # published for the lane's assertions, never in front of the api
      GOAPI_DEBUG_ADDR: ":6060"
    ports:
      - "8080:8080"
      - "6060:6060"
    volumes:
      - data:/data
    healthcheck:
      test: ["CMD", "/api", "-health"]
      <<: *hc
```

listing 204: the whole system: probe timing anchors, one service, two listeners, the volume (source: go/api/compose.yml, lines 8-30)

The environment block carries exactly what construction demands. The jwt secret must be present or the authn service refuses to start, loudly, which is the right failure mode for a container: a missing secret is a crash the orchestrator restarts and an operator reads, never a running service signing tokens with an empty key. The value in the file is a development constant, and both platform envelopes later in this chapter exist partly to replace it: the chart reads the secret from a kubernetes secret reference, the terraform slice from an ssm parameter, and neither carries the value in a file a reviewer diffs. The second listener is the same discipline for the observability surface, the metrics and pprof routes bind an internal address the guard never wraps, and the port mapping publishes it to the host so the lane can assert against it. The migrations rule from chapter 23 holds unchanged, `docker compose up -d` is the only command that touches the schema, because the version history arrives inside the store's snapshot and applies itself at startup under the migration registry.

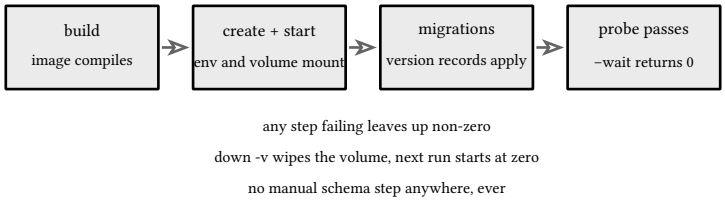


figure 181: what a green up proves, in order

29.3. the docker lane as a gate

The lane is one make target, and its shape mirrors the infrastructure book's two-gate rule, book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems) owns the compose depth and book 13, ch 3 (infrastructure: docker, databases, and testing, containers: processes, signals, pid 1) the pid 1 and signal story. The plain module gate never needs a docker daemon, and the docker lane never runs silently: a missing daemon is a loud red with the instruction to start it, never a skip:

```
verify-api-docker:
  @docker info >/dev/null 2>&1 || { echo "docker daemon not reachable." >&2; exit 1; }
  @cd books/go/api && trap 'docker compose down -v' EXIT && \
    docker compose up -d --build --wait && \
    go test -tags docker ./integration/...
```

fragment, illustrative, not executed

The suite behind the tag replays the contract's golden scenarios over the real listener. The part's in-process suites replay fourteen of the sixteen vectors by file, where the clock and id source are injected; the remaining pair, the register replay and the reports pair, is held here over the wire, and this suite asserts the layer under the frozen bytes: with a real clock, real uuids, and real wiring, the statuses, the header relations, and the byte-identical guarantees must still hold. The ETag rule is the sharpest example, a strong ETag is the sha256 of the exact response body, so the suite recomputes the hash over the wire bytes and compares:

```
func wantStrongETag(t *testing.T, r result) string {
  t.Helper()
  etag := strings.Trim(r.header.Get("ETag"), `"`)
  if etag == "" {
    t.Fatalf("no ETag header, body %s", r.body)
  }
  sum := sha256.Sum256(r.body)
  if got := hex.EncodeToString(sum[:]); got ≠ etag {
    t.Fatalf("ETag %s, want sha256 of body %s", etag, got)
  }
  return etag
}
```

listing 205: the etag check recomputed over live bytes, not frozen ones (source: go/api/integration/api_test.go, lines 115-126)

Six tests cover the scenarios the contract names. Health and routing pins both probes and the two routing envelopes, including the ruling that a 405 carries the not_found code plus an Allow header. Register and login walks vectors 01, 02, 04, and 05: the 201 with Location and ETag, the idempotent replay returning the stored snapshot byte for byte, the login pair, and the two 401 shapes that must not differ enough to enumerate accounts. The patch duel releases two concurrent merge patches on one If-Match and demands exactly one 200 and one 412:

```
  start := make(chan struct{})
  results := make(chan result, 2)
  patch := func(name string) {
    <-start
    results <- call(t, "PATCH", "/api/users/"+id, patchHeaders(v1),
      `{"display_name":"ops renamed `+name+`"}`)
  }
  go patch("a")
  go patch("b")
  close(start)
  first, second := <-results, <-results
```

listing 206: the duel over the wire: barrier channel, two patchers, one winner (source: go/api/integration/api_test.go, lines 295-305)

The remaining three are the rate limit, five wrong passwords drain the bucket to zero and the sixth reads 429 with a Retry-After in seconds, the reports pair, a 202 that reads into a completed report and replays it as the sanctioned 200 with identical bytes while the same key under a different body reads 422, and the metrics exposition, read off the internal listener the wired service binds it to, both series present with the route label the pattern table produces. That last one is the wiring lesson of the lane: the deny-by-default guard wraps the public mux, so an exposition route registered there would answer 403 to everyone, and metrics belongs on the internal listener beside pprof, which is exactly where the observability chapter said production puts it. The build tag keeps all of it out of `go test ./...`; the tagged vet that keeps the suite from rotting behind its tag, `go vet -tags docker ./integration/...`, runs in the ci workflow copy under `deploy/ci/`, the lane that always has the daemon.

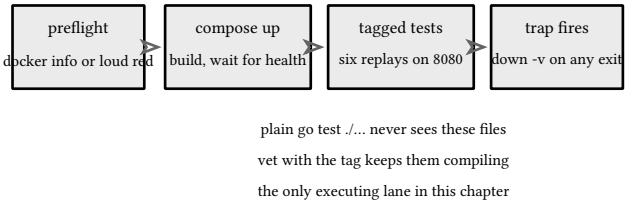


figure 182: the gate: one lane, one trap, no skips

29.4. ci on GitHub Actions

The workflow is the same gate wearing a badge. It runs the module legs exactly as the repository runs them locally, format check, vet, tests with the race detector, the tagged vet, so a green check means the same thing a green make means, and the compose lane follows on a runner where the docker daemon is already listening. The path filter keeps the workflow out of every commit that touches nothing under the service module:

```yaml
jobs:
  module:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: books/go/api
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-go@v6
        with:
          go-version: "1.27"
          cache-dependency-path: books/go/api/go.sum
      - run: test -z "$(gofmt -l .)"
      - run: go vet ./...
      - run: go test -race ./...
      - run: go vet -tags docker ./integration/...
```

listing 207: the module job: the local legs, verbatim (source: go/api/deploy/ci/ci.yml, lines 13-28)

```yaml
  lane:
    runs-on: ubuntu-latest
    needs: module
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-go@v6
        with:
          go-version: "1.27"
      - name: compose up, integration, down
        working-directory: books/go/api
        run: |
          docker compose up -d --build --wait
          trap 'docker compose down -v' EXIT
          go test -tags docker ./integration/...
```

listing 208: the lane job: compose up, tagged tests, the trap (source: go/api/deploy/ci/ci.yml, lines 30-43)

The image job exists to prove the artifact builds in a clean environment, one checkout and one docker build, with the commit sha passed as the VERSION build argument so the binary reports its build on /healthz, the versioned health route chapter 18 pinned. The job does not push anywhere: a registry, credentials, and a release policy are a repository decision, not a service one, and the workflow stops at proving the build.

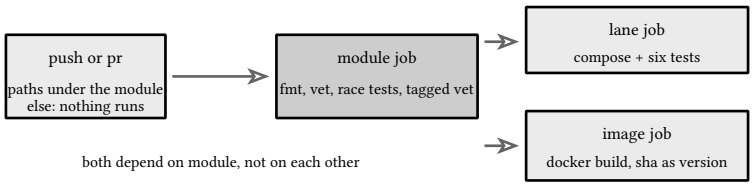


figure 183: push to ci to image: three jobs, one dependency edge

NOTE: why a taught copy

> The workflow lives at goapi/deploy/ci/ci.yml, not .github/workflows/. This repository keeps no .github directory, because a workflow there executes on push to the real remote, and corpus ci is not ratified. The file is byte for byte what a service repository would carry, taught line by line, and the make lane is what actually runs.

29.5. helm the service envelope

The chart is the service wearing kubernetes. Chart.yaml declares the name and the version pair, values.yaml is the entire configuration surface a reviewer diffs, and the templates render it into the three objects the platform needs, a deployment, a service, and the autoscaler, plus an ingress behind a flag. The deployment is where the service's own contracts surface: the probes split exactly along the line chapter 28 drew, liveness on /healthz answers is this process alive and gets a restart, readiness on /readyz answers is this process accepting traffic and gets a pull from the endpoints list, which is how a draining pod cooperates with a rolling deploy instead of dropping requests into it:

```yaml
apiVersion: v2
name: goapi
description: the goapi service, users sessions roles and reports
type: application
version: 0.1.0
appVersion: "0.1.0"
```

listing 209: the chart header: name, type, the two versions (source: go/api/deploy/helm/goapi/Chart.yaml, lines 1-6)

```yaml
image:
  repository: ghcr.io/example/goapi
  pullPolicy: IfNotPresent
  # empty falls back to .Chart.AppVersion
  tag: ""

replicaCount: 2

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi

probes:
  liveness: /healthz
  readiness: /readyz
```

listing 210: the reviewable surface: image, replicas, resources, probes (source: go/api/deploy/helm/goapi/values.yaml, lines 5-23)

```
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          ports:
            - name: http
              containerPort: 8080
          env:
            - name: GOAPI_JWT_SECRET
              valueFrom:
                secretKeyRef:
                  name: {{ .Release.Name }}-goapi
                  key: jwt-secret
          livenessProbe:
            httpGet:
              path: {{ .Values.probes.liveness }}
              port: http
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: {{ .Values.probes.readiness }}
              port: http
            periodSeconds: 5
            failureThreshold: 2
          resources: {{- toYaml .Values.resources | nindent 12 }}
```

listing 211: the container: secret from the release, both probes, the resources block (source: go/api/deploy/helm/goapi/templates/deployment.yaml, lines 26-48)

Three details match the image on purpose. The securityContext repeats user 65532, so the platform and the image agree on who runs the process even if the image changes. The jwt secret arrives through a secretKeyRef, never a values entry, keeping the reviewable file free of credentials. And the resources block is templated from values because the scheduler and the autoscaler both read it, cpu requests drive the horizontal pod autoscaler's utilization target, and a service without requests is a service the cluster cannot schedule honestly. The hpa template scales on that target between the min and max in values, the plain resource-metric answer, custom metrics being a platform decision rather than a service one.

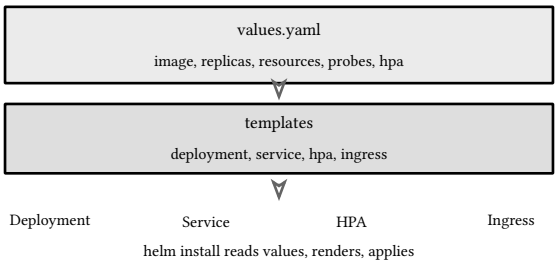


figure 184: the chart's layers: values flow down, objects render out

29.6. terraform for this service

Terraform the language, module structure, and the provider catalog belong to the cloud book: book 1, ch 23 (c23: the language, the machine, and the cloud, terraform: language and core workflow) for the language and core workflow, book 1, ch 24 (c23: the language, the machine, and the cloud, terraform modules and workflows) for how this slice becomes a module others call, book 1, ch 25 (c23: the language, the machine, and the cloud, aws essentials through terraform) for the resource catalog and account layout. This file is the service's slice of that practice, one fargate service behind one load balancer, one alarm on the tail, and it reads as a decision record for the service more than as a lesson in the tool:

```
terraform {
  required_version = "≥ 1.17"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.region
}
```

listing 212: the pin: terraform 1.17 plus the aws 6 provider (source: go/api/deploy/terraform/main.tf, lines 6-19)

The task definition runs the same image compose runs, with awsvpc networking and the secret pulled from ssm at container start. The secret's path is in the plan, its value never is beyond the one write, and the binary that receives it fails closed on an empty value, so the weakest link is the parameter store's own permissions rather than any copy of the value:

```
  container_definitions = jsonencode([
    {
      name      = var.name
      image     = var.image
      essential = true
      portMappings = [
        { containerPort = 8080, protocol = "tcp" }
      ]
      environment = [
        { name = "GOAPI_ADDR", value = ":8080" }
      ]
      secrets = [
        { name = "GOAPI_JWT_SECRET", valueFrom = aws_ssm_parameter.jwt_secret.arn }
      ]
```

listing 213: the container definition: image, env, the secret by reference (source: go/api/deploy/terraform/ecs.tf, lines 88-101)

The load balancer carries two service decisions. The target group probes /healthz, the same liveness route from the other two envelopes, with a deregistration delay of 30 seconds, which is the drain window chapter 28′s shutdown owns the inside of: a task leaving the pool gets half a minute to finish its in-flight requests, and the graceful drain inside the process is what makes that window sufficient rather than merely allocated. And desired_count defaults to 1, stated in the variable itself, because the store is one file set, snapshot plus log, on the task's volume and two tasks would be two databases wearing one name. That is the honest constraint of this service, and the fix is a store that moves off the local volume, not a count raised past the truth.

The alarm reads the tail off the platform instead of the service publishing a second copy: the load balancer already measures target response time, and cloudwatch computes the p99 extended statistic for it, so the pager watches the same number the load chapter defends, three one-minute periods above threshold, and pages the sns topic when the tail is a fact and not a spike.

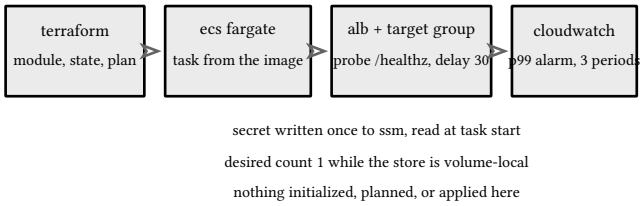


figure 185: apply to ecs to alb: the service's aws slice in one line

29.7. the shipped checklist

The ship lane ends where it started, with the property each chapter contributed. Every row is a behavior the lane or the module gate observes, not an aspiration, and every row names where it is proven, which is the difference between a readiness checklist and a slide. The suite chapter that follows hardens the whole set:

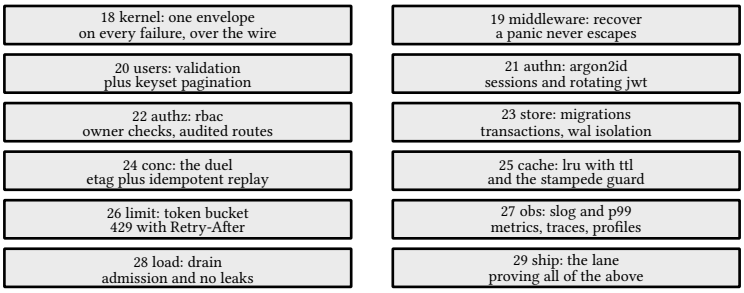


figure 186: the readiness table: twelve chapters, twelve properties, all proven

The chapter also declares what it deliberately does not teach again. Docker's storage model, the container runtime, and networking are the infrastructure book's chapters, the compose internals and the one-click philosophy are book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems), and the migration discipline the version records ride is book 13, ch 18 (infrastructure: docker, databases, and testing, migrations, test driven). The cloud side defers the same way, the terraform workflow and module design live in the cloud book's own chapters, and the aws resource catalog with it. What this chapter owns is the narrow band between a green module and a running service: the image, the command, the gate, and the two envelopes, each one small enough to read in a sitting and boring enough to trust.

sources: docker docs on multi-stage builds and `docker compose up --wait` semantics, at docs.docker.com; the GitHub Actions workflow syntax plus actions/checkout v6 and actions/setup-go v6, at docs.github.com/actions; the kubernetes api reference for probes, resources, and the autoscaling/v2 target types, at kubernetes.io/docs/reference/kubernetes-api; the aws provider 6 documentation for aws_ecs_service, aws_lb_target_group, and aws_cloudwatch_metric_alarm extended statistics, at registry.terraform.io; the terraform 1.17 language documentation, at developer.hashicorp.com/terraform. Accessed 2026-09-25. Verified by `go vet -tags docker` and a tagged compile of the integration suite, the lane itself run under `make verify-api-docker`; the helm chart and terraform are documentation-verified, the cloud contract inherited from the c-os-cloud book.

30. the test suite

The service part ends with the service tested, and this chapter turns the suite around and examines it as its own artifact. Chapter 14 taught the toolchain's test kinds, and every service chapter applied one technique to its own component. What is left is the discipline the whole suite runs on: invariants stated as properties and held by fuzz targets whose seeds ride the plain gate for free, a strict recording double that gives hand fakes the one thing they never had, the modern `testing.T` surface used where it earns its keep, and a merged coverage lane that says what it measures and no more. One rule governs everything: no third party is adopted, and every library the ecosystem reaches for by habit is taught as a dated fact instead.

30.1. what the suite already holds

The inventory first, because the chapter builds on all of it. The suite holds 182 test functions, 6 fuzz targets, 1 example, and 2 benchmarks, and all sixteen frozen vectors are now pinned by an in-process suite: the register pair in the concurrency chapter's flight tests, the login and refresh family in authn, the cursor walk in users, the 429 in limit, the roles matrix in authz, the metrics exposition in observability, the reports pair in analytics, with the docker lane replaying the load bearing scenarios again over the real wire. Around the vectors sit the techniques each chapter contributed, table tests with subtests, clocks and ids and hashes injected as function fields, channel choreography that parks one request inside another's window, synctest bubbles in the limiter and the janitor, and the child process race canary behind the `GOAPI_CANARY_UNGUARDED` environment key. The youngest technique is the leak check:

```
func TestHarnessRunLeaksNoGoroutines(t *testing.T) {
  ts := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
  }))
  h := New()
  h.BaseURL = ts.URL
  clock := &tickClock{now: t0}
  h.Now = clock.Now

  if _, err := h.Closed(context.Background(), 4, 5, 2); err ≠ nil {
    t.Fatalf("harness run: %v", err)
  }
  ts.Close()
  h.Client.CloseIdleConnections()
```

listing 214: the goroutineleak profile of go 1.27, asked directly after a real run (source: go/api/internal/loadtest/leak_test.go, lines 19-33)

The `goroutineleak` profile, new in go 1.27, has the runtime's collector decide reachability: a goroutine blocked on a primitive nothing live references is a leak, and the profile's own write runs the detection pass first, so the count reads the cycle that just happened. A closed harness run against a closed server must leave exactly zero:

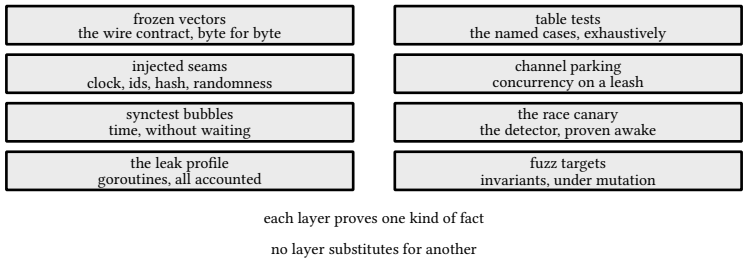


figure 187: the suite's layers, each against the one thing it proves

30.2. properties as fuzz targets

A property is a sentence that must hold for every input, and the ecosystem's habit is a property library: `rapid` v1.3.0 offers structured generators, shrinking, and state machine tests, `gopter` is its older sibling, and `testing/quick` sits frozen in the standard library as the ancestor nobody extends. None is adopted, on the program's standing rule: a third party is a maintenance liability, and the toolchain already ships a property engine, a fuzz target with an invariant inside its body. The seeds run as an ordinary table test under plain `go test`, which is why all six targets ride `make verify` at zero cost, and `go test -fuzz` wakes the coverage guided mutation engine on demand in the opt-in `api-fuzz` lane, never inside the gate, because a search is not a gate:

```
f.Fuzz(func(t *testing.T, capacity, perMilli, draws, elapsedMs int) {
  t.Attr("invariant", "bucket-bounds")
  if capacity < 1 || capacity > 1<<20 || perMilli <= 0 || draws < 0 || draws > 1<<16 || elapsedMs < 0 {
    return
  }
  b := NewBucket(capacity, float64(perMilli)/1000.0, t0)
  prev := b.Tokens(t0)
  if prev < 0 || prev > float64(capacity) {
    t.Fatalf("level %v at birth, want [0, %d]", prev, capacity)
  }
  now := t0
  for i := 0; i < draws; i++ {
    now = now.Add(time.Duration(elapsedMs%17) * time.Millisecond)
    refilled := b.Tokens(now)
    if refilled < prev-1e-9 {
      t.Fatalf("draw %d: level fell from %v to %v while the clock only advanced", i, prev, refilled)
```

listing 215: the bucket's arithmetic as a property: bounded, monotone, denial costs nothing (source: go/api/internal/limit/bucket_fuzz_test.go, lines 18-33)

The bucket is the cleanest demonstration because its invariants are its arithmetic: the level lives in `[0, capacity]` at every observation, advancing the clock never lowers it, a denial at a fixed instant costs nothing, and the decision's remaining never exceeds its limit. The seeds are the fixed-draw vectors' own shapes, the login burst, the sustained refill, the register rule's long idle, so plain `go test` re-runs the exact arithmetic the wire contract froze and the engine explores around shapes that already matter. Degenerate parameters are guarded by an early return rather than a skip, because a skipped invariant is an unproven one, and a failure found under `-fuzz` graduates the chapter 14 way, the input landing in `testdata/fuzz` only with a fix beside it:

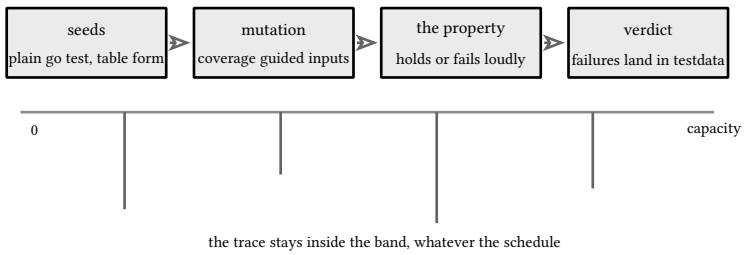


figure 188: seed to mutation to verdict, the bounds band drawn through a refill trace

30.3. the cache as a state machine

The state machine is the property library's signature move, and the cache target expresses it over the fuzz engine directly: each op byte dispatches by its low two bits to set, get, delete, or advance the clock one full ttl, with the key taken from the bits above them, so every op kind reaches every key and the engine chooses the collisions. The invariants read as a contract, the bound holds after every operation, a hit always answers the last write, and everything written before a tick is gone after it, because expiry is a cliff. The model beside the machine is deliberately naive, a plain map maintained in lockstep, since the point is a boring model that never agrees with a

wrong implementation, and divergence fails with both sides named. Two siblings complete the family: the cursor target pins the codec's round trip and its refusal to decode a client string into a silently empty key, the failure that would truncate a list walk at page two, and the envelope target is the next section's subject. Beside the machine, one plain test runs the janitor's `Sweep` under `t.Context()` in a synctest bubble, the context canceled at cleanup, so no defer is needed and the sweep goroutine must be gone when the bubble closes:

```
for i, op := range ops {
  k := "k" + strconv.Itoa(int(op>>2)%8)
  switch op & 0x03 {
  case 0:
    v := "v" + strconv.Itoa(i)
    c.Set(k, v)
    last[k] = v
  case 1:
    if v, ok := c.Get(k); ok && v ≠ last[k] {
      t.Fatalf("op %d: get %s = %q, want the last write %q", i, k, v, last[k])
    }
  case 2:
    c.Delete(k)
    delete(last, k)
  case 3:
    now = now.Add(ttl) // a full ttl passes: everything written before now is gone
    for key := range last {
      if _, ok := c.Get(key); ok {
        t.Fatalf("op %d: %s answered a hit past its ttl", i, key)
      }
    }
  }
  if n := c.Len(); n > max {
    t.Fatalf("op %d: len %d exceeds the bound %d", i, n, max)
  }
```

listing 216: the op dispatch: set, get, delete, or one full ttl of clock, bound checked after every step (source: go/api/internal/cache/cache_fuzz_test.go, lines 34-58)

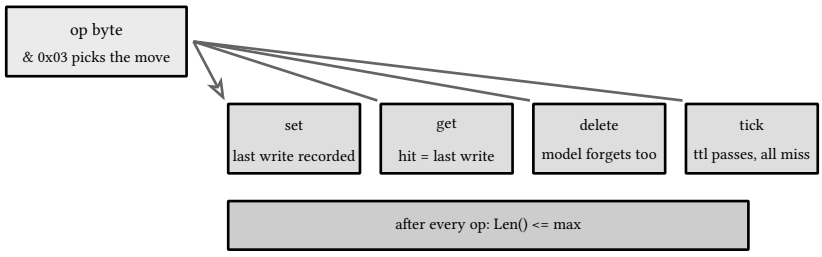


figure 189: ops and states with the bound guard under every step

30.4. forgery resistance as a target

The authn targets attack the two things an attacker would. `FuzzJWTParse` holds the verifier to its contract under mutation: `ParseHS256` either refuses with one of its five sentinels, a closed set the body enforces with a default branch that fails the test on any sixth error, or it accepted a token whose signature genuinely gates the bytes, which the body proves by flipping each of the first eight signature bytes and demanding every mutant fail, because a parse that survives a flipped signature byte was never checking the signature. The seeds are the forged token table's own shapes, the tampered payload under the original signature, the alg none header, an EdDSA token presented to the HS256 parser, truncations, four part splits, and pure garbage. The envelope target holds both of the kernel's shapes at once. Its encoding leg asserts a fixed point, marshal, unmarshal, re-marshal, byte identical, with a second decode equal to the first, deliberately the claim rather than field equality, because a fuzzed string

can carry invalid UTF-8 that the codec legitimately sanitizes once, and the stable property is that whatever the codec accepts it reproduces. Its ladder leg sends an arbitrary body and content type through the strict `Bind` the handlers use and demands the outcome land on exactly the closed set, clean decode, 415, 413, or 400, never a panic and never a raw error leaking past the `errors.AsType` check. One construction detail earns its sentence: the oversize seed is built before registration, because a seed only the mutation engine could synthesize would leave the 413 leg unexercised under plain `go test`:

```
f.Fuzz(func(t *testing.T, token string) {
  t.Attr("invariant", "jwt-forgery-resistance")
  _, err := ParseHS256(token, []byte(testSecret), t0)
  switch err {
  case nil:
    // acceptance implies verification gated the bytes: flip each
    // of the first 8 signature bytes, every mutant must fail
    p := strings.Split(token, ".")
    sig, derr := base64.RawURLEncoding.DecodeString(p[2])
    if derr ≠ nil || len(sig) == 0 {
      t.Fatalf("accepted token with an undecodable signature")
    }
    for i := 0; i < 8 && i < len(sig); i++ {
      mut := append([]byte(nil), sig...)
      mut[i] ^= 0xff
      m := p[0] + "." + p[1] + "." + base64.RawURLEncoding.EncodeToString(mut)
      if _, err := ParseHS256(m, []byte(testSecret), t0); err == nil {
        t.Fatalf("flipping signature byte %d still verified", i)
      }
    }
  case ErrMalformedToken, ErrBadSignature, ErrAlgMismatch, ErrExpired, ErrWrongIssuer:
  default:
    t.Fatalf("unknown error %v, the sentinels are a closed set", err)
  }
})
```

listing 217: refuse with a sentinel, or prove the signature gated the bytes (source: go/api/internal/authn/jwt_fuzz_test.go, lines 41-65)

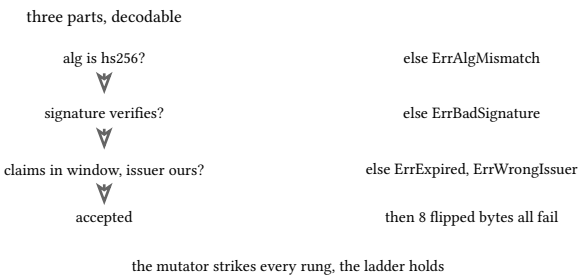


figure 190: the verify ladder under attack at every rung

30.5. strictness without a framework

The suite's doubles were all stateful until now: `countingKeys` signals every lookup, `countingHasher` proves the dummy verify ran, `countingMux` counts origin fills, `seqReader` feeds deterministic randomness, `stubClient` stands in for the transport. What none could do is fail a test for a call that should not have happened, or for an expected call that never arrived, and the strict recording double closes exactly that gap: it wraps the real in-memory session store, every call lands on a tape in arrival order, a call the test did not expect fails on the spot, and a cleanup armed at construction fails the test again if an expected call never came. Two interaction tests put it to work over the real routes, the logout test pinning `Create`, `Get`, then `Revoke` in that order, and the refresh reuse test pinning `GetByRefresh`

before `Rotate`, twice through, with the reused token answering the family revocation's 401. Sixty lines, hand-rolled, and it is the strictness generated mocks sell:

```
// newStrictSessions returns the double with its cleanup armed.
func newStrictSessions(t *testing.T) *strictSessions {
  s := &strictSessions{MemSessions: NewMemSessions(), t: t}
  t.Cleanup(func() {
    s.mu.Lock()
    defer s.mu.Unlock()
    if !reflect.DeepEqual(s.want, s.got) {
      s.t.Errorf("session tape drifted:\nwant %v\ngot  %v", s.want, s.got)
    }
  })
  return s
}

// stamp renders any recorded argument for the tape. It is the module's
// one generic method: a method declaring its own type parameter, legal
// since go 1.27 and barred from satisfying interfaces, which is why
// production code never grew one and this test helper did.
func (s *strictSessions) stamp[T hashArg](v T) string {
  return fmt.Sprintf("%x", v)
}
```

listing 218: the strict double: cleanup armed at birth, the tape judged at death (source: go/api/internal/authn/sessioncall_test.go, lines 42-61)

The landscape deserves its honest paragraph, because the ecosystem's habit is the reverse of this choice. `golang/mock` was archived by google and reanimated as `go.uber.org/mock`, v0.6.0, whose `mockgen` generates expectation-style mocks and whose workflow is the go 1.24 `tool` directive plus a `go generate` line; `mockery`, `moq`, and `counterfeiter` generate doubles in other dialects, and `testify` layers assertions over the plain surface. All of it is taught as fact and adopted as nothing, on the maintenance rule: a generated file outlives the interface churn it serves and a dependency's deprecation becomes your migration, while this module's interfaces are small and stable enough that one readable file costs less than the generator it replaces. The criteria are stated so the choice is a decision rather than a habit, a generated mock earns its keep against wide churning interfaces, call-order contracts many consumers assert, and teams large enough that uniformity beats reading sixty lines:

the axis	hand fake	generated mock	assertion library
strictness	what you write	expectations by default	none, you assert
drift	visible, one file	regenerate on churn	assert strings drift
readback	the tape, in order	violations reported	failures, formatted
dependency cost	zero	the toolchain plus a module	one module

this module: hand fake, zero cost, one file to read

figure 191: hand fake, generated mock, assertion library, four axes

30.6. the testing.T surface and the generics truth

The `testing.T` type grew methods the suite uses where they fit. `t.Context` returns a context canceled at cleanup, the whole lifetime contract in one call, and the sweep test uses it as the janitor's cancellation. `t.Cleanup` registers a function that runs even after `t.Fatal` has exited the test, which is why the strict double arms its tape judgment there rather than in a defer the panic path could skip. `t.Attr` attaches key value pairs to the json event stream, stamped inside the fuzz body on the `t` the engine hands it, so a failing input carries the invariant it broke, and every target names exactly one. `t.Chdir` changes and restores the working directory, the one method unused, because nothing here reads the cwd and forcing it would be ceremony, the same rule the middleware chapter drew for

its onion, adopt a layer when the job exists. The generics inventory closes the accounting with the truth: the production module uses the generic types `Cache[V]`, `entry[V]`, and `FillResult[V]`, the generic functions `NewCache[V]`, `Bind[T]`, `Fill`, `GetOrFill`, and `OncePair`, the constructor fixes `V` once and every later method call rides it, exactly one constrained generic, `NearestRank[T cmp.Ordered]`, and zero generic methods, because a method declaring its own type parameter cannot satisfy an interface and every seam this service defines is an interface a store or a fake implements. The one generic method in the module lives in test code, the strict double's `stamp[T hashArg]`, which needs no interface satisfaction. Production zero, tests one, and the reason is structural:

```
func (s *strictSessions) record(name, arg string) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.got = append(s.got, call{name: name, arg: arg})
  if len(s.got) > len(s.want) || s.want[len(s.got)-1] ≠ s.got[len(s.got)-1] {
    s.t.Errorf("unexpected session call %s(%s)", name, arg)
  }
}
```

listing 219: the tape judged against the queue head, one call at a time (source: go/api/internal/authn/sessioncall_test.go, lines 72-79)

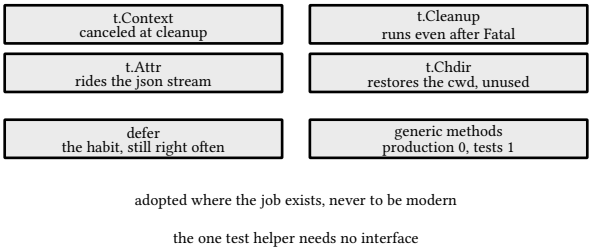


figure 192: the modern T methods against the defer habit, and the generics count

30.7. coverage without ceremony

Coverage answers what the suite executes, and the honest lane is opt-in exactly like the bench lane, never inside `make verify`, because a percentage measures the machine and the tree, not the code. The mechanism is two text profiles and a merge: the unit leg runs `go test -cover` over the module, the docker leg brings the compose stack up with the ship chapter's `--wait` contract and `down -v` trap, then runs the tagged integration suite with `-coverpkg` over the whole module, and the merge sums execution counts per block, the same semantics `go tool covdata merge` applies to binary profiles. The text route and not the binary one is a toolchain fact stated plainly: `GOCOVERDIR` collects from binaries built with `go build -cover`, and `go test` never routes its own collection through it. Two boundaries keep the number honest, the containerized server is an uninstrumented release binary so the report covers what the two host test binaries execute, a fact written in the script's own header, and no percentage appears anywhere in this book, because the figure moves with every edit and a pinned number would be a lie with a date on it:

```
go test -covermode=atomic -coverprofile=.cover/unit.out ./...
echo "== docker leg"
trap 'docker compose down -v' EXIT
docker compose up -d --build --wait
go test -tags docker -covermode=atomic -coverpkg=./... -coverprofile=.cover/docker.out ./integration/...
echo "== merge"
{ echo "mode: atomic"; grep -h -v '^mode:' .cover/unit.out .cover/docker.out |
  awk '{ c[$1 " " $2]+=$3 } END { for (b in c) print b, c[b] }'; } > .cover/merged.out
go tool cover -func=.cover/merged.out | tail -1
```

listing 220: both legs, the merge, and the honest total, the down -v trap armed before up (source: go/api/cover.sh, lines 16-24)

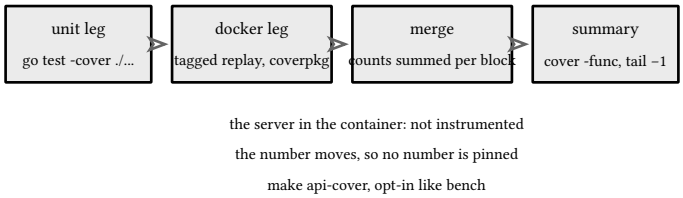


figure 193: two legs, two profiles, one merged summary

30.8. what the suite proves

The closer maps each layer to the fact it establishes, chapter 29's readiness table one chapter later: the vectors prove the wire contract, the tables the named cases, the injected seams determinism, the channel choreography that the concurrency windows close, the synctest bubbles the time dependent logic, the canary that the race detector is awake, the leak profile that the goroutines are accounted, the fuzz targets that the invariants survive mutation, and the strict double that the call orders hold. This surface is contract, not preference, every lane of the program carries it per the testing minimums, and the store direction is a decision already executed: the module owns an append-only write ahead log engine with torn tail recovery and a hand-rolled `database/sql` driver over it, its one community dependency modernc.org/sqlite gone from go.mod, while `golang.org/x/crypto`, `x/time`, and `x/sync` stay under the platform-official ruling, taught through the primitives they compose. The suite's last artifact is its quietest, the module's first example, an `Example` with an `// Output:` comment that pins the cursor's wire bytes as a test, executable documentation that cannot rot without failing the gate, and the existence of it answers the inventory's opening question, a suite that runs tests, examples, seed corpora, and benchmarks has no kind left unexercised:

```
// ExampleEncodeCursor pins the wire bytes of one issued cursor. The
// rendering is deterministic base64url of the two-member json payload,
// so the example doubles as a regression pin for the encoding itself.
func ExampleEncodeCursor() {
  s, err := EncodeCursor(CursorKey{T: 1790830803000, I: "01a0f5d5-7838-7103-8a7f-baa98e20f808"})
  if err ≠ nil {
    panic(err)
  }
  fmt.Println(s)
  // Output: eyJ0IjoxNzkwODMwODAzMDAwLCJpIjoiMDFhMGY1ZDUtNzgzOC03MTAzLThhN2YtYmFhOThlMjBmODA4In0
}
```

listing 221: the module's first example: the cursor's wire bytes, pinned as output (source: go/api/internal/user/pagination_fuzz_test.go, lines 42-52)

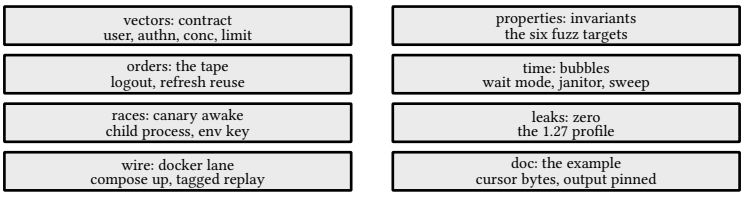


figure 194: the layer to property to where proven grid, closing the part

sources: pkg.go.dev/testing for `F.Fuzz`, `F.Add`, `T.Context`, `T.Chdir`, `T.Attr`, and `T.Cleanup`, and pkg.go.dev/runtime/pprof for the goroutineleak profile, accessed 2026-09-26, the fuzz and coverage behavior verified against go 1.27.0 locally, the two leg merge semantics read against `go tool cover` and `go tool covdata`; pkg.go.dev/pgregory.net/rapid v1.3.0, published 2026-03-30, go.uber.org/mock v0.6.0, published 2025-08-18, and the mockery, moq, counterfeiter, and testify documentation, all accessed 2026-09-26 and deliberately not adopted. Verified by the module's suite, 182 tests, 6 fuzz targets with their seeds riding plain `go`

`test`, 1 example, and 2 benchmarks under go vet, the plain gate, and the module-wide race leg, with `make api-fuzz` green across all six targets and `make api-cover` green over the standing compose stack.

31. numerical statistics in go

The service part taught one program against one database. The data part teaches the numbers a program computes about itself, and its case study is the user's own dota-helper engine, a tool that screens hundreds of hero and item pairs for win-rate effects and has to decide which of those effects are real. Every idea in this chapter runs there in production: the exact binomial tail, the 2x2 chi-square with its fallback gate, the Mann-Whitney AUC, Spearman correlation, a deterministic bootstrap, and the Holm and Benjamini-Hochberg corrections. Here they are rebuilt as `godata/internal/stats`, a second module beside the api, stdlib only, and the same math the kdd book derives in c, book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) and book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr) for the theory and this chapter for the go:

```
module godata

go 1.27
```

listing 222: the data part's module, stdlib only by the same ruling as the api (source: go/data/go.mod, lines 1-3)

31.1. why a game tool needs exact numbers

A win-rate screen is a family of tests, not a test. Five hundred hero pairs scored at the conventional 0.05 bar will cross it on noise alone about 25 times, one in twenty, because that is what the bar means. The tool that reports those 25 as discoveries is not lying about its arithmetic, it is asking the wrong question: the family needs a correction before any single p-value means anything. The measured version of the paragraph is a test in this package: a null family of 200 uniform p-values drawn from a pinned PCG stream crosses the 0.05 bar exactly 10 times, 5 percent on the nose, the smallest crossing sitting at 0.0124 far above the Bonferroni bar 0.00025, and neither correction rejects anything. The screen works, and the chapter builds it bottom up so every number in it is one a test asserted first.

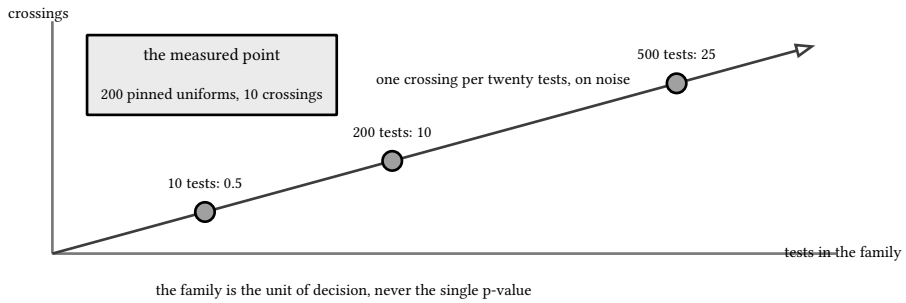


figure 195: expected false positives against family size at the 0.05 bar, with the measured null family

31.2. the binomial in log space

The workhorse question is does this count surprise this rate: 8 wins of 10 engagements against a 50 percent baseline. The binomial pmf answers it, and the only decision in its implementation is where the arithmetic lives. `100!` overflows an int64 and its float approximation loses the digits that matter, so the cell is evaluated entirely in log space, each factorial through `math.Lgamma`, and exponentiated once at the end. The one subtlety left over is the point masses: `p` 0 and 1 collapse the distribution to a single cell, and `Log(0)` is `-Inf`, so those cases short-circuit before the logarithms:

```
func BinomialPMF(k, n int, p float64) float64 {
  if !binomialArgsValid(k, n, p) {
    return math.NaN()
  }
  switch p {
  case 0:
    if k == 0 {
      return 1
    }
    return 0
  case 1:
    if k == n {
      return 1
    }
    return 0
  }
  logCell := lgamma(n+1) - lgamma(k+1) - lgamma(n-k+1) +
    float64(k)*math.Log(p) + float64(n-k)*math.Log1p(-p)
  return math.Exp(logCell)
}

// BinomialOneSided returns the exact upper tail P(X ≥ k), the sum of the
// pmf cells from k to n: no normal approximation and no continuity
```

listing 223: the pmf cell in log space: lgamma factorials, one Exp at the end (source: go/data/internal/stats/binomial.go, lines 28-50)

Both exact tails are sums of cells. The one-sided tail `P(X ≥ k)` adds the cells from `k` up, no normal approximation anywhere near it: for the 8-of-10 question that is `(45+10+1)/1024`, `7/128`, Pascal's row read straight off. The two-sided tail is the point-probability method, the same one the kdd book derives, book 14, ch 39 (knowledge discovery, hypothesis testing and p-values): the bar is the observed cell's own mass, 45/1024 here, and the p-value is the total mass of every cell at most as likely, the set {0,1,2,8,9,10} summing to `7/64`, both tails collected by likelihood rather than by any distance from a mean. The relative `tieTol` keeps a cell that ties the bar to within an ulp inside the region:

```
func BinomialTwoSided(k, n int, p float64) float64 {
  if !binomialArgsValid(k, n, p) {
    return math.NaN()
  }
  bar := BinomialPMF(k, n, p)
  total := 0.0
  for x := 0; x <= n; x++ {
    if cell := BinomialPMF(x, n, p); cell <= bar*(1+tieTol) {
      total += cell
    }
  }
  return clampProb(total)
}
```

listing 224: the point-probability two-sided tail: cells at most as likely as the observed one (source: go/data/internal/stats/binomial.go, lines 70-82)

Both sums leave through `clampProb`, and the clamp is not decoration. A sum of cells that is 1 in exact arithmetic accumulates rounding error like any other float sum, and the full row for `n = 141` fair flips lands at `1.0000000000000713`, seven times ten to the minus fourteenth past the bound. A p-value above 1 is not a p-value, and the BH step-up in the last section compares every cell against its bar, so an unclamped tail would poison the family. The test pins the overshoot on real input, then the clamp:

```
// clampProb bounds a summed probability to [0,1]. The pmf sums are 1 in
// exact arithmetic, but floating-point accumulation over dozens of cells
// can land a hair past it, the full row for n = 141 fair flips measuring
// 1 + 7.1e-14, and a p-value above 1 is not a p-value: the BH step-up
// compares every cell against its bar and a bar above 1 is unreachable
// nonsense. The excess is pure accumulation artifact, so the clamp carries
// no tolerance.
func clampProb(p float64) float64 {
  switch {
  case p > 1:
    return 1
  case p < 0:
    return 0
  default:
    return p
  }
}
```

listing 225: the clamp, no tolerance: the excess is accumulation artifact, never probability (source: go/data/internal/stats/binomial.go, lines 84-100)

```
func TestBinomialTailsWithinUnitInterval(t *testing.T) {
  raw := 0.0
  for x := 0; x <= 141; x++ {
    raw += BinomialPMF(x, 141, 0.5)
  }
  if raw <= 1 || raw > 1+1e-12 {
    t.Fatalf("raw n=141 row = %v, want an overshoot inside 1+1e-12", raw)
  }
  if raw-1 < 1e-15 || raw-1 > 1e-13 {
    t.Errorf("overshoot magnitude %v, want the measured 7.1e-14 class", raw-1)
  }
  if got := BinomialOneSided(0, 141, 0.5); got ≠ 1 {
    t.Errorf("oneSided(0,141,1/2) = %v, want exactly 1 after the clamp", got)
  }
```

listing 226: the overshoot demonstrated, not assumed: the raw row past 1, the tail exactly 1 (source: go/data/internal/stats/binomial_test.go, lines 140-153)

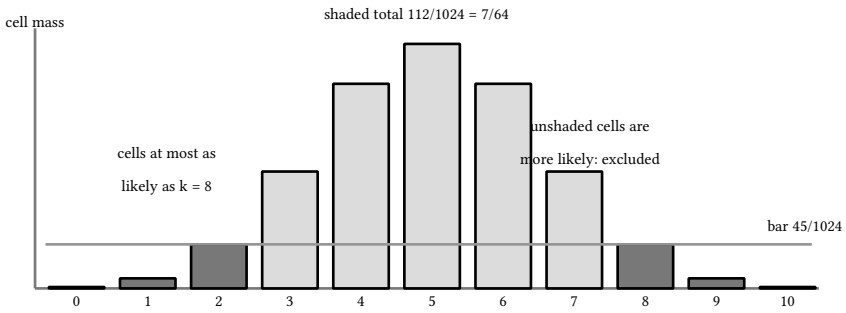


figure 196: the n = 10 fair-coin pmf, with the two-sided region of k = 8 shaded

31.3. the 2x2 chi-square and its floor

Two counting questions share one shape: wins and losses under two conditions, a 2x2 table. The chi-square test of independence scores the table against the counts the marginals imply, expected `row*col/n` per cell, and with one degree of freedom the upper tail closes as `erfc(sqrt(stat/2))`, which is why the NIST critical 3.8415 sits exactly at p 0.05: it is z 1.96 squared, and the erfc form is the two-sided normal tail in disguise. A hand table keeps the

arithmetic visible: observed `(15, 10, 10, 15)` against expected 12.5 per cell is four deviations of 2.5, a statistic of exactly 2, and p = `erfc(1)` = 0.1573, independence standing:

```
func ChiSquare2x2(a, b, c, d float64) (stat, p float64, ok bool) {
  n := a + b + c + d
  row1, row2, col1, col2 := a+b, c+d, a+c, b+d
  if nonfinite(a) || nonfinite(b) || nonfinite(c) || nonfinite(d) ||
    a < 0 || b < 0 || c < 0 || d < 0 || n == 0 ||
    row1 == 0 || row2 == 0 || col1 == 0 || col2 == 0 {
    return math.NaN(), math.NaN(), false
  }
  obs := [4]float64{a, b, c, d}
  exp := [4]float64{row1 * col1 / n, row1 * col2 / n, row2 * col1 / n, row2 * col2 / n}
  stat = 0
  ok = true
  for i := range obs {
    dev := obs[i] - exp[i]
    stat += dev * dev / exp[i]
    if exp[i] < minExpected {
      ok = false
    }
  }
  return stat, math.Erfc(math.Sqrt(stat / 2)), ok
}
```

listing 227: expected counts from marginals, the df=1 erfc tail, and the floor gate on ok (source: go/data/internal/stats/chisq.go, lines 21-41)

The third return value is the honest part. The chi-square statistic is an approximation, and the classic guidance is to trust it only when every expected count reaches 5; the function carries that as `ok` instead of deciding silently, and the caller falls back to the exact binomial tail of the previous section when it is false. The independence table `(2,2,2,2)` scores a perfectly ordinary statistic, 0, and still answers `ok = false` with expected counts of 2: the gate refuses the approximation, not the data. A negative count, a NaN, an infinity, a zero total, or a zero row or column leaves some expected count at a structural zero or no count at all, and the answer is the math package's `(NaN, NaN, false)`:

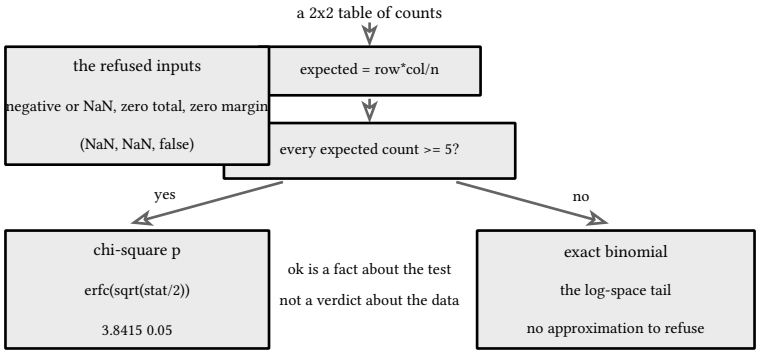


figure 197: the fallback decision: the gate on expected counts routes the table to the approximation or the exact tail

31.4. midranks, the shared tie primitive

Every rank-based statistic in this package answers the same question first: where does each value place, and what happens when values tie. The package answers it once. `midranks` sorts positions, walks the tie groups, and hands each group the mean of the rank span it covers, so {7, 1, 7, 7, 2} ranks as 4, 1, 4, 4, 2, the three sevens sharing the mean of ranks 3, 4, and 5. The AUC's half credits, Spearman's tie blocks, and the percentile positions below all flow through this one function, which is why tie policy is decided in one place instead of four quietly different ones:

```
func midranks(values []float64) []float64 {
  n := len(values)
  idx := make([]int, n)
  for i := range idx {
    idx[i] = i
  }
  // the comparator receives slice elements, which are themselves
  // positions into values, not positions into idx
  slices.SortStableFunc(idx, func(x, y int) int {
    return cmp.Compare(values[x], values[y])
  })
  out := make([]float64, n)
  for start := 0; start < n; {
    end := start
    for end+1 < n && values[idx[end+1]] == values[idx[start]] {
      end++
    }
    avg := float64(start+end+2) / 2 // 1-based mean rank of the tie group
    for _, i := range idx[start : end+1] {
      out[i] = avg
    }
    start = end + 1
  }
  return out
}
```

listing 228: positions sorted by value, each tie group assigned its mean rank (source: go/data/internal/stats/rank.go, lines 15-39)

One consumer deserves its eight lines. Percentile normalization maps each midrank to the plotting position `(r - 1/2)/n`, ties sharing one position, {1, 2, 2, 3} landing at 0.125, 0.5, 0.5, 0.875. The half offset is the whole point: every output stays strictly inside (0, 1), so an inverse-normal transform of the positions, the normal scores the regression chapter will want, never asks for `+Inf` at the extremes:

```
func PercentileRanks(values []float64) []float64 {
  ranks := midranks(values)
  out := make([]float64, len(values))
  for i, r := range ranks {
    out[i] = (r - 0.5) / float64(len(values))
  }
  return out
}
```

listing 229: the plotting position: midranks over n, half-offset inside (0,1) (source: go/data/internal/stats/rank.go, lines 45-52)

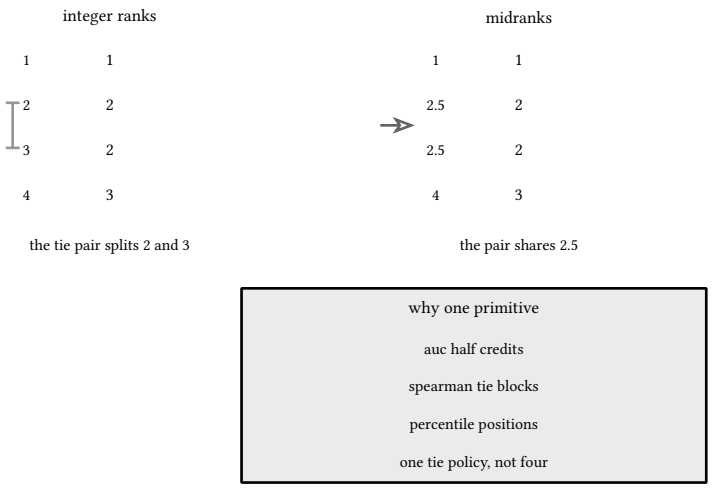


figure 198: one tie group under integer ranks and under midranks

31.5. auc as a rank statistic

A backtest scores rows and wants to know if the scores order the wins above the losses. The area under the ROC curve is the concordance fraction: the share of positive-negative pairs where the positive outscores the negative, a tie worth 1/2. Computing that as a pair loop is O(n^2) and a 20k-row backtest pays for it; computing it as the Mann-Whitney U statistic is O(n log n), one midrank pass plus one sum, `U = R+ - n1(n1+1)/2` over `n1*n0`, and the ties resolve themselves because the midranks already averaged them. The chapter fixture is six scores, three positives against three negatives, one positive losing: six concordant pairs of nine, AUC = 2/3:

```
func AUC(scores []float64, labels []bool) float64 {
  if len(scores) != len(labels) {
    panic("stats: AUC scores and labels differ in length")
  }
  n1 := 0
  for _, pos := range labels {
    if pos {
      n1++
    }
  }
  n0 := len(labels) - n1
  if n1 == 0 || n0 == 0 {
    return 0.5
  }
  ranks := midranks(scores)
  rplus := 0.0
  for i, pos := range labels {
    if pos {
      rplus += ranks[i]
    }
  }
  u := rplus - float64(n1)*float64(n1+1)/2
  return u / float64(n1*n0)
}
```

listing 230: the Mann-Whitney U over midranks: one sort, one sum, no pair loop (source: go/data/internal/stats/auc.go, lines 13-36)

The degenerate cases are contracts, not afterthoughts. A sample with only one class has no cross-class pair and the concordance is undefined; the API answers the 0.5 chance line, the value that claims no information, and the empty sample is that same case. Length-mismatched slices are a caller bug and panic, the loud failure the emit layer of the case study settled on too. The property tests carry the weight: the rank form is checked against the

O(n^2) definition it replaces on tied random data, five trials of 200 rows, and flipping every label reflects the curve through the chance diagonal, `AUC + AUC' = 1` to the last bit on 300 rows:

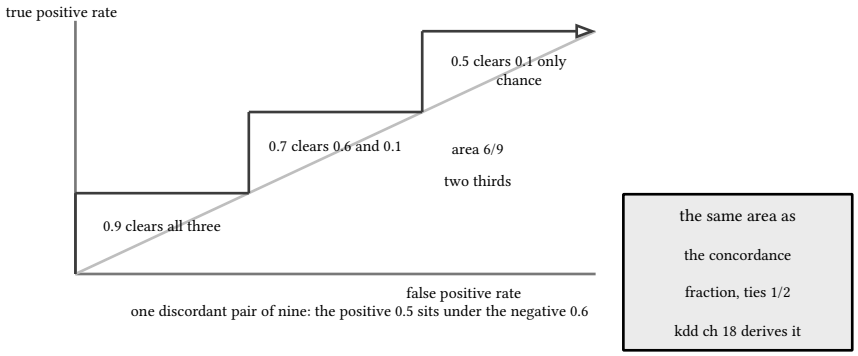


figure 199: the fixture's roc curve: a step per threshold, area 2/3 over the chance diagonal

31.6. spearman, pearson on ranks

The correlation the backtest wants is between ordinals, win-rate movement against pick-order movement, and the Pearson correlation of raw values would let one runaway match dominate. Spearman is Pearson run on the midranks instead: monotone input gives exactly +-1 whatever the values, a tie block is averaged once by the shared primitive, and the whole function is the familiar three sums over rank deviations. Constant input on either side has zero rank variance, 0 over 0, and returns NaN the way the math package answers its own degenerate cases, and the length mismatch panics like AUC's:

```
func Spearman(xs, ys []float64) float64 {
  if len(xs) != len(ys) {
    panic("stats: Spearman xs and ys differ in length")
  }
  rx, ry := midranks(xs), midranks(ys)
  var mx, my float64
  for i := range rx {
    mx += rx[i]
    my += ry[i]
  }
  n := float64(len(rx))
  mx /= n
  my /= n
  var sxy, sxx, syy float64
  for i := range rx {
    dx, dy := rx[i]-mx, ry[i]-my
    sxy += dx * dy
    sxx += dx * dx
    syy += dy * dy
  }
  if sxx == 0 || syy == 0 {
    return math.NaN()
  }
  return sxy / math.Sqrt(sxx*syy)
}
```

listing 231: Pearson's three sums over midranks: sxy over sqrt(sxx*syy) (source: go/data/internal/stats/rankcorr.go, lines 12-36)

The hand-worked tie block keeps the function honest. For xs {10, 20, 30, 40} against ys {7, 5, 7, 9}, the ys rank as 2.5, 1, 2.5, 4, the rank deviations are dx −1.5, −0.5, 0.5, 1.5 against dy 0, −1.5, 0, 1.5, and the three sums land at `sxy = 3`, `sxx = 5`, `syy = 4.5`, a correlation of `3/sqrt(22.5) = sqrt(2/5)`, all asserted bit for bit in the test. The exact +-1

on monotone input is the same discipline: squares, square roots, any strictly monotone map of xs correlates at 1, because ranks are invariant under monotone maps and the test says so:

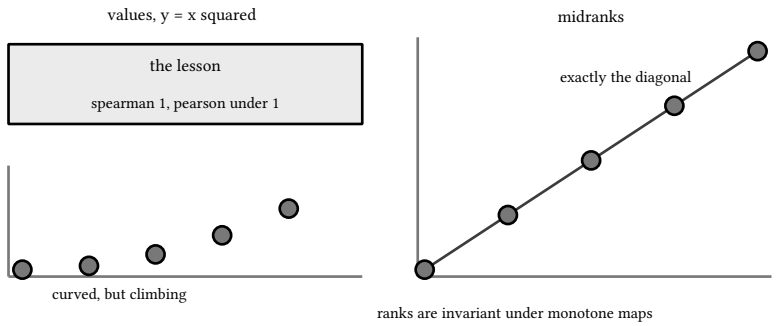


figure 200: values versus ranks: a nonlinear monotone pair flattens to the diagonal

31.7. the deterministic bootstrap

A p-value says an effect is there; a confidence interval says how big. The percentile bootstrap builds one without any distributional assumption about the metric: resample the rows with replacement, recompute the metric on each replicate, read the interval off the sorted replicates at the chosen percentiles. The go decision is that the resample must be reproducible, so the stream is a PCG generator from `math/rand/v2` seeded `(seed, seed)`, and the same seed replays a byte-identical interval, which turns any pinned interval into a permanent regression case. The metric arrives as a callback over a row-index slice, one buffer reused across replicates with a documented must-not-retain contract, the same shape a range loop's variable carries:

```
func PercentileCI(metric func(idx []int) float64, n, resamples int, seed uint64, loPct, hiPct float64) (lo,
hi float64) {
  if n < 1 || resamples < 1 {
    return math.NaN(), math.NaN()
  }
  rng := rand.New(rand.NewPCG(seed, seed))
  idx := make([]int, n)
  vals := make([]float64, resamples)
  for r := range resamples {
    for j := range idx {
      idx[j] = rng.IntN(n)
    }
    vals[r] = metric(idx)
  }
  slices.Sort(vals)
  return nearestRank(vals, loPct), nearestRank(vals, hiPct)
}
```

listing 232: the resample loop: PCG stream, reused index buffer, sorted replicates (source: go/data/internal/stats/bootstrap.go, lines 21-36)

The determinism test is the chapter's insurance policy: 80 uniforms from one pinned stream, resampled 200 times at seed 7, must return `(0.46601909707261074, 0.58610135085900195)` to the last bit on every run, and seed 8 must move it:

```
func TestBootstrapDeterminism(t *testing.T) {
  rng := rand.New(rand.NewPCG(33, 33))
  data := make([]float64, 80)
  for i := range data {
    data[i] = rng.Float64()
  }
  lo1, hi1 := PercentileCI(meanOf(data), 80, 200, 7, 2.5, 97.5)
  lo2, hi2 := PercentileCI(meanOf(data), 80, 200, 7, 2.5, 97.5)
  if lo1 ≠ lo2 || hi1 ≠ hi2 {
    t.Errorf("same seed differs: (%v,%v) vs (%v,%v)", lo1, hi1, lo2, hi2)
  }
  if lo1 ≠ 0.46601909707261074 || hi1 ≠ 0.58610135085900195 {
    t.Errorf("pinned interval drifted: got (%.17g,%.17g)", lo1, hi1)
  }
  lo3, _ := PercentileCI(meanOf(data), 80, 200, 8, 2.5, 97.5)
  if lo3 == lo1 {
    t.Error("different seed gave identical interval")
  }
}
```

listing 233: the pinned interval: full-precision equality, the seed is the only input (source: go/data/internal/stats/bootstrap_test.go, lines 25-43)

The percentiles are nearest-rank, the `ceil(p/100 * m)`-th smallest, clamped so 0 picks the minimum and 100 the maximum. The order of operations is load-bearing: the product `pct*m` is formed before the `/100`, so a dyadic percent with an exact integer rank, 2.5 percent of 400 resamples, lands on the 10th smallest instead of one ulp above it and silently the 11th. Each replicate draws n of n rows with replacement, so on average `1 - 1/e`, 63.2 percent, of the rows appear in a given replicate and the rest stay out of the bag; the measured first replicate at seed 1 over 500 rows touches 315 distinct rows, 63.0 percent, the same out-of-bag arithmetic the kdd evaluation chapter teaches, book 14, ch 17 (knowledge discovery, classifier evaluation):

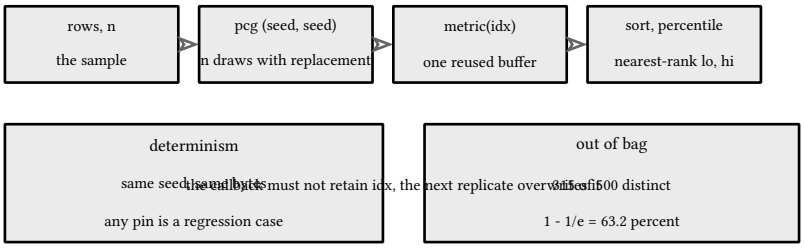


figure 201: the resample loop as a pipeline, determinism and buffer reuse pinned

31.8. correcting the family

Back to the opening number: the family is the unit of decision. Two classical corrections answer two different questions about it. Holm 1979 controls the family-wise error rate, the chance of even one false rejection: sort the p-values ascending, walk them against descending bars `alpha/(m-k+1)`, reject while each ordered p stays at or under its bar, and the first miss stops the family. Its first bar is `alpha/m`, plain Bonferroni, and the walk only loosens from there. Benjamini and Hochberg 1995 control the false discovery rate, the expected share of rejections that are wrong: the bars ascend as `(k/m)*q`, and the answer is the largest k whose p clears its bar, all k hypotheses below it rejected. One conservative guarantee about any mistake, one proportional guarantee about the mistake rate, and the verdicts come back in input order either way:

```
func Holm(ps []float64, alpha float64) []bool {
  validateFamily(ps, alpha)
  m := len(ps)
  order := ascending(ps)
  reject := make([]bool, m)
  for i, k := 0, 1; i < m; i, k = i+1, k+1 {
    if ps[order[i]] > alpha/float64(m-k+1) {
      break
    }
    reject[order[i]] = true
  }
  return reject
}
```

listing 234: Holm: step-down, the first miss stops the family (source: go/data/internal/stats/multiple.go, lines 18-30)

```
func BH(ps []float64, q float64) []bool {
  validateFamily(ps, q)
  m := len(ps)
  order := ascending(ps)
  reject := make([]bool, m)
  kstar := 0
  for i, k := 0, 1; i < m; i, k = i+1, k+1 {
    if ps[order[i]] <= float64(k)/float64(m)*q {
      kstar = k
    }
  }
  for i := range kstar {
    reject[order[i]] = true
  }
  return reject
}
```

listing 235: BH: step-up, the largest clearing k rejects below it (source: go/data/internal/stats/multiple.go, lines 38-53)

A four-hypothesis family shows why a tool needs both. At alpha 1/20, the sorted ps (0.01, 0.02, 0.03, 0.04): Holm's second bar is 1/60, about 0.0167, and 0.02 misses it, so exactly one hypothesis survives, while BH's fourth bar is 1/20 and 0.04 clears it, so all four do. Three of four verdicts differ. The boundary family in the tests sharpens the rule: at Holm's k = 3 the bar is `(1/20)/2 = 1/40` and the p is exactly `1/40`, equal to the last bit because halving a rounded double is exact, and the rule's ≤ decides the row, reject:

```
// the contrast family, m = 4 at alpha = 1/20, input order deliberately
// unsorted: H1 = 1/50, H2 = 1/25, H3 = 1/100, H4 = 3/100.
//
// Holm sorts to (1/100, 1/50, 3/100, 1/25) and walks the descending bars:
// k=1 bar 1/80 = 0.0125 rejects 1/100; k=2 bar 1/60 ~ 0.0167 stops at 1/50
// = 0.02, and the family is done: one rejection.
//
// BH walks the ascending bars: 1/80, 1/40 = 0.025, 3/80 = 0.0375, 1/20;
// every ordered p clears its bar, k* = 4, all four rejected. The two
// procedures disagree on 3 of the 4 hypotheses, which is the chapter's
// reason to carry both.
func TestContrastFamily(t *testing.T) {
  fam := []float64{1.0 / 50.0, 1.0 / 25.0, 1.0 / 100.0, 3.0 / 100.0}
  if got, want := Holm(fam, 1.0/20.0), []bool{false, false, true, false}; !slices.Equal(got, want) {
    t.Errorf("Holm(contrast) = %v, want %v", got, want)
  }
  if got, want := BH(fam, 1.0/20.0), []bool{true, true, true, true}; !slices.Equal(got, want) {
    t.Errorf("BH(contrast) = %v, want %v", got, want)
  }
}
```

listing 236: the contrast family: Holm one of four, BH four of four, derivation in the comments (source: go/data/internal/stats/multiple_test.go, lines 10-29)

The input policy is a panic, and the panic is the interesting part:

```


// validateFamily rejects p-values and levels outside [0,1], NaN included:
// a p-value is a probability, and the caller has no error channel, so an
// argument shaped like a probability that is not one deserves the loud
// failure now rather than a quietly wrong verdict set later.
func validateFamily(ps []float64, level float64) {
  if math.IsNaN(level) || level < 0 || level > 1 {
    panic(fmt.Sprintf("stats: level %v outside [0,1]", level))
  }
  for i, p := range ps {
    if math.IsNaN(p) || p < 0 || p > 1 {
      panic(fmt.Sprintf("stats: p[%d]=%v outside [0,1]", i, p))
    }
  }
}
```

listing 237: validateFamily: a p-value is a probability, and the caller has no error channel (source: go/data/internal/stats/multiple.go, lines 68-82)

NOTE: NaN answers and panic answers

This package returns NaN for a chi-square table with a zero margin and panics for a p-value of 1.5. Both are right. The zero-margin table is bad data, an answer of no-information in the math package's own idiom. The out-of-range p-value is a caller bug: a probability-shaped argument that is not a probability has no sane interpretation to return, an error return would be ignored at a screen site, and a loud failure at the call beats a quietly wrong verdict set in production.

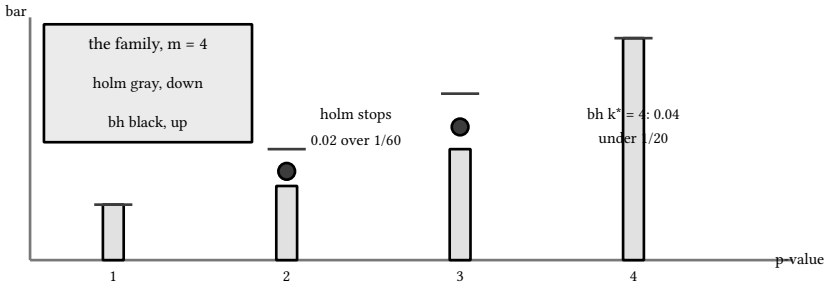


figure 202: the contrast family under both bar ladders: step-down stops early, step-up clears all four

31.9. the screening chain

The parts now compose into the case study's spine. The engine hands the package raw pair counts; each pair becomes a 2x2 table; the chi-square scores it fast and its gate routes thin tables to the exact binomial tail; the survivors' p-values enter Holm or BH as one family, and the tool chooses BH because a game balance tool prefers a bounded share of false leads over a guarantee of none, a hundred candidate buffs is a worklist, not a publication; the survivors' effect sizes then get bootstrap intervals, the deterministic kind, so a pinned run replays byte for byte; and the shrinkage step the next chapter builds keeps the screened effects from overfitting their own noise. Significance screens, correction prunes, intervals size, shrinkage stabilizes, in that order, because each stage's output is the next stage's contract:

```
ps := make([]float64, 0, len(pairs))
for _, p := range pairs {
  if _, pv, ok := stats.ChiSquare2x2(p.W, p.L, p.CW, p.CL); ok {
    ps = append(ps, pv)
  } else {
    ps = append(ps, stats.BinomialOneSided(p.W, p.W+p.L, 0.5))
  }
}
for i, keep := range stats.BH(ps, 0.05) {
  if keep {
    pairs[i].CI = stats.PercentileCI(pairs[i].Effect, len(rows), 400, pairs[i].Seed, 2.5, 97.5)
  }
}
```

fragment, illustrative, not executed

The kdd book owns the derivations this chapter compiled: the point-probability tail and the p-value grammar in book 14, ch 39 (knowledge discovery, hypothesis testing and p-values), the roc curve and the Mann-Whitney identity in book 14, ch 18 (knowledge discovery, roc and auc), the bootstrap and its out-of-bag third in book 14, ch 17 (knowledge discovery, classifier evaluation), and the correction theory, both ladders and their guarantees, in book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr). What go adds is the engineering: log space where factorials would overflow, one tie primitive where four statistics would each invent one, a seeded PCG where a flaky test suite would live, and a panic where a wrong verdict set would ship.

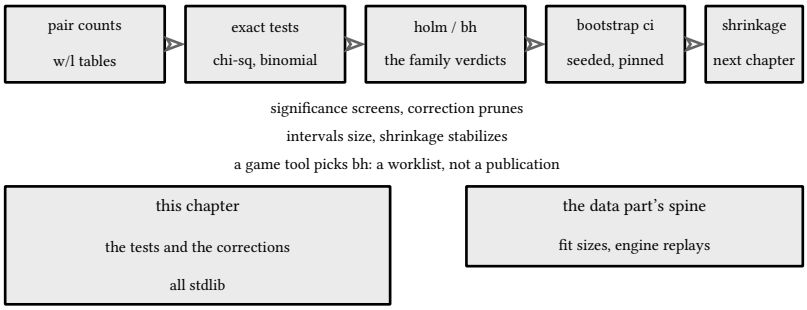


figure 203: the screening chain: each stage's output is the next stage's contract

sources: pkg.go.dev/math for Lgamma, Erfc, Log1p, and the NaN idiom, and pkg.go.dev/math/rand/v2 for the PCG source and its compatibility guarantee, both read against the go 1.27 module cache, accessed 2026-09-30; Holm, a simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6(2), pp 65-70, 1979, and Benjamini and Hochberg, controlling the false discovery rate, Journal of the Royal Statistical Society Series B 57(1), pp 289-300, 1995; the NIST SEMATECH e-Handbook of Statistical Methods for the df=1 critical 3.8415, accessed 2026-09-30; and the case study's own engine, github.com/lavantien/dota-helper, whose internal stats package these ideas were mined from. Verified by `go/data/internal/stats` tests, 48 of them under `go vet`, `go test`, and the race detector, gofmt clean over the module, every number in the prose pinned or banded by one of them.

32. fitting, backtesting, and guarded promotion

The mined tables hand over numbers, and the numbers are not equally trustworthy: a cell built from 8 matches and a cell built from 800 carry the same decimal places and wildly different evidence. This chapter builds the loop that decides which numbers deserve to live in the config hub, in `godata/internal/fit`: empirical-bayes shrinkage with its alpha measured from the data rather than tasted in, low-rank matrix completion for the thin cells, masked-cell cross-validation over a rank and ridge grid, two comparators that exist to be beaten, a bagged blend fitted by coordinate ascent, an order-aware replay backtest that cannot see the future, and a promotion guard that refuses to rewrite a config file without a bootstrap interval. The machinery is adapted from the dota-helper engine's eval and analytics packages, rewritten against the chapter 31 stats package so every statistic on these pages is one the corpus already owns.

32.1. shrinkage toward a prior

A win rate observed `n` times is a beta posterior in disguise. The fair prior says `p0`, the evidence says `w/n`, and the beta form blends them with `a` pseudo-counts: the posterior is `(w + a p0)/(n + a)`, which is the prior's `a` imaginary trials sitting next to the `n` real ones. Two boundary checks fall out of the arithmetic. With no evidence the posterior is exactly the prior, and as `n` grows past `a` the real trials outvote the imaginary ones and the posterior closes in on the observed rate. The mined tables store the displacement form, a cell's distance from fair rather than a raw rate, so the same blend written as a delta keeps only the `n/(n+a)` share of what was observed:

```
// Shrink blends an observed win count w over n trials toward the prior rate
// p0 with alpha pseudo-counts, the beta form (w + a*p0)/(n + a). n ≤ 0
// means no evidence, so the posterior is the prior untouched.
func Shrink(w, n, alpha, p0 float64) float64 {
  if n <= 0 {
    return p0
  }
  return (w + alpha*p0) / (n + alpha)
}

// ShrinkDelta shrinks an observed delta from the neutral rate: the same
// blend written as a displacement, where only the n/(n+a) share of the
// observed delta survives. It is the form the mined tables store, since a
// cell's value is its distance from fair, not a raw rate.
func ShrinkDelta(d, n, alpha float64) float64 {
  if n <= 0 {
    return 0
  }
  return d * n / (n + alpha)
}
```

listing 238: the beta blend and its displacement form, one posterior in two notations (source: go/data/internal/fit/shrink.go, lines 5-24)

Shrinking the value is half the job. A cell also needs its uncertainty, and the same alpha gives it: the per-trial sampling variance `sigma2` scales by the squared shrink factor, so the posterior variance is `sigma2 n/(n+a)^2`. The shape is the interesting part. It is 0 at `n = 0` because the prior is certain of itself, it peaks at exactly `n = a`, the one point where the observed delta and the prior carry equal weight, and it falls back toward 0 as evidence piles up. The test pins all three facts, including the peak value `sigma2/(4a)` at `n = a`:

```go
// PosteriorVar is the variance of the shrunk estimate given per-trial
// sampling variance sigma2: sigma2 * n/(n+a)^2, the sampling variance scaled
// by the squared shrink factor. It peaks at n = a, exactly where the
// observed delta and the prior carry equal weight, and falls toward 0 as
// evidence piles up. A cell's value says what to believe, its posterior
// variance says how much.
func PosteriorVar(n, alpha, sigma2 float64) float64 {
  if n <= 0 {
    return 0
  }
  return sigma2 * n / ((n + alpha) * (n + alpha))
}
```

listing 239: posterior variance: the value says what to believe, this says how much (source: go/data/internal/fit/shrink.go, lines 26-37)

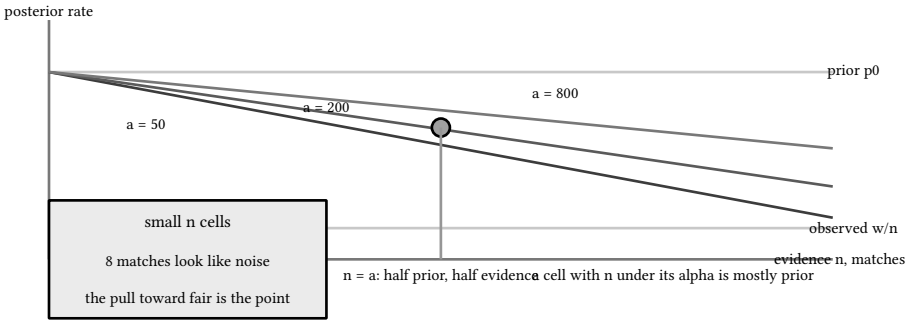


figure 204: posterior paths from the prior to the evidence, one curve per alpha, the fair rate between them

32.2. the alpha the data chooses

Pseudo-counts beg the obvious question: how many? A number typed into a config file is a taste with a decimal point, so the fit measures it. The population variance of the observed deltas decomposes into true between-cell spread `tau2` plus mean sampling noise `sigma2/n`, and the alpha whose displacement shrink reproduces that split is `sigma2/tau2`. When the spread is fully explained by sampling noise, `tau2` pins at 0, alpha would be infinite, and the family keeps its current value: no measurement should manufacture certainty out of variance. The estimator is ten lines over the family's deltas and counts:

```go
// MomentAlpha matches moments over one family of observed deltas d_i with
// counts n_i and per-trial sampling variance sigma2. The population variance
// of the deltas decomposes into true between-cell spread tau2 plus mean
// sampling noise sigma2/n, and the alpha whose ShrinkDelta reproduces that
// split is sigma2/tau2: prior strength measured from the data rather than
// tasted in. tau2 pinned at 0 means sampling noise alone explains the spread
// (alpha would be infinite), so kept reports the fallback to the current
// value instead of a number. Every n_i must be positive; a zero n reads as
// infinite sampling noise and forces the tau2 floor.
func MomentAlpha(ds, ns []float64, sigma2 float64) (alpha, tau2 float64, kept bool) {
  if len(ds) < 2 || len(ds) != len(ns) {
    return 0, 0, true
  }
  m := float64(len(ds))
  mean := 0.0
  for _, d := range ds {
    mean += d
  }
  mean /= m
  spread, sampling := 0.0, 0.0
  for i, d := range ds {
    spread += (d - mean) * (d - mean)
    sampling += sigma2 / ns[i]
  }
  tau2 = spread/m - sampling/m
  if tau2 <= 0 {
    return 0, 0, true
  }
  return sigma2 / tau2, tau2, false
}
```

listing 240: moment matching: spread minus sampling noise is tau2, and alpha is sigma2 over it (source: go/data/internal/fit/shrink.go, lines 39-68)

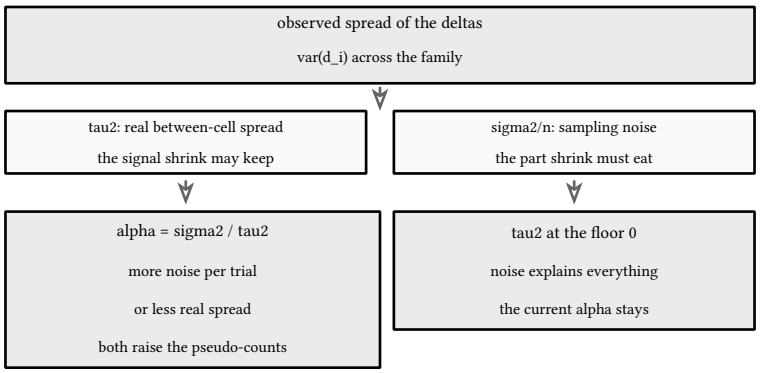


figure 205: the variance decomposition: observed spread splits into real signal and sampling noise

The measured number lands where the theory says it should. On the deterministic synthetic family the tests use, 200 cells at 150 to 250 trials each with true spread `tau2 = 0.0004` against the binomial worst-case `sigma2 = 0.25`, the moment match returns alpha 607.0 with tau2 0.000412 against the nominal 625, inside the test band. The moment estimate is not the last word: it sees wide counts, so the fit cross-checks it against a holdout win record by binomial deviance, twice the log-likelihood gap between held-out wins and the shrunk prediction, and lets that grid choice outvote the moment only when at least 30 cells qualify. On the seeded 40-cell synthetic the check picks grid alpha 200 at mean deviance 1.1871 against 1.4159 for a deliberately tiny current alpha of 10, the kind of mismatch the check exists to catch:

```
// CVAlpha sweeps an ascending grid by mean binomial deviance of the shrunk
// prediction p0 + ShrinkDelta(d, nAgg, alpha) against each cell's holdout
// win record; cells with nHold ≥ minHoldoutN qualify. Fewer than MinCVPairs
// qualifying cells marks the check inadequate and leaves every estimate
// untouched. Ties keep the earliest, smallest grid alpha, and the current
// value is scored on the same cells for the report.
func CVAlpha(grid []float64, obs []CVObs, minHoldoutN int, current, p0 float64) CVFit {
  var fit CVFit
  for _, o := range obs {
    if o.NHold ≥ minHoldoutN {
      fit.Pairs++
    }
  }
  if len(grid) == 0 || fit.Pairs < MinCVPairs {
    return fit
  }
  deviance := func(alpha float64) float64 {
    sum, n := 0.0, 0
    for _, o := range obs {
      if o.NHold < minHoldoutN {
        continue
      }
      sum += Deviance(float64(o.W), float64(o.NHold), p0+ShrinkDelta(o.D, o.NAgg, alpha))
      n++
    }
    return sum / float64(n)
  }
  fit.Adequate = true
  fit.Alpha, fit.MeanDeviance = grid[0], deviance(grid[0])
  for _, a := range grid[1:] {
    if d := deviance(a); d < fit.MeanDeviance {
      fit.Alpha, fit.MeanDeviance = a, d
    }
  }
  fit.CurrentDeviance = deviance(current)
  return fit
}
```

listing 241: the holdout cross-check: deviance of the shrunk prediction against real win records, ties keeping the smaller alpha (source: go/data/internal/fit/shrink.go, lines 112-148)

32.3. completing the thin cells

Shrinkage fixes the noisy cells, but a cell with no observations at all has nothing to shrink. The pool of heroes is small and the roster is large, so the pool-by-roster grid is mostly holes, and the holes are not random: a hero nobody picks against a hero nobody picks is a structural zero. Matrix completion refills those holes with a low-rank factorization, two thin matrices `u` and `v` whose product approximates the observed cells, trained by alternating least squares. Fix `v`, and every row of `u` is an independent ridge solve. Fix `u`, and every column of `v` is the same solve transposed. Alternate until the factors stop moving, break on the max-abs factor delta, and refill:

```
  // (seed, seed) is the corpus PCG convention: one stream per fit, every
  // consumer drawing its own
  rng := rand.New(rand.NewPCG(cfg.Seed, cfg.Seed))
  u := make([]float64, rows*r)
  v := make([]float64, cols*r)
  for i := range u {
    u[i] = rng.Float64()
  }
  for i := range v {
    v[i] = rng.Float64()
  }

  // scratch owned by this call and reused across every factor solve
  atA := make([]float64, r*r)
  b := make([]float64, r)
  rowObs := func(i, j int) bool { return m.Observed[i][j] }
  rowVal := func(i, j int) float64 { return m.Values[i][j] }
  colObs := func(i, j int) bool { return m.Observed[j][i] }
  colVal := func(i, j int) float64 { return m.Values[j][i] }

  for iter := 0; iter < cfg.MaxIter; iter++ {
    prevU := append([]float64(nil), u...)
    prevV := append([]float64(nil), v...)
    lsq(u, rows, cols, r, cfg.Lambda, cfg.PivotFloor, v, rowObs, rowVal, atA, b)
    lsq(v, cols, rows, r, cfg.Lambda, cfg.PivotFloor, u, colObs, colVal, atA, b)
    delta := maxAbsDiff(prevU, u)
    if d := maxAbsDiff(prevV, v); d > delta {
      delta = d
    }
    if delta < cfg.Tol {
      break
    }
  }

  for i := 0; i < rows; i++ {
    for j := 0; j < cols; j++ {
      if !m.Observed[i][j] {
        out.Values[i][j] = dot(u[i*r:(i+1)*r], v[j*r:(j+1)*r])
      }
    }
  }
  return out
```

listing 242: seeded init, fixed row-major order, one convergence test, then the refill (source: go/data/internal/fit/complete.go, lines 57-98)

Determinism is a property here, not an accident: the init draws from one PCG stream keyed `(seed, seed)` the corpus convention fixes, the sweep walks rows before columns in a fixed order, and the scratch buffers `AtA` and `b` are owned by the call so no allocator state leaks into the arithmetic. The per-factor solve assembles each side's normal equations with the ridge `lambda` on the diagonal, which is what keeps the system solvable when a row saw fewer cells than the rank, and hands them to a Gaussian elimination that refuses to divide by a pivot under the floor, leaving the previous estimate standing rather than manufacturing one from noise:

```
// lsq ridge-solves one factor side: for each of the count factors it
// minimizes over the observed cells against the current other-side factors,
// with lambda on the diagonal keeping the system solvable when a row saw
// fewer cells than the rank. A factor with no observed cells keeps its
// previous estimate. atA and b are caller-owned scratch reused across
// factors; solve destroys them.
func lsq(target []float64, count, otherCount, r int, lambda, pivotFloor float64, other []float64,
  obs func(i, j int) bool, val func(i, j int) float64, atA []float64, b []float64) {
  for i := 0; i < count; i++ {
    clear(atA)
    clear(b)
    has := false
    for j := 0; j < otherCount; j++ {
      if !obs(i, j) {
        continue
      }
      has = true
      ov := other[j*r : (j+1)*r]
      val := val(i, j)
      for p := 0; p < r; p++ {
        for q := 0; q < r; q++ {
          atA[p*r+q] += ov[p] * ov[q]
        }
        b[p] += val * ov[p]
      }
    }
    if !has {
      continue
    }
    for p := 0; p < r; p++ {
      atA[p*r+p] += lambda
    }
    if sol, ok := solve(atA, b, r, pivotFloor); ok {
      copy(target[i*r:(i+1)*r], sol)
    }
  }
```

listing 243: one factor side: normal equations with a ridge, caller-owned scratch, singular systems keep the old estimate (source: go/data/internal/fit/complete.go, lines 101-136)

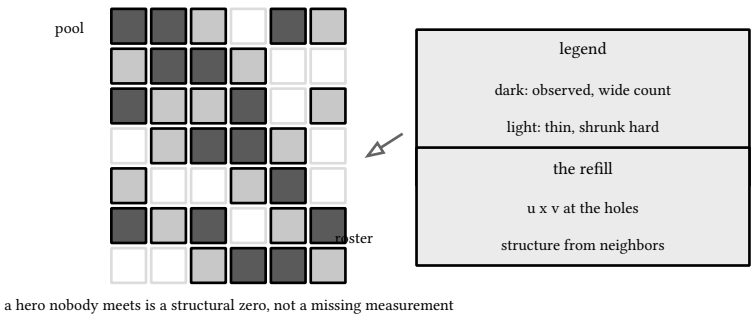


figure 206: the pool-by-roster grid: observed cells, thin cells, and holes refilled by low-rank structure

The tests grade the completion two ways. On an exact rank-1 grid with two masked cells and the ridge off, the refill recovers both cells under 1e-6, the one regime where recovery can be exact. On a rank-2 fixture under the production-sized ridge, ALS reaches rmse 1.0433 over the 4 masked cells where filling each hole with its column's observed mean scores 3.4473, and the observed cells pass through bit-identical: the completion may only ever add information where information is missing.

32.4. masked-cell cross-validation

Rank and ridge are two more constants that could be tasted in, and the fit refuses. It hides a seeded fraction of the observed cells, completes the holed matrix for every candidate on the grid, and scores each candidate by the Spearman correlation between its predictions and the held-out truth, the chapter 31 stats package's midrank Spearman reused as-is. Ties keep the smaller rank, then the smaller ridge. The current config runs through the same mask, both top candidates re-predict every held cell, and the winner-minus-current difference gets a percentile bootstrap interval over held-cell resamples through the same package's `PercentileCI`: one explicit reuse, two fewer statistics to re-derive and drift:

```
func CompletionCV(mats []Matrix, ranks []int, lambdas []float64, cur CompletionCfg,
  frac float64, seed uint64, resamples int) (CompletionReport, error) {
  var rep CompletionReport
  rng := rand.New(rand.NewPCG(seed, seed))
  train := make([]Matrix, len(mats))
  heldBy := make([][]HeldCell, len(mats))
  for i, m := range mats {
    train[i], heldBy[i] = Mask(m, frac, rng)
  }
  score := func(rank int, lambda float64) (float64, bool) {
    sum, n := 0.0, 0
    for i := range train {
      cc := cur
      cc.Rank, cc.Lambda = rank, lambda
      done := Complete(train[i], cc)
      var pred, actual []float64
      for _, c := range heldBy[i] {
        pred = append(pred, done.Values[c.Row][c.Col])
        actual = append(actual, c.Actual)
      }
      if len(pred) < 2 {
        continue
      }
      if r := stats.Spearman(pred, actual); !math.IsNaN(r) {
        sum += r
        n++
      }
    }
    if n == 0 {
      return 0, false
    }
    return sum / float64(n), true
  }
```

listing 244: the shared mask, then each grid point scored by reused Spearman against the held-out cells (source: go/data/internal/fit/crossval.go, lines 87-119)

```
  rep.Masked = len(rep.Held)
  rep.Frac = frac
  // a resample whose ranks go constant carries no Spearman and contributes
  // a flat 0, the convention that keeps the interval finite on tiny cells
  rep.DiffLo, rep.DiffHi = stats.PercentileCI(func(idx []int) float64 {
    var pw, pc, act []float64
    for _, i := range idx {
      pw, pc, act = append(pw, rep.Held[i].Winner), append(pc, rep.Held[i].Current), append(act,
rep.Held[i].Actual)
    }
    dw, dc := stats.Spearman(pw, act), stats.Spearman(pc, act)
    if math.IsNaN(dw) || math.IsNaN(dc) {
      return 0
    }
    return dw - dc
  }, len(rep.Held), resamples, seed, 2.5, 97.5)
```

listing 245: the diff CI: reused PercentileCI over held-cell resamples, constant ranks contributing 0 (source: go/data/internal/fit/crossval.go, lines 153-167)

On the seeded synthetic, two exact rank-3 grids at 75 percent observed with 30 percent of cells masked, the grid of ranks 1 through 4 against ridges 0.001 to 0.1 picks the true rank 3 with held-out Spearman 0.9991 against 0.8064 for a crippled current candidate at rank 1, over 86 masked cells, with the difference CI landing at [0.1072, 0.2655]: the guard evidence, computed, not asserted.

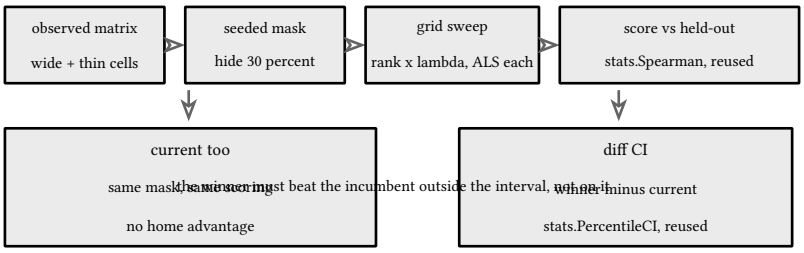


figure 207: the cv pipeline: mask, sweep the grid, score against the held-out truth, bootstrap the diff

32.5. comparators that never feed the live score

A fit that only ever races its own incumbent can drift into a local pride, so the backtest also runs two comparators built on deliberately different assumptions. Both read the same feature space, a draft's final pick set as ascending item ids, and both answer one question: does this set look more like the winners or the losers in train. The kNN is a retrieval engine, cosine similarity over one-hot sets, which for sorted id slices is the intersection count over the sqrt of the two sizes, a linear merge with no hash and no float feature vector. Its neighbors vote weighted by similarity, and similarity ties break by train id ascending so the vote never depends on the order drafts were handed in, the property the test proves by reversing the train slice and scoring again. The naive Bayes is a counting engine, per-item win and loss counts with a Laplace add-alpha rescue so a zero cell scores a finite ratio. Its one subtle rule is the out-of-vocabulary skip: an item counted zero in both classes contributes nothing, because scoring it would add another copy of the class prior per unknown item and a draft full of strangers would drift toward whichever class was rarer:

```
// cosineSim is the similarity between the query and one train set from
// their shared id count and the precomputed train norm: for one-hot sets
// this is |A intersect B| / sqrt(|A| * |B|). Both slices are sorted
// ascending, so the intersection is one linear merge.
func cosineSim(q, tr []int, trNorm float64) float64 {
  qNorm := math.Sqrt(float64(len(q)))
  if qNorm == 0 || trNorm == 0 {
    return 0
  }
  shared := 0
  for i, j := 0, 0; i < len(q) && j < len(tr); {
    switch {
    case q[i] < tr[j]:
      i++
    case q[i] > tr[j]:
      j++
    default:
      shared++
      i, j = i+1, j+1
    }
  }
  return float64(shared) / (qNorm * trNorm)
}
```

listing 246: cosine over one-hot sets as a sorted-int merge, the norm precomputed at train time (source: go/data/internal/fit/knn.go, lines 111-133)

```
// Score is the win log-odds of the draft: the prior log-odds plus, per
// present item, the log ratio of its class conditionals P(f|win)/P(f|loss),
// each cell Laplace-smoothed against the binary denominator Nclass + 2a.
// Positive favors a win, 0 is exactly the decision boundary, and the
// ascending-id sum makes equal sets bit-identical across calls. A train
// split holding only one label carries no discrimination and scores a flat
// 0, the AUC chance-line convention again.
func (nb *NaiveBayes) Score(s Set) float64 {
  if nb.nWin == 0 || nb.nLoss == 0 {
    return 0
  }
  score := math.Log(float64(nb.nWin) / float64(nb.nLoss))
  for _, ft := range s.ids {
    w, l := nb.winCnt[ft], nb.lossCnt[ft]
    if w == 0 && l == 0 {
      continue
    }
    pw := (float64(w) + nb.alpha) / (float64(nb.nWin) + 2*nb.alpha)
    pl := (float64(l) + nb.alpha) / (float64(nb.nLoss) + 2*nb.alpha)
    score += math.Log(pw / pl)
  }
  return score
}
```

listing 247: bernoulli log-odds with the laplace rescue and the unknown-item skip (source: go/data/internal/fit/naivebayes.go, lines 42-64)

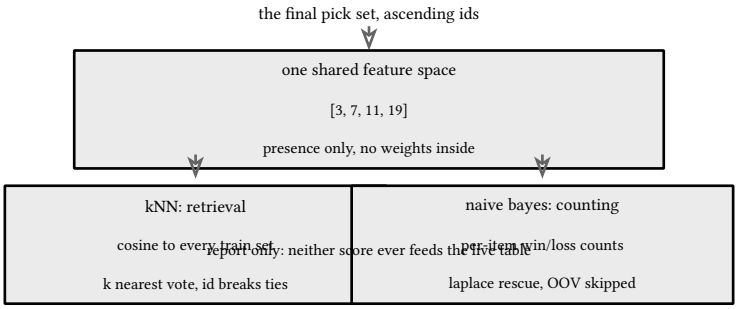


figure 208: two comparators, one feature space: retrieval by similarity versus counting by class

The hygiene lesson is the wiring, not the math. Both comparators score into the report and stop there: no comparator output reaches the live config, because a comparator's job is to lose informatively. The day a comparator beats the fitted model is the day the fit needs redoing, not the day the comparator gets promoted around the guard.

32.6. a bagged blend under coordinate ascent

The blend is where the fitted model meets the comparators: an authored draft-advantage component, the bagged naive Bayes score, the bagged kNN score, three non-negative weights over them. Bagging draws B bootstrap resamples of the train drafts, trains both comparators in every bag, and averages: a single comparator's split idiosyncrasies are noise, and the resampled average trades a little bias for a lot of stability, leaving roughly `1/e` of the drafts out of bag in each. The weights are fitted by coordinate ascent on train AUC, the stats package's Mann-Whitney AUC reused as the objective. Each weight scans the multiples of the step up to the cap with the others fixed, keeps a value only on strict improvement so ties keep the earlier, lower value, and a pass that improves nothing stops the ascent:

```
func TrainEnsemble(spec EnsembleSpec, train []TrainDraft, advantage map[int64]float64) *Ensemble {
  if spec.Bags <= 0 {
    panic("fit: ensemble bags must be positive")
  }
  e := &Ensemble{}
  rng := rand.New(rand.NewPCG(spec.Seed, spec.Seed))
  for _, idx := range bootstrapResamples(rng, len(train), spec.Bags) {
    bag := make([]TrainDraft, len(idx))
    for i, j := range idx {
      bag[i] = train[j]
    }
    e.nbs = append(e.nbs, TrainNB(bag, spec.NBAlpha))
    e.knns = append(e.knns, TrainKNN(bag, spec.KnnK))
  }
  nbS := make([]float64, len(train))
  knS := make([]float64, len(train))
  pkS := make([]float64, len(train))
  labels := make([]bool, len(train))
  for i, d := range train {
    nbS[i], knS[i] = e.NBScore(d.Set), e.KNNScore(d.Set)
    pkS[i] = advantage[d.ID]
    labels[i] = d.Won
  }
  e.weights = FitBlendWeights([][]float64{pkS, nbS, knS}, labels,
    spec.Step, spec.MaxWeight, spec.MaxPasses)
  return e
}

// bootstrapResamples draws bags replicates of n train slots with replacement
// from one rng stream, the same draw pattern the stats package's
// PercentileCI uses: every slot an independent uniform pick, so each bag
```

listing 248: bags from one PCG stream, comparators per bag, components handed to the ascent (source: go/data/internal/fit/ensemble.go, lines 40-70)

```
func FitBlendWeights(components [][]float64, labels []bool, step, maxWeight float64, maxPasses int)
[]float64 {
  w := make([]float64, len(components))
  if len(components) == 0 || len(labels) == 0 || step <= 0 || maxWeight <= 0 || maxPasses < 1 {
    return w
  }
  cur := make([]float64, len(labels))
  base := make([]float64, len(labels))
  cand := make([]float64, len(labels))
  best := stats.AUC(cur, labels)
  for pass := 0; pass < maxPasses; pass++ {
    improved := false
    for j, comp := range components {
      for i := range base {
        base[i] = cur[i] - w[j]*comp[i]
      }
      bestV := w[j]
      for i := 0; ; i++ {
        v := step * float64(i)
        if v > maxWeight+step*1e-9 {
          break
        }
        for k := range cand {
          cand[k] = base[k] + v*comp[k]
        }
        if a := stats.AUC(cand, labels); a > best {
          best, bestV, improved = a, v, true
        }
      }
      if bestV != w[j] {
        w[j] = bestV
        for i := range cur {
          cur[i] = base[i] + bestV*comp[i]
        }
      }
    }
    if !improved {
      break
    }
  }
  return w
}
```

listing 249: coordinate ascent on AUC: strict improvement only, early stop, ratios over levels (source: go/data/internal/fit/ensemble.go, lines 134-174)

One property of the objective deserves its sentence in the config review. AUC reads only the order of the blended scores, so scaling every weight by the same constant changes nothing: the absolute level is meaningless and only the ratios carry information. That scale invariance is why the grid may stay coarse, and the test states it as a property, scaling one component by 100 and watching the fitted AUC hold still.

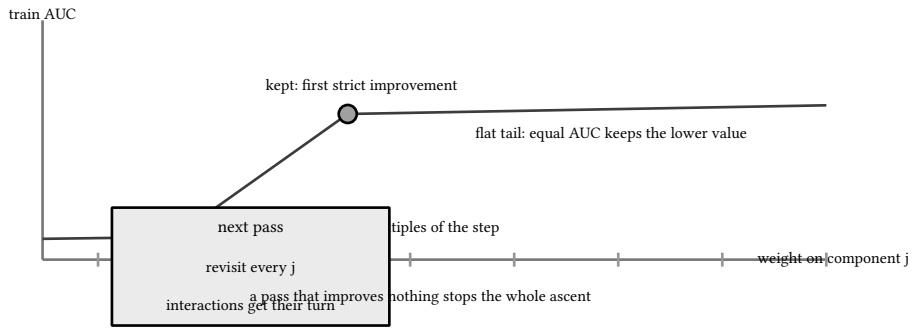


figure 209: one ascent over a two-weight slice: the sweep, the keep rule, the stop

32.7. replaying drafts without lookahead

Every metric so far scores tables against tables. The backtest that matters walks real drafts pick by pick and asks the only question a picker cares about: at this moment, with this much of the draft visible, where did the taken hero rank among the alternatives. The discipline is one function: the state at pick i is built from events strictly before i, same-side earlier picks as allies, opposing earlier picks as enemies, picks and bans together as taken. Nothing at or after i leaks in, and the property test holds the walker to it by permuting the draft's future at three cut points and asserting every earlier pick's score and rank come out bit-identical: a shuffled future is invisible or the backtest is lying:

```
// StateBefore builds the visible state of the event at position i: allies
// are the earlier same-side picks, enemies the earlier opposing picks, and
// taken every earlier pick and ban. Nothing at or after i leaks in, which is
// the lookahead discipline the whole backtest stands on: the future of the
// draft is exactly the part of the file the walker must never open.
func StateBefore(events []Event, i int) Visible {
  var v Visible
  ev := events[i]
  for _, prev := range events[:i] {
    v.Taken = append(v.Taken, prev.Item)
    if !prev.Pick {
      continue
    }
    if prev.Side ≠ ev.Side {
      v.Enemies = append(v.Enemies, prev.Item)
    } else {
      v.Allies = append(v.Allies, prev.Item)
    }
  }
  return v
}
```

listing 250: stateBefore: everything strictly earlier, split into allies, enemies, taken (source: go/data/internal/fit/replay.go, lines 31-51)

The rank itself is order-independent by construction: score descending, ties by ascending item id, over the roster minus the taken list with the picked item competing. The mean percentile and the top-k hit rate read directly off the ranks, skipping single-candidate fields where no ranking exists:

```
// Replay walks every draft in pick order and scores each pick against the
// alternatives its state left available: the whole roster minus the taken
// list, the picked item included. Every score reads only the visible
// allies, so no candidate and no rank can see the draft's future.
func Replay(m *ReplayModel, roster []int, drafts []Draft) []PickEval {
  var out []PickEval
  for _, d := range drafts {
    for i, ev := range d.Events {
      if !ev.Pick {
        continue
      }
      v := StateBefore(d.Events, i)
      taken := make(map[int]bool, len(v.Taken))
      for _, t := range v.Taken {
        taken[t] = true
      }
      sp := m.Score(ev.Item, v.Allies)
      rank, n := 1, 0
      for _, c := range roster {
        if taken[c] {
          continue
        }
        n++
        if c == ev.Item {
          continue
        }
        if sc := m.Score(c, v.Allies); sc > sp || (sc == sp && c < ev.Item) {
          rank++
        }
      }
      p := PickEval{DraftID: d.ID, Item: ev.Item, Rank: rank, N: n, Score: sp}
      if n > 1 {
        p.Percentile = (float64(n) - float64(rank)) / float64(n-1)
      }
      out = append(out, p)
    }
  }
  return out
}
```

listing 251: the walk: each pick scored against the alternatives its state left available (source: go/data/internal/fit/replay.go, lines 87-125)

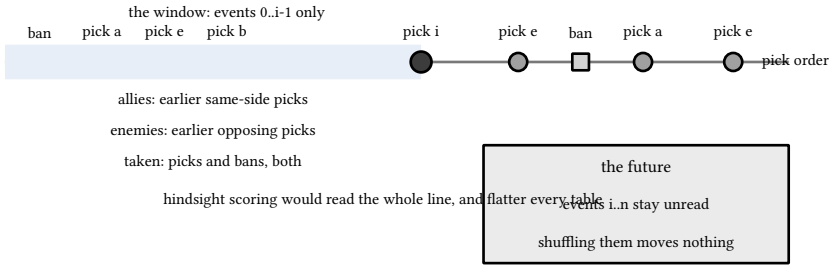


figure 210: one draft on a timeline: the visibility window at pick i is everything left of it

Scores also need calibration, a number that means the same thing on holdout as it meant on train. The fit slices train pick scores into decile edges, upper-edge inclusive, collapses repeated edges so heavy ties leave no zero-width bin, bins the holdout scores against those edges, and counts adjacent nonempty bins whose realized win rate

drops: monotonicity violations. On the seeded synthetic the climbing ladder scores 0 violations and its inversion scores 9, one per adjacent pair, and the count rides in the report where a promotion decision can read it:

```
// Calibrate bins holdout pick scores against quantile edges fitted on train
// pick scores, the decile calibration the holdout is graded on: a score
// that means what it says produces a monotone ladder and zero violations.
// Bins are upper-edge inclusive, a score exactly on an edge lands in the bin
// below it, and repeated edges from heavy ties collapse so no zero-width
// bin survives.
func Calibrate(trainScores, holdScores []float64, holdWon []bool, bins int) Calibration {
  if bins < 2 || len(trainScores) == 0 {
    return Calibration{}
  }
  sorted := append([]float64(nil), trainScores...)
  sort.Float64s(sorted)
  var edges []float64
  for i := 1; i < bins; i++ {
    e := quantile(sorted, float64(i)/float64(bins))
    if len(edges) == 0 || e > edges[len(edges)-1] {
      edges = append(edges, e)
    }
  }
  cal := Calibration{Edges: edges, Bins: make([]CalibrationBin, len(edges)+1)}
  for i := range cal.Bins {
    cal.Bins[i].Index = i
  }
  for i, s := range holdScores {
    b := sort.SearchFloat64s(edges, s)
    cal.Bins[b].N++
    if holdWon[i] {
      cal.Bins[b].Rate++
    }
  }
  prev := -1
  for i := range cal.Bins {
    if cal.Bins[i].N == 0 {
      continue
    }
    cal.Bins[i].Rate /= float64(cal.Bins[i].N)
    if prev ≥ 0 && cal.Bins[i].Rate < cal.Bins[prev].Rate {
      cal.Violations++
    }
    prev = i
  }
  return cal
}

// quantile is the nearest-rank percentile of ascending values, the same
```

listing 252: decile calibration: edges from train, rates from holdout, drops counted (source: go/data/internal/fit/replay.go, lines 175-220)

32.8. guarded promotion

Everything upstream writes proposals, never configs. The promotion is the single choke point where a fitted number may enter the hub, and it is guarded four ways. The proposal is re-read from disk, because the bytes that

get promoted must be the bytes that were audited. The baseline is pinned: the proposal records the live weights its gain was measured against, and a hub that moved since makes the numbers stale, so the guard refuses and asks for a rerun rather than comparing a fresh gain against an old truth. The holdout gain must clear its floor. And the bootstrap interval's low end must sit strictly above 0, because a mean gain that could be zero is no gain:

```
// Guard re-checks one proposal against the live weights and the gain floor.
// The weight vector must be present and finite, the baseline must equal the
// live weights, the holdout gain must clear the floor, and the bootstrap
// interval's low end must sit strictly above 0: a mean gain that could be
// zero is no gain. Every check runs before any file is opened for writing.
func Guard(p Proposal, live []float64, minGain float64) error {
  if len(p.Weights) == 0 || slices.ContainsFunc(p.Weights, notFinite) {
    return fmt.Errorf("promote: proposal carries no sane weight vector")
  }
  if notFinite(p.Gain) || notFinite(p.GainCiLo) || notFinite(p.GainCiHi) {
    return fmt.Errorf("promote: proposal gain interval is not finite")
  }
  if !slices.Equal(p.Baseline, live) {
    return fmt.Errorf("promote: config changed since the fit measured its gain, rerun the fit")
  }
  if p.Gain < minGain {
    return fmt.Errorf("promote: guard failed, holdout gain %.4f below the floor %.4f", p.Gain, minGain)
  }
  if p.GainCiLo <= 0 {
    return fmt.Errorf("promote: guard failed, gain CI low %.4f not above 0", p.GainCiLo)
  }
  return nil
}
```

listing 253: the guard: finite weights and interval, pinned baseline, gain floor, CI low above zero (source: go/data/internal/fit/promote.go, lines 24-46)

The write path is shaped so refusal is provably harmless. Every check runs before any file is opened for writing, and the single rewrite unmarshals the hub, replaces the blend weights, and re-indents at two spaces with sorted keys, so the same hub rewrites to the same bytes. The test states the property the operator actually needs: a gain below the floor returns an error and the config file's bytes compare identical before and after, not as a promise but as an assertion:

```go
// Promote applies one proposal to the config hub behind its guard: the
// proposal is re-read from disk, guarded against the live weights with
// nothing opened for writing, and only a passing guard reaches the single
// rewrite, which unmarshals the hub, replaces the blendWeights entry, and
// re-indents at 2 spaces with a trailing newline, keys sorted so the same
// hub rewrites to the same bytes. Every failing path returns before the
// write, so a refused promotion leaves the config bytes identical; the
// caller's proof is a byte comparison, not a promise.
func Promote(proposalPath, cfgPath string, live []float64, minGain float64) (Proposal, error) {
  b, err := os.ReadFile(proposalPath)
  if err ≠ nil {
    if os.IsNotExist(err) {
      return Proposal{}, fmt.Errorf("promote: no proposal at %s: run the fit first", proposalPath)
    }
    return Proposal{}, err
  }
  var p Proposal
  if err := json.Unmarshal(b, &p); err ≠ nil {
    return Proposal{}, fmt.Errorf("promote: parse proposal: %w", err)
  }
  if err := Guard(p, live, minGain); err ≠ nil {
    return p, err
  }
  cb, err := os.ReadFile(cfgPath)
  if err ≠ nil {
    return p, fmt.Errorf("promote: read config: %w", err)
  }
  hub := map[string]json.RawMessage{}
  if err := json.Unmarshal(cb, &hub); err ≠ nil {
    return p, fmt.Errorf("promote: parse config: %w", err)
  }
  wb, err := json.Marshal(p.Weights)
  if err ≠ nil {
    return p, err
  }
  hub["blendWeights"] = json.RawMessage(wb)
  out, err := json.MarshalIndent(hub, "", "  ")
  if err ≠ nil {
    return p, err
  }
  if err := os.WriteFile(cfgPath, append(out, '\n'), 0o644); err ≠ nil {
    return p, fmt.Errorf("promote: rewrite config: %w", err)
  }
  return p, nil
}
```

listing 254: promote: re-read, guard, then and only then the one rewrite (source: go/data/internal/fit/promote.go, lines 48-92)

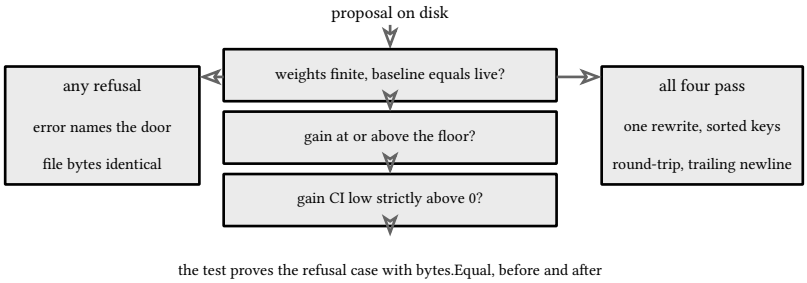


figure 211: the guard decision flow: four doors, and every refusal leaves the file untouched

32.9. the fit loop

The pieces close into a cycle. The fit measures against the current hub and writes proposals: an alpha per family with its cross-check, a completion candidate with its diff CI, a blend weight vector with its gain interval. The guard re-reads each proposal, re-checks it against the live hub, and either promotes through the single rewrite or refuses with the file untouched. A promoted hub changes the baseline every later fit measures against, which is the point: the loop's next pass must beat the promoted numbers, not the numbers they replaced. Every fitted constant carries its provenance, the procedure that produced it and the measurement it must answer to again, because a config file full of unexplained decimals is where models go to rot quietly.

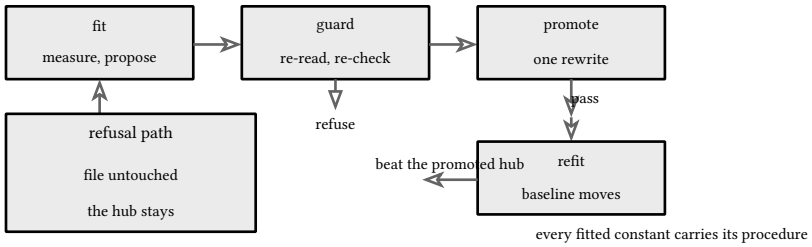


figure 212: the loop: fit measures the hub, guard re-checks, promote rewrites, refit must beat what promotion made live

sources: pkg.go.dev/math/rand/v2 for the PCG generator the seeded init and bags draw from, read against the go 1.27 module cache source, plus the dota-helper engine (`poolguide`, local repository) whose eval and analytics and mine packages this chapter adapts: shrinkage and posterior variance from analytics/shrink.go and mine/cell.go, the moment-matched alpha with its holdout deviance cross-check from eval/fitalpha.go, the ALS completion and masked-cell CV from analytics/complete.go and eval/fitcompletion.go, the comparators from eval/knn.go and eval/naivebayes.go, the blend from eval/ensemble.go, the replay and calibration from eval/replay.go and eval/metrics.go, and the guarded promotion from eval/promote.go, all accessed 2026-10-05. Verified by `go/data/internal/fit` tests under `go vet`, `go test`, and the race detector, with the Spearman, AUC, and PercentileCI machinery reused from `go/data/internal/stats`.

33. the offline engine and its js twin

Every number the previous two chapters mined had to arrive from somewhere and had to reach a browser somehow. This chapter builds both ends as one program, the offline engine in `go/data/internal/engine`: a client-side rate limiter and an http client that survives a hostile remote, a resumable backfill whose ledger outlives crashes, a reflection emitter that renders Go structs as javascript literals, and a browser-side twin that rescores the generated artifact in the page. The design adapts a real production engine, the dota-helper pool guide, whose ingest, emit, and picker layers this chapter reworks idea by idea rather than copying, and the honest debts are stated as they come. The package is 7 source files and 1177 lines, stdlib only, and its proof is 47 tests across 12 test files plus 8 node tests over the twin.

33.1. the offline frame

The engine runs as a pipeline with a hard wall in the middle. Everything before the wall happens offline, on a schedule, in a terminal: the crawl walks a graphql api one query at a time, every response is committed to a raw cache before anything is derived from it, the mine reads only committed files, and the emitter renders the mined tables into generated javascript. Everything after the wall is a static page: no server, no database, no runtime query, just the artifact plus a linear scorer and the authored gate rules, both small enough to read in one sitting. The wall is the point. Because the artifact is committed bytes, the whole product is reproducible from git, and because the browser only ever sums precomputed cells, the expensive parts cannot fail in front of a user:

```
// Package engine holds the offline-pipeline machinery the data part
// teaches: the client-side rate limiter with an injected clock, the
// retry policy that honors Retry-After under a cap, the resumable
// backfill manifest over append-only NDJSON, the reflection-based
// Go-to-JS emitter behind golden tests, and the browser-side twin the
// emitter's artifacts feed. Stdlib only.
package engine
```

listing 255: the package doc: one paragraph naming every stage the chapter builds (source: go/data/internal/engine/doc.go, lines 1-7)

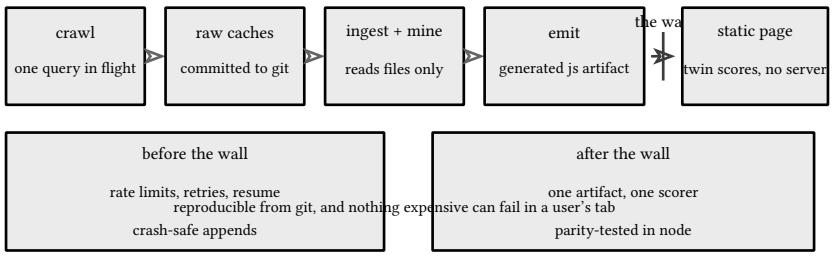


figure 213: the offline frame: everything left of the wall runs in a terminal, the browser only scores committed bytes

The second design decision is the one that reads as heresy after chapter 6: the engine is serial. Zero goroutine statements across the 7 source files, zero mutexes, zero channels, and the only `sync.Mutex` in the package lives in the test fake's request recorder, guarding the httptest handler's slice. That is not ignorance of the concurrency chapters, it is their cost accounting applied honestly. A crawl that must respect a rate limit of a few requests per second has no parallel speedup to buy: the wall clock belongs to the remote api, not the local cpu, and the mine is a batch job over committed files that finishes in seconds. Concurrency here would add a leak surface, a race surface, and a resume story that has to reason about interleaved appends, and it would buy nothing measurable. Chapter 6′s lesson was when goroutines pay; this chapter is the counterexample that completes it, and the crawl is never run with a parallel jobs flag on purpose.

One more honest note before the machinery. This module ships no fuzz targets, and neither does the dota-helper engine it adapts. The corpus's own fuzz culture, six stdlib fuzz targets over the api module's invariants in chapter 30, stops at the service part's border, and the reasoning is scope, not oversight: the engine's parseable inputs are

files it wrote itself plus remote bytes that all flow through bounded, loud failure paths, and the property that matters most here, cross-language score parity, is already machine-checked every verify by the fixture contract at the end of this chapter.

33.2. spacing plus a bucket

The client-side limiter answers a different problem than chapter 26's service-side one. There, the bucket protected a server from its callers. Here, the caller protects itself from the remote's limits, which arrive in two shapes at once: a minimum spacing between requests and a burst allowance per second. A client that respects only the spacing trips the burst cap the moment it runs long enough, and a client that respects only the bucket hammers the spacing floor, so the limiter enforces both and waits the larger of the two debts:

```
// Wait blocks until the next request is allowed. The delay is the larger of
// the spacing debt and the whole-token debt, and a caller that cannot pay
// yet spends nothing: the token is charged once, after the sleep.
func (l *Limiter) Wait() {
  now := l.c.Now()
  var wait time.Duration
  if !l.lastReq.IsZero() {
    if need := l.interval - now.Sub(l.lastReq); need > 0 {
      wait = need
    }
  }
  l.refill(now)
  if l.tokens < 1 {
    if need := time.Duration((1 - l.tokens) / l.rate * float64(time.Second)); need > wait {
      wait = need
    }
  }
  if wait > 0 {
    l.c.Sleep(wait)
    now = l.c.Now()
    l.refill(now)
  }
  l.tokens--
  l.lastReq = now
}
```

listing 256: Wait: the spacing debt and the whole-token debt compared, the larger one slept, the token charged once after (source: go/data/internal/engine/ratelimit.go, lines 46-70)

The arithmetic is chapter 26's bucket with the refill moved into the clock: `refill` converts elapsed time into tokens lazily on every call, capped at the capacity, so idle time earns at most one burst and there is no ticker and no background loop anywhere. The one new rule is the comparison. The spacing debt is `interval - elapsed`, the token debt is `(1 - level) / rate` seconds when the level sits under one token, and a caller that cannot pay spends nothing: the decrement happens once, after the sleep, which is why a denied request can never push the level negative.

Every time-dependent behavior in the package runs on an injected clock, the two-method `Clock` interface at the top of the file. Production passes the real clock, tests pass a fake pinned at unix zero that records each sleep and advances instantly, so the suite asserts a 7 second Retry-After backoff as one recorded duration instead of waiting seven seconds, and no test in the package sleeps on wall time:

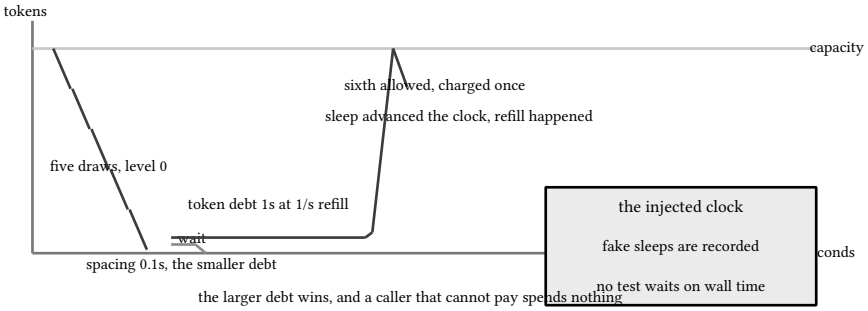


figure 214: one wait under both constraints: the bucket is empty and the spacing floor has 100ms left, the larger debt wins

33.3. the client and its retries

The http client is deliberately plain: `net/http`, one POST per query, a json envelope, a 30 second timeout. Two production facts shape everything else in the file. First, the plain client is answered 403 at the edge, because the remote fronts a bot filter that fingerprints headers, so the browser-like headers from the options ride on every attempt, set before the authorization header so nothing about the auth surface is ever the varying part. Second, the failures a crawl can outlive are not the failures a browser can: a 429 with a long Retry-After, a 502 from a dying edge, a connection killed before any response byte, a body that dies halfway through. All four retry. A 403 does not, because no number of attempts fixes a forbidden client, and failing fast costs one request instead of four:

```
// Query posts q, retrying the arms a crawl can outlive: 429 and 5xx answers
// back off by their Retry-After under the cap, transport and mid-body
// failures retry after one second, and every other status fails fast because
// no number of attempts fixes a 403. The bearer token is set on the request
// and appears in no log line and no error string, all of which flow through
// c.logf and the error types above.
func (c *Client) Query(q string) ([]byte, error) {
  payload, err := json.Marshal(map[string]string{"query": q})
  if err ≠ nil {
    return nil, err
  }
  var lastErr error
  for attempt := 0; attempt <= c.opt.MaxRetries; attempt++ {
    c.rl.Wait()
    body, err := c.attempt(payload)
    if err == nil {
      return body, nil
    }
    lastErr = err
    var se *statusError
    var te *transientError
    switch {
    case errors.As(err, &se) && se.seconds == 0:
      // a status no retry can fix: fail now, having spent one request
      return nil, err
    case errors.As(err, &se):
      c.logf("client: attempt %d/%d failed (%v), backing off %ds",
        attempt+1, c.opt.MaxRetries+1, err, se.seconds)
      c.opt.Clock.Sleep(time.Duration(se.seconds) * time.Second)
    case errors.As(err, &te):
      c.opt.Clock.Sleep(time.Second)
    default:
      // request construction failed, which no retry can fix either
      return nil, err
    }
  }
  return nil, fmt.Errorf("query failed after %d attempts: %w", c.opt.MaxRetries+1, lastErr)
}
```

listing 257: Query: one classification switch, retryable statuses back off, wire faults wait a second, everything else fails fast (source: go/data/internal/engine/client.go, lines 121-158)

The classification is three error types. A `statusError` carries the code, the bounded body excerpt, and the honored Retry-After in seconds when the status deserves one; zero seconds means final, and the switch returns it immediately. A `transientError` wraps a transport failure or a mid-body read failure, the two ways a remote dies that no status code can express. Anything else, a request construction error, is a programming bug and comes back unretried. The backoff itself honors the header but never trusts it, because the remote's hour and day rate buckets answer Retry-After 3600 and a crawl that slept an hour per attempt would look exactly like a hung one:

```
// statusError reads a Retry-After it is willing to honor, clamped by the
// cap: an api whose hour-long bucket answers Retry-After 3600 must fail
// visibly after the retries, not park the crawl for an hour per attempt.
func (c *Client) statusError(resp *http.Response, body []byte) error {
  se := &statusError{status: resp.StatusCode, body: excerpt(body)}
  if se.status != http.StatusTooManyRequests && se.status < 500 {
    return se
  }
  ra, _ := strconv.Atoi(strings.TrimSpace(resp.Header.Get("Retry-After")))
  if ra <= 0 {
    ra = 1
  }
  if ra > c.opt.RetryCap {
    c.logf("client: %d with Retry-After %ds, clamping backoff to %ds",
      se.status, ra, c.opt.RetryCap)
    ra = c.opt.RetryCap
  }
  se.seconds = ra
  return se
}
```

listing 258: the Retry-After read: floored at 1, clamped at the cap, the clamp said out loud (source: go/data/internal/engine/client.go, lines 189-208)

The clamp is the interesting contract. Honoring the header verbatim is correct per the spec and catastrophic per the crawl, so the client clamps to a configurable cap, 60 seconds by default, logs the clamp, and lets the retries exhaust. An hour-bucketed 429 then fails visibly within minutes with the attempt count in the error, which is the honest outcome: the operator reads that the budget is spent today, the code never silently parks. One more rule closes the file's security surface: the bearer token is set on the request and appears nowhere else. Every diagnostic flows through one logf, every error carries at most a code and a 200 byte excerpt, and a test walks every failure arm asserting the token appears in no log line and no error in the chain.

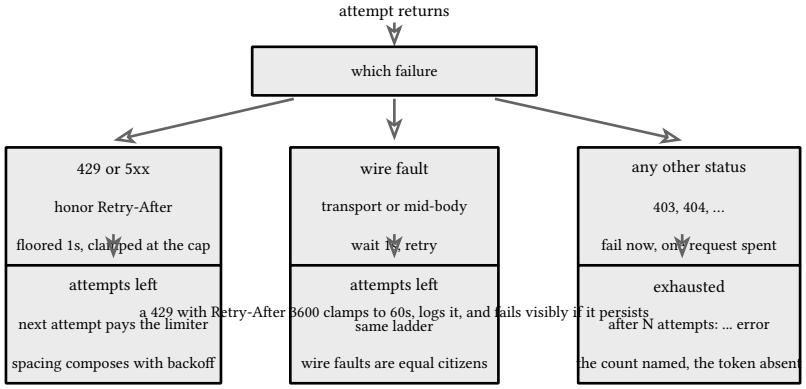


figure 215: the retry ladder: where each failure class lands

33.4. fault-injecting fakes

Testing that ladder against the real api would spend the token, burn the rate budget, and still never reproduce a mid-body disconnect on demand. The suite instead runs every wire test against an httptest server on loopback whose handler answers from a responder callback keyed by the request index, so a sequencing arm is a three-line closure and needs no counters of its own. The responder returns one canned reply per hit, and two flags on that reply manufacture the failures a status code cannot express:

```
    s.mu.Lock()
    s.seen = append(s.seen, payload.Query)
    hit := len(s.seen)
    s.mu.Unlock()
    rep := respond(payload.Query, hit, r)
    if rep.hijack {
      hj, ok := w.(http.Hijacker)
      if !ok {
        t.Errorf("fake server: response writer cannot hijack")
        return
      }
      conn, _, err := hj.Hijack()
      if err ≠ nil {
        t.Errorf("fake server: hijack: %v", err)
        return
      }
      conn.Close()
      return
    }
    if rep.retryAfter ≠ "" {
      w.Header().Set("Retry-After", rep.retryAfter)
    }
    w.WriteHeader(rep.code)
    if rep.abort {
      // flush a partial chunk then kill the stream: the client must
      // observe a mid-body read error, not a clean empty 200
      w.Write([]byte(rep.body[:len(rep.body)/2]))
      w.(http.Flusher).Flush()
      panic(http.ErrAbortHandler)
    }
    w.Write([]byte(rep.body))
```

listing 259: the responder: the hit index, the hijack arm that kills the connection pre-response, the abort arm that flushes half a body then panics (source: go/data/internal/engine/server_test.go, lines 107-137)

The hijack arm takes the connection back from the http server before any response byte and closes it, which the client observes as a plain transport error, exactly what a blocked port or a hard reset looks like. The abort arm writes half the body, flushes it down the wire, then panics with `http.ErrAbortHandler`, the documented way to abort a handler without taking the server down: the client's `io.ReadAll` receives real bytes and then an unexpected eof, which is the mid-body read error path no mock body can produce. Both arms are asserted through the same `Query` call as the happy path, and both retry to success because the ladder classifies them as wire faults.

The exported entry points construct their own client around the product endpoint constant, which is the right production shape and the wrong test shape, so the seam is the transport itself. A `roundTripFunc` swap replaces `http.DefaultTransport` with a function that rewrites only scheme and host toward the loopback server and delegates to the old transport for the real connection, restored in a test cleanup. `Fetch` then runs its exact production construction path, its constant endpoint included, and not one byte leaves the machine:

```
func (f roundTripFunc) RoundTrip(r *http.Request) (*http.Response, error) { return f(r) }

// redirectTraffic points every request that falls through to the default
// transport at srv, rewriting only scheme and host. The exported entry
// points build their own client around DefaultEndpoint, so this swap is the
// seam that runs their production path untouched while the suite owns the
// wire: no request can leave loopback, because the transport itself no
// longer knows where the real endpoint lives.
func redirectTraffic(t *testing.T, s *fakeServer) {
  t.Helper()
  target, err := url.Parse(s.url())
  if err ≠ nil {
    t.Fatal(err)
  }
  old := http.DefaultTransport
  http.DefaultTransport = roundTripFunc(func(r *http.Request) (*http.Response, error) {
    rr := r.Clone(r.Context())
    u := *r.URL
    u.Scheme, u.Host = target.Scheme, target.Host
    rr.URL, rr.Host = &u, u.Host
    return old.RoundTrip(rr)
  })
  t.Cleanup(func() { http.DefaultTransport = old })
}
```

listing 260: the DefaultTransport swap: the exported entry point's traffic rewritten to loopback, scheme and host only (source: go/data/internal/engine/server_test.go, lines 184-207)

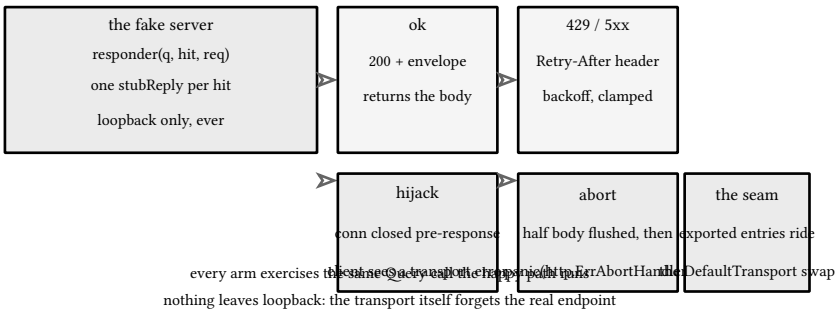


figure 216: the fault arm matrix: four ways a remote answers, one responder callback

33.5. the resumable backfill

A crawl over thousands of remote pages will be interrupted, by a rate budget, a network death, or an operator, and the only sane answer is that restarting costs the work not yet done and nothing else. The backfill keeps two files in one directory: an append-only ndjson ledger, one json object per line keyed by id, and a manifest, the resume record that names how far the walk got, what it kept and dropped and why, how many requests it spent, and whether the window is exhausted. Committed lines are never rewritten, so the ledger is safe to append across crashes, and the dedupe set is rebuilt from the committed bytes on every run, so a page the previous run already landed can never land twice.

The one crash case that needs actual repair is the torn tail. A process killed mid-append can leave a last line with no newline, either because the bytes never finished arriving or because the newline itself was the byte that did not land. The loader distinguishes the two: a complete row missing only its newline is kept and terminated, a torn fragment is dropped, and either way the file is rewritten once so later appends never glue onto a half line. The line it refuses to cross is just as deliberate: a corrupt line with committed lines after it is corruption, not a crash

tail, and resuming from a silently shrunken set would rewalk pages the ledger already owns, so that fails loudly with the line number and the bytes:

```
func ManifestPath(dir string) string { return filepath.Join(dir, "_manifest.json") }

// LoadSeen rebuilds the dedupe set from the committed ledger. An unterminated
// trailing line is the signature of a crash mid-append: a complete row that
// only lacks its newline is kept and terminated, a torn fragment is dropped,
// and either way the file is rewritten once so later appends never glue onto
// a half line. A corrupt line mid-file is corruption, not a torn tail, and
// fails loudly instead of silently shrinking the set.
func LoadSeen(path string) (map[int64]bool, error) {
  raw, err := os.ReadFile(path)
  if err ≠ nil {
    if os.IsNotExist(err) {
      return map[int64]bool{}, nil
    }
    return nil, err
  }
  s := string(raw)
  if s ≠ "" && !strings.HasSuffix(s, "\n") {
    idx := strings.LastIndexByte(s, '\n')
    head, tail := "", s
    if idx ≥ 0 {
      head, tail = s[:idx+1], s[idx+1:]
    }
    var r Row
    if json.Unmarshal([]byte(tail), &r) == nil && r.ID ≠ 0 {
      head += tail + "\n"
    }
    if err := os.WriteFile(path, []byte(head), 0o644); err ≠ nil {
      return nil, err
    }
    s = head
  }
  seen := map[int64]bool{}
  for i, line := range strings.Split(strings.TrimSuffix(s, "\n"), "\n") {
    if line == "" {
      continue
    }
    var r Row
    if json.Unmarshal([]byte(line), &r) ≠ nil || r.ID == 0 {
      return nil, fmt.Errorf("ledger line %d corrupt: %q", i+1, line)
    }
    seen[r.ID] = true
  }
  return seen, nil
}
```

listing 261: LoadSeen: the torn-tail repair, keep-and-terminate or drop, then the loud walk over committed lines (source: go/data/internal/engine/backfill.go, lines 38-83)

The walk itself is a loop with three exits, the target row count, the end of the window, and the per-run request budget, and the checkpoint lands after every page. A fetch error returns with the manifest already saved and the cursor parked at the failed page, so the rerun retries exactly that page and the ledger dedupes anything a half-written page left behind. The budget exit is what makes the crawl polite by construction: two pages now, the rest tomorrow, and the manifest is the whole conversation between the two runs:

```
func Run(dir string, target, budget int, fetch Page) (*Manifest, error) {
  manifestPath := ManifestPath(dir)
  m, err := LoadManifest(manifestPath)
  if err ≠ nil {
    return nil, err
  }
  if m.Done {
    return m, nil
  }
  seen, err := LoadSeen(NdjsonPath(dir))
  if err ≠ nil {
    return nil, err
  }
  requests := 0
  for !m.Done && m.Kept < target && requests < budget {
    rows, drops, more, err := fetch(m.NextPage)
    m.Requests++
    requests++
    if err ≠ nil {
      if saveErr := SaveManifest(manifestPath, m); saveErr ≠ nil {
        return m, saveErr
      }
      return m, err
    }
    for _, d := range drops {
      m.DropReasons[d]++
    }
    m.Dropped += len(drops)
    n, err := AppendRows(NdjsonPath(dir), rows, seen)
    if err ≠ nil {
      return m, err
    }
    m.Kept += n
    m.NextPage++
    m.Done = !more
    if err := SaveManifest(manifestPath, m); err ≠ nil {
      return m, err
    }
  }
  return m, nil
}
```

listing 262: Run: three exits, one checkpoint per page, the cursor never moves past a page it did not survive (source: go/data/internal/engine/backfill.go, lines 164-204)

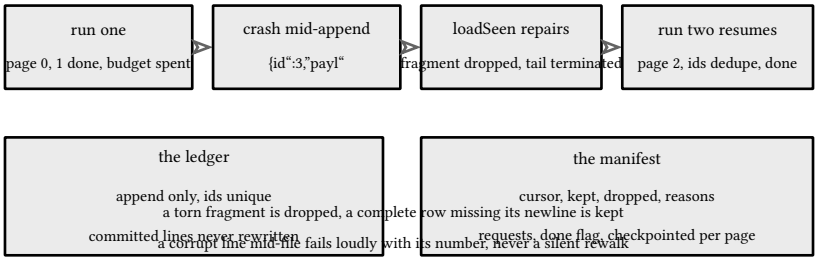


figure 217: crash mid-append, the repair, and the resume: what each run redoes

33.6. the store decision

The mine needs somewhere to land its tables, and this module's ruling is the plainest one that tells the truth: an in-memory store with provenance rows, rebuilt from the committed raw layer on every run, stdlib only. The store keeps one cell per key with the source that last wrote it and an audit trail of provenance rows, one per recorded fact, because every mined number eventually faces the question that is not what it is but where it came from:

```
// Provenance is one row of the run's audit trail: what kind of fact, about
// what subject, with what detail. Provenance rows are the store's answer to
// the question every mined number eventually faces, which is not what it is
// but where it came from.
type Provenance struct {
  Kind    string
  Subject string
  Detail  string
}

// Store is the in-memory landing zone the mine builds on. The stdlib-only
// ruling for this module makes it deliberately plain: the store is rebuilt
// from the committed raw layer on every run, so nothing here persists and
// nothing here needs a lock, the single-threaded engine is its only client.
// The trade is stated rather than hidden: no columnar scan, no on-disk
// footprint, and a size bounded by the roster, which this workload fits.
type Store struct {
  cells map[string]Cell
  prov  []Provenance
}

func NewStore() *Store {
  return &Store{cells: map[string]Cell{}}
}
```

listing 263: provenance rows and the in-memory landing zone, rebuilt per run, no lock because no second client exists (source: go/data/internal/engine/store.go, lines 12-35)

The ruling is scoped, not global. The dota-helper engine this chapter adapts mines through real analytics instead: duckdb behind the duckdb-go driver, imported unconditionally there and kept off the windows dev box by running every build and test inside a linux docker image. The corpus takes the build-tag road on its own duckdb lane, the `duckdb_use_lib` tag behind make verify-go, and teaches it where the workload justifies it, in book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices), whose capstone runs the whole discovery pipeline on duckdb. This book's other storage posture is chapter 23's wal engine, where durability is the product being taught. Here, neither applies: the data part's tables are roster-sized, tens of thousands of cells at most, the store is a cache in front of committed files, and a map plus a slice plus a sorted snapshot costs less than any of the honest alternatives would cost to justify. The `Cells` snapshot sorts by key so two runs over the same raw layer produce byte-identical snapshots regardless of Go's map iteration order, which is the same determinism discipline the emitter enforces one section later.

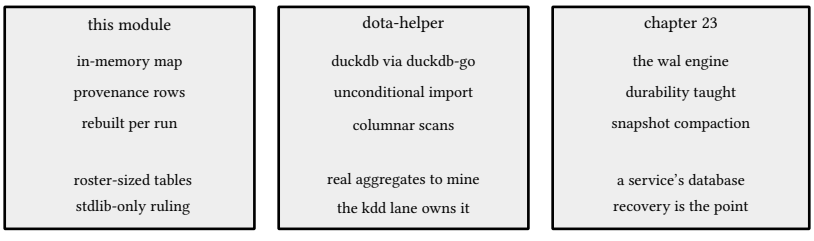


figure 218: three storage postures, one per workload

33.7. the go-to-js emitter

The artifact is a javascript file, and rendering it through `encoding/json` would be almost right, which is worse than wrong: json quotes with double quotes and escapes the wrong things for a script tag, map iteration order would leak Go's randomized map order into committed bytes, and the float formatter json picks is not the one the twin rescores against. So the emitter is 155 lines of reflection that walks any json-shaped value and renders the literal the artifact actually wants. Struct fields render in declaration order honoring json names, `omitempty` drops zero values, unexported fields and dash tags skip exactly what `encoding/json` skips, map keys sort ascending so the bytes out are a pure function of the value in, and NaN and both infinities render as null because the artifact is a script tag and none of the three has a literal:

```
case reflect.Struct:
  t := v.Type()
  var parts []string
  for i := 0; i < t.NumField(); i++ {
    f := t.Field(i)
    if f.PkgPath ≠ "" {
      // unexported, and encoding/json skips it too: what the
      // standard encoder cannot see, the emitter must not render
      continue
    }
    name := f.Name
    omitEmpty := false
    if tag, ok := f.Tag.Lookup("json"); ok {
      fields := strings.Split(tag, ",")
      if fields[0] == "-" && len(fields) == 1 {
        continue
      }
      if fields[0] ≠ "" && fields[0] ≠ "-" {
        name = fields[0]
      }
      for _, opt := range fields[1:] {
        if opt == "omitempty" {
          omitEmpty = true
        }
      }
    }
    fv := v.Field(i)
    if omitEmpty && fv.IsZero() {
      continue
    }
    parts = append(parts, quoteJS(name)+": "+renderJS(fv))
  }
  if len(parts) == 0 {
    return "{}"
  }
  return "{ " + strings.Join(parts, ", ") + " }"
}
```

listing 264: the struct arm: declaration order, json names, omitempty, the dash tag and unexported fields skipped like the standard encoder (source: go/data/internal/engine/js.go, lines 118-154)

The one function the whole parity contract rests on is smaller than its consequences. `RoundFixed` rounds a float by printing it at a fixed number of decimals and parsing the printed string back, which sounds circular until the artifact enters the picture: the packed cells the twin reads were printed at 4 decimals, so the engine rescores against the digits the artifact ships, not the unrounded doubles that produced them. Without this function the Go side would average values the browser never saw, and the two scores would drift in the fourth decimal forever:

```
func RoundFixed(v float64, dec int) float64 {
  n, err := strconv.ParseFloat(fmtFixed(v, dec), 64)
  if err ≠ nil {
    return v
  }
  return n
}
```

listing 265: RoundFixed: parse back exactly what fmtFixed printed, the artifact's digits are the truth (source: go/data/internal/engine/js.go, lines 55-62)

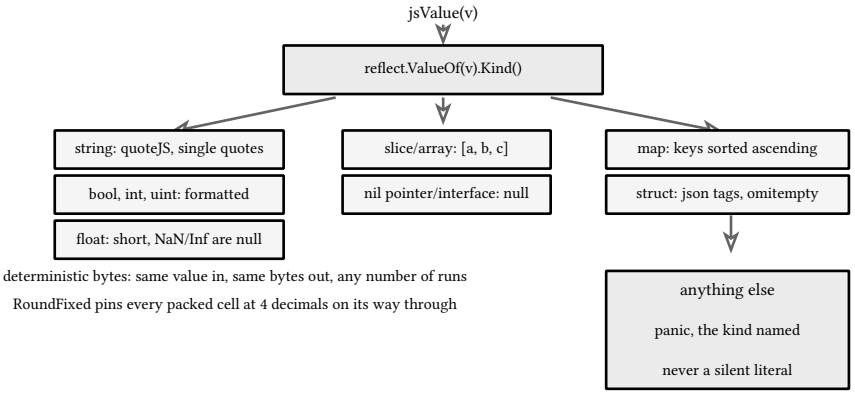


figure 219: the type walk: one switch, six shapes, one panic

33.8. golden tests

An emitter whose output is committed bytes needs a test that reads those bytes, and the corpus pattern is the update flag: a plain `flag.Bool` parsed by `go test`, false by default, true under `go test ./internal/engine -update`, and one check function that either compares or rewrites. The committed golden is the contract. Nobody edits it by hand, and a change to the payload, the emitter, or the fixture generator shows up as a one-line diff instead of two walls of generated bytes, because the failure path prints `firstDivergence`, the first line where want and got disagree, both lines quoted:

```
// updateGoldens rewrites the testdata golden files instead of comparing, so
// a payload or fixture change regenerates the pins with
// `go test ./internal/engine -update`. The committed bytes are the contract:
// a golden is never edited by hand.
var updateGoldens = flag.Bool("update", false, "rewrite golden files under testdata")

func checkGolden(t *testing.T, name string, got []byte) {
  t.Helper()
  path := filepath.Join("testdata", name)
  if *updateGoldens {
    if err := os.MkdirAll("testdata", 0o755); err ≠ nil {
      t.Fatal(err)
    }
    if err := os.WriteFile(path, got, 0o644); err ≠ nil {
      t.Fatal(err)
    }
    return
  }
  want, err := os.ReadFile(path)
  if err ≠ nil {
    t.Fatalf("read golden: %v (run go test ./internal/engine -update to generate)", err)
  }
  if string(want) ≠ string(got) {
    t.Fatalf("%s drifted from testdata/%s:\n%s", name, name, firstDivergence(string(want), string(got)))
  }
}
```

listing 266: checkGolden: the update flag, the compare path, the drift named by its first divergent line (source: go/data/internal/engine/golden_test.go, lines 12-38)

Two disciplines keep the pattern honest. The first is the cache: Go's test cache keys on the files a test reads at runtime, the recorded opens hashed into the key, so a golden edited with no code change busts the cache and the plain run fails honestly. A run about to trust a golden still passes `-count=1` when it wants execution rather than recall: the flag makes the run prove the suite ran on the tree in front of it, warm cache or not, the discipline the dota-helper emit suite adopted for its own note. The second is the generator's determinism, asserted directly: the parity fixture is built twice from one seed pair and the two artifacts must be byte-identical, which holds because the seeded PCG stream is stable by construction and the emitter sorts every map it walks. A golden over a nondeterministic generator would be a flaky test wearing a contract's clothes.

The generated artifact in this package is one file, `testdata/parity-data.js`, 1380 bytes, and its golden gate is what makes the next section's parity contract work: because the Go emitter's bytes and the committed file are proven identical, the node suite can read that file as the shared truth without any Go runtime in the room.

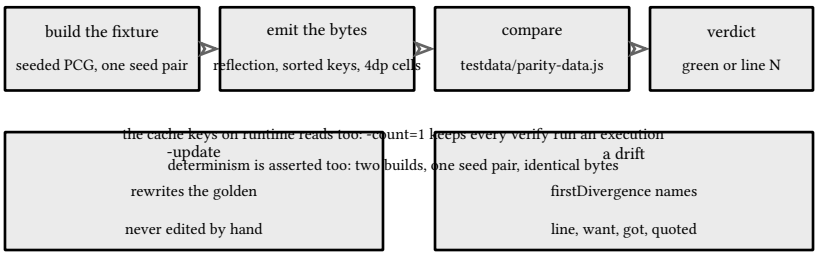


figure 220: the golden gate: emit, compare, and the one line a drift costs

33.9. the twin and the parity contract

The browser side of the frame is a 156-line javascript twin of the Go scorer, written as a dual-use module: the same file is a plain `<script>` that hangs `PoolScore` off the global object for the static page, and a CommonJS export

the node suite imports, the one pattern that serves both hosts without a bundler. It reads only the artifact's shape, and it fails loud on a corrupt payload rather than scoring on a duplicated stale copy of the constants: missing or nonpositive score constants throw, a role the synergy weights do not know throws, an unknown condition kind throws instead of quietly answering false and disarming a rule, and so does an unknown action. The Go side returns errors on exactly the same arms, which is the parity discipline applied to failure too:

```
function scoreCandidate(G, state, role, candidateIdx) {
  const pos = G.poolIdx.indexOf(candidateIdx);
  if (pos < 0) throw new Error('candidate not in pool: ' + candidateIdx);
  // score consts live in the artifact and the emitter always ships them, so
  // a payload without them is corrupt: fail loud rather than score on a
  // duplicated stale copy of the hub values
  const C = G.scoreConsts;
  if (!C || !(C.enemySlots > 0) || !(C.allySlots > 0)) {
    throw new Error('pool artifact missing positive scoreConsts');
  }
  const synW = G.weights && G.weights.synByRole && G.weights.synByRole[role];
  if (synW === undefined) throw new Error('weights.synByRole has no role ' + role);

  const muRow = parsePacked(G.mu[pos]);
  const synRow = parsePacked(G.syn[pos]);
  const muOf = i => pctAt(muRow, i);
  const synOf = i => pctAt(synRow, i);

  const knownMu = mean(state.enemies.map(muOf));
  const knownSyn = mean(state.allies.map(synOf));
  const prior = G.prior[pos];

  const taken = new Set(state.allies.concat(state.enemies, state.banned));
  taken.add(candidateIdx);
  const unseen = [];
  for (let i = 0; i < G.roster.length; i++) if (!taken.has(i)) unseen.push(i);
  const genericFit = popWeightedMean(unseen.map(synOf), unseen.map(i => G.pop[i]));
  const exposure = popWeightedMean(
    unseen.map(i => Math.max(0, C.midPct - muOf(i))),
    unseen.map(i => G.pop[i]));

  const gate = gateResult(G, state, role, candidateIdx);

  const k = state.enemies.length;
  const a = state.allies.length;
  const W = G.weights;
  // accumulation order mirrors the go scorer exactly for bit-level parity
  let score = W.knownMu * (k / C.enemySlots) * knownMu;
  score += synW * (a / C.allySlots) * knownSyn;
  score += W.prior * prior;
  score += W.genericFit * ((C.allySlots - a) / C.allySlots) * genericFit;
  score -= W.exposure * ((C.enemySlots - k) / C.enemySlots) * exposure;
  score += gate.delta;

  return { score, hardGated: gate.hard, firedRuleIds: gate.fired };
}
```

listing 267: the twin's scoreCandidate: same terms, same order, same operations, throws where the Go side errors (source: go/data/internal/engine/twin/score.js, lines 58-103)

The accumulation is the heart of it. Both sides sum six terms in one fixed order, known matchup weighted by visible enemies, known synergy weighted by the role and visible allies, the prior, the generic fit over unseen heroes

weighted by the unseen share, the exposure overhang weighted by the empty enemy share, then the gate delta, and every sub-expression matches operation for operation: the means sum ascending then divide, the popularity weighting interleaves numerator and denominator per element, and the Go line `score -= w * frac * exposure` is the twin's `score -= W.exposure * frac * exposure` with the operands in the same tree. IEEE 754 arithmetic is deterministic per operation, so identical operation sequences give identical doubles, which is why the contract can demand bit-identical scores rather than scores within an epsilon:

```
  k := len(st.Enemies)
  a := len(st.Allies)
  c := p.ScoreConsts
  score := p.Weights.KnownMu * (float64(k) / float64(c.EnemySlots)) * knownMu
  score += synW * (float64(a) / float64(c.AllySlots)) * knownSyn
  score += p.Weights.Prior * prior
  score += p.Weights.GenericFit * (float64(c.AllySlots-a) / float64(c.AllySlots)) * genericFit
  score -= p.Weights.Exposure * (float64(c.EnemySlots-k) / float64(c.EnemySlots)) * exposure
  score += gate.delta

  return Verdict{Value: score, HardGated: gate.hard, FiredIDs: gate.fired}, nil
}
```

listing 268: the Go accumulation the twin mirrors line for line (source: go/data/internal/engine/score.go, lines 212-223)

None of that is worth anything unless something checks it, so the parity contract is machine-checked through one committed fixture. The Go test builds the payload from a seeded PCG, one fixed seed pair, renders it through the emitter, and golden-gates the bytes as `testdata/parity-data.js`. It then scores a frozen draft board against that payload and asserts two exact constants. The node test reads the same committed file, evaluates it under a `window` shim exactly as a browser would, scores the same frozen board through the twin, and asserts the same two constants with plain `===` equality. Parity is transitive: Go equals the artifact, the artifact equals what node reads, and both suites pin the same 17 significant digits, `1.2223335616139586` for charlie with the nudge applied and `0.71824239654427635` for alpha under the hard gate, digits that round-trip a float64 exactly, so both literals parse to the same double:

```
test('parity: charlie scores the go constant exactly, nudge included', () => {
  const v = PoolScore.scoreCandidate(G, STATE, STATE.role, CHARLIE);
  // the same 17 significant digits go's parity test asserts: both literals
  // parse to the same double, and the comparison is plain equality
  assert.ok(v.score === 1.2223335616139586, 'got ' + v.score);
  assert.equal(v.hardGated, false);
  assert.deepEqual(v.firedRuleIds, ['bravo-nudge']);
});

test('parity: alpha is hard gated and scores the go constant exactly', () => {
  const v = PoolScore.scoreCandidate(G, STATE, STATE.role, ALPHA);
  assert.ok(v.score === 0.71824239654427635, 'got ' + v.score);
  assert.equal(v.hardGated, true);
  assert.deepEqual(v.firedRuleIds, ['delta-veto']);
});
```

listing 269: the node side of the contract: the committed artifact, the frozen board, plain equality on both constants (source: go/data/internal/engine/twin/score.test.mjs, lines 34-48)

The twin suite is plain `node:test` plus `node:assert/strict`, zero dependencies, deterministic, and it runs inside `make verify` through the `verify-data-twin` lane right after the companion gate, so the parity contract is checked on every verify of the corpus. A hand-computed fixture rides beside the generated one in both suites, every value a quarter or an eighth so the expected scores, `1.21875` for the gated alpha and `0.875` for charlie, are exact doubles a human

can verify with pencil and paper, which is what keeps the frozen constants honest: the pencil arithmetic and the seeded generator agree, in two languages, on every verify.

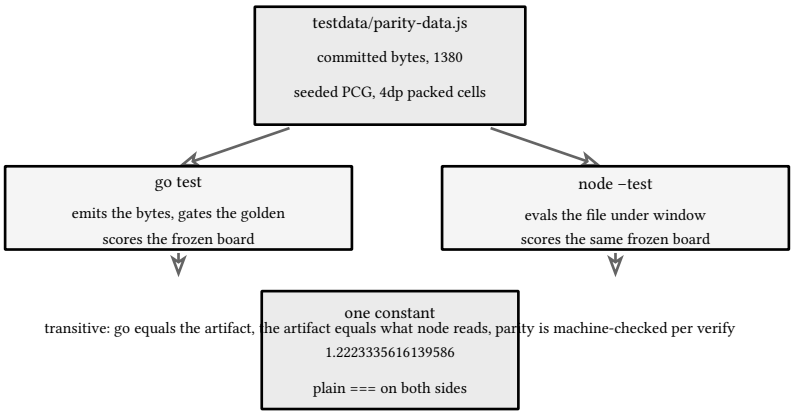


figure 221: the parity contract: one committed fixture, two suites, one constant

sources: pkg.go.dev for net/http including the DefaultTransport swap and http.ErrAbortHandler semantics, encoding/json for the struct tag rules the emitter honors, reflect for the value walk, and testing for the flag pattern inside go test, nodejs.org for node:test and node:assert/strict, plus the dota-helper repository's engine (internal/ingest, internal/emit, internal/order) and picker layer this chapter adapts, read at `C:\Users\lavantien\dev\github\dota-helper`, all accessed 2026-10-05. Verified by `go/data/internal/engine` tests, 47 of them across 12 files, under `go vet`, `go test`, and the race detector, and the twin, 8 node tests, under `make verify-data-twin`, with the parity artifact pinned byte for byte in `testdata/parity-data.js`.

34. capstone: concurrent crawler, indexer, and web ui

The capstone is a small search engine front half: a crawler walks a link graph under a concurrency cap, the indexer folds fetched pages into an inverted index with uuid stamped documents, the store writes documents and postings into sqlite through a pure Go driver, and a web ui serves search and crawl control over htmx fragments that templ components render. It lives in this repo at `go/capstone/`, its own go module, 32 tests, zero skipped, runnable from that folder with `go test`, stable across repeated runs because nothing in it sleeps, the ui's tests included: they wait on the crawl's done channel, never on a clock. Every chapter's material appears once, for real: the walkers are goroutines under a semaphore and a WaitGroup, the fetch transport is an interface, the errors wrap, the documents carry v7 uuids, the snapshots are deterministic, the result type uses a generic method, and the ui is the fragment contract chapters 16 and 17 taught, answered over the real pipeline.

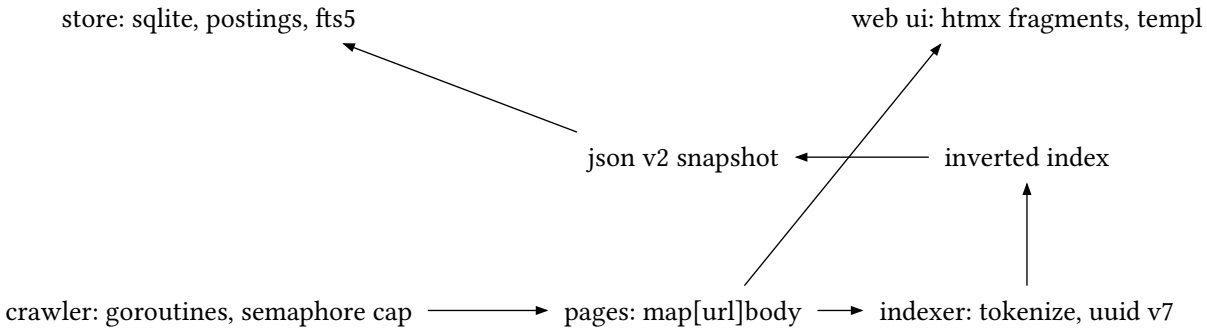


figure 222: capstone pipeline

34.1. the fetch boundary

Transport is an interface with one method, so the test suite drives the crawler over an in-memory web with fetch counters and a concurrency gauge, and production plugs in net/http without touching the walker:

```
type Page struct {
  URL   string
  Body  string
  Links []string
}

// Fetcher abstracts transport. The tests drive it with an in-memory
// fake, production plugs in net/http.
type Fetcher interface {
  Fetch(ctx context.Context, url string) (Page, error)
}
```

listing 270: page, fetcher, the whole boundary (source: go/capstone/crawler/crawler.go, lines 14-24)

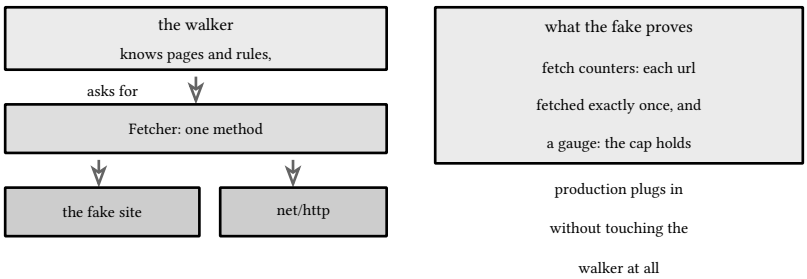


figure 223: the one method boundary: walker above, two transports below

The fake site in the test file is 34 lines and knows every page visit, which is how the suite proves each url is fetched exactly once even when two pages link it, and how it proves the concurrency cap holds.

34.2. result, the generic method

The walker's plumbing carries values-or-errors, and go 1.27 lets that type be honest with a method that adds its own parameter:

```
type Result[T any] struct {
  Value T
  Err   error
}

func ResultOf[T any](v T) Result[T] { return Result[T]{Value: v} }

func Failed[T any](err error) Result[T] { return Result[T]{Err: err} }

// UnwrapOr yields the value, or the fallback when the chain failed.
func (r Result[T]) UnwrapOr(fallback T) T {
  if r.Err != nil {
    return fallback
  }
  return r.Value
}

// Map transforms the value and short circuits the error. A method
// declaring its own type parameter: impossible before go 1.27.
func (r Result[T]) Map[U any](f func(T) U) Result[U] {
  if r.Err != nil {
    return Result[U]{Err: r.Err}
  }
  return Result[U]{Value: f(r.Value)}
}
```

listing 271: result of t, mapped to result of u (source: go/capstone/crawler/crawler.go, lines 27-51)

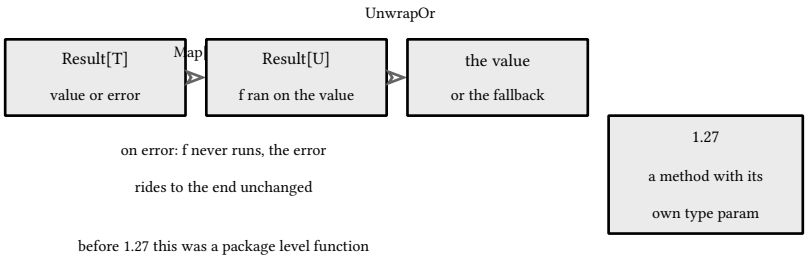


figure 224: result chains with the error riding along, map short circuits

`Map` transforms the value and short circuits the error, `UnwrapOr` terminates the chain with a fallback, and the tests pin both behaviors including that a failed result never invokes the mapping function. Before 1.27 this type needed a package level `MapResult` function, the workaround the release notes retired.

34.3. the walk

Breadth first, depth limited, concurrency capped, cancellation aware, dedup on enqueue:

```
// Crawl walks start's link graph breadth first up to depth hops,
// fetching at most workers pages concurrently. Each url is fetched at
// most once, fetch failures are recorded and skipped, and context
// cancellation stops the walk and surfaces in the returned errors.
func Crawl(ctx context.Context, start string, depth, workers int, fetch Fetcher) (map[string]string,
[]error) {
  var mu sync.Mutex
  pages := make(map[string]string)
  errs := make([]error, 0)
  visited := map[string]bool{start: true}
  sem := make(chan struct{}, max(workers, 1))
  var wg sync.WaitGroup

  var visit func(url string, depth int)
  visit = func(url string, depth int) {
    defer wg.Done()

    // a pre-cancelled context must never reach the fetcher, and a
    // select alone cannot promise that: with a free slot both
    // cases are ready and the choice is random
    if ctx.Err() != nil {
      return
    }
    select {
    case sem <- struct{}{}:
      defer func() { <-sem }()
    case <-ctx.Done():
      return
    }

    page, err := fetch.Fetch(ctx, url)
    mu.Lock()
    if err != nil {
      errs = append(errs, fmt.Errorf("fetch %s: %w", url, err))
    } else {
      pages[page.URL] = page.Body
    }
    mu.Unlock()
    if err != nil || depth <= 0 {
      return
    }
```

listing 272: the visit recursion under semaphore and waitgroup (source: go/capstone/crawler/crawler.go, lines 53-92)

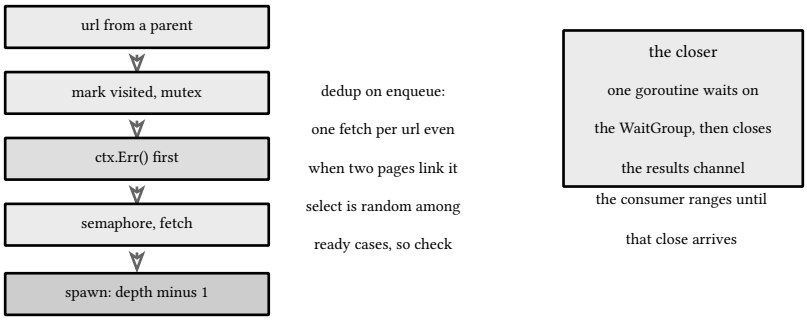


figure 225: one visit: dedup before fetch, cancel before select, spawn at lower depth

Four rules interact. The semaphore caps simultaneous fetches, and the test asserts the gauge never crosses the cap. The visited map is marked when a url is enqueued, not when fetched, so two pages linking the same target

cause one fetch, and the mutex around the mark is what makes the check-and-set atomic. The WaitGroup counts outstanding visits, a closer goroutine converts `wg.Wait` into a channel, and the outer select honors cancellation while draining. Failures append to a slice and the walk continues, because one dead link is not a dead crawl, while a cancelled context stops spawning and surfaces `context.Canceled` in the returned errors.

PITFALL: select is random among ready cases

The first draft guarded the fetch with a select over the semaphore slot and `ctx.Done()`. With a pre-cancelled context and a free slot both cases are ready, select picks randomly, and the test caught the crawler fetching pages it had no right to touch. The explicit `ctx.Err()` check before the select is the fix, a lesson the test suite wrote into this chapter.

34.4. the index side

Documents tokenize on whitespace, lowercase, trim edge punctuation, and take a v7 uuid so ids sort by creation:

```
// Doc is one indexed page.
type Doc struct {
  ID    uuid.UUID
  URL   string
  Words []string
}

// NewDoc tokenizes a body: whitespace split, edge punctuation trimmed,
// lowercased, tiny on purpose. Lowercasing the body once instead of
// each word turns one allocation per capitalized word into one for
// the whole body, measured in the profiled section of the capstone
// chapter. The id is uuid v7, so ids sort by creation time.
func NewDoc(url, body string) Doc {
  fields := strings.Fields(strings.ToLower(body))
  words := make([]string, len(fields))
  for i, f := range fields {
    words[i] = strings.Trim(f, ",.!?;:")
  }
  return Doc{ID: uuid.NewV7(), URL: url, Words: words}
}
```

listing 273: doc and tokenizer, the lowered-body form the profiled section pays for (source: go/capstone/index/index.go, lines 15-34)

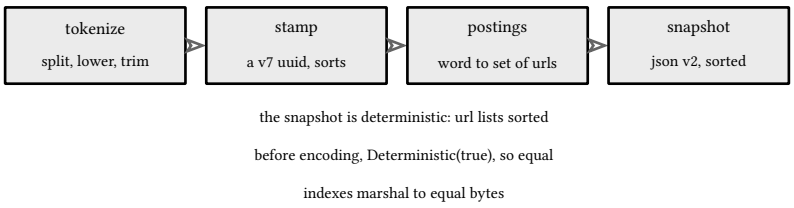


figure 226: body to doc to postings to snapshot, one fold each way

The index itself is the classic inverted structure, word to set of urls, built in one fold, queried in one lookup plus a sort:

```
// Index is an inverted index: word -> the set of urls containing it.
type Index struct {
  postings map[string]map[string]struct{}
}

// Build folds documents into postings.
func Build(docs []Doc) *Index {
  ix := &Index{postings: make(map[string]map[string]struct{})}
  for _, doc := range docs {
    for _, w := range doc.Words {
      set, ok := ix.postings[w]
      if !ok {
        set = make(map[string]struct{})
        ix.postings[w] = set
      }
      set[doc.URL] = struct{}{}
    }
  }
  return ix
}

// Search returns the urls containing word, sorted.
func (ix *Index) Search(word string) []string {
  set := ix.postings[strings.ToLower(word)]
  if len(set) == 0 {
    return nil
  }
  return slices.Sorted(maps.Keys(set))
}

// Vocabulary is the count of distinct indexed words.
func (ix *Index) Vocabulary() int { return len(ix.postings) }
```

listing 274: build, search, vocabulary (source: go/capstone/index/index.go, lines 36-67)

Snapshots go through json v2 with `Deterministic(true)` over url lists sorted before encoding, so the test asserts byte equality between two snapshots of equal indexes, and `Load` is the round trip that must survive:

```
// snapshotForm is the serializable shape: url lists sorted, so the
// bytes are identical for identical indexes.
type snapshotForm struct {
  Postings map[string][]string `json:"postings"`
}

// Snapshot serializes the index deterministically: sorted map keys
// from json.Deterministic, sorted url lists from here.
func (ix *Index) Snapshot() ([]byte, error) {
  form := snapshotForm{Postings: make(map[string][]string, len(ix.postings))}
  for word, set := range ix.postings {
    form.Postings[word] = slices.Sorted(maps.Keys(set))
  }
  return json.Marshal(form, json.Deterministic(true))
}

// Load rebuilds an index from snapshot bytes.
func Load(b []byte) (*Index, error) {
  var form snapshotForm
  if err := json.Unmarshal(b, &form); err != nil {
    return nil, err
  }
  ix := &Index{postings: make(map[string]map[string]struct{}, len(form.Postings))}
  for word, urls := range form.Postings {
    set := make(map[string]struct{}, len(urls))
    for _, u := range urls {
      set[u] = struct{}{}
    }
    ix.postings[word] = set
  }
  return ix, nil
}
```

listing 275: snapshot form, marshal, load (source: go/capstone/index/index.go, lines 69-100)

34.5. the store, sqlite without cgo

The index side lives in memory, and the store package makes the results survive a restart: documents and postings written to sqlite through `database/sql` over modernc.org/sqlite, a pure Go driver, so the module needs no cgo toolchain. The schema is three tables, documents keyed by the doc's uuid, the postings inverted index as a `WITHOUT ROWID` pair keyed term first, and an fts5 virtual table holding each doc's joined words. `Open` applies it idempotently, the dsn turns on foreign keys, a wal journal, and a busy timeout, and the pool holds a single connection because sqlite allows one writer, so `SaveDoc` transactions never meet the busy handler:

```
var schema = []string{
  `CREATE TABLE IF NOT EXISTS documents (
    id    BLOB PRIMARY KEY,
    url   TEXT NOT NULL UNIQUE,
    words TEXT NOT NULL
  )`,
  `CREATE TABLE IF NOT EXISTS postings (
    term   TEXT NOT NULL,
    doc_id BLOB NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    PRIMARY KEY (term, doc_id)
  ) WITHOUT ROWID`,
  `CREATE INDEX IF NOT EXISTS postings_doc_id ON postings(doc_id)`,
  `CREATE VIRTUAL TABLE IF NOT EXISTS documents_fts USING fts5(text, doc_id UNINDEXED)`,
}

// Open connects to the database at path, creating it when absent, and
// applies the schema: documents keyed by the doc's uuid, the postings
// inverted index, and an fts5 table over each doc's searchable text.
func Open(path string) (*Store, error) {
  dsn := "file:" + filepath.ToSlash(path) +
    "?_pragma=foreign_keys(1)&_pragma=journal_mode(WAL)&_pragma=busy_timeout(5000)"
  db, err := sql.Open("sqlite", dsn)
  if err ≠ nil {
    return nil, err
  }
  // sqlite allows one writer; a single pooled connection keeps
  // SaveDoc transactions from ever meeting the busy handler
  db.SetMaxOpenConns(1)
  for _, stmt := range schema {
    if _, err := db.Exec(stmt); err ≠ nil {
      db.Close()
      return nil, fmt.Errorf("apply schema: %w", err)
    }
  }
  return &Store{db: db}, nil
}
```

listing 276: schema and open, wal journal, one pooled connection (source: go/capstone/store/store.go, lines 23-58)

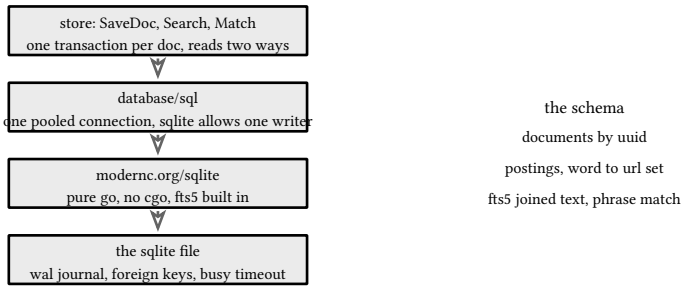


figure 227: the store stack: one interface, one pooled connection of one, one file

SaveDoc writes one document as one transaction: the row upserts on its uuid, a resave deletes the previous postings and fts row before writing new ones so replacement never appends, the words dedup into postings, and the joined text lands in the fts table, commit or rollback as a unit:

```
// SaveDoc writes one document, its postings, and its searchable text
// atomically. Resaving the same doc id replaces its previous words.
func (s *Store) SaveDoc(ctx context.Context, doc index.Doc) error {
  id := doc.ID[:]
  text := strings.Join(doc.Words, " ")

  tx, err := s.db.BeginTx(ctx, nil)
  if err != nil {
    return err
  }
  defer tx.Rollback()

  _, err = tx.ExecContext(ctx,
    `INSERT INTO documents (id, url, words) VALUES (?, ?, ?)
     ON CONFLICT(id) DO UPDATE SET url = excluded.url, words = excluded.words`,
    id, doc.URL, text)
  if err != nil {
    return err
  }
  for _, stmt := range []string{
    `DELETE FROM postings WHERE doc_id = ?`,
    `DELETE FROM documents_fts WHERE doc_id = ?`,
  } {
    if _, err := tx.ExecContext(ctx, stmt, id); err != nil {
      return err
    }
  }
  seen := make(map[string]struct{}, len(doc.Words))
  for _, w := range doc.Words {
    if _, dup := seen[w]; dup {
      continue
    }
    seen[w] = struct{}{}
    if _, err := tx.ExecContext(ctx,
      `INSERT INTO postings (term, doc_id) VALUES (?, ?)`, w, id); err != nil {
      return err
    }
  }
  _, err = tx.ExecContext(ctx,
    `INSERT INTO documents_fts (text, doc_id) VALUES (?, ?)`, text, id)
  if err != nil {
    return err
  }
  return tx.Commit()
}
```

listing 277: savedoc, one transaction for document, postings, fts (source: go/capstone/store/store.go, lines 63-107)

Reading goes two ways. `Search` is the persisted mirror of `index.Search`, postings joined to documents and ordered by url, the term lowercased first. `Match` runs text as an fts5 phrase with inner quotes doubled, so arbitrary crawled input is always a phrase and never fts5 syntax:

```
// Search returns the urls of documents containing word, sorted, the
// persisted mirror of index.Index.Search.
func (s *Store) Search(ctx context.Context, word string) ([]string, error) {
  return s.urls(ctx, `SELECT d.url
    FROM postings p JOIN documents d ON d.id = p.doc_id
    WHERE p.term = ?
    ORDER BY d.url`, strings.ToLower(word))
}

// Match runs text as an fts5 phrase query over the searchable text,
// quoted so arbitrary input is a phrase and never fts5 syntax.
func (s *Store) Match(ctx context.Context, text string) ([]string, error) {
  phrase := `"` + strings.ReplaceAll(text, `"`, `""`) + `"`
  return s.urls(ctx, `SELECT d.url
    FROM documents_fts f JOIN documents d ON d.id = f.doc_id
    WHERE documents_fts MATCH ?
    ORDER BY d.url`, phrase)
}
```

listing 278: search over postings, match as a quoted phrase (source: go/capstone/store/store.go, lines 109-126)

Six tests pin the package. One probes the driver, asserting `sqlite_version()` is at least 3.53, and another that the pinned modernc build accepts fts5 virtual tables at all. Reopen survival closes the store and searches again from disk, a resave replaces postings without duplicating the document row, `Match` agrees with `Search` on single terms, and the phrase narrows past them, “go notes” matching one page where “go” alone matches two.

34.6. the web ui

The ui is chapter 16’s contract answered with chapter 17’s components: one htmx page whose search box and status panel poll fragments, every fragment a templ component rendered to the response, htmx 4.0.0 vendored under `internal/web/static/` and pinned by the same SRI digest recipe, the served bytes hashed by a test. A `CrawlService` wraps the crawler behind a mutex, seeded with a fake site in the style of the crawler package’s own test fixture, so the crawl endpoint needs no network. A finished crawl folds its pages through `index.NewDoc` into the store, keeps the json snapshot for serving, and closes a done channel the tests wait on:

```
// CrawlService wraps the crawler behind a mutex. Start launches one
// crawl of the fake site on its own goroutine, Wait blocks until it
// finishes, and a finished crawl saves its documents through the
// store and keeps the index's json snapshot bytes for /snapshot.json.
type CrawlService struct {
  mu      sync.Mutex
  start   string
  depth   int
  workers int
  fetch   crawler.Fetcher
  store   *store.Store
  running bool
  pages   int
  errs    int
  snap    []byte
  done    chan struct{}
}
```

listing 279: the service: crawler, fetcher, and store behind one mutex (source: go/capstone/internal/web/crawl.go, lines 72-88)

The routes are six, method patterns again, so a wrong verb is a 405 from the mux itself. `cmd/serve` mounts this handler over a database file and a port, the whole binary:

```
// Handler routes the whole ui. Method patterns give the 405s.
func (u *UI) Handler() http.Handler {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /{$}", u.page)
  mux.HandleFunc("GET /search", u.search)
  mux.HandleFunc("GET /fragment/status", u.status)
  mux.HandleFunc("POST /crawl/start", u.crawlStart)
  mux.HandleFunc("GET /snapshot.json", u.snapshot)
  mux.Handle("GET /static/", u.static)
  return mux
}
```

listing 280: six routes: page, search, status, crawl, snapshot, static (source: go/capstone/internal/web/web.go, lines 49-59)

Search splits by word count: one word goes through the postings lookup, several through the fts5 phrase match, the store's two read paths from the previous section, and a missing q is a 400 whose body is the error row itself, swapped in 4.0 style:

```
// search answers the fragment the box polls: one word goes through
// the postings lookup, several go through the fts5 phrase match, and
// a missing q is a 400 whose body is the error row itself.
func (u *UI) search(w http.ResponseWriter, r *http.Request) {
  q := strings.TrimSpace(r.URL.Query().Get("q"))
  if q == "" {
    render(w, r, http.StatusBadRequest, SearchResults("", nil, "q is required"))
    return
  }
  var (
    hits []string
    err  error
  )
  if len(strings.Fields(q)) > 1 {
    hits, err = u.store.Match(r.Context(), q)
  } else {
    hits, err = u.store.Search(r.Context(), q)
  }
  if err != nil {
    render(w, r, http.StatusBadGateway, SearchResults(q, nil, err.Error()))
    return
  }
  render(w, r, http.StatusOK, SearchResults(q, hits, ""))
}
```

listing 281: the fragment route, both store read paths, the 400 (source: go/capstone/internal/web/web.go, lines 65-88)

The status panel is the polled fragment: a templ component carrying its own id, the interval, and the outerHTML swap that keeps the id stable across polls, exactly the shape chapter 16 hand built and chapter 17 compiled:

```
// StatusPanel is the polled fragment: it carries its own id and
// replaces itself whole every two seconds, outerHTML keeping the id
// stable across polls.
templ StatusPanel(s Status) {
  <div id="status" hx-get="/fragment/status" hx-trigger="every 2s" hx-swap="outerHTML">
    if s.Running {
      <p>state: crawling the fake site</p>
    } else {
      <p>state: idle</p>
    }
    <p>pages indexed: { s.Pages }</p>
    <p>fetch errors: { s.Errs }</p>
  </div>
}
```

listing 282: the polled fragment as a component, every 2s, outerHTML (source: go/capstone/internal/web/status.templ, lines 3-16)

The suite waits rather than sleeps. `Start` hands back nothing, but `Wait` blocks on the done channel the run closes, so the test starts a crawl, waits, then reads the fragment over the completed state, deterministic and fast:

```
func TestStatusFragmentAfterCompletedCrawl(t *testing.T) {
  ui, ts := newUI(t)
  if !ui.crawl.Start() {
    t.Fatal("first start refused")
  }
  ui.crawl.Wait()
  _, _, frag := get(t, ts.URL+"/fragment/status")
  if !strings.Contains(frag, "state: idle") {
    t.Fatalf("crawl never finished: %s", frag)
  }
  if !strings.Contains(frag, "pages indexed: 4") {
    t.Fatalf("fragment = %s, want 4 pages", frag)
  }
  if !strings.Contains(frag, "fetch errors: 0") {
    t.Fatalf("fragment = %s, want 0 errors", frag)
  }
}
```

listing 283: wait on the channel, then assert the fragment over the result (source: go/capstone/internal/web/web_test.go, lines 181-197)

The remaining tests pin the contract end to end: the SRI tag and the served digest, the seeded search fragment, the phrase narrowing past the single term, the hostile query escaped in the fragment, the crawl start's `HX-Trigger` header, the 409 for a second start while one runs, observed through a gated fetcher that holds the crawl mid flight, and the 405s. Twelve tests, all over real http through `httptest`, no browser.

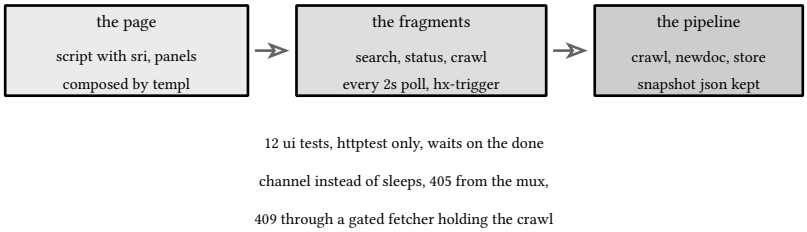


figure 228: the ui: fragments over the real pipeline, both taught chapters paying off

34.7. profiled, and why

The module measures itself. Four benchmark functions in `index/bench_test.go` run the real paths, tokenize, fold, lookup, serialize, over a fixed seeded corpus of 200 documents of roughly 60 mixed-case words from a 400-word vocabulary, and one real change went through the whole discipline the profiling chapter teaches:

```
            | bench-old.tmp |            bench-new.tmp             |
            |    sec/op     |    sec/op     vs base                |
NewDoc-20      4.937µ ± 27%   3.982µ ± 26%  -19.34% (p=0.009 n=10)
Build-20       1.695m ±  6%   1.662m ±  6%        ~ (p=0.353 n=10)
Search-20      2.301µ ±  5%   2.226µ ±  5%        ~ (p=0.247 n=10)
Snapshot-20    2.796m ±  2%   2.830m ±  6%        ~ (p=0.631 n=10)
geomean        85.66µ         80.35µ         -6.20%
```

listing 284: the A/B: one change, three flat controls (source: go/capstone/index/benchstat.txt, lines 11-17)

The change is in the tokenizer listing above: `strings.ToLower` moved from each word to the whole body before `Fields`, so one allocation replaces one per capitalized word. The table says 19.34 percent faster with p=0.009 over ten runs each, 62 allocations down to 3 at p=0.000, and bytes unchanged at p=1.000, the same 2.469 KiB reshaped from 62 small strings into 3 allocations, exactly the size class arithmetic chapter 10 predicts. The three flat rows are the honest part: Build, Search, and Snapshot do not touch NewDoc's body path, and benchstat reports them statistically unchanged instead of letting a geomean imply the whole engine improved. The suite pins the new count at exactly 3 with `testing.AllocsPerRun` over the same fixed body.

The next question is where the rest of the time goes, and the cpu profile over `BenchmarkBuild` answers it:

```
      flat  flat%   sum%        cum   cum%
     0.21s 10.10% 10.10%      0.21s 10.10%  internal/runtime/maps.memHashAES
     0.15s  7.21% 17.31%      0.15s  7.21%  internal/runtime/maps.ctrlGroup.matchH2 (inline)
     0.10s  4.81% 22.12%      0.17s  8.17%  runtime.preemptM
     0.10s  4.81% 26.92%      0.14s  6.73%  runtime.tryDeferToSpanScan
     0.09s  4.33% 31.25%      0.09s  4.33%  internal/runtime/maps.(*groupReference).key (inline)
     0.08s  3.85% 35.10%      0.83s 39.90%  runtime.mapassign_faststr
     0.08s  3.85% 38.94%      0.24s 11.54%  runtime.scanObject
```

listing 285: build's flat top: the map is the machine (source: go/capstone/index/pprof-flat.txt, lines 13-20)

Reading it through chapters 10 and 11: `mapassign_faststr` holds 39.90 percent cumulative, and the flat top is the swiss table's own parts, `memHashAES` at 10.10 percent hashing every word key and `matchH2` at 7.21 percent probing groups. Map growth and rehash take another 15.38 percent cumulative, and the gc helpers, `gcDrain` at 26.92 and `scanObject` at 11.54 percent cumulative, exist because Build allocates 2814 objects per op, 952.9 KiB, the postings sets themselves. The profile verdict: Build is not slow code, it is the data structure, an inverted index over 12000 word insertions is map work, and the remaining lever would be allocation shape, not algorithm.

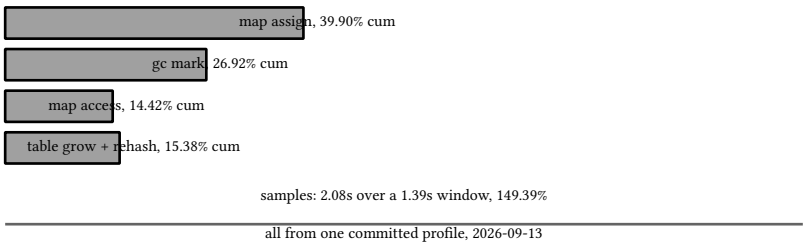


figure 229: where build's cpu goes, one profile, four buckets

34.8. what the capstone proves

The toolchain chapter's build tags and modules shape the two-module layout, one for samples, one for the capstone. Lexical rules show up in the tokenizer's string handling. The type system carries the maps and structs,

generics carries `Result[T]` with its method, and 1.27's inference lets the result helpers instantiate at call sites. Functions contribute defer, closures over the walk state, and method sets at the Fetcher boundary. Concurrency is the crawler itself, semaphore, WaitGroup, closer goroutine, mutex, cancellation. Errors wrap with `%w` and never panic. The runtime chapter pays off in the leak profile test, which writes the 1.27 `goroutineleak` profile after a crawl to prove the walker leaves nothing behind. The stdlib tours supply `slices.Sorted`, `maps.Keys`, json v2, and uuid. The framework chapters pay off in the ui: htmx 4 supplies the fragment contract the routes answer, polling, response headers, error fragments, and templ renders every fragment as a typed component with its codegen committed. The store was the one step past the tours when this capstone closed, `database/sql` against modernc's pure Go sqlite and then the only capstone ground no chapter covered, and the service part's store chapter has since taken that ground further, an owned wal engine registered under the same `database/sql` interface. Its six tests pin reopen survival, resave replacement, and the fts5 phrase query. The testing chapter's discipline is the suite itself, and the idioms chapter is the shape: accept interfaces, return structs, zero values usable, errors as values.

chapter	payoff	chapter	payoff
toolchain	two modules	errors	%w, no panic
lexical	tokenizer strings	runtime	leak profile test
types	maps and structs	stdlib	json v2, uuid, sorted
generics	Result[T].Map	testing	the suite itself
functions	defer, closures	idioms	the whole shape
concurrency	the walker	the store	past the tours
		frameworks	the ui: htmx, templ

32 tests, zero skipped, no sleeps anywhere

figure 230: every chapter pays off once, in place

34.9. the service part

This capstone closed at 32 tests, and the book then grew a service part around it, chapters 18 through 30, the user REST api built as a second go module at `books/go/api`, twelve chapters from the kernel to the ship chapter plus the suite. That part took the store, this chapter's one step past the tours: modernc's pure Go sqlite left, replaced by a wal engine owned bottom up and registered as the `database/sql` driver, and the module's go.mod now holds zero community requires. `make verify-go` runs the part beside this capstone, vet, the plain suite, and the race leg over every module, and its recorded closeout is 182 tests, 6 fuzz targets, 1 example, and 2 benchmarks, closed 2026-09-26.

sources: the capstone module in this repo, `go/capstone/`, tested with `go test` from its own directory and through `make verify-go`, 32 tests on go1.27.0, benchmarked and profiled with the chapter 11 toolchain, captures committed beside the numbers (`index/bench-old.txt`, `bench-new.txt`, `benchstat.txt`, `pprof-flat.txt`, `pprof-cum.txt`), all measured 2026-09-13 on this machine. The ui rides htmx 4.0.0 and templ v0.3.1020, pinned and sourced in chapters 16 and 17.

35. appendices: coverage and sources

The coverage matrices are this book's audit trail. All 25 keywords enumerated from the specification, every stdlib component family the book touches, each mapped to the chapter that teaches it, rendered here at milestone time. The build enforces honesty from the other side: every chapter listing reads a real file that a go module in this repo compiles and tests.

35.1. keyword coverage

Go reserves 25 words and freezes the list, so this matrix has one kind and no unmapped rows: 0. The program structure chapter carries the statement words, functions and concurrency split the rest, and the predeclared identifiers that look like keywords, `len` through `println`, live in chapter 2 as the lexical chapter's counterexample.

word	kind	taught in
break	reserved	ch 8
default	reserved	ch 8
func	reserved	ch 5
interface	reserved	ch 3
select	reserved	ch 6
case	reserved	ch 8
defer	reserved	ch 5
go	reserved	ch 6
map	reserved	ch 3
struct	reserved	ch 3
chan	reserved	ch 6
else	reserved	ch 8
goto	reserved	ch 8
package	reserved	ch 1
switch	reserved	ch 8
const	reserved	ch 8
fallthrough	reserved	ch 8
if	reserved	ch 8
range	reserved	ch 8
type	reserved	ch 8
continue	reserved	ch 8
for	reserved	ch 8
import	reserved	ch 1
return	reserved	ch 5
var	reserved	ch 8

35.2. stdlib coverage

Component families against chapters, with the status column marking the one documented-not-tested entry and the pointers that defer to the systems books.

component family	chapter	status
fmt verbs and Sprint family	ch 12	covered
errors: New, Is, As, Join	ch 7	covered

component family	chapter	status
slices toolkit and iterators	ch 12	covered
maps toolkit and iterators	ch 12	covered
cmp: Compare, Or	ch 12	covered
time: Time, Duration, parse	ch 12	covered
strings/bytes: CutLast, Builder, Fields	ch 12	covered
log/slog handlers and attrs	ch 12	covered
encoding/json/v2 core	ch 13	covered
encoding/json/jsontext tokens	ch 13	covered
uuid: New, NewV7, Parse	ch 13	covered
net/http client and server	ch 13	covered
crypto: sha256, rand, mldsa	ch 13	covered
simd vectors	ch 13	documented, GOEXPERIMENT gated
os files and TempDir	ch 13	covered
io interfaces and combinators	ch 13	covered
testing: tables, subtests, examples	ch 14	covered
testing: fuzzing	ch 14	covered
testing: benchmarks and b.Loop	ch 14	covered
testing/synctest bubbles	ch 14	covered
testing: property invariants via fuzz targets	ch 30	covered
tooling: merged coverage profiles across legs	ch 30	covered
runtime/pprof profiles	ch 9	covered
runtime/metrics gauges	ch 11	covered
runtime/trace and execution tracer	ch 11	covered
sync.Pool and per-P caches	ch 10	covered
math/rand/v2 seeded PCG	ch 10	covered
sync: Mutex, Once, WaitGroup	ch 6	covered
go/scanner and go/token	ch 2	covered
go:embed, go:generate, build tags	ch 1	covered
context cancellation	ch 6	pointer, deep use in capstone
database/sql with modernc.org/sqlite	ch 34	covered
database/sql driver implementation, Driver Conn Stmt Tx, and hash/crc32 Castagnoli	ch 23	covered
embed.FS served over http.FS	ch 16	covered
net/http mux method patterns, 405s	ch 16	covered
crypto/sha512 digest pinning	ch 16	covered

35.3. pinned sources

Every chapter ends with its own sources line naming the official sections consulted while writing it. The book level pins are:

topic	url	accessed
go 1.27 release notes	go.dev/doc/go1.27	2026-09-08
language specification	go.dev/ref/spec	2026-09-08
effective go	go.dev/doc/effective_go	2026-09-08
modules reference	go.dev/ref/mod	2026-09-08

topic	url	accessed
encoding/json/v2	pkg.go.dev/encoding/json/v2	2026-09-08
encoding/json/jsontext	pkg.go.dev/encoding/json/jsontext	2026-09-08
uuid	pkg.go.dev/uuid	2026-09-08
testing/synctest	pkg.go.dev/testing/synctest	2026-09-08
testing	pkg.go.dev/testing	2026-09-08
runtime and runtime/debug	pkg.go.dev/runtime	2026-09-08
diagnostics	go.dev/doc/diagnostics	2026-09-08
generic methods proposal	github.com/golang/go/issues/9859	2026-09-08
htmx 4.0 reference and migration notes	four.htmx.org/docs	2026-09-21
templ syntax, components, and security	templ.guide	2026-09-21

NOTE: version drift rule

These pins were fetched 2026-09-08 against go 1.27.0 on windows with the module pinned `go 1.27`. If an upstream page changes, the delta gets recorded here rather than silently rewriting chapters, so this table is also the book's changelog for documentation movement. The simd row is the honest boundary: documented from the release notes and pkg.go.dev, untested because the experiment gate would pin the module to a toolchain configuration.